Data2Evidence

Architecture Blue Book

Peter Hoffmann

First Edition — April 2026

Preface

This book is the technical reference for the Data2Evidence platform — a microservices system for clinical data analytics built on the OMOP Common Data Model. D2E is difficult to summarize concisely: components have dense interdependencies, many design decisions are shaped by constraints from the OHDSI ecosystem, and most workflows cross several service boundaries. A short overview cannot explain why the plugin system exists, what the four-stage query pipeline buys, or how a Vue [74] micro-frontend came to be nested inside a React [34] portal. This book aims to answer those questions.

Audience

This book addresses several readers:

- **Technical architects** who need to understand and reason about D2E's structural choices — why Trex, why a plugin system, why the four-stage query pipeline.
- **New backend engineers** who must extend the platform without breaking it. They need to know which constraints are accidental and which are load-bearing.
- **Frontend engineers** who work across the React portal shell and the Vue analytics module.
- **DevOps and SRE staff** responsible for deployment, observability, and incident response.
- **Product managers and technical writers** who need to communicate accurately about the platform without working in the codebase directly.

What this book is, and is not

This is a *conceptual* reference. It explains how the platform's pieces fit together and why they are shaped the way they are. It is not an API reference — those exist in the codebase, generated from route declarations, and they evolve faster than print. It is also not a tutorial: it does not walk a new user through their first analytical query, nor an operator through their first deployment.

Where a design decision has consequences that are not obvious from looking at the code, the book explains them. Where a design choice was made for a specific historical reason, the book records that reason so future engineers can evaluate the constraint before changing it.

Structure

The book is divided into five parts. Part I covers the platform as a whole: its layered architecture, its plugin system, its data flows, and its security model. Part II covers the frontend application — the portal shell, the Patient Analytics module, and the supporting tools for data management and notebooks. Part III covers the cloud functions, organized by domain: the Query Engine, data management, OHDSI integration, and the AI and infrastructure services. Part IV documents the integrations: FHIR, Jupyter, and Prefect workflows. Part V covers operational concerns: authentication, deployment, the CLI, and testing.

Each part opens with a short introduction that places its chapters in context. Each chapter follows a consistent structure: *why it exists*, *how it is structured*, *how it integrates with its neighbours*,

and *its design trade-offs*. The book focuses on concepts and design rationale rather than reference details. Route maps, environment variables, configuration tables, and operator runbooks live in the codebase and the platform's operational documentation; this book describes what those things are *for*.

Conventions

API route paths and code identifiers are shown in monospace: `/analytics-svc/api/services/query`. Service names use prose capitalization (Analytics Service); package names are monospace (`d2e-functions`). The terms "function" and "service" have specific meanings: a *function* is a Deno [8] worker isolate hosted inside Trex, while a *service* is a standalone Docker container. Many service names retain the historical "alp-" prefix; the prefix has no functional significance and can be ignored.

Cross-references between chapters are deliberate. When a chapter mentions a concept defined elsewhere, it links rather than re-explains. The glossary at the back defines the platform's domain vocabulary.

Acknowledgments

Data2Evidence is the work of many contributors across multiple organizations and several years. This book draws from their decisions and their code. Any errors of description are the author's; the platform itself is the result of its builders' work.

Contents

PART I

Architecture & Infrastructure

Part I treats D2E as an organism rather than a catalogue of parts. The aim is not to enumerate every component but to show how the platform's pieces are held together by a small number of structural decisions: a single runtime that hosts every cloud function, a plugin system that defines what those functions are, a layered request pipeline that imposes consistent authentication and routing, and a credential model that keeps each tenant's data separate without trusting the network.

Chapter 1 orients readers who have not encountered D2E before. Chapter 2 describes the layered architecture — the path a request takes from browser to database, the Docker network topology, the ways services communicate. Chapter 3 explains the plugin system; this is the chapter to read if you want to understand why D2E's behaviour is defined more by configuration than by code. Chapter 4 walks the data flows that animate the platform: how a query becomes SQL, how a notebook session is born, how a Prefect flow gets dispatched. Chapter 5 covers the security architecture — authentication, authorization, credential encryption, and the boundaries between tenants. Chapter 6 documents the core infrastructure: PostgreSQL [69], Redis [57], Caddy [6], Supabase Storage [66], Logto [33], and the runtime engine that ties them together.

Read these six chapters in order if you are new to the platform. The plugin system chapter is the conceptual crux; everything that follows assumes you understand it.

1 Introduction

1.1 Purpose

Data2Evidence (D2E) is an open-source, microservices-based [30] healthcare data analytics platform built on the Observational Medical Outcomes Partnership (OMOP) Common Data Model [22, 41, 54]. It enables researchers, clinicians, and data analysts to query, filter, aggregate, and export observational health data across multiple database technologies. The platform integrates OHDSI analytical tools [42], AI-assisted features, FHIR interoperability, and workflow orchestration into a unified backend that can be deployed on-premises or in the cloud.

This document provides a comprehensive reference for the entire D2E platform. Part I covers the platform architecture — the plugin system that assembles the platform, end-to-end data flow patterns, the security model, and core infrastructure. Part II documents the frontend application — the micro-frontend architecture, Patient Analytics interface, data management tools, and administration UIs. Part III documents each cloud function — from the query engine and analytics services through user management, dataset administration, OHDSI tools, and AI capabilities. Part IV covers integrations with external systems (FHIR, Jupyter, Prefect workflows and flows). Part V addresses deployment and operations.

1.2 Target Audience

This document is intended for technical architects, DevOps engineers, system administrators, product managers, and new team members who need to understand the D2E backend and how its services collaborate. It is written at a conceptual and architectural level, focused on service responsibilities, data flows, and design decisions.

1.3 Scope

The document covers all backend components: the core infrastructure services (Trex runtime, Caddy [6] reverse proxy, Logto [33] identity provider, PostgreSQL [69], Redis [57], Supabase Storage [66]), the 26 cloud functions deployed via Trex (analytics, query generation, user management, dataset management, OHDSI tools, AI services, and more), the FHIR service module,

the 24 Prefect [55] flows for ETL and analysis, the Jupyter Enterprise Gateway [56] for notebook execution, and the authentication, security, and deployment architecture that ties everything together.

1.4 Conventions

Throughout this document, API route paths are shown in monospace (e.g., `/analytics-svc/api/services/query`). The term "function" refers to a Deno [8] worker isolate hosted inside the Trex runtime — these are the cloud functions that handle business logic. The term "service" refers to a standalone Docker container (Trex, Caddy, Logto, PostgreSQL, etc.). Many service names carry an "alp-" prefix inherited from the platform's development history; the prefix has no functional significance.

2 Design Principles

Before describing how D2E is built, it is worth naming the principles that govern the building. Every architecture has invariants — rules the design enforces, constraints it accepts, decisions it has stopped revisiting. The platform's invariants are not stated in any one place in the code; they are visible only when you have read enough of it to recognize the same shape recurring. This chapter names them, so the rest of the book can reference them without restatement.

The principles are listed in rough order of consequence: the ones that govern the most code first.

2.1 1. Composition over construction

Every capability D2E offers arrives through a plugin manifest. The Trex runtime contains no business logic of its own — not a single cloud function, not a single UI route, not a single Prefect deployment is hardcoded into core. Adding a capability is an act of declaration, not construction: you write a `package.json` that says "this is what I contribute," and the runtime wires it into the running system at startup.

This forbids: hardcoding service routes in core, embedding business logic in the runtime, central registries that have to be edited when a new function is added. It allows: independent deploy cycles per plugin, mixing platform versions across plugins, removing capabilities by removing packages.

2.2 2. The credential boundary is the platform boundary

Database credentials are encrypted with an RSA key pair whose private half lives only inside Trex's process. They are decrypted on demand for each request and never leave the runtime in plaintext form. This is the platform's deepest security commitment, and everything else in the security architecture — the layered authentication, the per-request credential resolution, the dataset-scoped authorization — exists to keep this commitment intact.

This forbids: tenant code receiving raw database credentials, services running with master credentials and acting on behalf of users, trust boundaries inside Trex. It allows: multi-tenant deployments without per-tenant infrastructure, credential rotation without service-by-service updates, audit trails that meaningfully distinguish "who" from "what."

2.3 3. One process, many isolates

Every cloud function runs in its own Deno [8] worker isolate inside the same Trex process. The isolation is at the V8 [14] level — separate environment variables, separate memory limits, separate CPU budgets — but the transport between isolates is in-process IPC, not network calls.

This forbids: relying on operating-system-level process isolation for security, sharing global state between functions, assuming a function can read another function's environment. It allows: low-latency calls between co-located functions, per-function resource limits enforced by the runtime, and a single deployable artifact that hosts arbitrary numbers of functions.

2.4 4. Declarative over imperative

The plugin manifest describes *what* a plugin contributes; the runtime decides *how* those contributions are wired in. Routes are declared, scopes are declared, environment variable namespaces are declared, even Prefect flow deployments are declared. The runtime's job is to read declarations and produce the running system.

This forbids: imperative startup hooks that mutate the platform's state, plugins that register themselves through code calls, configuration that diverges between declaration and deployment. It allows: every contribution to be inspected statically, the same package to behave identically across environments, and the platform's behaviour to be predicted from package contents alone.

2.5 5. Layered authorization

Every authenticated request passes through four authorization layers, each rejecting independently: token validation (the JWT [25] must be valid and signed by Logto [33]), scope check (the route's required scopes must be in the token), role check [60] (the user's roles must grant those scopes), and dataset check (where applicable, the user must have access to the specific dataset referenced in the request).

This forbids: skipping any layer for "internal" requests, combining authorization with business logic, treating authentication and authorization as a single check. It allows: each layer to be tested and audited in isolation, scope changes without schema changes, and access decisions that depend on the specific dataset being acted on rather than only on the user's coarse role.

2.6 6. Dialect-independent analytics

The Query Engine compiles user intent [1] through a four-stage pipeline (IFR $\rightarrow$ FAST $\rightarrow$ AST $\rightarrow$ SQL) that delays dialect-specific decisions until the final stage. The same IFR runs against PostgreSQL [69], HANA, BigQuery (via TrexSQL caching), and DuckDB [9] without rewrites; only the Stage 4 renderer changes.

This forbids: inlining dialect-specific SQL in higher pipeline stages, leaking PostgreSQL semantics into the FAST representation, optimization passes that assume a particular execution engine. It allows: adding support for a new database backend with a single new renderer, optimizing at the AST level once instead of per-dialect, and testing pipeline stages in isolation against synthetic inputs.

2.7 7. Internal IPC for co-located, HTTP for crossing boundaries

Cloud functions hosted inside the same Trex process communicate via internal message-passing IPC. Calls that leave the process — to Prefect [55], to Logto, to Supabase Storage [66], to Enterprise Gateway [56] — go over HTTP through the SERVICE_ROUTES map. The two transports are interchangeable from the calling function's perspective; both produce a Request/Response pair, and the runtime decides which to use based on whether the target is co-located.

This forbids: hardcoding the choice of transport in calling code, assuming a target is always remote or always local, exposing the IPC mechanism to plugins. It allows: deploying a function inside Trex or as an external service without changing its callers, low-latency local calls without the network stack, and a uniform call surface across topology choices.

2.8 8. Identity outside, authorization inside

Logto owns user identity, including federated sign-in through Microsoft Entra ID [35] and similar providers. D2E owns dataset-scoped authorization — the rule that researcher Alice can analyse the diabetes dataset but not the cardiology one. The boundary is at the token: Logto issues tokens that carry the user's identity claims; D2E's authorization layer reads those tokens and consults its own data to decide what the user is allowed to do.

This forbids: encoding dataset access in OIDC [52] scopes, querying Logto for per-dataset permissions, treating Logto as a general-purpose authorization service. It allows: swapping identity providers without rewriting authorization code, dataset-scoped rules that change frequently without involving Logto, and audit trails that distinguish "who they are" (Logto's responsibility) from "what they can do" (D2E's responsibility).

2.9 9. Object storage is canonical, database is curated

Files — notebook attachments, Parquet [68] exports, FHIR bundles, ETL inputs, encrypted recontact packages — live in Supabase Storage. Structured records — patient demographics, condition occurrences, drug exposures, configurations, bookmarks — live in PostgreSQL. The two are not interchangeable. A file is a thing the platform stores but does not interpret; a database row is a thing the platform reasons about.

This forbids: storing files as database BLOBs, storing structured analytical data only in object storage, mixing the two access patterns in a single service. It allows: backups and replication strategies that match each storage type's needs, scaling object storage independently of the database, and a clear answer to "where should this data go?"

2.10 10. The plugin manifest is the security policy

Roles, scopes, and the URL-to-scope mappings that gate them are all declared in plugin manifests. There is no central RBAC config separate from plugin contributions. When a plugin adds a new route, it simultaneously declares the scope required to access it and assigns that scope to the appropriate roles. The platform's security policy is the union of every plugin's manifest.

This forbids: editing roles or scopes outside of plugin manifests, central security configuration that diverges from what plugins declare, scope tables that are out of date with deployed code. It allows: removing a plugin and removing its entire authorization surface in one motion, security-policy changes that ride the plugin's deploy cycle, and a single source of truth for what the platform allows.

2.11 Reading the rest of the book through these principles

The chapters that follow do not restate these principles; they assume them. When the system architecture chapter says "services communicate via internal IPC for co-located calls," it is appealing to principle 7. When the security architecture chapter describes the authorization layers, it is unfolding principle 5. When the cloud-function chapters describe how each plugin contributes its scopes, they are showing principle 10 in action. The principles are a vocabulary; the rest of the book is written in it.

3 System Architecture

3.1 High-Level Architecture

The D2E backend follows a layered architecture. External clients connect to Caddy [6], a reverse proxy that handles TLS termination. Caddy forwards requests to Trex, the Deno [8]-based runtime engine that serves as the API gateway for all backend operations. Trex hosts 37+ cloud functions as isolated Deno worker processes and also proxies requests to external services (Prefect [55], Enterprise Gateway [56], Logto [33], Supabase Storage [66], and others). All persistent data is stored in PostgreSQL [69], with Redis [57] for caching and Supabase Storage [66] for file/object storage.

The following diagram illustrates the platform's compositional structure using FMC notation [29], where rectangles represent active agents and rounded shapes represent passive data stores:

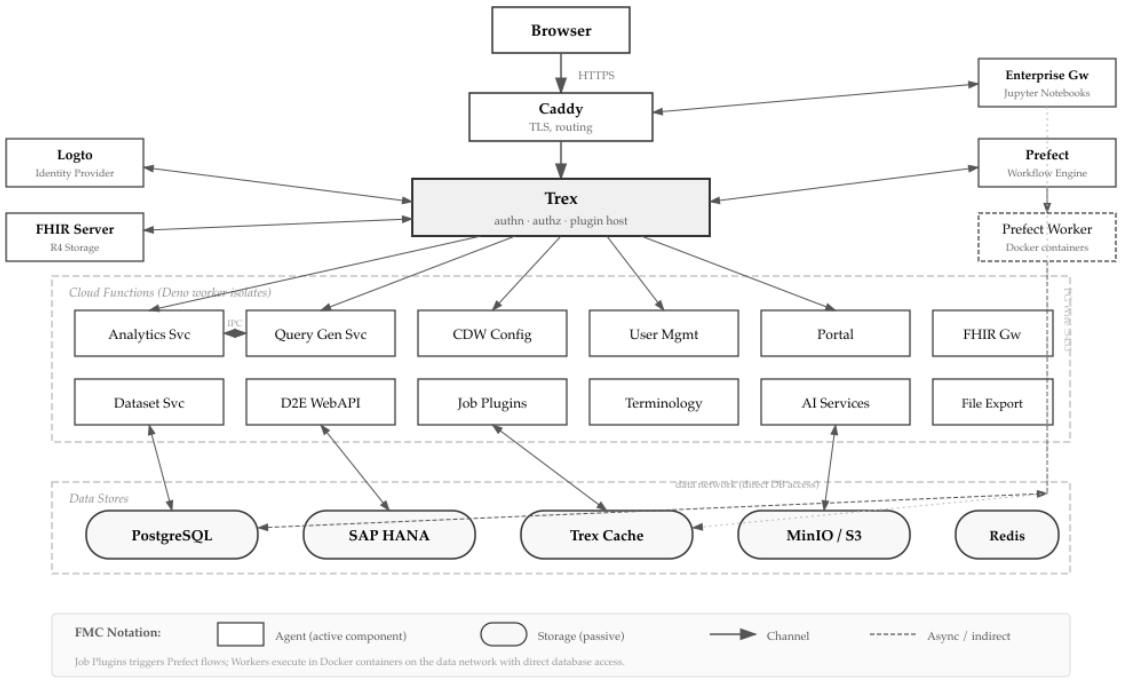

Figure 1: D2E Platform Architecture (FMC Block Diagram)

The service topology, showing the routing paths between components:

3.2 Docker Network Topology

The platform uses four Docker networks to isolate traffic:

Network	Purpose
alp	Primary service mesh connecting all backend services. All inter-service HTTP communication flows through this network.

Network	Purpose
data	Database and data-processing plane. Prefect worker containers and flow execution containers use this network to access databases and the Trex volume.
enterprise-gateway	Jupyter kernel management for researcher access (read/write notebooks).
enterprise-gateway-viewer	Jupyter kernel management for viewer access (read-only notebooks).

3.3 Service Communication Patterns

Services communicate through several mechanisms:

SERVICE_ROUTES Configuration

A JSON map injected into Trex via environment variables maps logical service names to internal URLs. When a cloud function needs to call another service, it looks up the target in this map rather than hardcoding a network address. The map covers the analytical services, the OHDSI surface, the platform infrastructure, the integration endpoints, and the AI surface — in short, every service the platform can route to. New entries are added through plugin manifests rather than by editing the configuration directly.

Trex IPC

Cloud functions hosted within Trex can communicate via in-process channels (Trex.createRequestListener / Trex.userWorkers.create), bypassing HTTP overhead entirely. This is used for performance-critical internal calls between co-located functions.

PG Wire Protocol

Trex exposes port 5433 via a DuckDB [9] pgwire extension, allowing SQL clients (including Jupyter R kernels) to query cached CDW data using the PostgreSQL wire protocol. This provides a familiar interface for data analysts without requiring them to use the HTTP API.

HTTP Proxy

For external services (Prefect, Logto, Enterprise Gateway, Supabase Storage, DICOM [38] server), Trex uses its _addService() mechanism to proxy requests with authentication and authorization middleware applied. The proxy strips the source path prefix when configured via the rmsrc flag.

3.4 Plugin System

Trex loads its functionality through a plugin system. The PLUGINS_SEED environment variable lists npm packages that define the platform's capabilities:

- d2e-functions — the core cloud functions (analytics, user management, dataset, portal, etc.)
- d2e-ui — the frontend web application
- d2e-flows — Prefect workflow definitions
- d2e-atlas — OHDSI Atlas [44] integration
- d2e-fhir-server — FHIR service module

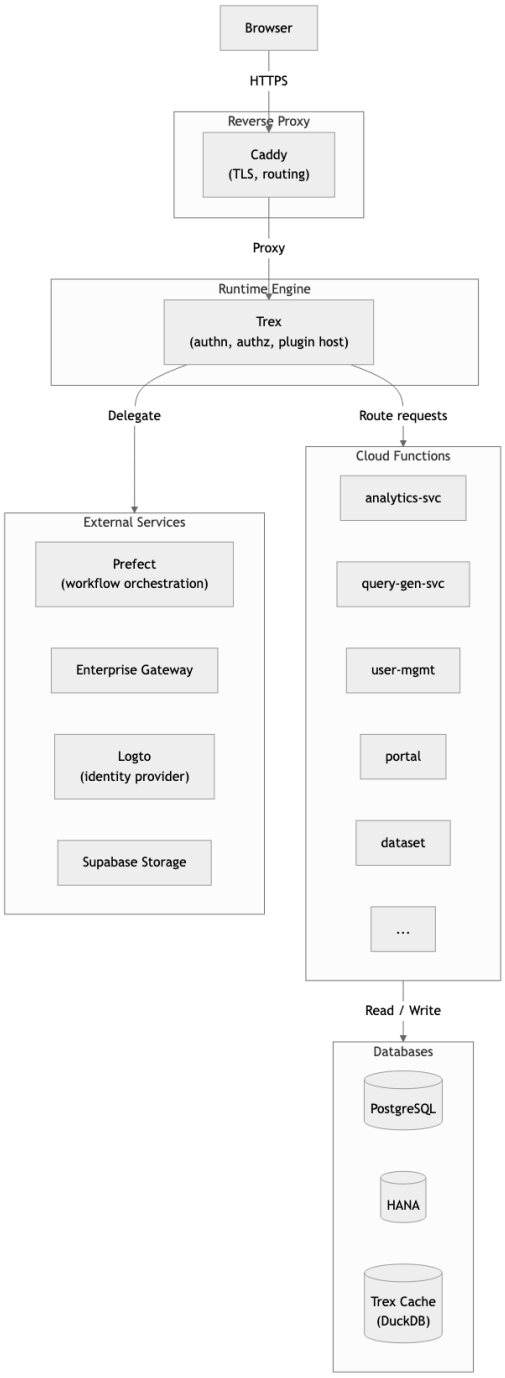

Figure 2: D2E Service Topology

- data-transformation-flow, hades-flow — additional workflow packages

Plugin Manifest Structure

Each plugin provides a manifest with five sections:

- **init** – One-time startup tasks (database migrations, schema creation). Init functions run sequentially before API routes are registered, with optional waitfor URLs and inter-function delays.
- **api** – Route registrations mapping URL paths to function entry points. Each route is wrapped with authn (authentication) and authz (authorization) middleware.
- **roles** – Logto role definitions created during plugin initialization. These extend the platform's RBAC system dynamically.
- **scopes** – Authorization scopes that map roles to URL path + HTTP method permissions.
- **env** – Environment variable isolation per function. Each function receives only the variables it needs, preventing cross-function data leakage.

Function Registration

When Trex processes a plugin manifest, it calls _addFunction() for each API entry. This creates a Deno worker isolate with configurable resource limits:

Parameter	Init Functions	API Functions
Memory Limit	1000 MB	1000 MB
Worker Timeout	3 minutes	30 minutes
CPU Soft Limit	100 seconds	1000 seconds
CPU Hard Limit	200 seconds	2000 seconds
Filesystem Access	Allowed	Allowed
Sloppy Imports	Enabled	Enabled

The isolation ensures that a misbehaving function cannot affect other functions or the Trex process itself. Workers support TypeScript with ES module imports and metadata decorators.

Sloppy Imports is a Deno compatibility feature that relaxes strict ES module resolution rules. By default, Deno requires fully specified import paths including file extensions (e.g., `import {foo} from "./utils.ts"`). With sloppy imports enabled, Deno also resolves imports without extensions and with `index.ts` directory resolution — matching the behavior of Node.js and TypeScript's `moduleResolution: "node"` setting. This is enabled for all D2E cloud functions because many shared libraries and npm packages use Node-style imports that would otherwise fail under Deno's strict resolution.

Environment Variable Substitution

The plugin system supports bash-style variable substitution in environment values, including default values (:-, -), required values (:?, ?), and conditional substitution (:+, +). Circular references are detected with a maximum of 10 resolution iterations.

4 Plugin Architecture

The plugin system is the mechanism by which D2E is assembled. Trex, the platform's Deno [8]-based runtime, contains no hard-coded business logic of its own. Every cloud function, every UI application, every Prefect [55] workflow, and every DuckDB [9] extension enters the system through a single, uniform plugin mechanism. The result is a platform whose capabilities are defined entirely by configuration—by the contents of npm packages whose `package.json` files describe what they contribute and how they should be wired into the running system.

This design choice has far-reaching consequences. It means that the same Trex binary can host a minimal development environment with two plugins or a full production deployment with a dozen. It means that new capabilities—a FHIR service, an AI code-suggestion endpoint, a Strategus [48] results viewer—can be added without modifying a single line of Trex's core code. And it means that the boundary between "platform" and "application" is defined not by a compiled binary but by a JSON array in an environment variable.

Understanding the plugin architecture is essential for anyone who needs to extend D2E, debug a startup failure, or reason about why a particular function receives certain environment variables and not others.

4.1 Plugin Discovery

The entry point for the entire plugin system is a single environment variable: `PLUGINS_SEED`. This variable contains a JSON array of npm package names. When Trex starts, it parses this array and iterates through each name, installing and registering the packages in order.

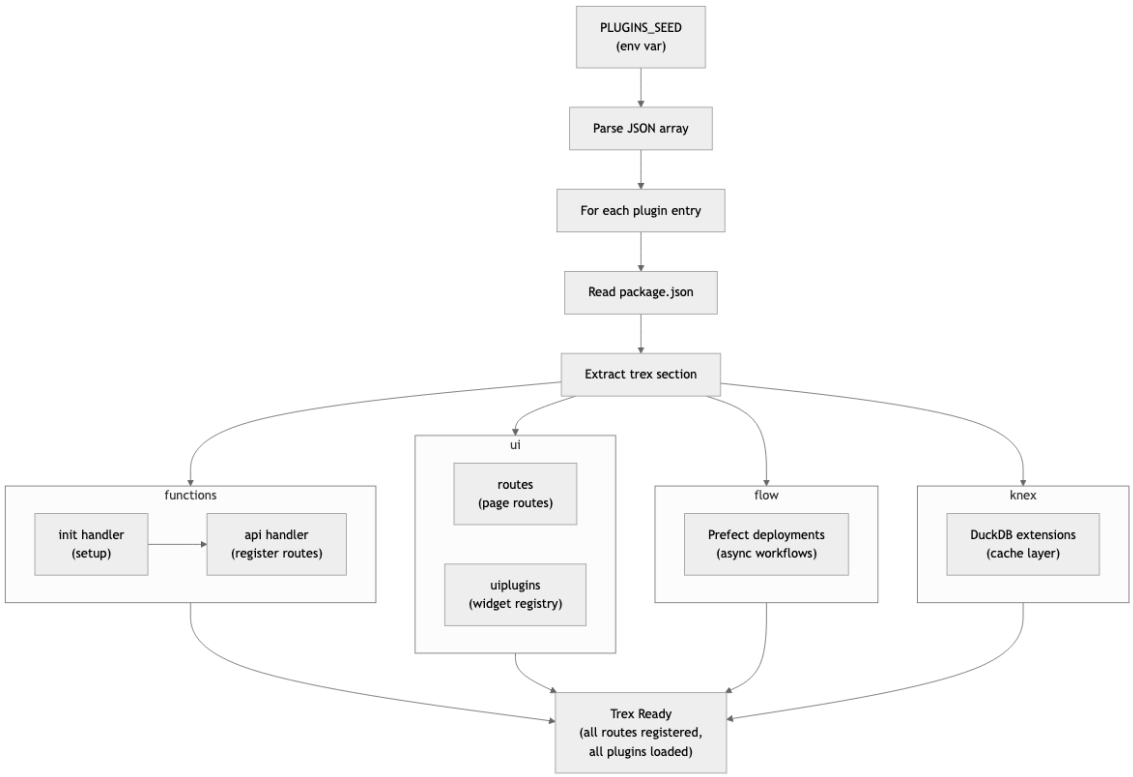

Figure 3: Plugin Loading Process

The discovery process follows a cache-first strategy. For each package in the seed list, Trex checks whether the plugin is already installed by querying a plugin registry table in PostgreSQL. If the plugin exists in the registry and no forced refresh has been requested, Trex skips

the npm installation entirely and loads the plugin's manifest directly from the database record. This prevents redundant network fetches on restart and ensures that a deployment that has already bootstrapped can come back online quickly.

If the plugin is not yet installed—or if a refresh is forced—Trex uses its internal package manager to install the package from the GitHub npm registry. The package is installed into a configured plugin directory under the organization namespace. If no explicit version is specified and a platform-wide API version is configured, that version is appended automatically, allowing operators to pin all plugins to a coordinated release.

Once the package is on disk, Trex reads its `package.json` and extracts the `trex` section—the plugin manifest. The manifest is then dispatched to type-specific handlers based on the top-level keys it contains. There are four recognized plugin types:

- **functions** — Cloud functions implemented as Deno worker isolates. This is the most common type, used to register the platform's backend services.
- **ui** — Frontend web applications served as static assets, with navigation entries injected into the portal shell.
- **flow** — Prefect workflow definitions that are registered as deployments with the Prefect orchestration server.
- **core** — Internal extensions loaded into Trex, such as the pgwire extension that exposes a PostgreSQL [69] wire-protocol interface.

A single package can contain multiple types. Trex iterates over every key in the `trex` section and dispatches each to the appropriate type-specific handler. After all handlers have run, the plugin's manifest is persisted to the registry table via an upsert, recording the package name, URL, version, and the full manifest payload as JSON. This database record serves as both a cache (to skip reinstallation on restart) and an audit trail of what is currently loaded.

In development mode, an additional discovery path is available. Trex scans a configured local directory, treating each subdirectory as a plugin. It reads the local `package.json` and registers the plugin through the same handler pathway. This allows developers to work on plugins without publishing them to a registry.

4.2 The Plugin Manifest

The plugin manifest is the `trex` section of a package's `package.json`. For a functions plugin—the most elaborate type—it contains up to five sections: `init`, `api`, `roles`, `scopes`, and `env`. Each section serves a distinct purpose in wiring the plugin into the running system.

Init Functions

The `init` array defines one-time startup tasks that must complete before the plugin's API routes become available. Each entry specifies a Deno function to execute, along with its import map and environment variable namespace. A typical functions package includes init entries covering database migrations, schema seeding, Prefect flow registration, and analysis setup.

Init functions support two mechanisms for controlling execution order. The `waitfor` field specifies an HTTP endpoint that must be reachable before the function runs. Trex polls this URL in a loop until it gets a successful response. This is used, for example, to ensure the Prefect server is healthy before attempting to register flow deployments. The `delay` field introduces a pause after an init function completes, giving downstream services time to stabilize before the

next init function begins. Init functions are processed sequentially—each must finish before the next starts.

Each init function runs as a Deno worker with its own V8 [14] isolate, receiving only the environment variables specified by its manifest entry. The function is invoked once at startup and then its worker is discarded.

API Entries

The `api` array defines the routes that the plugin exposes to external clients. Each entry maps a URL path prefix to either a function (a Deno worker) or a service (an HTTP proxy target).

For function-backed routes, the entry specifies the URL source path, the Deno entrypoint on disk, an import map, and an environment variable group name. When Trex processes such an entry, it registers the route on the Hono [21] application with authentication and authorization middleware, and also inserts the function into the internal function dispatch map for inter-worker calls.

For service-backed routes, the entry specifies the URL source path and a logical service name from the service routing map. Trex creates a proxy route that forwards requests to the resolved service URL, with optional path-prefix stripping before forwarding. The Jupyter gateway proxy, for instance, strips its source prefix so that the downstream service receives clean paths. Both function and service routes are wrapped with authentication and authorization middleware, ensuring that every request—whether handled locally or proxied—passes through the same security checks.

Roles

The `roles` object maps role names to arrays of scope strings. When Trex processes this section, it merges the scopes into an in-memory role-to-scopes map (deduplicating entries), and for any role that does not yet exist, it calls the identity provider API to create it. This means that the platform's RBAC structure is defined entirely by plugin manifests, not by manual identity-provider configuration.

A typical functions package defines roles including a system administrator role (with scopes covering dataset management, dataflow orchestration, portal administration, and user management), a researcher role (with scopes for patient analytics, concept set management, and notebook access), a tenant viewer role (with read-only scopes), and several special-purpose roles for machine-to-machine service accounts.

Machine-to-machine role names are resolved to actual identity provider client IDs from environment variables before being registered, allowing the same manifest to work across environments where the client IDs differ.

Scopes

The `scopes` array defines URL-to-scope mappings that the authorization middleware uses to enforce access control. Each entry contains a regex path pattern, an array of required scopes, and optionally an HTTP methods array to restrict the rule to specific verbs. Some entries also include a dataset identifier field for dataset-scoped authorization.

When Trex processes the scopes section, it appends these entries to a global required-scopes array and calls the identity provider API to ensure the corresponding API resources and scopes exist. At runtime, when a request arrives at a protected route, the authorization middleware

matches the request path and method against this array, checks whether the authenticated user's token contains the required scopes, and rejects the request if any are missing.

This design means that a single plugin manifest can define a complete security policy: the roles, the scopes each role carries, and the URL patterns that require those scopes. When a new plugin adds a new API endpoint, it can simultaneously declare the scope needed to access it and assign that scope to the appropriate roles.

Environment Variables

The `env` object is a two-level structure that defines the environment variables each function receives. The top level contains keys corresponding to function names (matching the `env` field in init and API entries), plus a special `_shared` key for variables common to all functions.

The shared namespace typically defines variables covering database connection parameters, identity provider configuration, Supabase Storage [66] object storage settings, and other cross-cutting concerns. Each function-specific namespace then adds or overrides variables for its particular needs—an analytics service adds its port and SQL configuration, while a dataflow init function adds Prefect, OHDSI, and FHIR-related variables.

The environment values support bash-style variable substitution. The substitution engine supports standard shell parameter expansion operators for providing defaults, preserving empty values, raising errors on missing variables, and selecting alternate values based on whether a variable is set. Substitution is recursive—nested expressions are resolved iteratively up to a configurable depth, with a warning if the maximum is reached (suggesting a circular reference). The engine walks the entire env structure, applying substitution to every string value, including those nested inside arrays and objects.

This substitution mechanism is what allows a single `package.json` to work across development, staging, and production environments. The manifest specifies the structure; the environment provides the values.

4.3 Function Isolation

Every cloud function in D2E runs in its own Deno worker isolate, a design that provides security boundaries, resource limits, and environment variable isolation between functions that share the same Trex process.

When Trex processes a function-backed API entry, it registers both an HTTP route and an internal dispatch entry. When a request arrives (either via HTTP or via internal IPC), Trex constructs a Deno worker with carefully controlled parameters: the entrypoint path, the import map, and a set of resource constraints.

The memory limit is set to 1000 MB per worker, ensuring that no single function can exhaust the host's memory. CPU time limits are enforced at two levels—soft and hard—with API-serving workers receiving generous limits and init workers receiving tighter ones since they perform bounded initialization work. The worker timeout—the maximum wall-clock time a request can take—is 30 minutes for normal workers and shorter for development watch-mode workers.

Each worker receives its environment variables through a layered merge: the shared namespace is applied first, then function-specific overrides from the manifest, and finally system-injected variables such as the inter-service routing map and the function's filesystem path. This layered construction means that a function like the analytics service sees all the shared database connection parameters plus its own overrides, but it does not see variables belonging to user management or the portal. The isolation is enforced at the worker level—each V8

context has its own set of environment variables, and there is no mechanism for one worker to read another's.

Workers are granted filesystem access and network access, since cloud functions routinely need to read configuration files and make HTTP calls to databases and external services. For functions that use pre-bundled code, an eszip option allows loading a compressed module bundle directly, bypassing the normal module resolution process for faster cold starts.

4.4 Internal Function Calls

One of the most important consequences of running all cloud functions inside a single Trex process is that they can call each other without HTTP. The mechanism for this is the internal function dispatch map and a global message listener.

The dispatch map is populated during plugin registration. Each function-backed API entry creates an entry that maps a function identifier—combining the plugin name and the function path—to a handler closure. Every cloud function registered through the plugin manifest gets an entry.

The global message listener receives internal requests from other workers. When a worker wants to call another function, it sends a message containing a service name and a request object. The listener looks up the service name in the dispatch map, invokes the corresponding handler, and sends the response back through the same IPC channel.

This is how, for example, the analytics service calls the query generation service to produce SQL. Rather than making an HTTP request through the full network stack, the analytics worker sends an internal message that is dispatched directly to the query generation worker's handler. The target function still runs in its own V8 isolate with its own environment variables, but the communication bypasses the HTTP stack entirely—no TCP sockets, no serialization to HTTP wire format, no TLS handshake.

The internal call path is complementary to the external HTTP path defined by the service routing map. Functions that need to call external services (Prefect, the identity provider, Enterprise Gateway) use HTTP routes. Functions that need to call co-located functions within the same Trex process use the internal dispatch map. The two mechanisms are interchangeable from the function's perspective—both produce a Request/Response pair—but the internal path has significantly lower latency and no network overhead.

Error handling in the internal path mirrors the HTTP path. If the target function throws an exception, the listener returns a 500 response with the error message. If the request message is malformed, a 400 response is returned. The caller receives a structured response object with status, headers, and body fields, identical in shape to what it would get from an HTTP fetch.

4.5 UI Plugins

UI plugins serve the platform's frontend web applications as static assets and register navigation entries that appear in the portal shell.

The `routes` array in a UI plugin's manifest maps URL path prefixes to directories containing built frontend assets. Each entry has a source path and a target directory relative to the plugin. Trex serves these as static files, with path rewriting to strip the source prefix.

The UI plugin handler also installs client-side routing fallbacks for known portal paths, serving the portal's `index.html` so that the single-page application can handle its own routing.

The `uiplugins` section is where things become architecturally interesting. This object is keyed by role category—researcher, system administrator, setup—and each key maps to an array of navigation entries. Each entry describes a UI module: its name, route, feature flag, plugin path (a JavaScript module to load), required roles, and whether it should be auto-mounted.

When Trex processes the uiplugins section, it merges the entries into a global plugin registry structure. The merge is sophisticated: if a navigation entry with the same route already exists (from a previously loaded plugin), the existing entry is updated rather than duplicated, and children are merged recursively. This allows multiple plugins to contribute navigation entries to the same section of the UI without conflicts.

The final plugin registry undergoes domain substitution, replacing placeholder tokens with the actual public domain name from the deployment configuration. This allows UI plugins to embed absolute URLs that are correct for the deployment environment.

At runtime, the portal shell reads the plugin registry and renders the navigation entries appropriate for the current user's roles. This is how the researcher sees "Concepts" and "Cohorts" while the system administrator sees "Datasets" and "Dataflows"—the navigation structure is assembled from plugin contributions at startup and filtered by role at render time.

4.6 Flow Plugins

Flow plugins define Prefect workflow deployments. Unlike function and UI plugins, which operate within the Trex process, flow plugins configure an external orchestration system.

Before registering any flows, the handler performs two readiness checks. First, it polls until the Prefect server responds to health checks. Then it polls for the existence of the configured worker pool, waiting until the pool is available.

Once Prefect is ready, the handler iterates over the `flows` array in the manifest. For each flow, it creates the flow object in Prefect, then creates a deployment for it. The deployment includes the flow's name, its Python entrypoint, the work pool and queue, job variables specifying the Docker image and network configuration, and any tags.

The Docker image for a flow is determined by a layered resolution. Each flow can specify its own image, or fall back to the package-level default image. The image tag comes from the platform configuration. An image rewriting mechanism allows organizations that mirror container images to an internal registry to substitute the repository URL transparently.

Flow deployments also support concurrency limits. A flow entry can include concurrency limit options, where each option specifies a tag name and a maximum concurrent execution count. The handler ensures these concurrency limits exist in Prefect, creating or updating them as necessary. This prevents resource-intensive operations like cache building from overwhelming the source database.

Flows that define a parameter schema have Prefect validate flow run parameters against that schema before execution, catching configuration errors early.

The platform registers flows covering the full data lifecycle: OMOP CDM schema creation, cache building from source databases, Achilles [43] data characterization, Data Quality Dashboard [45] checks, cohort generation, FHIR caching, and interactive Shiny applications. Additional flow packages contribute their own deployments through the same mechanism.

4.7 Extension Points

The plugin architecture makes D2E extensible without modifying Trex. The following describes how to add each type of capability.

Adding a New Cloud Function

To add a new cloud function, create a Deno entrypoint with an import map in the functions package. Then declare the function in the package's plugin manifest. In the `api` array, add an entry mapping the desired URL path to the new function's entrypoint and environment variable group. In the `env` object, add a namespace with whatever environment variables the function needs. If the function requires its own database schema, add an init entry with a migration function that runs at startup.

To secure the function, add the necessary scope strings to the relevant role arrays in the `roles` section, and add a scope entry in the `scopes` array mapping the function's URL pattern to those scopes. Once the package is rebuilt and deployed, Trex will automatically discover the new function, create the identity provider roles and scopes, run any init migrations, register the HTTP route, and add the function to the internal dispatch map.

Adding a New Prefect Flow

To add a new flow, create a Python module with a Prefect flow function in the flows package. Then add an entry to the flows array in the package's manifest, specifying the flow name, its Python entrypoint, any parameter schema for validation, optional tags, and optional concurrency limit options. If the flow needs a custom Docker image, specify it; otherwise it inherits the package-level default.

When Trex next starts and the dataflow init function runs, it will process the updated flows manifest and register the new deployment with Prefect. The flow then appears in the Prefect UI and can be triggered via the platform's job management API.

Adding a New UI Plugin

To add a new UI module, build the frontend application and place its assets in the UI package's resources directory. Add a route entry mapping a URL path to the asset directory. Then add a navigation entry to the appropriate role category in the `uiplugins` section, specifying the module's name, route, feature flag, plugin path, and required roles.

On the next deployment, Trex will serve the static assets at the configured path and inject the navigation entry into the plugin registry. Users with the appropriate roles will see the new module in their portal navigation, gated by the specified feature flag.

The Broader Pattern

In each case, the pattern is the same: declare what you are contributing in a `package.json` manifest, and Trex will wire it into the system at startup. There is no plugin registration API to call, no configuration database to populate manually, no Trex source code to modify. The manifest is the contract between the plugin and the platform, and the plugin system's dispatch logic ensures that every type of contribution—function, UI, flow, extension—is handled through a consistent, predictable process.

This architecture reflects a deliberate trade-off. The plugin system adds a layer of indirection that makes the startup sequence more complex to debug. But it buys something valuable: the ability to compose the platform from independent packages, each versioned and deployed on

its own schedule, without tight coupling to the core runtime. For a platform that must integrate OHDSI tools, FHIR services, AI capabilities, and custom analytics into a single coherent system, that composability is not a luxury—it is a structural necessity.

5 Data Flow Patterns

Understanding how data moves through D2E end-to-end—crossing process boundaries, Docker networks, and storage layers—is essential for reasoning about the system's behavior under real workloads. The previous sections described the architecture in static terms: which services exist, how they are configured, what roles they play. This section tells the dynamic story. It traces five distinct patterns through which data enters, transforms, and exits the platform, from the millisecond-scale synchronous query path to the hour-scale asynchronous job path, and from the multi-step dataset lifecycle to the analytical cohort lifecycle that researchers interact with daily.

5.1 The Synchronous Query Path

The synchronous query path is the most latency-sensitive flow in the system. It begins when a researcher interacts with the Patient Analytics UI and ends when chart data renders in the browser. Every hop along the way adds latency, so the architecture is designed to minimize unnecessary boundary crossings while maintaining strict security invariants at each one.

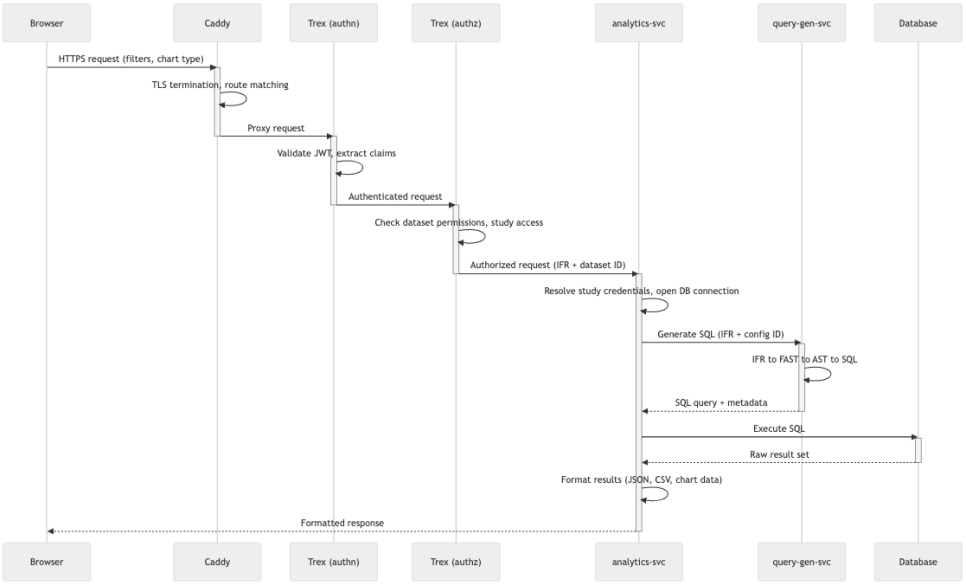

Figure 4: Synchronous Query Path

TLS Termination and Path Routing

The journey starts at Caddy [6], the reverse proxy that sits at the edge of the platform. Caddy terminates TLS for all inbound HTTPS connections, presenting a certificate for the configured public FQDN. Once the TLS handshake completes, Caddy examines the request path. Requests to `/api/*` and `/oidc/*` are routed directly to Logto [33] for authentication flows. WebSocket upgrade requests on `/jupyter/*` undergo a sub-request authentication check before being forwarded to Enterprise Gateway [56]. Everything else—the vast majority of API traffic—is forwarded to Trex on its HTTPS listener at port 33000.

Caddy's role is deliberately narrow. It does not interpret request bodies, validate tokens, or enforce authorization. It handles connection-level concerns: TLS, HTTP/2, request buffering, and path-based dispatch. This separation means that Caddy can be replaced or reconfigured without affecting the security model, which is enforced entirely within Trex.

Trex Global Middleware

When a request arrives at Trex, the Hono [21] application applies several transformations before any route matching occurs. The `getPath` callback in the Hono constructor strips the `/d2e/` prefix from all incoming paths, normalizing URLs so that cloud functions do not need to know about the public URL structure. This prefix stripping is the only path transformation that occurs at the framework level; all subsequent routing operates on the normalized path.

Trex's route registration process, driven by the plugin manifest, wraps every API endpoint with two middleware functions: `authn` and `authz`. These are not optional and cannot be bypassed by individual functions. The only exceptions are explicitly configured public URLs, which are checked against a whitelist before middleware execution.

Authentication: JWT Verification

The `authn` middleware extracts the bearer token from the `Authorization` header. If no header is present, it falls back to checking cookies for an `authtoken` or `fhirtoken` value—this dual extraction supports both programmatic API access and browser-based sessions where the token is stored in an HTTP-only cookie.

The extracted token is verified against Logto's JWKS [24] endpoint using the `jose` library's `jwtVerify` function. This verification confirms three things: the token [25] was signed by Logto's private key, the token has not expired, and the token's audience matches the platform's resource API identifier. If any check fails, the middleware returns a 401 response immediately, and the request never reaches a cloud function.

The JWKS is fetched once at startup via `createRemoteJWKSet` and cached in memory, with automatic rotation when Logto publishes new keys. This means that token verification is a purely local operation after the initial fetch—no network call is required per request.

Authorization: Scope and Dataset Validation

The `authz` middleware implements a two-phase authorization check. In the first phase, it decodes the JWT (without re-verifying the signature, since `authn` already did that) and extracts the user's subject identifier. It then calls the User Management API to retrieve the user's group memberships, role assignments, and dataset-level permissions. These are assembled into a structured `MriUser` object that carries the user's complete authorization context.

The second phase matches the request's URL and HTTP method against the `REQUIRED_URL_SCOPES` table, a compiled list of path patterns and their required scopes built during plugin initialization. If the matched route requires researcher-level access, the middleware additionally extracts a `datasetId` from the request—searching the query parameters first, then the JSON body, then the request headers—and verifies that the user holds the `STUDY_RESEARCHER` role for that specific dataset. This per-dataset check is what prevents a researcher with access to Dataset A from querying Dataset B, even though both requests arrive on the same API endpoint.

Worker Execution

Once authentication and authorization pass, the Hono route handler invokes `_callWorker`, which spawns (or reuses) a Deno [8] user worker to execute the target cloud function. The worker receives the raw HTTP request object, an environment variable map scoped to the function, and an import map for module resolution. Workers are configured with a 1 GB memory limit, a 30-minute execution timeout (reduced to 1 minute in watch mode for development), and CPU time soft and hard limits of 1,000 and 2,000 seconds respectively.

The worker isolation model means that each cloud function runs in its own V8 [14] isolate with its own global scope. Functions cannot access each other's memory, environment variables, or module state. The only way for a function to communicate with another function is through the `fnmap` in-process channel or via HTTP calls through `SERVICE_ROUTES`.

The Analytics Query Flow

Consider the most common synchronous path: a patient count query. The analytics-svc function receives the request and begins by resolving the study context. The `StudyDbCredential` middleware has already looked up the dataset's metadata from the Portal service, resolved the database credentials from the environment converter, and attached the connection details to the request object.

Analytics-svc then forwards the IFR (Internal Filter Representation) to query-gen-svc for SQL generation. This call uses the `Trex.tokioChannel` mechanism, which is the in-process `fnmap` channel. The call appears in code as `Trex.tokioChannel("d2e-functions/query-gen-svc")`, and it routes through the `Trex.createRequestListener` callback registered in the function module. The request is serialized to JSON, dispatched to the query-gen-svc worker, and the response is deserialized back—but no TCP socket, no HTTP parsing, and no TLS handshake are involved. The data stays within the Trex process.

Query-gen-svc transforms the IFR through its four-stage pipeline (IFR to FAST to AST to SQL, as described in Chapter 20), produces an executable SQL string, and returns it to analytics-svc. Analytics-svc then executes the SQL against either the TrexSQL cache (if the dataset has a cached copy) or the PostgreSQL [69] database directly, formats the results into the chart data structure expected by the frontend, and returns the response.

The entire chain—Caddy to Trex to authn to authz to analytics-svc to query-gen-svc and back—completes in the same HTTP request/response cycle. The correlation ID, generated by the `AddCorrelationId` middleware if not already present in the `x-req-correlation-id` header, propagates through every hop, appearing in all log entries and enabling end-to-end tracing of a single user interaction.

5.2 The Asynchronous Job Path

Not all operations can complete within the lifetime of an HTTP request. ETL pipelines that load millions of clinical records, R-based statistical analyses that fit complex models, data quality checks that scan every row of every table—these operations can run for minutes or hours. The asynchronous job path exists to handle them.

Why Asynchronous Execution Exists

The synchronous path is optimized for interactive latency: sub-second response times for queries that touch pre-aggregated or cached data. But several categories of work fundamentally cannot meet this constraint. OMOP CDM schema creation involves executing hundreds of DDL statements. Achilles [43] characterization computes over 4,000 statistical measures across the entire dataset. Strategus [48] analyses orchestrate 20+ R analytical modules, each of which may run for minutes. Data loading flows parse CSV files, validate records against the CDM schema, and insert them in bulk.

These operations also have different resource profiles. R analyses require the R runtime and OHDSI package libraries. Python flows need scientific computing packages. Both may need direct database access with administrative credentials that should not be exposed to the inter-

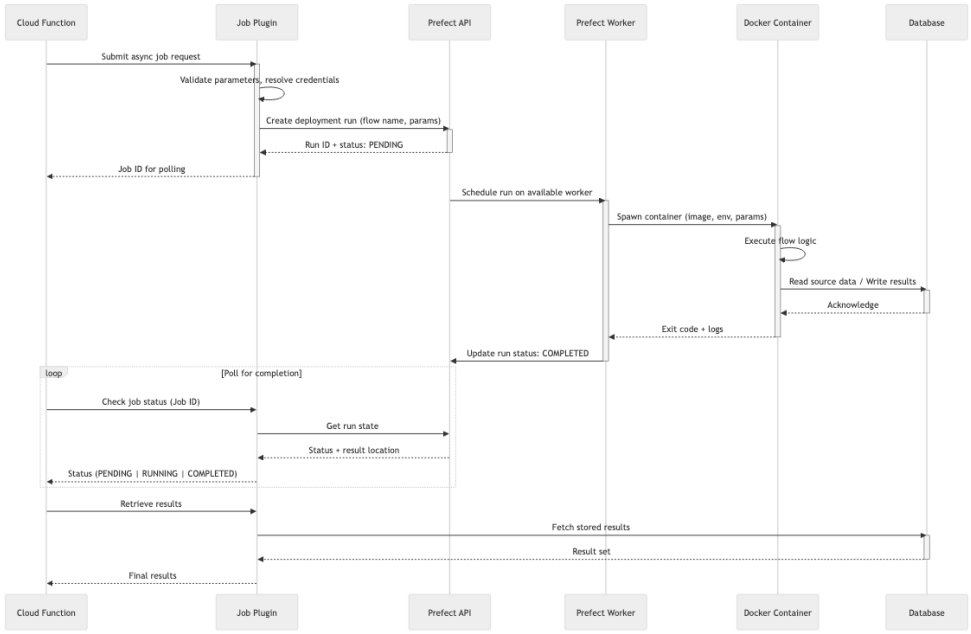

Figure 5: Asynchronous Job Path

active Trex environment. The asynchronous path addresses all of these concerns by delegating long-running work to containerized Prefect [55] flows that run on the Docker data network.

Job Submission

The submission path begins like any synchronous request: browser to Caddy to Trex to authn to authz. The request reaches a cloud function—typically the Job Plugins service—which validates the parameters and determines which Prefect deployment to invoke. The Job Plugins service maintains a registry that maps logical operations to Prefect flow deployments: cohort materialization, TrexSQL cache creation, Achilles data characterization, Strategus analytical studies, and others.

The function constructs a parameter object and calls the Prefect API to create a deployment run. This call goes through the service routing map to the Prefect server. The Prefect API returns a flow run ID, which becomes the handle for all subsequent status queries.

Token Propagation

Immediately after creating the flow run, the function calls `createInputAuthToken`, which posts the user's bearer token (and any third-party tokens embedded within it) to the Prefect flow run's input endpoint. This is the mechanism by which the user's identity propagates into the asynchronous context. The containerized flow, when it starts, reads this input to obtain a token it can use to call back into D2E APIs if needed. A cleanup timer deletes the token from Prefect's storage after five minutes, limiting the window of exposure.

Container Execution on the Data Network

The Prefect worker picks up the scheduled run and spawns a Docker container. The container is attached to the `data` network (`${PROJECT_NAME}_data`), which provides direct access to PostgreSQL and to the Trex volume where TrexSQL cache files are stored. This is a deliberate architectural choice: flow containers bypass Trex entirely for database operations. They connect

directly to PostgreSQL using credentials resolved from Prefect block documents, and they read and write DuckDB [9] files on the shared `trex` volume mounted at `/app/duckdb_data`.

The data network bypass exists for two reasons. First, bulk data operations—scanning millions of rows, writing hundreds of thousands of inserts—would create unnecessary overhead if routed through the HTTP API layer. Direct PostgreSQL wire protocol access is an order of magnitude more efficient for these workloads. Second, flow containers need administrative database credentials (for DDL operations like creating schemas) that the interactive API layer deliberately does not expose to end users.

Completion and Result Retrieval

The submitting function can poll for completion using `pollFlowRunCompletion`, which checks the flow run state at one-minute intervals for up to ten minutes. The polling checks for terminal states: `COMPLETED` indicates success, while `FAILED`, `CRASHED`, `CANCELLED`, and several other states indicate failure. For longer-running flows, the frontend polls the Job Plugins status endpoint periodically, and the user sees progress reflected in the UI's job management panel.

Results from asynchronous flows are stored in one of three locations depending on the flow type: directly in PostgreSQL tables (for cohort materialization, data quality results), as TrexSQL files on the Trex volume (for cache creation), or in Supabase Storage [66] (for large artifacts like Strategus result packages). The function that initiated the job knows where to find the results based on the flow type and retrieves them accordingly.

5.3 The Dataset Lifecycle

A dataset in D2E is not a single object but an evolving aggregate of schemas, metadata, cached views, quality assessments, and search indexes. The lifecycle from creation to active querying involves multiple services and asynchronous flows, each building on the output of the previous step.

Schema Creation

The lifecycle begins when an administrator triggers dataset creation through the Dataset Service. The request specifies a schema option—create a new OMOP CDM schema, use an existing one, or create a custom schema—along with the target database, data model, and organizational metadata. For the common case of creating a new CDM schema, the Dataset Service calls Job Plugins to launch a `data_management_plugin` Prefect flow. This flow connects to the target PostgreSQL database and executes DDL to create three schemas: the CDM data schema (named with a generated identifier like `cdm_studycode_timestamp`), a vocabulary schema (which may be shared across datasets), and a results schema (for storing computed outputs like cohort tables and characterization results).

Portal Registration and Metadata

Once the schema exists, the Dataset Service registers the dataset in the Portal. The Portal stores the dataset's identity (a UUID), its human-readable name and description, the mapping to database and schema names, the data model type, visibility status, and organizational attributes like tags and tenant assignment. This Portal record is the canonical reference that all other services use to resolve a dataset identifier into a concrete database location.

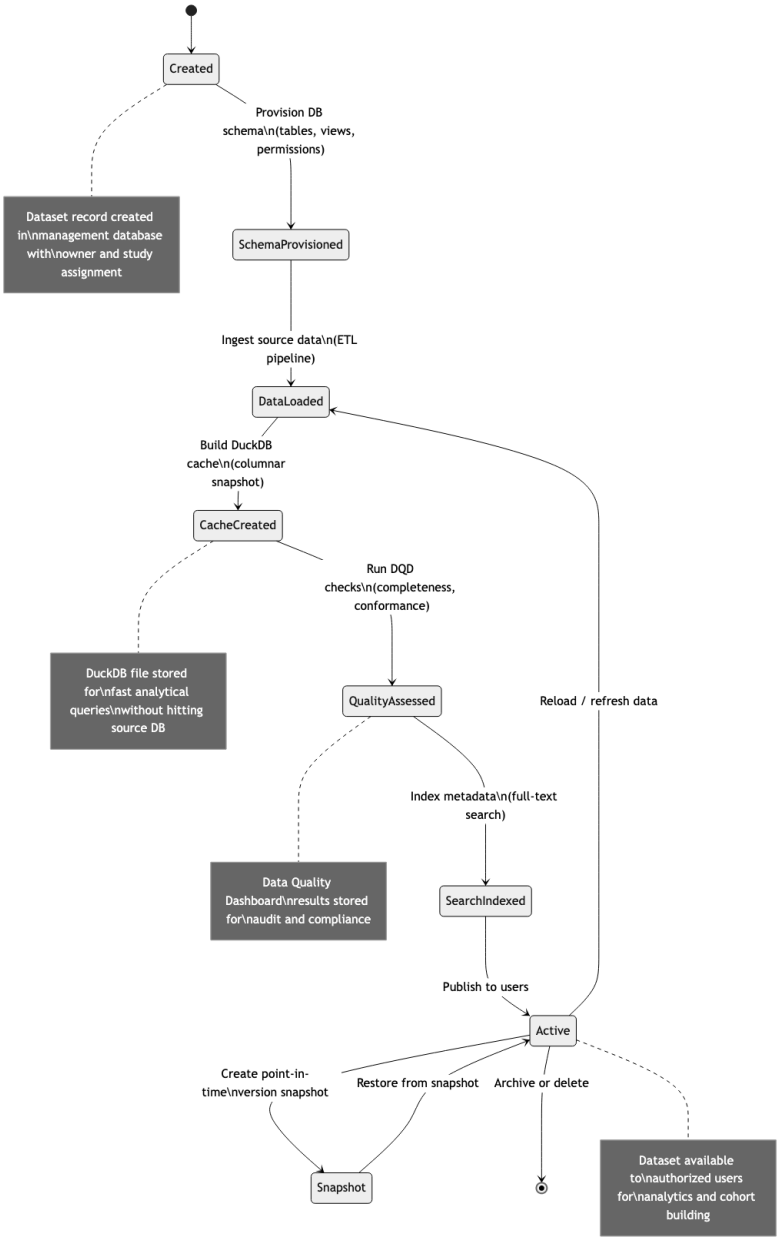

Figure 6: Dataset Lifecycle

Data Loading

An empty schema becomes useful only when data is loaded into it. D2E supports several loading paths. CSV upload triggers a `data_management_plugin` flow that parses the files, validates records against the CDM table definitions, and bulk-inserts them into the appropriate schema tables. FHIR resource ingestion uses the FHIR-to-OMOP mapping pipeline, where FHIR bundles are transformed through StructureMap definitions and loaded via a dedicated `data_load_fhir` flow. For demonstration and testing, the MIMIC-to-OMOP conversion flow transforms MIMIC-IV clinical data into the OMOP CDM format.

Cache Creation

After data is loaded, the administrator typically triggers TrexSQL cache creation via the `create_cachedb_file_plugin` Prefect flow. This flow reads the CDM tables from the source database and writes them to the Trex volume. The TrexSQL cache serves as the primary query target for all non-HANA datasets—its columnar storage and vectorized execution engine provide sub-second response times for the aggregation queries that analytics-svc generates, without requiring indexes or materialized views in the source database. The cache concurrency is governed by `CACHE_FLOW_LEVEL_CONCURRENCY` and `CACHE_TABLE_LEVEL_CONCURRENCY` settings, which prevent cache creation from overwhelming database resources.

Quality Assessment

With data loaded and cached, the dataset undergoes quality assessment through two complementary flows. The `data_characterization_plugin` flow runs OHDSI Achilles, which computes over 4,000 statistical measures—record counts by domain, temporal distributions, concept frequency rankings, and many others—and stores the results in the results schema. The `dqd_plugin` flow runs the OHDSI Data Quality Dashboard [45], which evaluates the data against a comprehensive set of quality checks (completeness, conformance, plausibility) and produces a structured quality report. Together, these assessments give researchers confidence in the data before they begin analysis.

Search Optimization

To support semantic search across datasets, the `search_embedding_plugin` flow computes vector embeddings from dataset metadata and concept descriptions. These embeddings are stored alongside an HNSW (Hierarchical Navigable Small World) index that enables approximate nearest-neighbor search. When a researcher types a natural-language query into the dataset search interface, the system encodes the query into the same embedding space and uses the HNSW index to find relevant datasets in sub-linear time.

Active Querying and Snapshot

Once all preparation steps are complete, the dataset is ready for interactive querying. Researchers use the Patient Analytics UI to build queries, which flow through the synchronous query path described earlier. The analytics-svc function preferentially queries the TrexSQL cache for performance, falling back to direct PostgreSQL access when the cache is unavailable or the query requires data not present in the cache (such as results schema tables that are updated by ongoing analyses).

At any point, an administrator can create a snapshot—a point-in-time copy of the dataset's CDM schema into a new schema. The Dataset Service coordinates this through the Portal (to register the new dataset record) and Job Plugins (to launch a `datamart_plugin` flow that performs the schema copy). Snapshots provide reproducibility: a researcher can run analyses

against a frozen copy of the data, confident that concurrent data loading or quality updates will not affect their results.

5.4 The Cohort Lifecycle

Cohorts are the fundamental unit of clinical research in D2E. A cohort is a defined set of patients who meet specific criteria, and the cohort lifecycle spans from initial definition through materialization, comparison, and downstream analysis.

Definition

A researcher builds a cohort definition on the Patient Analytics canvas by dragging filter cards that represent clinical criteria: a diagnosis of Type 2 Diabetes, an age range of 40–65, a drug exposure to metformin within 30 days of diagnosis. Each card contributes constraints to the IFR, and the canvas assembles these constraints into a complete query specification that defines the target patient population.

Bookmarking

Before materializing the cohort, the researcher can save the current filter state as a bookmark via the Bookmark Service. A bookmark captures the complete IFR—all filter cards, their attribute constraints, temporal relationships, axis definitions, and combination logic—as a serialized JSON document associated with the user and dataset. Bookmarks serve as named query definitions that can be recalled, modified, and shared. They are the persistent representation of a cohort definition before it becomes a materialized table.

Materialization

Materialization is the process of converting a cohort definition into a concrete table of patient identifiers. D2E supports two materialization paths, reflecting its dual heritage as both a native analytical platform and an OHDSI-compatible research environment.

The native path uses query-gen-svc to compile the IFR into SQL through the standard four-stage pipeline. The generated SQL includes a cohort insert statement that writes matching patient identifiers, along with cohort start and end dates, into the `cohort` table in the results schema. This path is fast and tightly integrated with the Patient Analytics UI, but it is limited to the query expressiveness of the IFR format.

The OHDSI path delegates materialization to the `cohort_generator_plugin` Prefect flow, which runs the OHDSI CohortGenerator R package inside a containerized environment. This path accepts cohort definitions in the OHDSI JSON format (which the `ifr-to-extcohort` module can produce from an IFR) and leverages the full expressiveness of the OHDSI cohort definition language, including complex temporal logic, nested criteria groups, and concept set expressions that may not have direct IFR equivalents. The flow receives a scoped authentication token via the Prefect input mechanism, connects to the database on the data network, and writes the materialized cohort to the results schema.

Comparison and Analysis

With materialized cohorts, researchers can perform comparative analysis. The Patient Analytics UI supports loading multiple bookmarks simultaneously and running the same chart configuration across different cohort populations. The cohort compare endpoint in analytics-svc generates parallel queries—one per cohort—and returns results that the frontend overlays on the same visualization.

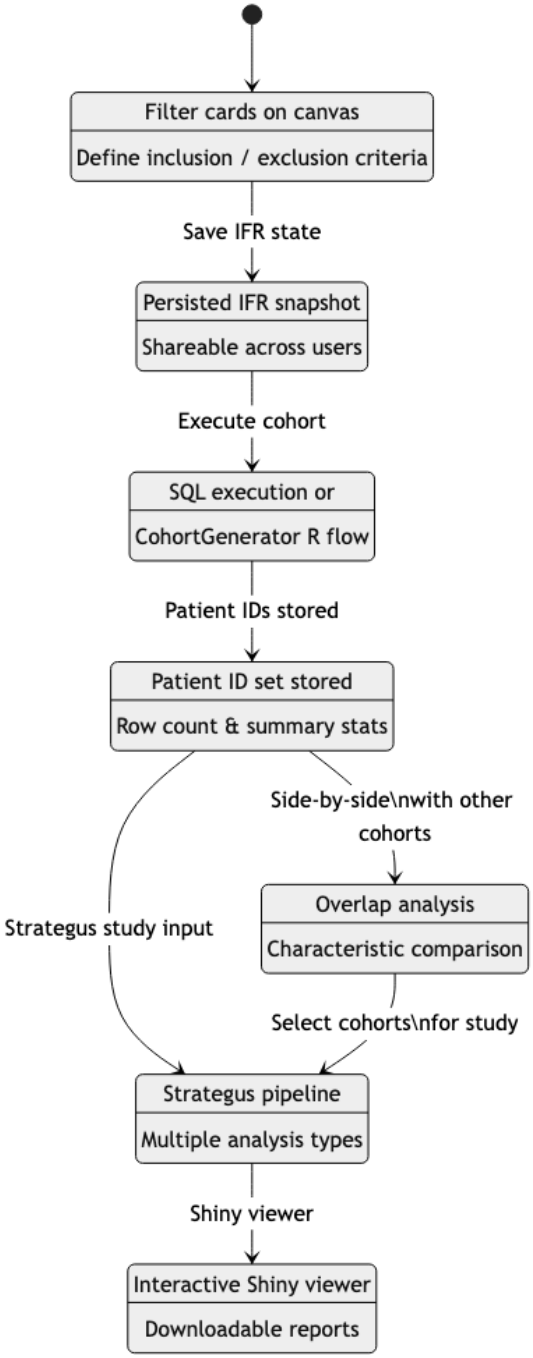

Figure 7: Cohort Lifecycle

For advanced analysis, a Strategus study can use one or more cohorts as inputs to a battery of 20+ OHDSI analytical modules (incidence rate estimation, cohort characterization, comparative cohort analysis, patient-level prediction, and others). The `strategus_plugin` Prefect flow orchestrates the execution of these modules, which run as R processes within a container on the data network. When the study completes, results are stored in the database and can be viewed through a Shiny application container that Caddy proxies via the `/strategus-results/` path prefix, with the study identifier embedded in the URL to route to the correct container.

5.5 Service-to-Service Communication

The patterns described above rely on three distinct communication mechanisms, each optimized for a different set of constraints.

In-Process Channels (fnmap)

When two cloud functions are co-located within the same Trex process—which is the case for all functions in a standard deployment—they can communicate through the `fnmap` registry. The calling function obtains a channel via `Trex.tokioChannel("plugin-name/function-name")`, and the target function registers a handler via `Trex.createRequestListener`. The call is dispatched as a structured message containing the HTTP request, and the response flows back through the same channel.

This mechanism provides zero-copy semantics for the message envelope (the actual request body is serialized as JSON, but no network stack is involved). It is the preferred path for performance-critical internal calls. The most important example is the analytics-svc to query-gen-svc channel: every patient count, bar chart, and patient list query flows through this path, and eliminating the HTTP overhead saves several milliseconds per call—significant when the goal is sub-second interactive response times.

HTTP via SERVICE_ROUTES

For calls that cross container boundaries, functions use HTTP requests routed through the `SERVICE_ROUTES` configuration map. This JSON map, injected as an environment variable, resolves logical service names to internal URLs. A function calling the Prefect API looks up `services.prefect` and gets `http://d2e-dataflow-gen-1:41120/d2e/api`. A function calling the Logto token endpoint looks up `services.logto` and gets `http://d2e-logto-1:3001`.

Most `SERVICE_ROUTES` entries point back to Trex itself (at `trex:33001`) because the target functions are co-located. In these cases, the HTTP call traverses the loopback network interface and re-enters Trex's Hono router, passing through authn and authz again. This may seem redundant, but it maintains a critical invariant: every API call is authenticated and authorized, regardless of whether it originates from a browser or from another function. The `Authorization` header is forwarded on every call, propagating the user's identity through the entire call chain.

Cross-container calls—to Prefect, Logto, Enterprise Gateway, Supabase Storage, and the DICOM [38] server—traverse the `alp` Docker network. These calls carry the same `Authorization` header and `x-req-correlation-id` header as the originating request, enabling the target service to verify the caller's identity and enabling operators to trace requests across service boundaries.

PG Wire Protocol

The third communication path bypasses HTTP entirely. Trex exposes port 5433 via an extension that speaks the PostgreSQL wire protocol. This allows SQL clients—most importantly,

the R kernels running in Jupyter notebooks via Enterprise Gateway—to connect to Trex as if it were a PostgreSQL database and issue SQL queries against cached TrexSQL data.

The PG Wire interface authenticates using a static username and password (`TREX__SQL__USER` and `TREX__SQL__PASSWORD`), which are shared with Enterprise Gateway via environment variables. This is a simpler authentication model than the JWT-based system used for HTTP APIs, appropriate because the Jupyter kernel lifecycle is already authenticated and authorized at the notebook session level by Enterprise Gateway's own token validation.

The PG Wire path is essential for the notebook-based analytical workflow. Researchers write R or Python code in Jupyter notebooks, issue SQL queries through database driver libraries (using standard PostgreSQL connection parameters), and receive tabular results that they can process with statistical packages. The engine behind the wire protocol provides the same columnar performance as the HTTP-based query path, but through an interface that data scientists find more natural.

Correlation and Tracing

All three communication paths support request correlation through the `x-req-correlation-id` header. When a request enters the system without this header, the analytics-svc `AddCorrelationId` middleware generates a UUID and attaches it. Every subsequent call—whether through fnmap, HTTP, or logged alongside a PG Wire query—includes this identifier. The correlation ID appears in log entries across all services, enabling operators to reconstruct the complete execution trace of a single user action, from the initial browser click through query generation, SQL execution, and result formatting, even when the path spans multiple workers and containers.

6 Security Architecture

Security in the D2E platform is not a single gateway or a bolt-on layer. It is a set of interlocking mechanisms distributed across every tier of the system, from the TLS handshake that greets an external browser to the RSA decryption that occurs in Trex's process memory when a cloud function needs a database password. Understanding the security architecture requires understanding where trust is established, how identity is proven, how permissions are resolved, and how secrets are managed throughout their lifecycle.

This section traces those concerns from the outside in, beginning with the network boundaries where identity and integrity are verified, then moving to the authentication and authorization protocols that govern every request, and finally to the credential storage and configuration mechanisms that keep sensitive data out of reach.

6.1 Trust Boundaries

A trust boundary is any point in the system where an identity claim must be verified or where data integrity must be re-established. The D2E platform defines five such boundaries, each enforcing a different kind of guarantee.

Browser to Caddy

The outermost boundary is the one between the external world and the platform. All client traffic — whether from a researcher's browser, a REST client, or an automated pipeline — arrives at Caddy [6], the reverse proxy that serves as the platform's public front door. Caddy terminates TLS here, using either auto-generated certificates from its built-in ACME integration or operator-provided certificates for environments with their own PKI. No plaintext HTTP traffic

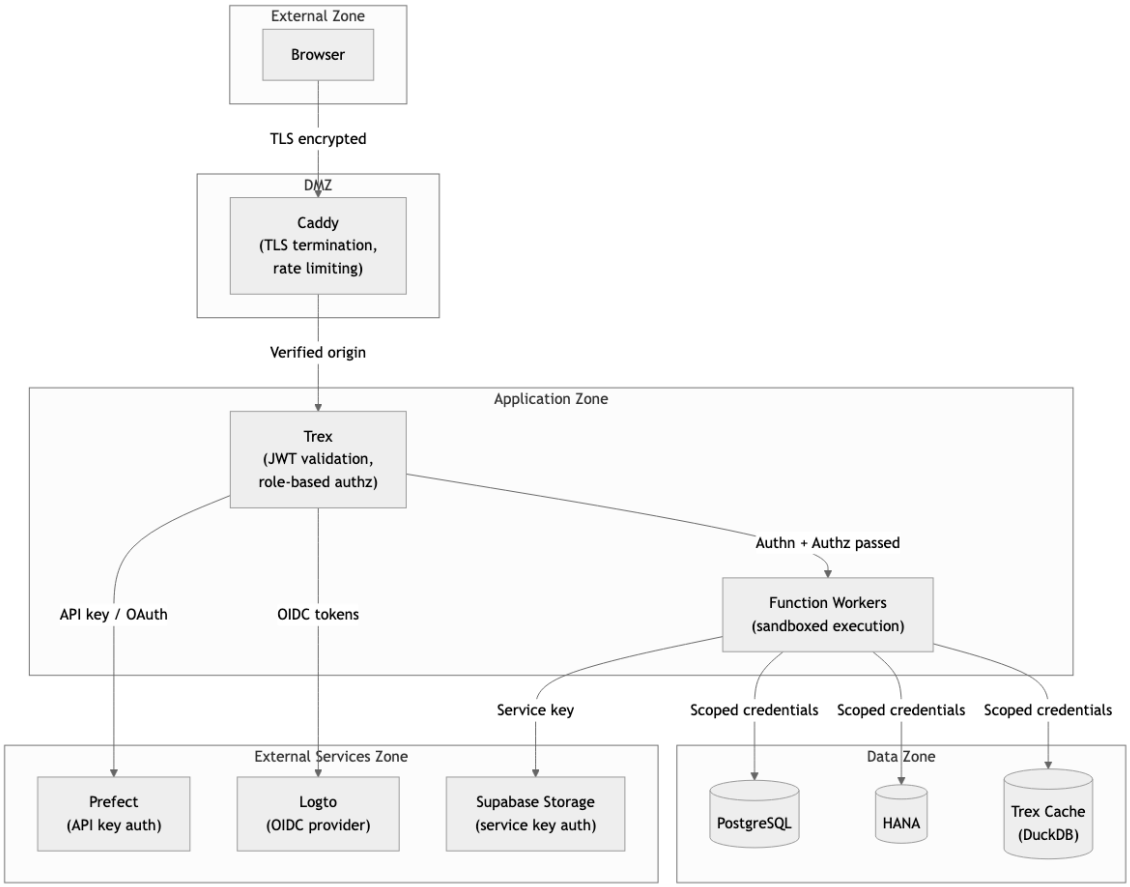

Figure 8: Trust Boundaries

is accepted. This boundary is where the platform asserts that the connection is encrypted and that the client is speaking to the genuine server, not an intermediary. Everything downstream of Caddy assumes that the transport has already been secured.

Caddy to Trex

Traffic between Caddy and Trex does not travel in the clear either. Caddy operates its own internal certificate authority, generating a root CA and an intermediate CA at startup. Service certificates are issued from this chain and used to encrypt all internal communication. The platform uses an internal domain for service resolution, and Trex presents a certificate signed by Caddy's intermediate CA when accepting forwarded requests. This means that even if an attacker gained access to the Docker network, they could not eavesdrop on or tamper with the traffic between the reverse proxy and the application server without possessing the internal CA's private key. The certificate chain — root CA, intermediate CA, service certificate — follows the same hierarchical trust model used by public certificate authorities, but scoped entirely to the deployment.

Trex to Function Workers

The boundary between Trex and its cloud functions is unusual because it is not a network boundary at all. Cloud functions run as Deno [8] V8 [14] isolates within the Trex process itself. There is no HTTP hop, no socket, no serialization over a wire. Instead, Trex spawns each function as a Web Worker [76] — an isolated JavaScript execution context backed by V8's memory sandboxing. Each worker has its own heap, its own event loop, and its own set of environment variables, defined by the plugin manifest's `env` section. A function handling analytics queries cannot read the environment variables of the user management function, even though both run within the same operating system process. The isolation is enforced by V8's security model rather than by network segmentation, but the effect is the same: one function cannot access another function's state, credentials, or memory.

CPU and memory isolation are also enforced at the isolate level. A runaway function cannot starve other workers of resources because Deno's worker implementation inherits V8's per-isolate resource accounting. This design avoids the overhead of container-per-function architectures while preserving the security properties that matter: memory isolation, environment isolation, and fault containment.

Trex to External Services

When Trex needs to communicate with external services — Prefect [55] for workflow orchestration, Logto [33] for identity management, Supabase Storage [66] for object storage — it does so over authenticated channels using machine-to-machine OAuth [16] tokens. Each external service relationship is established through the OAuth 2.0 Client Credentials flow: Trex presents a client ID and client secret to the service's token endpoint, receives a scoped access token, and includes that token in subsequent API calls. The trust relationship is bilateral: Trex trusts the external service because it verified the TLS certificate; the external service trusts Trex because it verified the client credentials.

Each external service has its own client ID and secret pair, stored as environment variables in Trex's configuration. There is no shared token or universal service account. If the Prefect credentials were compromised, the attacker would gain access to workflow orchestration but not to Logto's user directory or Supabase's object store. This compartmentalization limits the blast radius of any single credential leak.

Functions to Databases

The final trust boundary lies between cloud functions and the databases they query. This boundary is notable for what the function *does not* receive: it never sees the raw database password. When a function needs a database connection, it requests one through Trex's credential manager. The credential manager holds the RSA private key and decrypts the credential on demand within Trex's own process memory. The function receives a ready-to-use connection object with the connection already established, but the password that was used to establish that connection exists only in the credential manager's memory and is never exposed to the worker's environment or return values.

This design means that even if a cloud function were compromised — through a code injection vulnerability, a malicious dependency, or a logic error — the attacker would be able to execute queries through the existing connection but would not be able to extract the database password and use it from outside the platform. The credential never crosses the isolate boundary.

6.2 Identity and Authentication

Every request that reaches Trex must carry proof of identity, with narrow exceptions for health checks and public endpoints. The platform supports two identity types — human users and machine clients — each authenticated through a different OAuth 2.0 flow, but both validated through the same middleware chain.

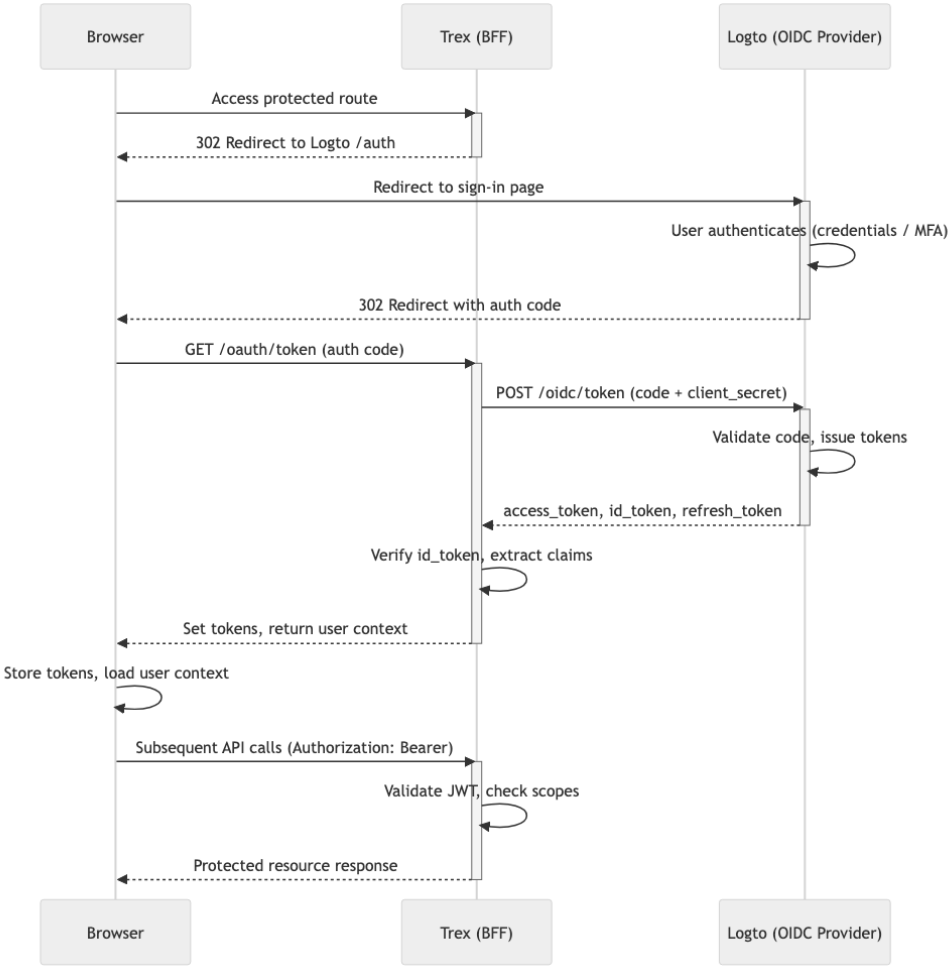

Figure 9: Authentication Flow

Human User Authentication

Human users authenticate through the OpenID Connect [52] Authorization Code flow, with Logto serving as the identity provider. The sequence begins when a user's browser is redirected to Logto's sign-in page. Logto supports multiple authentication methods: its own built-in username/password authentication for standalone deployments, and Microsoft Entra ID [35] federation for enterprise environments where single sign-on is required. After the user successfully authenticates, Logto issues an authorization code and redirects the browser back to the platform.

Trex's token endpoint receives this authorization code and exchanges it with Logto's token endpoint for three artifacts: an access token, an ID token, and a refresh token. The exchange is performed server-side, appending the client secret and resource identifier before posting to the token URL. Sensitive parameters — client IDs, client secrets, and the tokens themselves — are redacted in all log output to prevent accidental credential exposure in log aggregation systems.

The access token is a short-lived JWT [25] that the browser includes in subsequent API requests via the `Authorization: Bearer` header. The refresh token has a 14-day time-to-live and supports rotation: each time the client uses a refresh token to obtain a new access token, the old refresh token is invalidated and a new one is issued. This rotation policy limits the window of exposure if a refresh token is intercepted.

Trex's authentication middleware validates every incoming request's JWT against the identity provider's JSON Web Key Set (JWKS) [24]. The middleware fetches and caches the provider's public signing keys via the JWKS endpoint, then verifies that each token's signature is valid, that it has not expired, and that it was issued by the expected authority. Because the keys are fetched remotely and cached, key rotation at the identity provider takes effect automatically without redeploying the platform. If verification fails for any reason — expired token, invalid signature, malformed JWT — the middleware returns a 401 response. No request proceeds past authentication without a valid, verifiable token.

Machine Client Authentication

Machine-to-machine communication uses the OAuth 2.0 Client Credentials flow. Two M2M applications are registered in the identity provider — one for internal service calls and one for data pipeline operations — each with its own client ID and secret. When Trex needs to act as a client (for example, when calling the identity provider's admin API to synchronize user roles), it authenticates with the service client credentials and receives a scoped access token. Prefect flow containers use the same mechanism: they authenticate with their own client credentials at startup and include the resulting token in all callbacks to Trex.

Machine tokens are distinguished from human tokens during authorization. The authorization middleware constructs the user object differently for client credential tokens, extracting roles directly from the token's claims rather than looking them up through the user management API.

Cookie Fallback for WebSocket Connections

Certain platform features — Jupyter notebooks via Enterprise Gateway, Shiny applications, and FHIR server interactions — rely on WebSocket connections. The WebSocket protocol's upgrade handshake does not support custom HTTP headers after the initial request, which means the `Authorization: Bearer` pattern cannot be used for ongoing communication. To accommodate this, Trex's authentication and authorization middleware checks for authentication cookies as a fallback when no Authorization header is present. The cookie contains

the same JWT that would otherwise appear in the header; the validation logic is identical. For FHIR server paths specifically, a dedicated FHIR cookie is always checked regardless of whether an Authorization header exists, ensuring consistent behavior for FHIR clients that rely on cookie-based sessions.

Public URL Bypass

Not every endpoint requires authentication. Health check endpoints, OIDC discovery endpoints, public dataset listings, and public configuration endpoints are accessible without a token. The bypass is implemented as configurable lists of URL patterns—separate lists for authentication and authorization—each containing regular expressions tested against the incoming request path. If a request's path matches any pattern in the relevant list, the middleware passes the request through without performing token validation. These lists are defined in Trex's environment configuration and can be extended by operators to accommodate deployment-specific requirements.

6.3 Authorization Model

Authentication establishes *who* is making a request. Authorization determines *what* they are allowed to do. The D2E platform implements a three-level authorization model that composes URL-scoped permissions from role assignments, with an additional dataset-level enforcement layer for research data access.

The Two Configuration Objects

Authorization decisions are driven by two data structures, both populated from plugin manifests at startup.

The first is a required-URL-scopes array—a list of entries, each mapping a URL regex pattern and an optional set of HTTP methods to one or more required scope names. When a request arrives, the authorization middleware finds the first entry whose pattern matches the request URL and whose methods (if defined) include the request method. If no entry matches, the request is rejected with a 404 — the rationale being that an unrecognized URL is not merely unauthorized but undefined. This design ensures that every accessible endpoint has an explicit authorization rule; there is no default-allow behavior.

The second is a role-to-scopes map—a dictionary from role names to arrays of scope strings. Roles are coarse-grained labels (system administrator, user administrator, researcher, tenant viewer, and others), while scopes are fine-grained permission tokens tied to specific API capabilities. The role-to-scopes map is the bridge between the two: it defines which capabilities each role grants.

Scope Resolution

When a non-public request passes authentication, the authorization middleware decodes the JWT and determines the user's roles. For Entra ID users, roles come directly from the token's claims. For regular users authenticated through Logto, the middleware calls the user management API to retrieve the user's group memberships, which encode role assignments, tenant associations, and per-study researcher permissions.

The middleware then maps group memberships to platform roles and resolves each role through the role-to-scopes map to produce the user's effective scope set. The middleware compares this scope set against the required scopes for the matched URL. Authorization succeeds

only if the user's scopes include *every* scope required by the endpoint—all required scopes must be present, not merely any one of them.

Dataset-Level Enforcement

Some endpoints handle data that is partitioned by dataset (study). For these endpoints, knowing that a user has the researcher role is not sufficient — the system must also verify that the user has been granted access to the *specific dataset* referenced in the request.

The middleware checks whether any of the matched endpoint's required scopes belong to the study researcher role's scope set. If they do, the middleware extracts the dataset identifier from the request context, following a three-step cascade: first the query parameter, then the JSON request body, then a request header. If no dataset identifier is found, the request is rejected. If a dataset identifier is found, it is checked against the user's list of authorized dataset IDs. Access is permitted only if the requested dataset ID appears in that list.

This two-tier check — first role-based [60] scopes, then dataset-level membership — ensures that a user who has researcher privileges on Study A cannot accidentally or deliberately access Study B's data, even if both studies are served by the same API endpoints and the same database infrastructure.

Entra ID Group Synchronization

For deployments integrated with Microsoft Entra ID, the platform maintains a mapping between Entra ID security group IDs and platform roles, configured through an environment variable. On each login by an Entra ID user, the platform retrieves the user's current group memberships via the Microsoft Graph API [36] and compares them against this mapping. Platform roles are added for groups the user belongs to and removed for groups they have left. This synchronization ensures that role assignments in the D2E platform reflect the organization's directory state without requiring manual role management. Changes in Entra ID group membership take effect on the user's next login.

6.4 Credential Lifecycle

The management of database credentials illustrates the platform's defense-in-depth approach. Credentials pass through four distinct phases — encryption, storage, decryption, and use — and the system is designed so that the plaintext password is exposed only at the moment of use, and only within a single process's memory.

Encryption and Storage

When a database credential is registered with the platform (through the Portal's database configuration interface or the admin API), the password is encrypted using RSA-OAEP with SHA-256 padding before it leaves the client. The encryption uses a per-deployment public key that is distributed as part of the platform configuration. The encrypted credential, along with the username, user scope, service scope, and a salt value, is stored as a JSON structure in the credential table in PostgreSQL [69]. The table also holds connection metadata — host, port, database name, dialect, vocabulary schemas, and any extra configuration.

The critical property of this design is asymmetry: the public key used for encryption is widely available, but the private key required for decryption is held exclusively by Trex. No other service, container, or user has access to this key. An attacker who gained read access to the PostgreSQL database would obtain only encrypted blobs; without the private key, the passwords are computationally infeasible to recover.

Runtime Decryption

The credential manager is implemented as a singleton, instantiated once during Trex's startup. On initialization, it connects to PostgreSQL, retrieves all credential records, and decrypts each password using RSA private key decryption with OAEP-SHA256 padding. The salt that was prepended to the password during encryption is stripped after decryption. The resulting plaintext credentials are maintained in process memory for connection pooling.

When credentials are updated — a new database is registered, or an existing credential is modified — the credential manager re-decrypts the full credential set and updates both the in-memory cache and the Prefect block document. This ensures that running functions always use the current credentials without requiring a process restart.

Prefect Integration

Prefect flow containers execute outside of Trex's process and therefore cannot access the in-memory credential cache. When credentials are updated, the credential manager stores the decrypted credentials as a Prefect block document marked as a secret block. Prefect encrypts these values within its own storage layer. When a flow container starts, it retrieves the block document through the Prefect API, which requires the container to have authenticated with valid M2M credentials. This creates a two-hop trust chain: the flow container trusts Prefect because it authenticated via client credentials; Prefect trusts the credential data because it was provided by Trex, which is the only entity capable of decrypting the original values.

Per-Study Isolation

Each dataset in the platform maps to a database code — an identifier that corresponds to a specific entry in the credential table. When a cloud function processes a request for a particular dataset, the credential resolution is tied to that dataset's database code. There is no global connection pool shared across datasets; each dataset's credentials are resolved independently. This means that a function handling a request for Study A will receive a connection authenticated with Study A's credentials, and a function handling Study B will receive a different connection with different credentials. The isolation is enforced by the data model itself: the request carries a dataset identifier, the identifier maps to a database code, and the database code maps to a unique credential set. There is no path through which one study's function invocation could resolve another study's password.

6.5 Configuration Store Security

The Portal service manages a configuration store that holds both public settings (display preferences, feature flags, system metadata) and secret values (API keys, service account tokens, integration credentials). These two categories coexist in the same storage backend but are treated differently by the read and write APIs.

When a client reads a configuration that contains secret fields, the API returns the secret values replaced with a `[REDACTED]` placeholder string. The actual value is never transmitted to the browser. This redaction is applied server-side, in the Portal function, before the response is serialized. The client sees enough information to know that a secret is configured — the field exists, and its value is `[REDACTED]` rather than empty — but cannot recover the original value.

The write path complements this design with a round-trip safety mechanism. When a client submits an updated configuration, the Portal checks each field: if a field's value is `[REDACTED]`, the existing stored value is preserved unchanged. This means that a user can modify the non-secret fields of a configuration, submit the update, and the secret fields will survive the round

trip without being overwritten or blanked. Without this mechanism, every configuration edit would risk destroying secrets, because the client would be submitting `[REDACTED]` as the literal value.

Only system administrators can bypass redaction and read unredacted secret values, and only through dedicated admin endpoints that are protected by the system administrator role scope. This separation ensures that routine configuration management — performed by operators, researchers, or automated tooling — never exposes secrets, while administrators retain the ability to audit or rotate credentials when necessary.

7 Core Infrastructure

The core infrastructure of D2E is a small set of well-known services, each with a single clear role. There is a runtime engine that hosts cloud functions and routes API traffic; an edge router that terminates TLS and dispatches to internal services; an identity provider that owns authentication and the role-and-scope model; a primary relational database; a session and ephemeral-cache store; and an object store for files. The architectural commitment is that each role is held by exactly one component and that the components do not bleed into each other's responsibilities. New capabilities are added by writing cloud functions on top of this base, not by adding new infrastructure pieces.

7.1 Trex — Runtime Engine

Trex is the runtime engine that hosts every cloud function in the platform as an isolated worker, and it is the single API gateway for all traffic that enters those functions. Built on a Rust foundation with a TypeScript server layer above it, Trex spawns a fresh isolate per function invocation, which is what makes it possible for hundreds of independently-deployed functions to share a single process boundary without sharing memory, file descriptors, or runtime state.

Every request to Trex passes through a layered middleware pipeline: identity extraction, token validation, scope check, dataset check, route matching, worker spawn, function execution, response. The shape is the same as the layered authorization pattern — each layer can reject the request without the next layer having to know it existed; functions only run when every preceding layer has passed — and the pipeline is what makes the layers enforceable. A function does not have to defend itself against unauthenticated callers because it does not run unless authentication has already succeeded; a function does not have to check dataset access because it does not run unless dataset access has already been verified.

Trex also exposes a PostgreSQL [69] wire-protocol endpoint, served by a DuckDB [9]-backed extension. This is the seam that lets standard SQL clients — including the R kernels that run in researchers' notebooks — query cached CDW data using ordinary PostgreSQL connection strings. The architectural idea is that the platform's caching and query-acceleration layer should be reachable through the same wire protocol any analyst already speaks, not through a bespoke client library.

OAuth [16] code exchange is also handled at the Trex layer: the platform's web-facing token endpoint takes an authorization code, appends server-side client credentials, forwards to the identity provider, and returns access and refresh tokens. Keeping the exchange inside Trex means the client secret never has to leave the server.

7.2 Caddy — Edge Router

Caddy [6] sits in front of Trex as the platform's TLS terminator and edge router. It generates an internal certificate authority, issues certificates for every internal service, and terminates

external TLS at a single public endpoint. Internal services do not deal with public-facing TLS material directly; Caddy holds it on their behalf.

Caddy routes requests by URL prefix to the appropriate downstream service, with the default backend being Trex. The architectural idea is that Caddy holds the public-facing routing concern in one place; downstream services know nothing about the public URL space. When a new service is added, the public-facing routing change is one configuration block in Caddy — the service itself does not need to know which prefix it answers to externally. WebSocket-upgrading routes (the notebook kernels, the per-study result viewers) sit alongside the standard HTTP routes; the routing model is uniform whether the downstream protocol is HTTP or WebSocket.

7.3 Logto — Identity Provider

Logto [33] is the externalized identity provider. Authentication, the role-and-scope model, OAuth flows, and sign-in experience all live inside Logto rather than inside the platform's own code. The architectural commitment is that D2E does not implement identity itself; it consumes a standard OIDC [52] provider and owns only the configuration that adapts that provider to the platform's specific needs.

Three OAuth client applications are registered: one for general service-to-service traffic, one for data-access-specific service-to-service traffic, and one for the user-facing web application. The split exists so that data-access traffic can be independently audited and revoked from non-data service traffic. A compromised data-access credential can be rotated without disturbing service-to-service traffic that has nothing to do with data access; a misbehaving service can be silenced without taking the data-access path down.

A small set of default roles is provisioned — platform admin, user admin, dashboard viewer — and additional roles are contributed dynamically by plugins through their manifests. Each role maps to a set of authorization scopes that the runtime engine's scope-check layer enforces against URL patterns. The roles and scopes live in Logto; the enforcement lives in Trex; the connection between them is the JWT [25].

For enterprise deployments, a custom Entra ID [35] connector handles single sign-on. The connector exchanges Entra authorization codes for tokens, fetches the user's group memberships from Microsoft Graph [36], maps those groups to Logto roles via configuration, auto-provisions the Logto user if they do not already exist, and synchronizes role assignments on every sign-in. The architectural idea is that group membership in the enterprise directory drives role assignment in the platform — administrators do not maintain a parallel role list.

Logto post-init configures the identity provider for D2E's specific needs: registering the client applications, defining roles and scopes, creating the administrative user, and optionally wiring up the Entra ID social connector. This must complete before Trex starts; Trex's authorization model assumes Logto is already configured.

7.4 PostgreSQL

PostgreSQL is the platform's primary relational database, hosting every schema that core services persist to. The init path provisions a tiered set of database users — a superuser for emergency access, a manager that owns DDL and schemas, a writer that handles application-level read/write traffic, and a separate manager isolated to the identity provider's schema. The tiering is the architectural idea: routine application traffic does not run as a superuser, and the identity-provider schema is isolated from application schemas so that a compromised application credential cannot rewrite identity data.

The init path also creates the schemas that the platform's services own — one for each major service domain (platform configuration, CDM configuration, query engine, user management, portal, files, terminology, identity, and several specialized stores for FHIR, jobs, Strategus, and external tooling). Each schema is owned by exactly one service. The architectural commitment is that schema ownership is explicit and does not drift: services do not share schemas, and schemas are not amended ad hoc by services that do not own them.

7.5 Redis

Redis [57] provides session management and short-lived caching. It is the platform's home for ephemeral state that does not need to survive restarts or replicate to other regions — session tokens, short-lived cache entries, transient coordination data. Keeping ephemeral state out of PostgreSQL is the architectural idea: the relational database is for things that must persist; Redis is for things that must be fast and may evaporate.

7.6 Object Storage

File storage — uploads, downloads, DICOM [38] images, analytical exports — runs on an S3-compatible object store fronted by a JWT-authenticated API. The platform does not store binary blobs in the relational database; large objects live in object storage with a metadata reference in PostgreSQL. The split keeps the relational database from becoming a blob bucket and lets file traffic scale independently of the database.

8 Database and Configuration

8.1 Supported Database Dialects

Database	Key Characteristics
SAP HANA	Session authentication for user-level access control. HANA-specific SQL (DAYS_BETWEEN, anonymous blocks). Uppercase schema names. Session variables for context propagation.
PostgreSQL	Standard SQL with connection pooling (configurable min/max/idle timeout). Schema name adaptation. TLS support.
TrexSQL	The primary database layer for all non-HANA deployments. Provides high-performance analytical query execution for PostgreSQL and BigQuery-backed datasets through its caching and query acceleration layer. Also exposes a PostgreSQL wire-protocol interface that allows SQL clients and Jupyter notebooks to query data directly.

8.2 Connection Pooling and Management

The Analytics Service manages connections through a pooling system:

- Connections allocated per-request and returned to the pool upon completion.
- Configurable minimum/maximum thresholds and idle timeout.
- Alternative Trex database manager for optimized co-located connections.
- Cleanup guaranteed through middleware that runs after every request.

8.3 The Placeholder-to-Table Mapping System

1. Each CDM configuration includes a placeholder map resolving logical placeholders to physical database objects.
2. The map includes column mappings for standard fields (patient ID, event ID, dates, type).
3. Table names include a schema placeholder (`$$SCHEMA$$`) resolved at execution time.
4. Specialized schemas (`$$VOCAB_SCHEMA$$`, `$$RESULT_SCHEMA$$`) are resolved separately.
5. A "guarded" mapping exists for access-controlled environments, restricting queries to patient-level records.

8.4 Cross-Backend Query Consistency

The same analytical request produces semantically equivalent results regardless of database backend, achieved through:

- The placeholder system abstracting physical table differences.
- The multi-stage pipeline generating SQL in a database-independent form before dialect adaptation.
- Standardized OMOP-based placeholder mappings for consistent clinical concept interpretation.
- Dialect-specific SQL formatting adapting functions while preserving logical intent.

PART II

Frontend Application

Part II covers the frontend application: the React [34]-based portal shell that mounts everything, the Vue [74]-based Patient Analytics module that is the platform's signature interface, the data and admin tools that surround them, and the shared component layer that keeps the visual language consistent.

The frontend is unusual in two ways. First, it is a micro-frontend system: the portal shell loads independent applications at runtime based on the user's roles, mixing React modules and Vue modules in the same browser tab. Second, the most important application — Patient Analytics — predates the rest of the platform and is built on a different framework than its host. This part explains how that came to be, how it works in practice, and what it costs.

The opening chapter introduces the frontend architecture: the micro-frontend model, the role-driven plugin registry, the build and deployment pipeline. Subsequent chapters cover the portal shell — routing, navigation, the role-aware layout — and Patient Analytics, the chapter on which is long because the module is dense. The remaining chapters cover the data tools, analysis notebooks, admin tools, and the shared component library that all the above depend on.

Read this part if you write or maintain frontend code, or if you need to understand what the platform's users see and click.

9 Frontend Architecture

9.1 Purpose

The D2E frontend is not a single application. It is a host shell plus a collection of independently built and independently deployed modules, mounted into the shell at runtime according to the role set carried by the current user's session. What the browser displays at any given moment is an assembly: a React [34] portal that owns the URL, the chrome, and the authentication context, with one or more child modules drawn from a manifest and grafted into named DOM containers. The shell decides which children to load by reading the user's roles; the children are otherwise unaware of one another and of the shell's internals beyond a small contract of injected props. This chapter describes that assembly. The chapters that follow describe the modules it composes.

The shape of the architecture is a direct consequence of the platform's history. Patient Analytics — the cohort builder described in Chapter 11 — predates the rest of the D2E platform by several years. By the time the wider portal was being designed, Patient Analytics was already a mature Vue 3 [74] application with hundreds of components, a deeply entrenched Vuex store, and idioms refined over years of clinical use. The portal team chose React for the new surfaces it was building. Rewriting Patient Analytics in React to match would have cost months of effort and produced an application no better than the one already shipping; it would have been pure technical migration with no functional gain for users. The alternative — forcing the rest of the portal to adopt Vue — was equally unappealing, since the React skill base, component libraries, and routing conventions had already shaped the team's design choices for the new modules.

The decision was to refuse the choice. The portal became a React shell, and the modules became framework-agnostic: any application that produces a static-asset bundle exposing a known mount API is a valid module, whether it is built in React, in Vue, or in something else again. This is the local form of principle 1 of 2 (composition over construction) applied to the frontend: capabilities arrive as bundles registered in a manifest, and the runtime composes them at load time rather than at compile time. The plugin system, the SystemJS and single-spa [64] loaders, and the role-driven navigation registry described in the rest of this chapter are the mechanisms that make that composition work.

The cost is real and worth naming. Bundle size is larger than it would be with a single framework: every browser session that visits Patient Analytics ships both the React runtime (for the shell) and the Vue runtime (for the analytics module). Two router systems must be reconciled — the portal's React Router owns the top-level URL namespace, and Patient Analytics's internal router owns the path segments below its mount point, and the boundary between them is a contract that has to be upheld by both sides. Shared state crosses framework boundaries through DOM events and a token-getter callback, which is a documented seam but a seam nonetheless. A developer debugging an end-to-end issue must be comfortable reading both React and Vue idioms within a single page load.

What the architecture buys is independence. Each module ships on its own cadence; the Patient Analytics team can release without coordinating with the portal team, and the portal team can release without holding the Vue module hostage to a framework migration that would serve no one. Modules added in the future can choose their own frameworks if there is a good reason, without disturbing what already works. The platform's heterogeneous history — UI5 modules inherited from upstream, the Vue analytics application, the React portal, and whatever comes next — presents as one coherent product to the user, because the assembly model accepts heterogeneity as a precondition rather than fighting it.

Understanding this architecture requires examining four layers: the micro-frontend composition model that stitches applications together at runtime, the plugin system that governs what users see based on their roles, the build system that compiles thirteen disparate applications into a single deployable artifact, and the deployment model that serves everything through Trex without a separate web server.

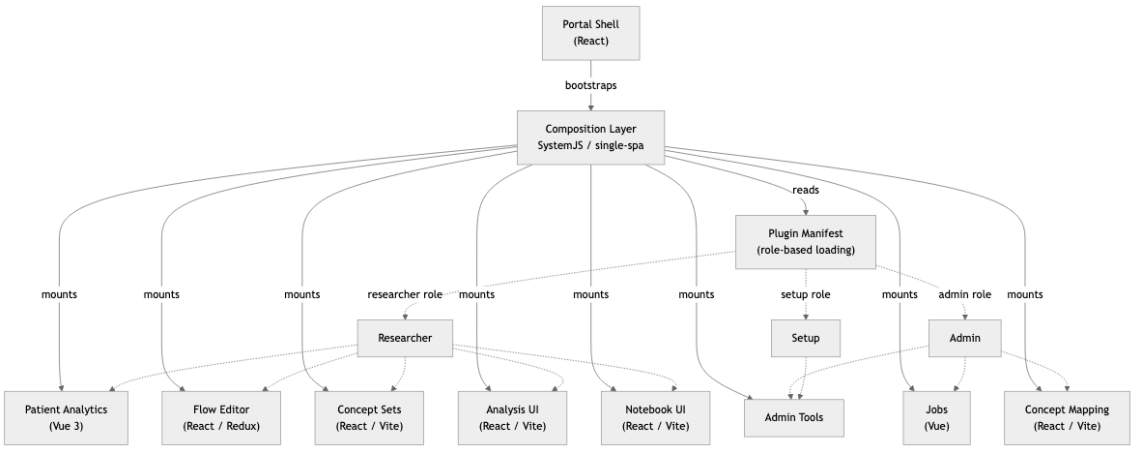

Figure 10: Frontend Micro-Frontend Composition

9.2 Micro-Frontend Composition

The portal application is the only application the browser loads directly. It is a React 18 application bootstrapped via Create React App, served from `/d2e/portal`. Every other application in the system — concept set management, notebook editing, ETL flow design, cohort analysis — exists as a separately built bundle that the portal loads at runtime.

Two distinct loading mechanisms coexist within this architecture, reflecting the system's evolution from a monolithic plugin model to a true micro-frontend approach.

Legacy Plugins via SystemJS

The original plugin mechanism uses SystemJS 0.21.6, a universal module loader that predates native ES module support in browsers. When the portal needs to render a legacy plugin, the `pluginLoader` imports the module through SystemJS, which handles the AMD/UMD module format that these older bundles export. Before any plugin can load, the loader pre-registers shared dependencies — React, ReactDOM, React Router DOM, and Emotion — as SystemJS dynamic modules. This ensures that every plugin uses the same React instance as the portal, preventing the duplicate-React bugs that plague naive micro-frontend setups.

Legacy plugins export a `plugin` object containing a `page` property — a React component that the portal renders inline. The portal passes a `metadata` object to this component containing the authentication token getter, the active dataset ID, the tenant ID, and any plugin-specific data. This contract is simple but limited: the plugin's lifecycle is entirely controlled by React's component mounting, and the plugin cannot manage its own routing.

Many of the built-in plugins — the system admin panels for user management, dataset management, study configuration, and setup pages — still use this legacy mechanism. They are compiled directly into the portal bundle as code-split chunks, registered in `builtInPlugins.ts` as dynamic imports keyed by their plugin path strings. When the plugin loader encounters a path matching a built-in plugin, it resolves the dynamic import rather than fetching a remote bundle.

Single-SPA Micro-Applications

The newer loading mechanism uses single-spa, a micro-frontend framework that gives each application independent lifecycle management. Applications loaded through single-spa export `bootstrap`, `mount`, and `unmount` lifecycle functions. The portal's `singleSpaRegistry` orchestrates registration: it creates an activity function based on the application's base path, loads the module through `window.System.import`, and delegates lifecycle management to single-spa's routing engine.

Each single-spa application gets its own DOM container — a `div` element whose ID is derived from the application's registration name. The `SingleSpaAppContainer` component manages the loading state, polling single-spa's status registry until the application transitions to `MOUNTED`. Custom properties — the authentication token getter, the active dataset ID, the user's locale, feature flags — flow into the application through single-spa's `customProps` mechanism. When properties change (the user selects a different dataset, for instance), the registry updates the props store and dispatches a `custom-props-changed` DOM event that the child application can listen for.

The concept sets app, notebook UI, analysis UI, and wizards app all use this mechanism. Their plugin manifest entries include `"type": "app"` to signal that the portal should use single-spa registration rather than legacy SystemJS loading. The ETL flow application is loaded as an external single-spa module from its own URL (`/flow/module.js`), separately built with Webpack 5.

Import map overrides provide a development escape hatch. The portal includes the `import-map-overrides` library, which creates a `systemjs-importmap` script tag mapping module URLs. Developers can override any module URL at runtime through the browser's developer tools, pointing a production-registered application at a locally running development server without rebuilding the portal.

9.3 Technology Stack

The frontend is deliberately polyglot, a consequence of the platform's evolution through multiple technology generations.

The portal shell and most micro-applications use React 18 with TypeScript. The portal itself depends on Material UI 5 for its component library, React Router v6 for routing, Axios for HTTP communication, and the `@axa-fr/react-oidc` library for OpenID Connect [52] authentication. The shared component library `@portal/components` and the plugin type definitions `@portal/plugin` live in the `libs/` directory and are consumed as workspace dependencies.

Patient Analytics — the cohort builder — is implemented in Vue 3 through the `vue-mri-ui-lib` application, originally a UI5 application that was progressively rewritten in Vue. It communicates with the portal through the same custom property mechanism, receiving authentication tokens and dataset context through props. Its UI5 ancestors persist in the `mri-pa-ui` directory, which still contains UI5 XML views and controllers for the patient analytics configuration interface.

Build tooling varies by application age. The portal uses Create React App (`react-scripts`), which provides zero-configuration Webpack bundling with Babel transpilation. The ETL flow application uses a custom Webpack 5 configuration. All newer micro-applications — concept sets, notebook UI, analysis UI, wizards — use Vite, which provides faster development builds through native ES module serving and faster production builds through Rollup. The Vue application also uses Vite. Legacy UI5 modules are built using the `@ui5/cli` build tool.

9.4 The Plugin System

The plugin system is the mechanism by which the portal decides what navigation items to show, what routes to create, and what bundles to load. It is configured entirely through the `uiplugins` section of the root `package.json`, which defines three plugin categories corresponding to user roles: `researcher`, `systemadmin`, and `setup`.

Plugin Manifest Structure

Each plugin entry in the manifest contains a name, a route path, a plugin path pointing to its loadable module, an enabled flag, and optional metadata including feature flag dependencies, required roles, internationalization keys, and plugin-specific data. The portal reads this manifest at startup through the `REACT_APP_PLUGINS` environment variable (injected at build time or overridden at runtime through `window.ENV_DATA`), parses it into an `IPlugin` structure containing three arrays, and stores it in memory.

The `loadPlugins` utility is intentionally simple: it parses the JSON from the environment variable and returns the three plugin arrays. There is no plugin discovery protocol, no dynamic registration at runtime, no plugin marketplace. The manifest is baked into the deployment, and changing the available plugins requires updating the configuration and redeploying.

Plugin Types and Role-Based Visibility

Researcher plugins appear in the left navigation when a user with the `RESEARCHER` role selects a dataset. The current manifest includes Concepts (concept set exploration), Cohorts (the Vue-based cohort builder), Notebooks (Starboard notebook interface), Analysis (Strategus analysis workflows), and Wizards (guided analysis wizards). Each researcher plugin can be gated behind a feature flag — Concepts requires `conceptSets`, Cohorts requires `cohort`, Notebooks requires `starboard`, Analysis requires `strategus`. The Researcher container further filters plu-

gins based on dataset type, mapping certain dataset types to allowed plugin sets through a feature map.

System admin plugins appear when a user with `ALP_SYSTEM_ADMIN` or `ALP_USER_ADMIN` roles navigates to `/systemadmin`. These include Users (user and role management), Datasets (dataset configuration), Studies (study management), Jobs (background job monitoring), ETL (the dataflow editor), Setup (a container for setup sub-plugins), and Logs (Docker container log viewer, gated behind the `d2eLogs` feature flag).

Setup plugins are nested within the system admin Setup page. They provide configuration interfaces for database connections, Git settings, feature flags, dataset metadata, cohort builder configuration, CDM configuration, overview descriptions, hybrid search settings, Trex plugin management, and demo dataset provisioning. Each setup plugin is rendered through the `SetupOverview` component, which presents them as cards in a grid layout.

Plugin Rendering Pipeline

When the portal needs to render a plugin, the rendering pipeline diverges based on the plugin's type. For legacy plugins (those without `"type": "app"` in their manifest), the `ResearcherStudyPluginRenderer` or `SystemAdminPluginRenderer` calls `loadPlugin`, which checks the built-in plugin registry, the module cache, and finally falls back to SystemJS import. The loaded module's `page` component is rendered inline with the metadata props.

For single-spa applications, the `ResearcherStudyPluginRenderer` takes a different path. On mount, it generates a deterministic application ID from the plugin path, calls `registerSingleSpaApp` with the URL, base path, and initial custom properties, and then starts the single-spa routing engine. The application's DOM container is pre-rendered as a `div` whose visibility is toggled by route matching — when the URL matches `/researcher/{route}`, the container is displayed; otherwise it is hidden. This approach avoids unmounting and remounting single-spa applications on every route change, preserving their internal state.

The portal pre-renders all single-spa app containers simultaneously. Each container's visibility is controlled by comparing the current URL path against the plugin's route. Legacy plugins, by contrast, use React Router's `Route` component and are mounted and unmounted as the user navigates.

9.5 Build System

The build system uses Bun as the package manager and NX as the task orchestration layer. The workspace is configured with two workspace directories: `libs/*` for shared libraries and `apps/*` for applications. NX provides dependency-aware task execution, build caching, and parallel builds.

The build pipeline follows a strict dependency order defined in `nx.json`. The `targetDependencies` configuration specifies that a project's `build` target depends on the `build` target of its dependencies, ensuring that shared libraries compile before their consumers. The NX cache recognizes twelve cacheable operations including `build`, `build-mri`, `lint`, `test`, and `clean`.

The full build sequence, orchestrated by the `build-all` script, proceeds in phases. First, the two shared libraries compile: `@portal/components` (the shared UI component library) and `@portal/plugin` (the plugin type definitions and utilities). These must complete before any application can build, because every application imports from them. Second, all applications build in parallel — the portal, concept sets, notebook UI, analysis UI, jobs, mapping, flow, wizards, and the Vue MRI library. Third, the legacy UI5 modules compile using the `@ui5/cli`

builder. Fourth, Starboard notebook assets are assembled. Fifth, the gateway proxy builds. Sixth, the wizards app runs its tests and builds.

Each application's build output lands in a subdirectory of `resources/`: the portal builds to `resources/portal`, concept sets to `resources/concept-sets`, the flow editor to `resources/flow`, and so on. The complete `resources/` directory is the final build artifact, containing every static asset the frontend needs.

9.6 Deployment Model

The frontend has no dedicated web server. All static assets are bundled into the Trex Docker image, which serves them directly. Trex is both the backend API gateway and the static file server, using route configuration from the `trex.ui.routes` section of the root `package.json` to map URL paths to filesystem paths within the `resources/` directory.

The route mapping is extensive. The portal is served from `/portal` mapping to `/resources/portal`. The flow editor maps `/flow` to `/resources/flow`. The legacy UI5 modules use the original path conventions inherited from the upstream platform: `/hc/mri/pa/ui` maps to `/resources/mri-ui5/mri-pa-ui/mri/`, `/hc/hph/core/ui` maps to the UI5 core utilities, and `/ui` maps to the UI5 runtime resources. There are over twenty-five route entries accommodating the various path conventions accumulated across the system's history.

Caddy [6], the reverse proxy sitting in front of the platform, routes `/d2e/portal/*` requests to Trex. The portal's React Router is configured with `basename="/d2e/portal"`, and all internal navigation uses paths relative to this base. When a single-spa application needs to load its assets, it uses absolute paths like `/resources/concept-sets/lifecycles.js` which Caddy routes to Trex, which resolves them against the static file mapping.

Runtime environment injection happens through `window.ENV_DATA`, a global object that Trex populates when serving the portal's HTML. This object carries the OIDC configuration (identity provider URLs, client IDs, scopes), the locale setting, feature flags, the plugin manifest JSON, and other deployment-specific values. The portal's `env.ts` module merges `window.ENV_DATA` with `process.env` (build-time values), giving runtime values precedence over build-time defaults. This mechanism allows the same built artifact to be deployed to different environments — development, staging, production — without rebuilding.

For local development, the `resources/` directory can be mounted into the Trex container via a Docker Compose volume override. Developers build a single application locally, and the volume mount makes the fresh build immediately available through the running Trex instance. The portal's Create React App development server can also proxy API requests to a local Trex instance, allowing hot-reload development of the shell while other applications are served from the container.

9.7 Integration Points

The decisions in this chapter ripple through every later chapter in Part II. The micro-frontend composition model is what allows the portal shell (Chapter 10) to mount Patient Analytics, the data tools, the analysis and notebook surfaces, and the admin tools as if they were sibling pages of a single application; each of those chapters describes one mounted module and inherits the loading, routing, and lifecycle contracts established here. The role-driven plugin registry is what determines which of those modules a given user actually sees, and so it is the implicit precondition for any chapter that talks about "the researcher view" or "the system admin view." The build pipeline and the `resources/` layout are the reason the shared component library (Chapter 15) can be consumed as a workspace dependency rather than a

published npm package. And the deployment model — Trex serving every static asset, with `window.ENV_DATA` carrying runtime configuration — is what lets the same built artifact appear in development, staging, and production without rebuilding, which is the assumption every operations chapter in Part V relies on.

10 Portal Application Shell

10.1 Purpose

The portal shell is the application that loads first when a user opens D2E. It owns the OIDC redirect, the role-aware navigation, the layout chrome, and the micro-frontend mounting machinery. Everything a user sees in their browser is either the shell itself or a module the shell has mounted into one of its containers. The shell is small in surface area — a handful of React [34] components, an authentication boundary, an Axios client, a global store, and a router — but it is on the critical path for every interaction the platform supports.

The architecture had a credible alternative: every module owns its own shell. In that model, each frontend module would handle its own OIDC redirect, render its own header and navigation, manage its own dataset selector, and present its own visual identity. This is how D2E's predecessors had largely worked, and it is how heterogeneous frontends often grow when no one is watching. The team rejected it for two related reasons. The first was user experience: when each module draws its own chrome, the platform looks like a directory of unrelated applications rather than one coherent product, and visual consistency becomes a matter of per-module discipline rather than an architectural property. The second was security and complexity: every module would have to re-implement the OIDC authorization code flow, manage its own token refresh, and handle its own session expiry. Auditing five different OIDC implementations is harder than auditing one, and any of the five could quietly drift from the others as libraries are upgraded on different cadences.

The decision was one shell, many modules. The shell handles the cross-cutting concerns once — authentication, the active dataset, the role-driven navigation, the Axios client with its token-attaching interceptors — and exposes them to mounted modules through a small set of injected props and DOM events. Modules focus on their domain. A notebook module renders notebooks; it does not render a header, it does not redirect to Logto [33], it does not maintain its own dataset selector. This is the same composition pattern principle 1 of 2 establishes for the platform as a whole: capabilities are added by registering bundles into a host runtime, and the host owns the shared concerns that each capability would otherwise have to re-invent.

The cost of consolidation is concentration. The shell is on the critical path for every page load, which means a bug in the shell affects every module simultaneously. A regression in the Axios retry interceptor can break every module's data loading at once; a regression in the OIDC event handler can prevent any module from receiving fresh tokens after a silent refresh. The shell is also the bottleneck for releases that touch shared infrastructure: changes to the global state shape or the custom-property contract require coordinated thinking across every consumer, even when the modules themselves are otherwise independent. This is the price of having one place where these things live.

What the consolidation buys is significant. Visual consistency across the platform falls out of the architecture rather than depending on per-module discipline; a module cannot accidentally draw the wrong header because it never draws a header at all. There is one OIDC implementation to audit, one token-refresh path to reason about, one place where session expiry is handled. The role-driven plugin registry gives the platform a single answer to the question "what can this user do?" — the shell reads the user's role assignments from the User Management Ser-

vice, filters the plugin manifest accordingly, and the resulting navigation is the authoritative view. The remaining chapters in Part II describe what the shell mounts; this chapter describes what makes mounting possible.

The portal application, located in `apps/portal/`, is the entry point for every authenticated user interaction with D2E. It is responsible for four concerns that no individual micro-application can handle on its own: authenticating the user through OpenID Connect [52], maintaining global application state across all plugins, providing a unified HTTP communication layer with automatic token management, and routing users to the correct experience based on their roles.

10.2 Application Bootstrap

The portal boots in `index.tsx`, where it creates a React 18 root, wraps the application in a `BrowserRouter` with basename `/d2e/portal`, applies the Material UI theme, and renders the root `App` component. Before React initializes, a synchronous deep-link extraction step runs: the code parses the current URL for whitelisted deep-link parameters (used by cohort and wizard paths) and persists them to session storage. This must happen before the OIDC redirect, which would otherwise destroy the URL parameters.

The `App` component is a thin wrapper that renders `OidcApp`, the OIDC authentication boundary. Everything downstream of this component exists inside the authentication context.

10.3 OIDC Authentication Flow

Authentication is implemented through the `@axa-fr/react-oidc` library, which provides an `OidcProvider` component that manages the full OpenID Connect authorization code flow with PKCE [59].

Provider Configuration

The `OidcApp` component reads its OIDC configuration from the `REACT_APP_IDP_OIDC_CONFIG` environment variable, which contains a JSON string specifying the identity provider's authorization endpoint, token endpoint, client ID, redirect URIs, and scopes. The configuration supports a template variable `{window.location.origin}` that is replaced at parse time with the actual origin, allowing the same configuration to work across environments.

The `OidcProvider` wraps the entire application tree and is configured with four lifecycle components: `OidcAuthenticating` (shown during the redirect to the identity provider), `OidcError` (shown when authentication fails), `OidcCallbackSuccess` (shown briefly during the token exchange after redirect), and `OidcSessionLost` (shown when the session expires, offering a logout button that clears local storage and redirects to the identity provider's logout endpoint).

The Authentication Sequence

When an unauthenticated user arrives, the `OidcAppInternal` component detects that `isAuthenticated` is false and renders the `PublicApp`. The public app displays public dataset information and a login route. When the user clicks login, the `OidcLogin` component calls the OIDC provider's `login()` method, which redirects the browser to the identity provider (typically Logto).

Before the redirect, the portal saves the user's current path as the post-login redirect URI. This is accomplished in `OidcAppInternal`: if the user is not authenticated and the current path is a valid redirect target, it is stored in the `AppState` and persisted to local storage.

After the user authenticates with the identity provider, the browser returns to the callback URL. The `OidcCallbackSuccess` component renders a loading indicator while the OIDC library exchanges the authorization code for tokens. Once the token exchange completes, the OIDC provider updates its internal state, and `isAuthenticated` becomes true.

Post-Authentication User Loading

The transition from authenticated OIDC state to a fully loaded user session happens in the `OidcLoginSilent` component, which is rendered as a permanent invisible component inside `PrivateApp`. On its first render after authentication, it extracts the ID token and its claims from the OIDC provider, stores them in the `AppState` via the `setIdToken` and `setIdTokenClaim` actions, and then calls the User Management API to fetch the user's group memberships and role assignments.

The `loggedIn` callback extracts the user's identity provider user ID from the token claims (using a configurable claim property name), calls `api.userMgmt.getUserGroupList` to retrieve their role map, and dispatches `setUserGroup` to populate the user state. If this call fails with a 403 status, the user is redirected to the no-access page; other failures redirect to logout.

A module-level flag `firstTimeLoggedIn` ensures this loading sequence runs exactly once per page load, preventing duplicate API calls during React's strict mode double-rendering in development.

Token Refresh and Session Management

The OIDC provider handles silent token refresh automatically. When tokens are acquired or renewed, the `OidcApp` component's `onEvent` handler fires. It dispatches an `oidc:token_refreshed` custom DOM event carrying the new access token, which single-spa [64] child applications can listen for to update their own token references.

For Entra ID [35] configurations, the event handler additionally validates that a required claim (configured via `REACT_APP_IDP_REQUIRED_CLAIM`) is present in the token payload, displaying an error snackbar if the third-party token claim is missing.

When the session expires and cannot be silently renewed, the OIDC provider renders the `OidcSessionLost` component, which displays an expiration message and a button that clears local storage and invokes the logout flow.

The logout flow itself (`OidcLogout`) clears all local state — user data, tokens, disclaimer acceptance — from both the React state and local storage, then calls `oidcLogout`, which instructs the OIDC library to redirect to the identity provider's logout endpoint with a post-logout redirect URI pointing back to `/d2e/portal`.

10.4 State Management

The portal's global state is managed through React's `useReducer` hook, wrapped in a context provider pattern. The `AppProvider` component creates two contexts: `AppContext` for reading state and `AppDispatchContext` for dispatching actions.

State Shape

The `AppState` interface defines the complete shape of the portal's global state:

- **token.** The OIDC token state, containing the raw `idToken` string and parsed `idTokenClaims` with fields for `sub`, `oid`, `name`, `username`, `email`, and token expiry.

- **user.** The user's role state: `userId`, `idpUserId`, and boolean flags `canAccessSystemAdminPortal`, `canAccessResearcherPortal`, `isResearcher`, `isUserAdmin`, `isSystemAdmin`, and `isDashboardViewer`. A `isDatasetResearcher` dictionary maps dataset IDs to boolean access flags.
- **activeDataset.** The currently selected dataset, stored as an `id` and `releaseId` pair.
- **translation.** The current locale string and a dictionary of translation key-value pairs.
- **feedback.** An optional feedback state for displaying success or error snackbar messages.
- **conversationHistory.** An array of chat items for the AI assistant conversation, using the NLUX chat item format.
- **disclaimer.** Tracks whether the platform disclaimer should be displayed and whether the user has accepted it.
- **postLoginRedirectUri.** The URL path the user was attempting to access before being redirected to login, used to restore their navigation after authentication.

Reducer and Actions

The reducer uses a map-based dispatch pattern rather than a switch statement. Sixteen action types are defined in the `ACTION_TYPES` enum: `SET_FEEDBACK`, `CLEAR_FEEDBACK`, `CHANGE_LOCALE`, `SET_ACTIVE_DATASET_ID`, `SET_ACTIVE_RELEASE_ID`, `SET_ID_TOKEN`, `SET_ID_TOKEN_CLAIM`, `CLEAR_TOKEN`, `SET_USER`, `CLEAR_USER`, `SET_POST_LOGIN_REDIRECT_URI`, `CLEAR_POST_LOGIN_REDIRECT_URI`, `SET_CONVERSATION_HISTORY`, `SET_DISCLAIMER_ACCEPTED`, `SET_SHOULD_DISPLAY_DISCLAIMER`, and `CLEAR_DISCLAIMER`. Each action type maps to a pure function that takes the current state and a payload, returning the next state.

Each state slice has its own action file and custom hook. The `useToken` hook provides `setIdToken`, `setIdTokenClaim`, and `clearToken`. The `useUser` hook provides `setUserGroup` and `clearUser`. The `useActiveDataset` hook provides dataset selection. These hooks abstract the dispatch mechanism, providing a clean API to components that need to read or modify state.

State Persistence

The `usePersistedReducer` hook adds selective local storage persistence. It accepts a whitelist of state keys — currently `activeDataset` and `postLoginRedirectUri` — and synchronizes only those slices to local storage under the key `d2e_app`. On initialization, it hydrates the reducer's initial state from local storage if available. On every state change, it compares the new state to the previous state using deep equality (via `fast-deep-equal`) and writes the whitelisted slices if they differ. This ensures that the user's dataset selection survives page refreshes without persisting sensitive data like tokens to local storage.

10.5 Backend Communication

All authenticated HTTP requests flow through a centralized Axios client defined in `axios/request.ts`. The client is configured with two interceptors that implement the portal's network communication contract.

Request Interceptor

The request interceptor runs before every outgoing request. It checks the request URL against a whitelist of public endpoints — `dataset/public/list`, `config/public`, and

`config/public/overview-description` — that do not require authentication. For all other requests, it calls `getAuthToken(false)` to retrieve the current OIDC access token (the `false` argument suppresses automatic logout on token failure) and attaches it as a `Bearer` token in the `Authorization` header.

Response Interceptor

The response interceptor implements automatic retry logic for network errors. When a request fails with `ERR_NETWORK` or a message containing `ERR_NETWORK_CHANGED` — conditions that occur when Docker containers restart during operation — the interceptor waits ten seconds and retries up to three times. This provides resilience during deployment rollouts and container restarts, particularly valuable during end-to-end testing where service containers may cycle.

Request Memoization

The exported `request` function wraps the raw Axios call with `memoizee` memoization using a maximum age of 800 milliseconds. Identical requests made within this window return the cached promise rather than issuing a new HTTP call. The normalizer serializes the full request options to JSON for cache key generation. This brief memoization window prevents duplicate requests during React's rendering lifecycle — where effects may fire multiple times, or parent and child components may independently request the same data — without stale data risks.

Service Layer

The `api` object in `axios/api.ts` aggregates service classes, each encapsulating the HTTP operations for a specific backend domain: `UserMgmt` for user and role management, `Gateway` for the analytics gateway, `Dataflow` for ETL pipeline operations, `SystemPortal` for system configuration, `DbCredentialsMgr` for database credential management, `Trex` for Trex server operations, `StudyNotebook` for notebook management, `Translation` for i18n strings, `Demo` for demo dataset provisioning, `StrategusResults` and `StrategusAnalysis` for OHDSI Strategus workflows, and `FhirGateway` for FHIR server operations. Each class uses the shared `request` function, ensuring consistent token attachment, retry behavior, and memoization.

10.6 Routing

The portal's routing uses React Router v6 with basename `/d2e/portal`. The top-level routing decision depends on authentication state: unauthenticated users see the `PublicApp`, which provides routes for public dataset browsing and login; authenticated users see the `PrivateApp`.

The `PrivateApp` determines the default route based on the user's roles. If a post-login redirect URI was saved before the OIDC flow, the user is sent there. Otherwise, users who can access the researcher portal default to `/researcher`; users who can only access system admin default to `/systemadmin`; users with neither role are sent to `/no-access`.

Within the researcher section, the `Researcher` component sets up a dual routing model. Single-spa applications have their DOM containers pre-rendered with CSS visibility toggling, while legacy plugins are rendered through React Router `Route` elements. The researcher overview, dataset information, and account pages use standard React routing. Plugin routes are generated dynamically from the filtered plugin manifest.

Within the system admin section, the `SystemAdmin` component filters the plugin manifest based on enabled flags and feature flags (fetched from the system features API), then generates routes for each active plugin. The first enabled plugin's route becomes the default redirect for the

system admin landing page. Setup plugins are nested within the Setup route, rendered as a card grid by `SetupOverview` with individual plugin routes for each configuration panel.

This routing architecture means that the portal manages the top-level URL namespace while delegating path segments to individual micro-applications. A URL like `/d2e/portal/researcher/concepts/some/deep/path` is handled by React Router at the portal level (matching `/researcher/concepts/*`), which shows the concepts single-spa container, and then by the concepts application's own internal router for the remaining path segments.

10.7 Integration Points

The shell is the integration point. Every other frontend module in Part II is mounted, fed, and authenticated by it. The shell depends on the User Management Service for the role data that decides which plugins appear in the navigation, and on Logto for the authorization code redirect that produces a session in the first place; neither is optional, and a failure in either prevents the rest of the frontend from rendering anything beyond the public landing routes. Outbound, the shell injects four custom properties into every single-spa application — the `getAuthToken` callback, the active dataset ID, the user locale, and the feature flag map — and dispatches `custom-props-changed` DOM events when any of them change. Modules that mount inside the shell rely on this contract: they do not perform their own OIDC flow, they do not maintain their own dataset selector, and they do not fetch their own role assignments. They read what the shell tells them and trust the shell to keep it current.

11 Patient Analytics

Patient Analytics is the primary researcher-facing tool in the D2E platform. It provides an interactive visual query builder that allows clinical researchers to define patient cohorts, apply analytical constraints, and visualize results — all without writing SQL. The interface translates a researcher's clinical questions into structured queries that flow through the Query Engine pipeline, returning charts, survival curves, and patient lists. Understanding how this tool works requires following data from the researcher's intent through several layers of abstraction: from visual filter cards on a canvas, through an intermediate query representation, across network boundaries to the Query Engine, and back again as rendered visualizations.

11.1 Library Structure

Patient Analytics is built on Vue [74], which stands in deliberate contrast to the React [34]-based portal shell that hosts the rest of the D2E frontend. The module was already a mature, feature-rich Vue application with hundreds of components and a deeply entrenched state-management layer when the broader portal adopted React for its micro-frontend architecture. Rewriting it in React would have offered no functional gain, so the pragmatic decision was to wrap the Vue application as a micro-frontend that mounts within the React portal shell.

The library exports a root component that orchestrates a split-pane layout: a left panel for filter construction and cohort management, and a right panel for chart rendering and axis configuration. The split pane lets researchers adjust the relative space devoted to query building versus result visualization, and the left pane can be collapsed entirely to maximize chart real estate.

11.2 The Filter Card Canvas

The central interaction metaphor of Patient Analytics is the filter card canvas. A filter card represents a clinical event type — a diagnosis, a procedure, a drug administration, a lab measurement — drawn from the Patient Analytics configuration (PA config) that defines which

clinical concepts are available for a given dataset. When a researcher opens Patient Analytics against a dataset, the system loads the PA config, which enumerates every interaction type and its attributes, along with metadata about data types, allowed operations, and default binning.

The canvas begins with a single mandatory card: a Basic Data card that represents patient-level demographic attributes such as age, gender, and vital status. This card cannot be removed. Additional filter cards are added from a dropdown menu listing all interaction types defined in the PA config. Each menu item corresponds to a configuration path that uniquely identifies the clinical concept within the data model.

When the researcher selects an interaction type, the system creates a new filter card model bound to that interaction's metadata and assigned an instance number, allowing multiple cards of the same type to represent different clinical events — for example, "first diagnosis" and "second diagnosis." Each card carries a stable instance identifier that survives bookmark save and restore cycles.

11.3 Filter Card Lifecycle

The lifecycle of a filter card follows a well-defined progression from configuration to constraint capture. When a card is added to the canvas, the UI consults the PA config to determine which attributes are available for that interaction type. Each attribute becomes a constraint control within the card.

Each attribute renders as a control type-matched to its data type — text-with-autocomplete for coded values, range inputs for numerics, date pickers for temporal attributes, and a vocabulary picker for concept-set attributes. The control choice falls out of the PA config; the component layer dispatches accordingly. Autocomplete values are fetched on demand from the Analytics Service, which queries the live data warehouse so researchers only see clinically meaningful options for the current dataset.

Each constraint modification updates the canvas state immediately. The state is held in a normalized form that lets a single-constraint edit propagate without re-rendering the entire filter tree — a design choice discussed at the end of the chapter.

11.4 IFR Construction

The Internal Filter Representation (IFR) is the structured query format that bridges the visual canvas and the Query Engine. Understanding how the UI assembles the IFR from canvas state is essential to understanding the entire query flow.

The IFR is constructed on demand whenever the query state changes. A traversal of the canvas state assembles a nested IFR object whose top level carries configuration metadata — the PA config identifier and version — and a cards collection organized as a nested boolean container hierarchy. The hierarchy encodes the combination logic that the researcher has chosen:

- **Match All (AND) groups.** Filter cards within the same boolean filter container are combined with OR logic at that level — meaning any one card in the group must match. The boolean filter containers themselves are combined with AND logic at the outer boolean container level, meaning all groups must be satisfied.
- **Match Any (OR) groups.** When the researcher creates multiple boolean filter containers, these represent alternative patient pathways. The Query Engine computes the cartesian product of these groups, generating separate SQL queries for each combination and unioning the results.

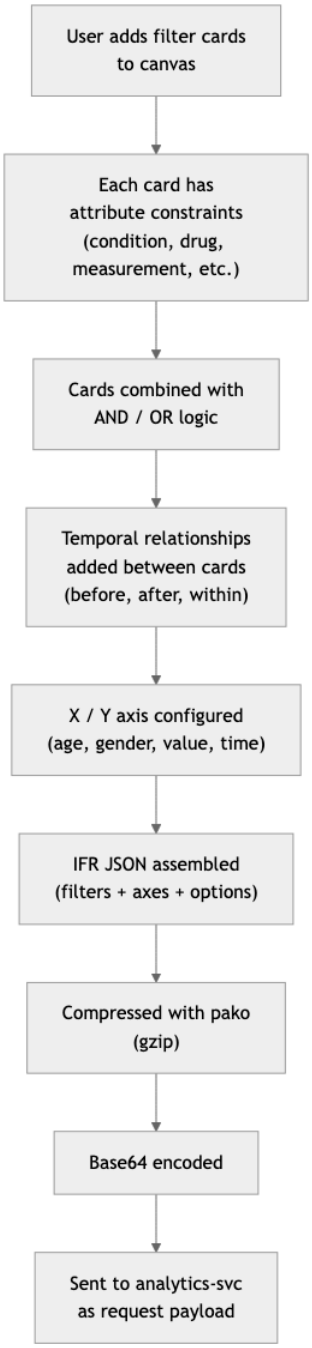

Figure 11: IFR Construction from UI State

Each filter card in the IFR carries its configuration path (identifying the clinical concept), an instance number (distinguishing multiple cards of the same type), a stable instance identifier, and an attributes collection that contains the researcher's constraints. The attributes are themselves wrapped in boolean containers: each attribute's constraint expressions are OR-combined (the patient must match at least one specified value), and the attributes within a card are AND-combined (all specified attributes must be satisfied simultaneously).

Filter cards also carry several flags that modify their behavior. An exclusion flag wraps a card in a logical negation, converting it from an inclusion criterion to an exclusion criterion — the resulting SQL will require that matching patients do *not* have the specified clinical event. An inactive flag allows a card to remain on the canvas without participating in the query, useful for iterative refinement. Entry and exit flags mark cards as cohort entry or exit criteria for temporal cohort definitions.

Temporal relationships between cards are captured through two mechanisms. A successor relationship defines minimum and maximum days between two clinical events, producing days-between calculations in the generated SQL. An advance time filter provides more complex temporal expressions combining multiple time-based conditions with boolean logic.

11.5 The Query Execution Flow

When the researcher triggers a query — either by clicking the execute button or by changing a filter that triggers auto-refresh — the IFR undergoes a series of transformations before reaching the database.

The IFR is serialized, compressed, and base64-encoded, then attached as the `mriquery` query-string parameter on the request to the Analytics Service. The compression matters: IFR payloads for complex queries with many filter cards and constraints can exceed URL length limits in their uncompressed form. The Analytics Service decompresses the payload and routes it to the Query Generation Service, which transforms the IFR through its multi-stage pipeline (IFR to FAST to AST to SQL), executes the resulting SQL against the clinical data warehouse, and returns chart-shaped JSON.

The response shape varies by chart type — categories and stacked traces for bar charts, quartile statistics for box plots, paginated rows for patient lists, and time-indexed survival probabilities for Kaplan-Meier — but the wire contract is uniform: the chart layer receives JSON keyed to the active chart type and renders it.

11.6 Chart Rendering

The chart layer is a dispatcher: a single chart-router component selects the right visualization for the active chart type. Each visualization is a self-contained component that consumes the chart-shaped JSON the Analytics Service returns. This dispatch model is what allows new chart types to be added without modifying the query path — the IFR and the Analytics Service do not need to know which chart will render the result, only that some chart will.

11.7 Axis Configuration

The axis system determines what the charts display. Patient Analytics supports up to four X-axis dimensions and one Y-axis measure, though typical usage employs one or two X-axes and one Y-axis. Each axis tracks a selected attribute path, the originating filter card, and optional parameters such as bin size for continuous variables.

A hierarchical axis menu lists every chartable filter card and its numeric or categorical attributes. A filter card is chartable if it participates in the current query (is not inactive or

excluded). Selecting an attribute updates the axis model and triggers a new query. Bin sizes default from the PA config and can be overridden per axis; the Query Engine uses the bin size to discretize continuous values into histogram buckets.

11.8 Bookmark and Cohort Management

The bookmark system provides persistence for query state. A bookmark captures the complete analytical context: the IFR filter tree, axis selections with bin sizes, chart type, sort configuration, Kaplan-Meier settings, and patient list column selections. This is stored as a versioned JSON structure with dataset binding.

Saving a bookmark snapshots the canvas, axis, and chart state and writes it to the bookmark service with the researcher's user identifier, a chosen name, the PA config and CDM config identifiers, and the dataset identifier. Restoring reverses the path: the stored filter structure is parsed back into an IFR, loaded into the canvas state, and the axis and chart settings are replayed.

Bookmarks can be marked as shared, making them visible to other researchers working on the same dataset. Shared bookmarks are read-only for non-owners, who can load a shared bookmark and save a personal copy under a new name.

Bookmarks are tightly integrated with cohort definitions. A cohort definition represents a materialized patient population — the actual set of patient identifiers that match a given set of criteria at a point in time. The bookmark list displays three kinds of entry: standalone bookmarks (query state without materialization), materialized cohorts (patient sets usable for downstream analysis), and Atlas [44]-style cohort definitions. Materialization is conditional on dataset capability and triggers a backend job that executes the query and persists the resulting patient identifiers. Materialized cohorts can then be compared side-by-side, viewed as patient lists, renamed, or deleted, with the same analytical axes applied to multiple saved cohorts at once.

The chapter also supports OHDSI Atlas-format cohort definitions through a parallel editor that round-trips with the Atlas tooling; the round-trip preserves concept sets, demographics, nested groups, and temporal windows. The dual-mode capability lets researchers work in whichever paradigm — the D2E filter card canvas or the Atlas cohort definition model — best fits their workflow, while sharing the same underlying cohort materialization and comparison infrastructure.

11.9 Data Export

Patient Analytics provides multiple data export pathways. CSV downloads are available for every chart type, each carrying the same compressed query payload as the chart rendering request, ensuring that exported data exactly matches what the researcher sees on screen.

For patient list exports that may contain large volumes of data, a ZIP pathway streams entity-separated CSV files — one per interaction type — in parallel and packages them into a single archive. This avoids the denormalization overhead that would occur if all interaction types were flattened into a single table. The researcher can choose between exporting only the currently selected columns or every column available for the matching interactions.

Chart images can be exported for inclusion in reports and presentations. The PDF export path renders the current chart at a specified resolution and waits for the chart to signal that it has finished rendering before capture.

11.10 State Management Architecture

The canvas state is normalized: filter cards, attributes, constraints, and the boolean containers that group them are held as flat entity maps keyed by stable identifiers, rather than as a single nested tree. Normalization is the architectural answer to a performance question. When a researcher tweaks a single constraint on a single filter card, only that constraint entity changes; the reactivity system re-renders only the components that depend on it. A naive nested-tree state would have to clone and replace the whole filter tree for every keystroke, triggering cascading re-renders across every card on the canvas. With dozens of cards and hundreds of constraints, that cost is the difference between a responsive canvas and a sluggish one.

The state is split along the same lines as the architectural concerns: query building (what the researcher is asking), visualization (how the answer is displayed), and persistence (how queries are saved and shared) live in separate stores. This separation lets the same IFR be rendered as a bar chart, a box plot, or a patient list by changing only the visualization state — the query state never has to know.

11.11 Trade-offs of the Vue-in-React Choice

The choice to keep Patient Analytics in Vue and mount it inside a React shell is the most consequential architectural decision the frontend has made, and it deserves to be examined honestly. The choice is defensible only if the costs are named, because costs that are not named tend to be paid silently and resented later. Four costs are worth being explicit about.

The first is bundle size. Every browser session that loads the researcher portal and navigates into Patient Analytics ends up with both the React runtime (for the portal shell, the navigation, the dataset selector, and every other researcher module) and the Vue runtime (for the Patient Analytics canvas, the chart components, and the analytics state store). The two frameworks coexist in memory, share no code, and each carries its own reactivity system, virtual DOM, and component lifecycle machinery. On a cold load this manifests as additional kilobytes over the wire and additional megabytes in memory; on a warm load the cache absorbs most of the impact, but the duplication never goes away. A single-framework portal would not pay this cost.

The second is state coordination. The portal's React-side state — the active dataset, the current OIDC token, the user's role assignments, the locale — is owned by the shell's reducer store. Patient Analytics's state — the filter card canvas, the IFR, the chart configuration, the bookmark list — is owned by the analytics store. Neither store can read the other directly. The bridge is a token-getter callback injected as a single-spa [64] custom property and a custom-props-changed DOM event that fires when shell-side state changes. This is a documented seam, but a seam: when a researcher selects a different dataset in the shell's selector, the shell dispatches the event, and Patient Analytics has to listen for it, react to it, and decide whether to discard its current canvas or keep it. The contract works, but it is more delicate than a single-store architecture would be, and bugs at the boundary tend to manifest as subtle staleness rather than loud failures.

The third is developer ergonomics. A new team member who is asked to debug an end-to-end issue — a researcher reports that selecting a new dataset does not refresh the chart, say — has to be comfortable reading both React and Vue idioms within a single page load. They must trace from a React event handler, through the shell's reducer, through the custom-props mechanism, into a Vue component's watcher, into a state-store action, and out to the analytics service. The architecture does not let anyone specialize in just one framework if their work touches the seam.

The fourth is routing. The portal's React Router owns the path prefix `/d2e/portal/researcher/cohorts`, and Patient Analytics's internal routing owns the segments below that. The two routers must agree on which paths each owns; mistakes produce blank pages (when both routers think the other is responsible) or duplicate-render bugs (when both try to render the same path). The boundary is explicit in the configuration, but it is not enforced by the type system, and adding a new sub-route inside Patient Analytics has historically required confirming that the shell's React Router is not also trying to handle it.

What the choice buys is what justifies the costs. Patient Analytics ships independently — its release cadence is its own, decoupled from the shell's. It evolves independently — the Vue team can adopt new Vue features, refactor state modules, and rework the canvas without coordinating with the React-portal team. And, most importantly, it continues to leverage years of accumulated Vue idioms: hundreds of components, a normalized entity model refined through real clinical use, a chart-rendering pipeline that has been hardened against the long tail of dataset shapes researchers actually load. None of that needed to be redesigned, ported, or risked in order for the wider portal to exist. The seam is the price of preserving that work, and the price is paid willingly.

12 Data Management Tools

12.1 Purpose

The data management tools are the administrative surface for managing what data the platform contains — the datasets, the ETL configurations, the source-to-OMOP mappings, and the curated concept sets that researchers later draw on. They are deliberately distinguished from the analytical tools described elsewhere in Part II: the analytical tools answer questions about data, the data management tools decide what data is available to be questioned. A researcher composing a cohort in Patient Analytics never opens these tools; a data steward preparing a new institution's clinical extract lives inside them.

This separation matters because the audiences and the workflows are fundamentally different. The analytical surfaces optimize for exploration and iteration — visual canvases, fast feedback, low-commitment edits. The data management tools optimize for correctness and provenance: every concept set decision is a deliberate act, every column mapping carries downstream consequences, every dataflow revision is preserved. Treating them in their own chapter keeps that distinction visible and lets the chapter speak directly to the people who use these tools without diluting it with concerns that belong to the cohort builder.

Beyond the analytical capabilities of Patient Analytics, the D2E frontend provides a suite of data management tools that support the full lifecycle of clinical data preparation: from vocabulary exploration and concept set curation, through source-to-standard concept mapping, to visual ETL pipeline construction. These tools are implemented as separate micro-frontend applications, each tailored to a specific phase of the data harmonization workflow, and each built with the technology stack best suited to its interaction model.

12.2 Concept Sets

The Concept Sets application is a React [34]-based tool built with Vite that provides researchers and data stewards with the ability to browse, create, and manage reusable collections of clinical vocabulary concepts. A concept set defines a group of related concepts—for example, "all diabetes-related diagnoses" or "statin medications"—that can be referenced throughout the platform wherever a vocabulary-aware filter is needed.

The Concept Set Expression Model

The fundamental data model behind concept sets is the expression: a list of concept entries where each entry specifies a concept identifier along with three boolean flags that control how that entry participates in the resolved set.

The *include* flag determines whether a concept adds to or subtracts from the set. Most entries are inclusive, meaning they contribute their concepts to the final collection. Excluded entries operate as refinements—they remove specific concepts that would otherwise be captured by a broader inclusion rule.

The *descendants* flag controls hierarchical expansion. When enabled, the concept and all of its descendants in the vocabulary hierarchy are included (or excluded). This is the mechanism that makes concept sets practical for large vocabularies: rather than manually selecting hundreds of individual ICD-10 [77] codes for diabetes, a researcher includes the parent concept "Diabetes mellitus" with descendants enabled, and the Terminology Service recursively expands it to capture every specific subtype.

The interplay between these flags produces an expressive algebra. A concept set for "all diabetes diagnoses except gestational diabetes" would include the parent diabetes concept with descendants enabled, then add an exclusion entry for gestational diabetes (also with descendants enabled to capture its subtypes). The resolution engine evaluates inclusions first, building the full expanded set, then subtracts the expanded exclusions.

Expression Resolution

When a concept set expression needs to be resolved into a concrete list of concepts, the UI calls the Terminology Service through the D2E WebAPI. The resolution process expands each entry according to its flags, performs the inclusion and exclusion algebra, and returns the final list of concept identifiers with their display names, vocabulary domains, and validity status. This resolution can be triggered on demand when viewing a concept set's contents, or implicitly when a concept set is referenced in a Patient Analytics filter card.

The Terminology Browser

The Terminology component is a full-featured vocabulary browser that serves multiple contexts across the platform. It operates in several modes: `CONCEPT_SEARCH` for general-purpose vocabulary exploration, `CONCEPT_SET` for building and editing concept set expressions, `CONCEPT_MAPPING` for selecting target concepts during source-to-standard mapping, and `CONCEPT_MULTI_SELECT` for choosing multiple concepts in a single session.

The browser presents a tabbed interface. The Search tab provides a text search with filterable columns for domain, vocabulary, concept class, and standard/non-standard status. When a dataset is selected, the search queries the Terminology Service to find concepts matching the search terms within the vocabularies loaded for that dataset. Results are displayed in a paginated table with concept ID, name, domain, vocabulary, class, standard designation, and validity period.

The Selected Concepts tab shows the concepts currently included in the concept set expression, with controls for toggling the include/exclude and descendant flags on each entry. For Atlas [44]-mode concept sets, the tab also supports the Atlas concept set expression format, providing compatibility with OHDSI tools.

The Detail view for a selected concept shows its full metadata, hierarchical relationships (parents and children in the vocabulary tree), and cross-references to related concepts in other

vocabularies. This contextual information helps researchers make informed decisions about which concepts to include in their sets.

Concept sets support sharing: the creator can mark a set as shared, making it visible to other users. An administrative configuration controls whether sharing is restricted to administrators or available to all users. Concept sets are scoped to datasets, ensuring that vocabulary references remain valid within their data context.

12.3 Concept Mapping

The Concept Mapping application provides the interface for mapping source vocabulary codes to standard OMOP concepts. This is a critical step in the ETL process: raw clinical data arrives with institution-specific codes (local drug formulary IDs, custom procedure codes, facility-specific diagnosis labels), and these must be mapped to standardized OMOP vocabulary concepts before the data can participate in cross-institutional analyses.

Source Data Import

The mapping workflow begins with importing a source code list, typically as a CSV file containing the source codes along with contextual metadata such as source names, frequencies (how often each code appears in the source data), and descriptions. The `CsvReader` component parses the uploaded file and presents an import dialog where the user maps CSV columns to the required fields: source code, source name, frequency, description, and optionally a domain ID that constrains the target vocabulary domain.

Once imported, the source codes are displayed in the `MappingTable`—a Material React Table with columns for mapping status, source code, source name, frequency, description, and the mapped OMOP concept ID and name. Each row represents a source code that needs a standard concept mapping. The status column indicates whether a mapping has been assigned, providing a visual progress indicator for large mapping tasks.

Manual Mapping Workflow

To map a source code, the researcher clicks on a row in the mapping table. This selection triggers the `MappingDrawer` component, which dispatches a custom DOM event (`alp-terminology-open`) to open the Terminology browser in `CONCEPT_MAPPING` mode. The browser is pre-populated with the source name as the initial search term and pre-filtered to show only standard concepts, optionally restricted to the domain specified in the column mapping configuration.

When the researcher selects a target concept from the Terminology browser, the selection callback dispatches a `SET_SINGLE_MAPPING` action to the concept mapping reducer, which associates the source code row with the chosen OMOP concept ID, name, domain, and vocabulary system. The mapping table updates to show the assigned concept, and the status changes to indicate completion.

This manual workflow is designed for precision: each mapping decision is a deliberate act by a domain expert who understands the semantic relationship between the source code and the target concept. For large source vocabularies, however, manual mapping of every code would be impractical.

Embedding-Based Suggestions

For source codes without a known standard mapping, the Concept Mapping application requests suggestions from the Terminology Service. The source code's name (and any provided

description) is encoded into the same sentence-embedding space used for semantic concept search, and the nearest standard concepts by cosine similarity are returned as a short candidate list. The researcher reviews the candidates and selects the one that best fits; no mapping is committed automatically. This keeps the precision of expert review while removing the burden of locating candidates by hand.

Persistence and Export

Completed mappings are saved through the Concept Mapping backend service. For large mapping sets, the payloads are compressed before transmission to handle the volume of data efficiently. The mapping table also supports CSV export of the current mapping state, allowing researchers to download their work for external review or archival. The export includes all source columns alongside the mapped concept identifiers and names.

12.4 ETL Dataflow Editor

The Dataflow Editor is the most architecturally complex of the data management tools. It provides a visual directed acyclic graph (DAG) editor where users compose data transformation pipelines by connecting processing nodes, then execute those pipelines against the Job Plugins backend service, which orchestrates execution through Prefect [55] workflow engine.

Technology Stack

Unlike the other Vite-based data tools, the Dataflow Editor uses Webpack 5 as its bundler and Redux Toolkit for state management—a deliberate choice driven by the complexity of the graph editing interaction model. The visual canvas is built on React Flow, which provides the node-and-edge graph editing primitives: draggable nodes with typed input/output handles, edge connections with validation, zoom and pan controls, and layout management. Redux Toolkit's entity adapters manage the node and edge collections with normalized state, while RTK Query handles the API layer for dataflow persistence and execution.

The Redux store is structured around three primary concerns. The `flow` slice (managed by `createSlice`) tracks the canvas state: the current dataflow ID, revision ID, the collections of nodes and edges (each managed by `createEntityAdapter` for efficient normalized updates), flow variables, imported libraries, dialog visibility states, and the current flow run status. The `dataflowApi` slice (managed by RTK Query's `createApi`) provides the full CRUD API for dataflow persistence, execution, and result retrieval. A separate `WhiteRabbitApi` slice handles the White Rabbit [50] scan report integration.

The Node Type System

The editor supports a rich palette of node types, each representing a different data processing capability:

Python and **R** nodes provide code editors (powered by Monaco, the same editor engine behind VS Code) where users write transformation logic. Python nodes include a default `exec` function signature that receives input data and returns transformed output, along with a `test_exec` function for the test mode. R nodes follow the same convention with R syntax.

SQL nodes allow users to write SQL queries that execute against a configured database connection. The Monaco editor provides SQL syntax highlighting and the node configuration includes connection selection.

CSV and **Generic File** nodes handle file-based data ingestion. CSV nodes are configured with a file path, delimiter character, header flag, column definitions, and character encoding. File

uploads are managed through dedicated API endpoints that store files on the backend and associate them with the node ID.

Database Reader and **Database Writer** nodes provide direct database connectivity—the reader executes a SQL query and outputs the result as a dataframe, while the writer takes a dataframe input and persists it to a target database table.

Rabbit in a Hat (Data Mapping) nodes embed the Mapping application within the dataflow canvas, allowing source-to-target column mapping using WhiteRabbit scan report context. This integrates the ETL mapping workflow directly into the pipeline definition.

Concept Mapping nodes embed the Concept Mapping application, enabling vocabulary mapping steps within the dataflow.

White Rabbit nodes trigger database schema scanning, producing scan reports that characterize source data tables, columns, and value distributions. The scanned schema information feeds into the Data Mapping nodes as context for column mapping decisions.

Transform FHIR Data nodes handle FHIR data transformation using configurable mapping rules, supporting healthcare interoperability workflows.

Each node type defines typed input and output handles that control which connections are valid. Handles are typed as `Any`, `Dataframe`, or `Object`, and the connection validation ensures type compatibility—a Dataframe output can connect to a Dataframe input, and Any handles accept all types.

Pipeline Composition

Building a pipeline follows a drag-and-drop workflow. The user opens the node type selection dialog, which presents the available node types with descriptions and stability tags (Stable or Experimental). Selecting a type places a new node on the canvas with default configuration. The user then configures the node—writing code, selecting files, configuring database connections—through a drawer panel that opens when the node is selected.

Edges are created by dragging from an output handle on one node to an input handle on another. Each edge represents a data dependency: the target node receives the output of the source node as its input. The DAG structure is enforced by React Flow's connection validation, preventing cycles that would create infinite execution loops.

The canvas supports grouping nodes into subflows for organizational clarity, flow-level variables that can be referenced across nodes, and import library declarations that specify Python packages to install before execution.

Execution Model

Pipeline execution follows a save-then-run protocol. When the user clicks the run button, the system first checks whether the current canvas state has been saved. If the flow is in "draft" status (modified since last save), the save dialog is presented. Once saved, the execution request is sent to the Job Plugins service at `/prefect/flow-run/{dataflowId}`, which translates the DAG into a Prefect flow and submits it for execution.

The system uses a polling mechanism to track execution progress. The `usePollingEffect` hook periodically queries the flow run state endpoint at `/prefect/flow-run/{flowRunId}/state`, updating the Redux store with the current status. The state machine progresses through states including Pending, Running, Completed, Failed, and Crashed. While running, the button displays a loading indicator; on failure, it shows an error state with the failure reason.

A test mode allows users to execute pipelines without persisting them, useful for iterative development. Test execution sends the current canvas state (nodes, edges, variables, and import libraries) directly to the `/prefect/test-run` endpoint, bypassing the save requirement.

Execution results are retrieved per-node via the `/dataflow/{id}/flow-run-results` endpoint. Each node's result (or error) is mapped back to the canvas, allowing users to inspect outputs and diagnose failures at the individual step level.

Versioning and Collaboration

Dataflows support revision history. Each save creates a new revision, and users can browse previous revisions, compare changes, and restore earlier versions. The duplication feature allows creating a copy of an existing dataflow (at a specific revision) as a starting point for variations. Dataflows can also be created from templates—predefined pipeline patterns for common ETL scenarios—and synchronized with remote sources when working in collaborative environments. The remote diff check endpoint detects divergence between local and remote versions, and the overwrite-from-remote action resolves conflicts by accepting the remote state.

12.5 Data Mapping

The Mapping application provides a specialized interface for source-to-target column mapping in the context of ETL transformations. Unlike the Concept Mapping tool (which maps vocabulary codes to standard concepts), the Data Mapping tool maps source database columns to target CDM (Common Data Model) table columns, defining how raw data fields flow into the standardized schema.

Visual Mapping Interface

The mapping interface uses React Flow to present a two-panel graph layout. Source tables appear as nodes on the left side, and target CDM tables appear on the right. Each table node exposes its columns as connection handles. The user creates mappings by drawing edges between source column handles and target column handles, establishing the column-level correspondence that will drive the ETL transformation.

The interface operates at two levels. The `TableMapLayout` displays the table-level view, showing source and target tables as nodes with edges representing table-to-table relationships. Double-clicking a table edge drills down to the `FieldMapLayout`, which shows the individual column mappings within that table pair. This two-level design keeps the high-level data flow comprehensible even when the source schema contains dozens of tables, while providing the column-level precision needed for accurate ETL specification.

WhiteRabbit Integration

The Data Mapping tool integrates with the WhiteRabbit database profiling tool, which is part of the OHDSI tool suite. WhiteRabbit scans a source database and produces a scan report that characterizes each table and column: data types, value distributions, null frequencies, and sample values. This profiling information provides essential context for mapping decisions—a data steward can see what values actually exist in a source column before deciding which target column it should map to.

Within the Dataflow Editor, a White Rabbit node can scan a database by providing connection credentials (server, port, database, schema, username, password) and selecting which tables to scan. The scan runs as a backend job through the Job Plugins service, and the results populate the node's scanned schema state. This scanned schema is then passed to downstream Data

Mapping nodes, which use it to populate the source table and column information in the visual mapping interface.

The scan report can also be downloaded as an Excel file for offline review, through the WhiteRabbit API's `scan-report/result-as-resource` endpoint. Connection testing is available before committing to a full scan, allowing users to verify credentials and connectivity without waiting for the potentially lengthy scan process.

ETL Report Generation

Once column mappings are defined, the Data Mapping tool can generate an ETL specification document. The `FieldMapLayout` component transforms the current mapping state into an ETL model that captures the source schema, target CDM schema, table relationships, and field-level mappings. This model is submitted to the WhiteRabbit backend, which generates a formatted Word document (DOCX) describing the complete ETL specification. The generation runs asynchronously as a flow run, and the tool polls for completion before downloading the resulting document. This generated specification serves as both documentation for the ETL process and a communication artifact between data engineers and clinical domain experts.

12.6 Integration Points

Each data tool talks to a specific backend cloud function rather than to a shared aggregator, and the boundary is sharp enough that a failure in one tool does not propagate to the others. The Concept Sets application calls the Terminology Service through the D2E WebAPI to resolve concept set expressions and to drive the vocabulary browser; concept set persistence is handled by the same service. The Concept Mapping application reads and writes through the Concept Mapping backend service, with the Terminology browser opened in `CONCEPT_MAPPING` mode for target selection and the Terminology Service's embedding index queried for candidate concepts when the researcher asks for a suggestion. The Dataflow Editor talks to the Job Plugins service for dataflow persistence and to Prefect (via Job Plugins) for execution and run-state polling, while the WhiteRabbit API hosts schema scans and ETL specification generation. The Data Mapping tool consumes WhiteRabbit scan reports as upstream context and submits its mapping model back to WhiteRabbit for DOCX generation. All four tools authenticate through the portal shell's token getter and inherit the active dataset from the shell's custom-props bridge — they do not maintain their own session or dataset state.

13 Analysis and Notebooks

13.1 Purpose

Notebooks are how researchers escape the canvas. Patient Analytics expresses a great deal — cohort definitions, attribute filters, axis selections, Kaplan-Meier configuration — but its grammar is bounded. The moment a research question requires Python, R, or SQL beyond what the canvas exposes, the work has to move somewhere else. This chapter describes the two "somewhere else" surfaces the frontend offers: a node-based composition environment for OHDSI Strategus [48] studies and an in-browser notebook environment built on Starboard. Both are paths for analytical work that the visual query builder does not represent.

This is a frontend chapter, and the boundary is deliberate. It covers the kernel-launch handshake with Jupyter Enterprise Gateway [56], the cookie-based authentication that the embedded runtimes rely on, and the way notebook sessions inherit dataset access from the user's role. It does not cover the kernel infrastructure itself — container images, R library volumes,

language-server configuration, idle culling — which belongs to Part IV. The split keeps the discussion focused on the parts that a frontend developer or a portal operator can change without rebuilding the platform.

The D2E frontend provides two complementary environments for analytical work: a structured, form-driven interface for configuring OHDSI Strategus studies, and a flexible notebook environment for exploratory analysis with code. Both are implemented as independent microfrontend applications that mount into the portal shell through the single-spa lifecycle, yet they share a common design philosophy: expose the full power of the underlying analytical engines while keeping the mechanics of execution invisible to the researcher.

13.2 Strategus Study Configuration

The analysis-ui application is a Vite-built React [34] app that gives researchers a visual canvas for composing Strategus analysis specifications. Rather than requiring users to hand-craft the JSON configuration that Strategus expects, the UI represents each analytical module as a node on a directed acyclic graph, rendered via the ReactFlow library. The researcher builds an analysis by adding nodes, configuring their parameters through form-based drawers, and connecting them with edges that define data flow relationships.

The Node-Based Composition Model

The system defines over twenty distinct node types, each corresponding to a concept in the OHDSI analytical framework. These include CohortGenerator for cohort construction, CohortMethod for comparative effectiveness studies, SelfControlledCaseSeries for within-person designs, TreatmentPatterns for medication pathway analysis, Characterization for baseline feature extraction, PatientLevelPrediction for machine learning models, and KaplanMeier for survival analysis. Each node type carries a default configuration with sensible starting values—for example, a CohortMethodAnalysis node initializes with a washout period of 183 days, first-exposure-only logic, and a Cox proportional hazards model.

Nodes communicate through a typed handle system. Each node declares its input and output handles with specific types—`HandleIOType.ModuleSpecification`, `HandleIOType.Cohort`, `HandleIOType.TargetComparatorOutcomes`, and so forth. The ReactFlow canvas enforces type compatibility: a user cannot connect a Cohort output to an input that expects CovariateSettings. This typed wiring prevents malformed analysis specifications at the UI level rather than discovering them at execution time. The handle type system also drives automatic node coloring, so researchers can visually distinguish module categories on the canvas.

The culminating node is the Strategus node itself, which accepts ModuleSpecification inputs from the various analytical modules. When the researcher connects CohortGenerator, CohortMethod, and Characterization nodes into a Strategus node, the UI understands that the final analysis specification should include all three modules. The canvas supports the full ReactFlow interaction model: drag-to-position, snap-to-grid alignment at ten-pixel intervals, zoom with constraints between 0.5x and 0.7x, a minimap for orientation in large analyses, and a cycle-detection algorithm that prevents circular dependencies.

Configuration Drawers

When a researcher clicks on a node, a side drawer slides open to reveal the module's configuration form. Each node type has a dedicated drawer component—CohortMethodDrawer, TreatmentPatternsDrawer, SelfControlledCaseSeriesDrawer—that renders the appropriate form fields for that module's parameters. The drawer reads the node's current data from the Re-

dux store, presents it through standard form controls from the shared component library, and dispatches updates back to the store on change.

The form data model mirrors the Strategus JSON specification closely. A TreatmentPatterns node, for instance, exposes fields for include-treatments strategy (start date or end date), index date offset, minimum era duration, era collapse size, combination window, and censoring parameters. The UI translates these form values directly into the nested JSON structure that Strategus expects, eliminating the need for a separate serialization layer.

State Management and Persistence

The application uses Redux Toolkit with RTK Query for both local state management and server communication. The Redux store tracks the current dataflow ID, the set of nodes and edges on the canvas, the selected revision, flow run state, and UI flags such as whether to upload results. RTK Query's `dataflowApiSlice` defines the full CRUD lifecycle against the backend's `jobplugins/analysisflow` endpoints: listing all dataflows, fetching the latest revision of a specific dataflow, saving (which creates a new revision), duplicating, and deleting both entire dataflows and individual revisions.

The revision model deserves attention. Every save creates a new revision rather than overwriting the previous state. The FlowRevisions component lets researchers browse the history of an analysis, compare configurations across revisions, and revert to earlier versions. This design treats analysis specifications as versioned artifacts, which is essential for reproducibility in clinical research.

Execution and Result Monitoring

When the researcher clicks the run button, the UI submits the analysis to the Prefect [55] orchestration engine via `prefect/analysis-run/{id}`. The submission includes the dataflow ID, the target dataset ID (injected from the portal's plugin metadata), and a flag indicating whether results should be uploaded to the Strategus results database.

After submission, a polling mechanism tracks execution progress. The `usePollingEffect` hook implements a recursive setTimeout pattern—not setInterval—with an eight-second default interval. This distinction matters: setTimeout-based polling waits for the previous fetch to complete before scheduling the next one, preventing request pileup when the server is slow. The hook polls the `prefect/flow-run/{id}/state` endpoint, watching for terminal states: COMPLETED, FAILED, CRASHED, or CANCELLED.

When the flow run reaches a terminal state, the ResultsPolling component fetches the per-node results from `analysisflow/{id}/flow-run-results`. The `processFlowRunResults` thunk matches each result to its node by name and updates the node's data with the result payload, the timestamp, and any error information. The researcher sees results appear directly on the canvas nodes, and can click any node to open the ResultsDrawer—a side panel that renders the full output or error message in a read-only Monaco editor, timestamped to the execution moment.

The UI also supports a test mode where the analysis specification is submitted directly as a JSON payload to `prefect/test-run` without requiring a saved dataflow, enabling rapid iteration during study design.

13.3 Notebook Integration

The notebook-ui application provides an in-browser notebook environment built on Starboard, a lightweight notebook runtime that executes code cells directly in the browser without requir-

ing a remote kernel. This architectural choice eliminates the infrastructure cost and startup latency of Jupyter kernels for many common analytical tasks.

The Starboard Runtime

The core of the notebook experience is the `StarboardEmbed` web component from the `@data2evidence/d2e-starboard-wrap` package. When a user opens a notebook, the application creates a new StarboardEmbed instance, configures it with the notebook content, a JWT [25] token for API authentication, the user's identity, and the active dataset ID, then mounts it into a dedicated DOM container. The embed loads its runtime from a separate `starboard-notebook-base/index.html` asset, which provides the cell execution engine, syntax highlighting, and output rendering.

Starboard notebooks use a plain-text format rather than Jupyter's JSON-based `.ipynb` format. However, the application supports importing Jupyter notebooks through a conversion pipeline: the `jupystar` library parses the `.ipynb` JSON, converts cell types and magic commands, and produces Starboard-compatible text content. This conversion handles v3 and v4 notebook formats and translates LaTeX content appropriately.

Notebook Lifecycle Management

Notebooks are persisted through the `system-portal/notebook` API. The StudyNotebook service class provides the full lifecycle: creating blank notebooks, creating from templates, saving with content and sharing flags, deleting, and listing all notebooks for a dataset. Each notebook is scoped to a dataset and a user, with an `isShared` flag that controls visibility to other researchers working on the same dataset.

The template system allows administrators to pre-configure notebook starting points. When a researcher creates a new notebook, a template dialog presents available templates fetched from the `/templates` endpoint. Templates carry a name, description, and pre-populated notebook content, so researchers can start with common analysis patterns rather than blank cells.

Git Synchronization

Notebooks support synchronization with a remote Git repository. The `SyncFromRemoteButton` component triggers a diff check against the remote version via `/{id}/remote-diff-check`. If differences exist, the researcher can overwrite the local notebook from the remote version. Administrators can also perform bulk synchronization across all notebooks through the Git Configuration setup page, which stores repository URLs, personal access tokens, and branch names for both dataflow and notebook repositories.

AI-Assisted Authoring

The notebook interface includes an AI chat assistant, implemented as a floating action button that opens a collapsible chat panel. Built on the NLUX library with streaming adapter support, the chat takes the current notebook content and dataset context as input, allowing the AI to generate code cells, explain results, or suggest analytical approaches aware of the notebook's current state.

13.4 Results Management

Analysis results flow through several presentation mechanisms depending on the analysis type. For Strategus analyses, the ManageViewerDialog in the SystemAdmin plugin provides a viewer management interface. The `useKernelViewer` hook tracks the lifecycle of result viewer

containers—starting, running, stopping, and failed states—against the Strategus Results API. Administrators can start a viewer for a specific dataset, which provisions a container running the appropriate visualization code (Shiny server, R, or Python), and stop it when no longer needed.

For the node-based analysis flow, results appear inline on the canvas. Each node displays its execution status and, upon completion, makes its output accessible through the ResultsDrawer. The drawer presents results in a Monaco editor configured for read-only viewing, with timestamps indicating when the output was generated. Error states are distinguished visually and in the drawer header, so researchers can quickly identify which modules in a complex analysis succeeded and which require attention.

Data Quality Dashboard [45] results follow a different path, flowing through the DQD plugin where researchers can view overview and detail tabs, examine job history, and trigger new quality assessments against selected datasets. The DQD component manages its own job polling lifecycle, fetching updated job states and presenting results through a tabbed interface with dataset selection.

13.5 Integration Points

The Strategus surface integrates with the Job Plugins service for analysis flow persistence (`jobplugins/analysisflow` for CRUD and revision history) and with Prefect (via Job Plugins) for execution: `prefect/analysis-run/{id}` submits a run, `prefect/flow-run/{id}/state` reports its progress, and `analysisflow/{id}/flow-run-results` returns per-node outputs once the run reaches a terminal state. The Strategus Results API governs the lifecycle of result-viewer containers used by administrators to present completed studies. The notebook surface integrates with Jupyter Enterprise Gateway for kernel sessions: the frontend performs a kernel-launch handshake against the gateway, and authentication flows through the cookie pattern described elsewhere in Part II — the portal writes a short-lived JWT cookie scoped to the gateway's path, and the iframe-hosted runtime reads it to establish its WebSocket connection. Notebook persistence is handled by the `system-portal/notebook` API, with optional Git synchronization against a configured remote repository. Both surfaces inherit the active dataset and the user's role through the portal shell's custom-props bridge, which means a notebook session sees exactly the datasets that its launching user is permitted to read — there is no separate dataset-access model for notebooks.

14 Administration Tools

14.1 Purpose

The administration tools are the surface for the platform's superusers — the people who provision datasets, assign roles, configure database connections, manage feature flags, and approve schema migrations. They exist in their own chapter because they serve a different audience than the data tools described earlier in Part II. A data steward curates concept sets and authors ETL pipelines; an administrator decides which users may exist, which datasets they may see, and which Git repository the platform synchronizes notebooks against. The interfaces look superficially similar — both are tables with dialogs — but the operations they expose carry platform-wide consequences that the data tools do not.

The chapter is also where the platform's setup model is described from the frontend's perspective: the boundary between "what the platform does" (system administration) and "what the platform is configured to be" (setup). Both sit behind the admin and user-admin role gates, and both are surfaced as portal plugins like any other UI module. The architectural idea worth

holding onto is that admin tools are not a special-cased part of the frontend — they are plugins that happen to be gated by privileged roles. New administrative capabilities are added the same way new researcher capabilities are added: as plugins.

The dialog-over-table pattern recurs throughout these tools. Every administrative listing is a table of platform entities — users, datasets, databases, feature flags — and every administrative operation is a dialog that opens over the listing. The dialog is the unit of permission gating: each dialog can be hidden or disabled based on the operator's role and the platform's identity-provider configuration. New operations land as new dialogs without disturbing the surrounding listings, which is what keeps the admin surface evolving without the listings themselves needing constant rework.

14.2 User Management

The user-management surface is a table of platform users with their assigned roles, fed by the User Management Service. The table distinguishes platform-level roles (operator-style accounts such as user admin or dashboard viewer) from data-administration roles (the privileged operators who manage running platform state) so that role-based filtering remains useful as the user list grows.

The role system is hierarchical and tenant-scoped: a user can hold different roles across different tenants — system admin in one, viewer in another, researcher in a third. Editing a user's role assignments is a single dialog that surfaces both data-admin and platform-level role choices, so administrators do not have to navigate away from the user list to make a change.

The interface adapts to the active identity provider. When an external enterprise IdP (Entra ID [35]) owns user provisioning, the local affordances — add user, edit, change password — are hidden, because the source of truth is external. When local identity management is active, the full lifecycle (add, delete, password change, role edit, activate, deactivate) is exposed.

A subtle reconciliation runs alongside the user list. The set of tenant identifiers referenced by users is compared against the set of tenants that actually exist; orphaned group memberships left behind by deleted tenants are cleaned up automatically. The architectural idea is that authorization data is not allowed to drift silently when tenants are removed.

14.3 Dataset and Study Management

The dataset and study surface is the most complex administrative listing in the platform. It is an expandable table of all datasets, each row revealing study-level details and an action menu that opens dialogs for the full dataset lifecycle: create, modify, set permissions, copy across schemas, delete, version a release, and apply schema migrations. The dialog-over-table pattern carries the weight here: the listing is one component, and every operation that mutates a dataset is its own dialog gated by the operator's role.

Dataset schema versioning follows a two-attribute model. Each dataset records its current schema version, and the system maintains a separate notion of the latest available schema. When the two diverge, the UI surfaces an update prompt, allowing administrators to apply schema migrations through a dedicated dialog. The system also tracks database dialect per dataset, which determines which SQL features and migration paths are available.

A separate dialog manages the per-dataset result viewer — a code editor where administrators write visualization code (typically Shiny, R, or Python) that a containerized runtime executes to present study results. The dialog supports multiple named viewer configurations per dataset, template selection for common visualization patterns, and attaching SQL queries to the viewer.

The architectural idea is that result viewers are configured per dataset and provisioned on demand: starting one spins up a container; stopping one tears it down.

Strategus [48]-format studies have their own set of dialogs covering analysis runs, external result uploads, artifact cleanup, and creation of studies pre-configured for the Strategus execution model. The pattern is the same as elsewhere in the chapter — one listing, several dialogs — but the operations are specific to the Strategus workflow.

14.4 CDM Configuration Editor

The CDM Configuration editor provides the interface for creating and managing Clinical Data Model configurations — the same configurations described in Chapter 22. Through this editor, administrators define the logical data model that the Query Engine operates on: which patient attributes exist, what clinical interactions (diagnoses, procedures, medications, lab results) are available, what database expressions map each attribute to its physical table and column, and what default filters distinguish different record types within the same table.

The editor follows the configuration lifecycle defined by the backend: configurations begin as drafts that can be edited iteratively without validation, then pass through a validation step that checks every expression against the live database, and finally are activated for use by the analytics frontend. The editor surfaces validation results inline, highlighting attributes whose SQL expressions fail to execute or whose data types do not match the declared type. This feedback loop allows administrators to build configurations incrementally, testing each attribute against real data before committing.

A key capability is the placeholder mapping editor, which lets administrators define how logical placeholders map to physical database tables. The editor provides auto-suggest functionality that scans the target database schema and proposes table and column mappings, which the administrator can accept or override. This is particularly useful when onboarding a new database whose OMOP schema uses non-standard naming conventions.

The editor also supports per-interaction overrides of the default placeholder mappings, enabling configurations that span multiple data model patterns — for example, standard OMOP tables for clinical data alongside custom tables for genomics or imaging data.

14.5 PA Configuration Editor

The PA Configuration editor manages Patient Analytics configurations — the presentation layer that sits on top of CDM configurations and controls how the analytics frontend displays and behaves. Every PA configuration depends on a CDM configuration, and the editor enforces this dependency by requiring administrators to select a CDM configuration before they can begin editing.

The editor provides form-based interfaces for each aspect of the PA configuration: filter card definitions (which interactions appear as filter cards, their display order, visibility, and initialization state), attribute settings within each filter card (category versus measure designation, reference text flags, default bin sizes, patient list column settings), chart options (which chart types are enabled, their download and display settings), and panel-level options (add-to-cohorts, domain value limits, cohort entry/exit tracking).

Before activation, the editor runs a validation pipeline that checks consistency against the dependent CDM configuration: every filter card source path must resolve to an existing interaction, initial attributes must be visible, at least one chart type must be enabled, and display orders must be unique. Validation results appear inline with detailed error descriptions, allowing administrators to fix issues before attempting activation.

The editor supports import and export of configurations as JSON files, enabling promotion workflows from development to production environments or sharing between institutions. When importing, the editor resolves CDW-enriched metadata dynamically against the target environment's CDM configuration, surfacing any incompatibilities for the administrator to address.

14.6 Data Quality Dashboard

The data-quality dashboard [45] is a multi-tab surface for monitoring and managing data-quality assessments. The administrator picks a dataset, then moves between four views: overview metrics, granular per-check results, run status, and statistical characterization of dataset contents. Quality assessment runs use the same job-execution model as analysis flows; the dashboard exposes a run-history table that supports both visible refreshes (with loading indicators) and silent background refreshes that do not disrupt the operator's current view.

14.7 Job Management

Job management takes a different architectural approach from the other administrative surfaces. Rather than rebuilding job monitoring inside the React [34] portal, the platform embeds the standalone jobs application — itself a Vue [74] micro-frontend — inside the portal shell. The embedded application loads its own asset manifest, mounts into a designated container, and unmounts cleanly when the operator navigates away.

The jobs application presents flow runs in a paginated, filterable list, with drill-downs into per-run task graphs, logs, and artifacts. Communication between the portal shell and the embedded application is minimal: a portal API object injected into the mount node provides a token-getter and a base URL, so the embedded app makes authenticated calls without managing its own authentication. The architectural idea is the same one Patient Analytics demonstrates — a self-contained tool can be hosted inside the portal as a micro-frontend without the portal having to know what it does internally.

14.8 Docker Log Viewer

Container logs are exposed through Dozzle, an open-source log viewer, embedded as an iframe inside the portal. The implementation is small but the authentication pattern is worth naming: before mounting the iframe, the portal writes a path-scoped JWT [25] cookie that the embedded application reads to authenticate its WebSocket connection. The architectural problem is general: iframed applications cannot reach the parent frame's JavaScript-managed tokens, but they can read cookies set on their path. Path-scoped cookie authentication is the seam that lets the platform embed third-party tools without forcing them to reimplement the platform's auth model.

14.9 Setup Configuration

The setup plugins configure the infrastructure that the platform itself depends on. They follow the same dialog-over-table pattern as the system-administration plugins, but the entities are infrastructure rather than running platform state.

Database connections are managed through a listing of registered databases (code, name, host, dialect) with separate dialogs for connection topology and authentication credentials — a deliberate split, because connection topology and secrets have different access patterns and audit requirements. Different dialects expose different subsets of the dialog options; cloud databases that authenticate via service-account keys, for example, hide the username/password edit affordance.

Git integration manages two independent repository configurations, one for analysis dataflows and one for notebooks, each capturing a repository URL, a personal access token, and a target branch. Beyond storing settings, the plugin exposes bulk-synchronization actions that pull the latest content from the configured repository and overwrite local copies, with confirmation dialogs that warn local changes will be lost.

Feature flags are a flat list of boolean toggles, each rendered as a checkbox. The list is deterministic: known features appear in a defined order, with any unrecognized flags (added by backend updates the frontend has not caught up with) sorted alphabetically at the end. The architectural idea is that the frontend tolerates new flags without code changes — it does not assume it knows the complete set.

The metadata plugin defines the attribute types and tag categories that can be attached to datasets, composing two CRUD listings: typed attributes (text, dropdown, date) and categorical tags. Demo-environment provisioning is a small page with one-click actions for sample databases and pre-configured phenotype definitions, useful for new deployments that need to populate the platform with example data for training.

14.10 Integration Points

The administration tools talk primarily to three backends. The User Management Service is consulted by every user-facing operation: it owns the user-and-role data model, the role and tenant edit dialogs write through it, and the orphaned-group reconciliation reads its tenant registry. The system Portal Service backs every dataset, study, attribute, tag, feature flag, database, and Git configuration that the setup pages and dataset surface manipulate; it is the persistence layer for all administrative metadata. The Logto [33] admin API is reached indirectly: when local identity management is active, account creation, password changes, and activation pass through Trex into Logto's management endpoints, while enterprise-IdP deployments hide those affordances entirely because user provisioning is owned externally. Beyond those three, job management embeds the Prefect [55] UI as a micro-frontend, log viewing authenticates Dozzle by writing a path-scoped JWT cookie before mounting its iframe, and the per-dataset result viewer reaches a results-API to provision and tear down viewer containers.

15 Shared Component Library

15.1 Purpose

Every other frontend module in Part II depends on this one. The buttons, tables, modals, drawers, snackbars, and design tokens that the shared library exports are what keep the visual language consistent across React and Vue surfaces, across plugins authored years apart, across applications that do not even share a build tool. The shared component library is why D2E feels like one product even though it is built from many.

The chapter exists in its own right because the alternative would scatter platform-wide design decisions across thirteen module-specific descriptions, each repeating the same ground. The component library and the companion `@portal/plugin` package together define both the visual idiom and the structural contract by which every plugin integrates with the portal shell — the typed metadata contracts, the icon system, the theme inheritance model, the SCSS-through-Rollup pipeline, and the implicit promise that any application importing `@portal/components` is binding itself to the platform's evolution. Treating these as a single chapter keeps the platform's design substrate in one place, where it can be referenced from the modules that consume it rather than re-explained alongside each one.

A platform composed of thirteen independently built micro-frontend applications faces an immediate design challenge: how to maintain visual consistency and interaction coherence when each app has its own build pipeline, its own dependency tree, and its own development cadence. The D2E frontend solves this through two shared libraries—`@portal/components` for visual building blocks and `@portal/plugin` for structural contracts—that together define the visual language and integration protocol of the platform.

15.2 The Component Library

The `@portal/components` package provides over thirty reusable React [34] components built on Material-UI v5. These are not thin wrappers; each component encodes platform-specific styling decisions, accessibility defaults, and API conventions that would otherwise drift across thirteen separate codebases. When a developer imports `Button` from `@portal/components` rather than from `@mui/material`, they get a button that already conforms to the platform's visual standards without consulting a style guide.

Component Categories

The library organizes its exports into several functional categories, each addressing a different layer of the interface.

Form controls provide the primary means of user input. The `Button` component supports text labels, loading states, and variant selection (contained, outlined, text). `IconButton` pairs an icon with click behavior for toolbar actions. `TextInput`, `TextField`, and `TextArea` handle text entry at different scales, while `InputLabel` provides accessible labeling. `Select` and `Autocomplete` offer choice selection with different interaction models—dropdown versus type-ahead. `Checkbox`, `Switch`, and `Slider` round out the input primitives for boolean, toggle, and range values respectively.

Layout components structure the page. `Box` provides the fundamental flex container with MUI's sx-prop styling. `Card` creates elevated content regions. `Drawer` slides panels in from the edge of the viewport, used extensively for node configuration in the analysis UI and detail views throughout the portal. `CollapsibleDrawer` adds expand/collapse behavior for panels that should remain accessible but not always visible. `Dialog` provides modal overlays for confirmations, forms, and alerts. `Tabs` and `Tab` organize content into switchable views, as seen in the DQD [45] plugin's multi-view interface.

Data display components present structured information. `TableRow`, `TableCell`, and `TablePaginationActions` extend MUI's table primitives with consistent styling and pagination behavior. The UserOverview, Database Configuration, and Study Overview plugins all build their primary views from these table components, ensuring that tabular data looks and behaves identically regardless of which plugin renders it. `List`, `ListItem`, and `ListItemButton` provide vertical list layouts. `Chip` renders compact labeled elements, used for displaying role badges in user management and tags in dataset views.

Feedback components communicate system state. `Snackbar` displays transient success, error, and informational messages with configurable auto-close timing—the pattern used by every plugin's `setFeedback` calls. `ErrorBoundary` catches rendering errors in child component trees and presents a fallback UI with error details through the companion `ErrorDetail` component, preventing a crash in one section from taking down the entire page. `Loader` renders a loading indicator for asynchronous operations. `Tooltip` wraps elements with contextual hover text.

Utility components serve specialized needs. `ZipFileUpload` handles file upload with ZIP extraction, used for bulk data import workflows. `FormControl` provides the MUI form control

wrapper with platform-standard styling. `Menu` and `MenuItem` build dropdown menus for action selectors, as seen in the database management interface where each row offers Details and Credentials actions through a Select-based menu.

The Icon System

The component library includes approximately sixty custom SVG icons, managed through an automated build pipeline. Source SVG files live in the `assets/` directory. During the build process, the SVGR tool (`@svgr/cli`) converts each SVG file into a typed React component with title-prop support for accessibility. A custom index template generates the barrel export file that maps each icon to a named export following the convention `IconNameIcon`—so `Edit.svg` becomes `EditIcon`, `Download.svg` becomes `DownloadIcon`, and so forth.

The prebuild step is destructive by design: it deletes the entire `src/components/Icons/` directory and regenerates it from the SVG assets. This ensures that the generated code is always in sync with the source artwork and prevents manual edits to generated files from persisting. Consuming applications import icons alongside other components:

```
import { EditIcon, TrashIcon, SearchIcon } from "@portal/components";
```

This approach provides several advantages over alternative icon strategies. Unlike icon fonts, SVG components can be tree-shaken—an application that uses only five icons does not bundle all sixty. Unlike inline SVG strings, the components carry TypeScript types and can accept standard React props. And unlike runtime SVG loading, the icons are compiled into the JavaScript bundle, eliminating network requests and rendering flicker.

15.3 Build and Distribution

The library uses Rollup to produce two output formats from a single source. The CommonJS build at `dist/cjs/index.js` serves applications that use `require()` syntax or older bundlers. The ES module build at `dist/esm/index.js` serves modern bundlers that can leverage static import analysis for tree-shaking. TypeScript declaration files are generated in a third Rollup pass using `rollup-plugin-dts`, producing a consolidated `dist/index.d.ts` that gives consuming applications full type information for every component and prop.

The Rollup configuration applies several transformations: `rollup-plugin-peer-deps-external` excludes React and MUI from the bundle (they are provided by the consuming application), `rollup-plugin-postcss` processes SCSS stylesheets into the bundle, and `rollup-plugin-terser` minifies the output for production. Icons receive a separate build entry point that produces only a CommonJS bundle, since the icon components have simpler dependencies and do not require the full plugin chain.

The package is published as an internal npm package under the `@portal` scope. Consuming applications declare it as a dependency in their `package.json`, and the monorepo's workspace resolution ensures they always build against the latest source. There is no separate version-bump-and-publish cycle; changes to the component library are immediately available to all applications in the next build.

15.4 The Plugin Contract

The `@portal/plugin` package defines the TypeScript interfaces that govern how feature plugins integrate with the portal shell. This is not a runtime framework—it is a compile-time contract that ensures type safety across the plugin boundary.

Plugin Types

Three plugin classes correspond to the three extension points in the portal:

`SystemAdminPagePlugin` wraps components that appear in the System Administration section. Its metadata interface provides the user ID, an async token getter for API authentication, and the system identifier. Plugins like UserOverview, StudyOverview, Jobs, DQD, and D2ELogs all export instances of this class.

`SetupPagePlugin` wraps components for the Setup section. Its metadata is simpler—just user ID and token getter—since setup pages operate at the platform level rather than within a specific system context. The Database, GitConfig, Feature, Metadata, and DemoSetup plugins use this type.

`ResearcherStudyPlugin` wraps components that appear within a study context. Its metadata is the richest: user ID, token getter, tenant ID, study ID, release ID, arbitrary data, a `fetchMenu` callback for registering navigation items, and a `subFeatureFlags` map for conditional feature rendering. This plugin type serves the analysis, notebook, concept mapping, and other study-scoped features.

Each plugin class extends `BasePlugin` and holds a single `page` property—a React component type parameterized by `PageProps<TMetadata>`. The `PageProps` interface simply wraps the metadata in an optional `metadata` property, providing a uniform prop contract across all plugin types. The portal shell reads the plugin's `page` property, renders it as a React component, and passes the appropriate metadata based on the current navigation context.

Plugin Registration

Each plugin directory contains a `module.ts` file that instantiates the appropriate plugin class:

```
import { SystemAdminPagePlugin } from "@portal/plugin";
import StudyOverview from "./StudyOverview";

export const plugin = new SystemAdminPagePlugin(StudyOverview);
```

This pattern makes plugin registration explicit and discoverable. The portal's routing system imports these module files and maps them to URL paths, so adding a new administration page requires creating the component, wrapping it in a plugin instance, and adding a route entry. The TypeScript compiler verifies that the component's props match the expected metadata interface, catching integration errors before runtime.

15.5 Theme and Styling

Visual consistency extends beyond shared components to a shared theme. The platform's MUI theme customization centers on a primary blue (`#2E74B5`) that matches the documentation's color scheme, creating visual continuity between the platform and its written materials.

Each micro-frontend application creates its own `ThemeProvider` at its root—the analysis-ui wraps its content in a ThemeProvider with a custom theme, the notebook-ui does the same with CssBaseline for consistent typography defaults. The shared component library does not embed a ThemeProvider; instead, it inherits the theme from whatever provider wraps it in the consuming application. This design means the same Button component can render with slightly different theme values in different applications if needed, though in practice all applications use the same theme constants.

Styling within components uses MUI's `sx` prop for one-off style adjustments and SCSS files for more complex layout rules. The component library processes SCSS through Rollup's PostCSS plugin, embedding the styles in the JavaScript bundle. Applications that need additional layout utilities can import Bootstrap 5 classes, which some portal plugins use for grid layouts and spacing alongside MUI's Box-based flex model. This dual approach—MUI for component-level styling, Bootstrap for page-level layout—reflects the platform's pragmatic evolution rather than a prescriptive architectural decision, and both systems coexist without conflict because they target different CSS specificity levels.

15.6 Integration Points

The shared component library is imported by every other frontend module in the workspace. The portal shell, the analysis-ui, the notebook-ui, the concept sets app, the wizards app, the jobs viewer wrapper, and every administrative plugin all declare `@portal/components` as a workspace dependency and consume its components and icons directly — there is no parallel design system, no second-source button, no plugin allowed to ship its own modal style. The plugin contract package `@portal/plugin` is similarly universal: `SystemAdminPagePlugin`, `SetupPagePlugin`, and `ResearcherStudyPlugin` are the only legitimate ways for a plugin to declare itself to the portal shell, so every chapter in Part II that mentions a plugin instance is implicitly importing from this package. The library's most subtle integration is the design-token bridge between React and Vue [74]: Patient Analytics, written in Vue 3, does not import `@portal/components` directly, but it inherits the same MUI theme constants (primary blue `#2E74B5`, typography defaults, spacing scale) through its own `ThemeProvider` configuration so that a button rendered in Vue is visually indistinguishable from a button rendered in React. That coordination is what makes the platform's polyglot frontend feel like a single product to its users.

PART III

Cloud Functions

Part III is the longest part of the book. It documents the cloud functions that implement D2E's business logic, organized by domain. Each function runs as a Deno [8] worker isolate inside Trex, with its own environment variables, memory budget, and CPU time limits, and each is wired into the system through the plugin manifest mechanism described in Chapter 4.

The functions are grouped into four families:

Query Engine. The five functions that turn a researcher's intent into a chart: the Analytics Service (gateway and orchestrator), the Query Generation Service (the SQL compiler), the CDW Service (clinical data model registry), the PA Configuration Service (UI configuration), and the Bookmark Service (saved-query persistence).

Data management. User Management (identity, roles, access), the Dataset Service (dataset lifecycle), and the Portal Service (the metadata hub that almost everything else reads from).

OHDSI integration. The D2E WebAPI (OHDSI cohort and concept endpoints), the Terminology Service (vocabulary search), and the OHDSI Tools collection (White Rabbit [50], Perseus [47], Strategus [48]).

Infrastructure. Job Plugins (the Prefect [55] orchestration gateway), Concept Mapping (source-to-OMOP vocabulary mapping), AI Services (LLM-backed code suggestion, MCP [3], AI ETL mapping), and the File and Export Services.

The chapters in this part follow a consistent shape: *why the function exists*, *how it is structured*, *the request lifecycle*, *the data and contracts it owns*, *how it integrates with neighbours*, and *how it fails and what to look at first*. Where a function is small and self-contained (the Bookmark Service, the File Export Service) the chapter is short. Where a function is intricate (the Query Generation Service with its four-stage pipeline, the Analytics Service with its multi-study request lifecycle) the chapter goes deep.

Readers who want to extend D2E with a new cloud function should read Chapter 4 first — it explains the plugin manifest — then read the chapter for the function whose role most closely matches what you are building.

16 System Overview

The Query Engine is a clinical analytics platform that allows users to query, filter, aggregate, and export healthcare data from clinical data warehouses. It is designed to work with data conforming to the OMOP Common Data Model, enabling standardized analysis across institutions and datasets.

At its core, the system translates high-level analytical requests - such as "show me the age distribution of patients diagnosed with diabetes who were also prescribed metformin" - into optimized SQL queries that run against the appropriate database backend. The results are then formatted and returned to the user as charts, tables, patient lists, or downloadable exports.

16.1 Supported Databases

The Query Engine supports multiple database backends, enabling organizations to use the system regardless of their data warehouse technology:

- SAP HANA - with support for session authentication and HANA-specific SQL constructs
- PostgreSQL [69] - with connection pooling and schema adaptation

- TrexSQL - the primary analytical database layer for all non-HANA deployments. Also serves as the execution dialect for Google BigQuery deployments via the TrexSQL caching layer

The multi-dialect capability is achieved through the placeholder system and database-specific SQL formatting, which are described in detail in later sections.

17 Architecture

17.1 Service Interaction Model

The services interact in a well-defined request lifecycle. The following diagram illustrates the typical flow of an analytical query from user request to data response:

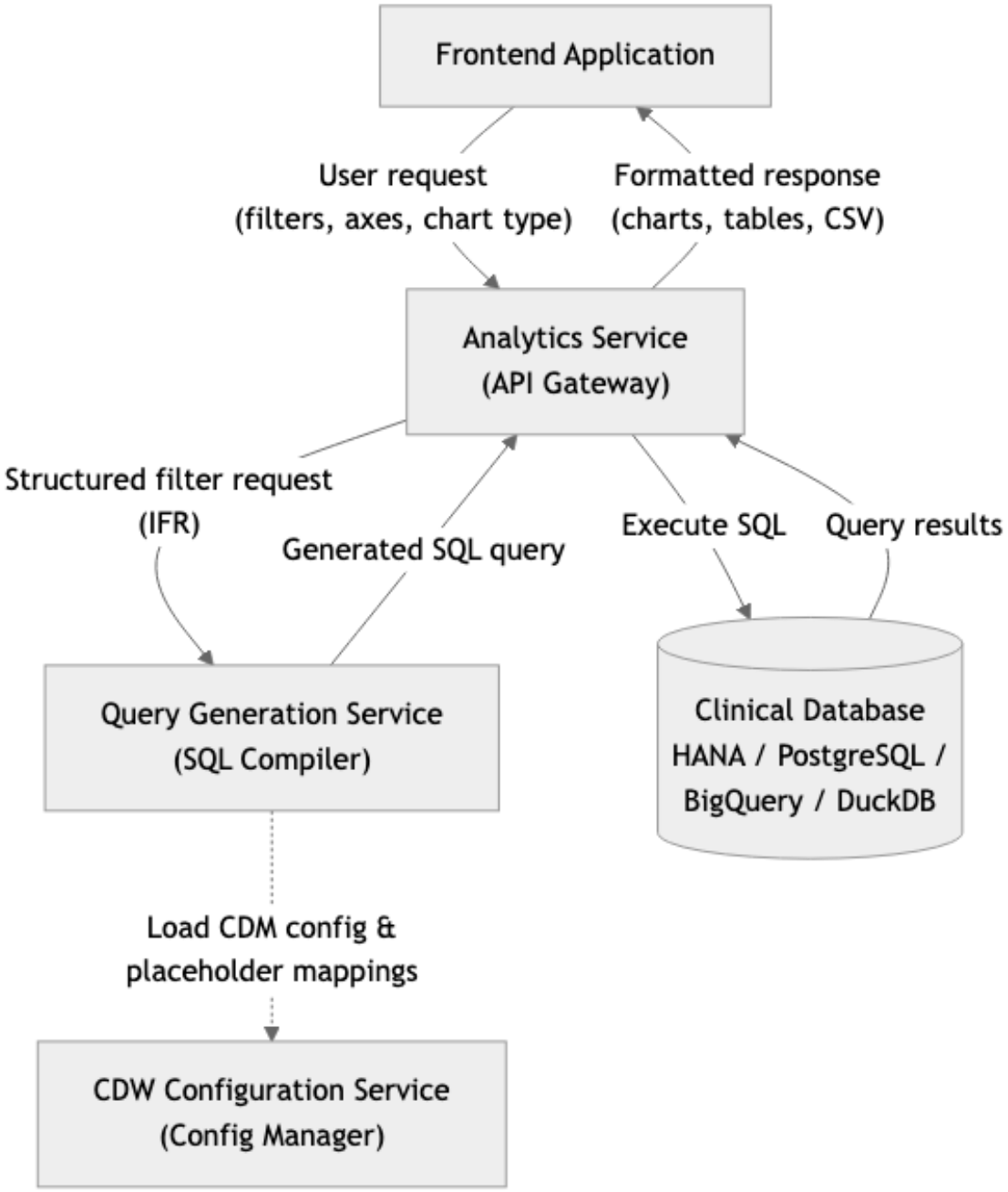

Figure 12: Service Interaction Model

17.2 Request Lifecycle

A typical analytical query follows the sequence shown below:

The key steps are:

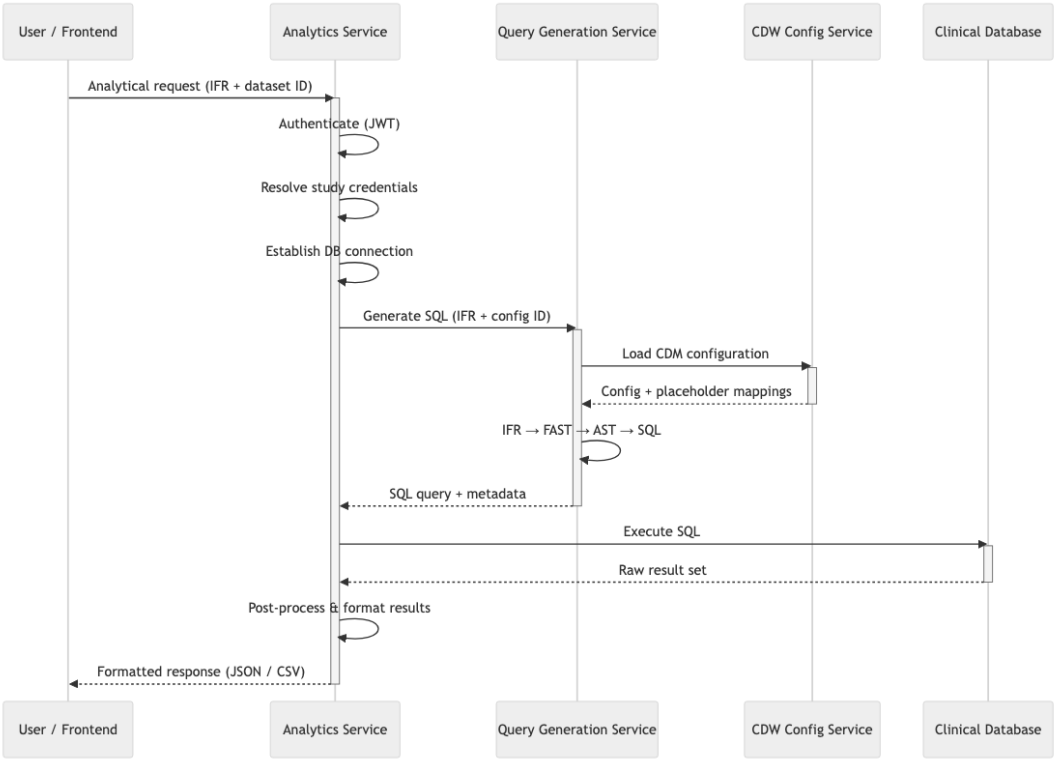

Figure 13: Request Lifecycle Sequence

1. The user constructs a query in the frontend application by selecting filters, grouping axes, and chart types.
2. The frontend sends the request to the Analytics Service, which acts as the API gateway.
3. The Analytics Service authenticates the user, resolves the appropriate database credentials for the requested study, and establishes a database connection.
4. The Analytics Service forwards the structured filter request to the Query Generation Service.
5. The Query Generation Service loads the relevant CDM configuration (which defines the data model mapping), transforms the filter request through its multi-stage pipeline, and produces an optimized SQL query.
6. The SQL query is returned to the Analytics Service, which executes it against the clinical database.
7. The Analytics Service processes the raw results (formatting, aggregation post-processing, CSV conversion if needed) and returns them to the frontend.

17.3 Microservice Communication

The services communicate via internal HTTP calls, with support for an optimized inter-process communication channel (Trex/Tokio) when services are co-located. Each request carries a correlation ID that propagates across all service calls, enabling end-to-end request tracing for debugging and observability.

All inter-service calls include the original user's authorization token, ensuring that access controls are enforced consistently throughout the request chain.

18 Analytics Service

18.1 Origins and Rationale

The Analytics Service is the platform's front door for every analytical operation a researcher initiates. It is the component the frontend talks to whenever the canvas changes — queries fire automatically on filter and axis edits, not from an explicit button press — the component that owns the request's database connection for its lifetime, the component that decides which study's credentials apply, and the component that turns SQL result rows into the chart-shaped JSON the frontend renders. Without it, the Query Engine has no way to be invoked.

The shape of the service is dictated by a single observation about analytical traffic: every request is for a *specific* dataset, and the credential, the connection, and the authorization are all coupled to that dataset. Authentication cannot be resolved in isolation, because the right answer depends on which study the user is asking about. Credential resolution cannot be performed without the authenticated user, because access is dataset-scoped. The database connection cannot be opened until the credential is decrypted, and it cannot outlive the request without leaking authority. These three concerns share a single key — the dataset identifier — and a single lifetime — one analytical request.

The alternative architecture, considered and rejected during early design, was three separate services: an authentication service that minted per-request session tickets, a credential-resolution service that exchanged tickets for decrypted database secrets, and a connection-pool service that handed out connections by credential hash. Each was conceptually clean. Together they required a three-way protocol in which all three services agreed on the same dataset identifier on every request, and any drift between them produced silent authorization failures. The Analytics Service collapses that protocol into a single function's middleware pipeline, where the dataset identifier is extracted once and consumed by all three concerns in sequence.

A second alternative was simpler still: let the frontend talk to the Query Generation Service directly, and have the frontend resolve credentials. This would have made the architecture flatter but it would have violated principle 2 of 2 — the credential boundary is the platform boundary — because the frontend is tenant code and the credentials are the platform's deepest secret. A frontend that knows database credentials is a frontend that can be compromised into exfiltrating them; the platform's threat model rules this out. The Analytics Service exists in part because somebody server-side has to hold credentials so the frontend never has to.

The cost of the choice is real. The Analytics Service is heavier than any other cloud function on the platform: it holds state for the duration of a request rather than acting as a pure transformer, it owns connection-pool slots that can be exhausted by long queries, and a single instance does the work that three separate services would split. The runtime's 30-minute worker timeout and the connection-pool backlog are both consequences of this concentration of responsibility. The benefit is that there is exactly one auditable path through which analytical traffic flows. Access logs, correlation IDs, and incident-response forensics all converge on the same code path; "who queried what when" is a question with a single answer rather than a join across three services. For a platform whose multi-tenant story rests on auditable per-dataset access, that single path is worth the weight.

18.2 Operations Surface

The service mediates a small but consequential set of operations: aggregation queries that produce bar charts and histograms; patient counting (sometimes across multiple studies in parallel); patient list retrieval with pagination; Kaplan-Meier survival analysis; box-plot statistics; data characterization reports; cohort creation, listing, comparison, and deletion; domain

value lookups for UI autocompletes; and concept lookups against the platform's terminology layer. Each is a different shape of question, but each lands at the service through the same authentication, credential resolution, and connection-management pipeline. The variety lives at the edges; the center is uniform.

Cohorts and Materialization

A cohort is a defined set of patients meeting specific criteria, persisted in the database for reuse. Cohort definitions are the unit of computable phenotyping in OHDSI-style observational research, where reusable, network-portable definitions are essential for results that travel between sites; the OHDSI network study by Hripcsak et al. [23], which characterized treatment pathways across 250 million patients in eleven data sources, is the canonical demonstration of why this matters. The cohort lifecycle is one of the more elaborate flows the Analytics Service manages.

When a user creates a cohort, the service forwards the user's filter criteria to the Query Generation Service, receives the SQL that identifies the matching patient identifiers, executes that SQL against the study's database, and writes the resulting patient IDs into a dedicated cohort table in the database's results schema. The cohort metadata — creator, name, definition reference, creation timestamp — is stored alongside, allowing the cohort to be listed, compared, or deleted later.

An alternative path exists for OHDSI-compatible cohorts. The service can convert IFR filter definitions into OHDSI cohort definitions and submit them to the Job Plugins Service for execution by an external Prefect flow. This path is used when researchers want their cohorts to round-trip with OHDSI's Atlas [44] tooling, and it is also what makes downstream cohort-quality assessment — for example, evaluating sensitivity, specificity, and positive predictive value of a phenotype algorithm with PheValuator [67] — possible without leaving the platform. The two paths produce semantically equivalent cohorts but materialize them through different mechanisms, and the metadata records which path was used.

Listing, filtering, and deletion follow the patterns expected of any persistent resource, with one twist: deleting a cohort that is referenced by a saved bookmark cascades a notification to the Bookmark Service so the bookmark's stale-cohort flag is set. Cohort comparison is implemented at the analytical layer rather than the storage layer — a comparison query simply applies the same axes to multiple cohorts in parallel and returns combined results.

Kaplan-Meier Survival Analysis

Kaplan-Meier [26] analysis is handled through an asynchronous processing model because survival computations can be resource-intensive:

1. The user specifies a target cohort, an outcome event of interest, and optionally a competing outcome and stratification cohorts.
2. The Analytics Service submits the analysis as a job to an external dataflow processing service, which returns a job run identifier.
3. The service then polls for results at regular intervals (every 10 seconds) until the job completes, with a maximum timeout of 20 minutes.
4. Once complete, the survival curve data is retrieved from the processing service's artifact store and returned to the user.

Data Characterization

Data characterization reports answer the "what is in this dataset" question that researchers ask before they ask anything else. The reports cover a high-level dashboard, density and demographic profiles, treemap visualizations of conditions, procedures, drugs, measurements, observations, and visits, and a mortality analysis. Each maps to one or more pre-built SQL queries that execute in parallel against the clinical database; their results are normalized into the JSON shapes the frontend's report views expect, with optional drill-down endpoints for per-concept breakdowns.

18.3 Export Surfaces

Every chart type the platform renders has a corresponding CSV export endpoint. The aggregation, box plot, Kaplan-Meier, and patient-list exports each follow the same pipeline: the request carries the same compressed query parameter as the chart-rendering call, the SQL is executed with CSV-specific formatting flags, and the resulting rows are streamed through a CSV writer that double-quotes every value, auto-generates headers, and prefixes formula-triggering characters to defend against spreadsheet injection. Filenames follow a predictable pattern (`PatientAnalytics_<chart-type>_<timestamp>.csv`) so downloaded files self-document.

Two specialized export paths handle cases the standard pipeline cannot. Parquet [68] snapshots, used for downstream analytical tools that want columnar data, are pre-generated by Prefect [55] flows and stored in Supabase Storage [66]; on request the Analytics Service streams them directly from object storage to the user's browser. Encrypted exports, used in recontact scenarios where re-identifiable data leaves the platform, package the export with ChaCha20-Poly1305 [40] authenticated encryption keyed by a per-recipient secret; only the intended recipient can decrypt.

18.4 Multi-Study Support

A critical capability of the Analytics Service is its support for multi-study operations. Organizations often have multiple clinical studies or datasets, each with its own database schema and access credentials.

Study Metadata Resolution

At the start of every request, the service contacts a central Portal Service to load metadata for all studies the user has access to. This metadata includes the database code, schema names (data schema, vocabulary schema, results schema), authentication mode, and configuration identifiers for each study.

Credential Isolation

Each study maps to a specific database via its database code. The middleware extracts the dataset identifier from the request and matches it to the correct study metadata. This isolation ensures:

- A request targeting study A uses study A's credentials and can only access study A's data.
- Credentials are never exposed to the frontend; they are resolved server-side.
- Each study can use a different database dialect independently.

Cross-Study Queries

For operations like multi-study patient counting, the service executes the same query against each requested study's database in parallel and aggregates the results.

18.5 Middleware Pipeline

Every request to the Analytics Service passes through a layered middleware pipeline that handles cross-cutting concerns:

Figure 14: Analytics Service Middleware Pipeline

Middleware Layer	Responsibility
Security Headers	Applies standard HTTP security headers (via Helmet) protecting against common web vulnerabilities.
Request Parsing	Parses JSON and URL-encoded request bodies with a 50MB size limit.
Cache Prevention	Sets Cache-Control, Expires, and Pragma headers to prevent cached responses.
Correlation Tracking	Generates a UUID v4 correlation ID for each request and propagates it across downstream calls.
Performance Monitoring	In development environments, measures and logs request execution duration.
Study Resolution	Contacts the Portal Service to load metadata for all studies the user can access.
Credential Resolution	Extracts the dataset identifier and resolves database credentials.
Connection Management	Establishes a database connection for the appropriate dialect and authentication mode.
Cleanup	Ensures database connections are returned to the pool after request completion.

18.6 Integration Points

The Analytics Service sits at the center of every analytical request, which means it talks to almost everything. It calls the Query Generation Service to compile filter definitions into SQL — via the internal IPC dispatch map, since both are co-located inside Trex. It calls the Portal Service at the start of every request to load the user's accessible studies and their metadata. It calls the Bookmark Service when cohort lifecycle events warrant cascade. For Kaplan-Meier and other compute-intensive analyses, it submits jobs to the Job Plugins Service, which routes them to Prefect.

In the other direction, the Analytics Service is called primarily by the Patient Analytics frontend, which addresses it directly under the `/analytics-svc` route prefix. Other internal services do not call it; the service is a leaf in the internal call graph, even though it is the root of the user-facing one.

18.7 Failure Modes

The Analytics Service has two characteristic failure modes. The first is credential resolution failure: the Portal Service is unavailable, returns stale data, or the user's study membership has just changed. The middleware pipeline catches this case and returns a 401, but a wave of these on a healthy deployment usually points to a Portal Service or Logto [33] issue rather than to the Analytics Service itself. The second is database-connection exhaustion: a slow query holds a connection, queued requests pile up, and the Trex worker hits its 30-minute timeout. This shows up as 503s on the route and as a backlog in the connection pool metrics.

The trade-off the service's shape reflects is that putting authentication, credential resolution, and connection management in one process makes the service heavy — a single instance is doing a lot — but it ensures that every analytical request goes through a single, auditable path. The alternative architecture, where each function manages its own credentials, was rejected during early design as unaffordably difficult to audit.

19 Data Flow Scenarios

This section walks through common scenarios to illustrate how the services work together.

19.1 Aggregation Query

Scenario: A researcher wants the age distribution of patients diagnosed with Type 2 Diabetes, grouped by gender.

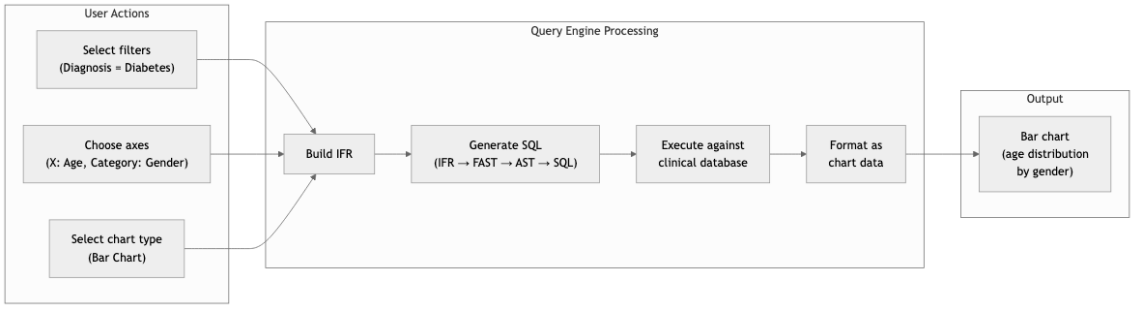

Figure 15: Aggregation Query Data Flow

1. The researcher selects filters, sets Age as X-axis, Gender as category, and chooses a bar chart.
2. The frontend constructs an IFR and sends it to the Analytics Service.
3. The Analytics Service authenticates, resolves credentials, and forwards the IFR to the Query Generation Service.
4. The Query Generation Service transforms IFR through its pipeline and returns an aggregation query.
5. The Analytics Service executes the SQL and returns formatted chart data.

19.2 Streaming and Parquet Notes

Two implementation details are worth naming. CSV streaming uses two writers depending on the connection type — a standard Node-pipeline writer for ordinary database connections and a custom Trex transform-stream writer for Trex's connection layer. Both produce identical output and the choice is invisible to the caller. Parquet exports are not generated at request time; they are snapshots produced by Prefect flows and stored in Supabase Storage, and the request path simply streams the bytes through with authentication enforced. For multi-entity patient-list exports, a temporary table holds the matched patient IDs while parallel per-entity queries run against it; the temporary table auto-disposes when the database session ends.

20 Query Generation Service

20.1 Purpose

The Query Generation Service exists because the rest of the platform should not know SQL. Frontend code constructs filter definitions in clinical-domain terms; the Analytics Service routes them; the database executes whatever it is given. Between those concerns sits a translation problem — turning a researcher's intent, expressed in OMOP-CDM concepts, into an optimised SQL statement that runs against whichever clinical database the study happens to use. The Query Generation Service owns that translation. Everything else in the chapter explains how.

The Query Generation Service is the intellectual core of the Query Engine. Its job is to take a structured representation of a user's analytical request and compile it into an optimized SQL query that can be executed against the target database. It achieves this through a sophisticated four-stage transformation pipeline.

20.2 Why a Four-Stage Pipeline

The translator did not begin life as four stages. The first version was a direct IFR-to-SQL renderer: a single traversal that walked the filter representation and emitted SQL strings as it went. It worked. It produced queries that ran correctly against one database dialect, and for a brief period the platform shipped against that dialect alone. The trouble arrived the moment a second backend had to be supported. The single-pass translator had tangled dialect-specific decisions — how to express `DAYS_BETWEEN`, when to wrap values in `UPPER()`, whether to use anonymous blocks or CTEs — with semantic decisions about cohort relationships, exclusion logic, and aggregation. Adding a new dialect did not mean adding a renderer; it meant rewriting the translator from scratch and hoping the semantic decisions came out the same way the second time. That approach was abandoned.

The pipeline that replaced it is structured as a compiler. IFR is the source language, the form the frontend produces. FAST is the semantic intermediate representation that emerges after analysis: filter cards are resolved, temporal relationships are normalized, axes are bound to measures and groupings, and what remains is a database-independent description of the query's meaning. AST is the abstract syntax tree where optimization passes run — alias management, scope resolution, default-filter injection, concept-hierarchy expansion — producing a tree that knows the shape of the SQL it will become but is still independent of any particular dialect. SQL is the target language, emitted by a per-dialect renderer that consumes the AST and produces the syntax the database actually accepts. The shape of the pipeline is the shape of every textbook compiler [1, 4]: source, semantic IR, AST, target. The Query Engine is a compiler, and structuring it as one is what made it possible to keep extending.

Each stage is independently testable. A FAST tree can be constructed and inspected without a database. An AST tree can be optimized and serialized without committing to a dialect. A renderer can be exercised against a fixed AST input and have its output diffed against a golden file. Optimization is centralized: a join-reordering pass [15, 62], a redundant-CTE eliminator, a constant-folding pass — all of these run once at the AST level and benefit every dialect that follows.

The cost is paid in cognitive load. There is more code to read, more concepts to learn, and more vocabulary to internalize before a developer can debug a query that produces wrong results. When a chart shows the wrong number, the bug could live in the IFR-to-FAST transformation, in the FAST-to-AST build, in an AST optimization pass, or in the dialect renderer; tracking it down means inspecting three intermediate forms rather than one. The platform pays this cost

willingly because dialect independence is non-negotiable for its multi-tenant story — one customer's HANA installation, another's PostgreSQL, a third's BigQuery, and the same analytical question must produce the same answer on all three. This is principle 6 of 2: dialect-specific decisions are delayed until Stage 4, and everything earlier in the pipeline is the same code regardless of which database the query will eventually hit.

20.3 The Four-Stage Pipeline

The transformation from user request to executable SQL occurs in four distinct stages, each with a clear purpose and well-defined input/output contract:

Stage 1: Internal Filter Representation (IFR)

The IFR is the structured request format that arrives from the frontend application. It captures the user's intent in terms of clinical concepts: which patient populations to include or exclude, what filters to apply, what to group by, and what to measure.

Filter Cards The fundamental building block of an IFR request is the filter card. Each filter card represents a clinical concept that the user wants to include in or exclude from their query. Each filter card contains:

- A configuration path identifying the clinical concept
- An instance number distinguishing multiple cards of the same type
- A list of attribute constraints that narrow down the records
- An optional successor relationship defining temporal ordering between events
- Flags indicating whether the card represents an exclusion, or marks a cohort entry or exit point

Filter Combination Logic Filter cards are organized into two combination modes:

- **Match All (AND logic).** All filter cards must be satisfied simultaneously.
- **Match Any (OR logic).** Filter cards are organized into alternative groups. When multiple Match Any groups exist, the system computes the cartesian product, creating a separate patient context for each combination combined via SQL UNION.

Formally, given Match All filters M and Match Any groups $G_1, G_2, \ldots, G_n$, the resulting patient contexts are:

$$\mathcal{C} = \{M \cup \{g_1, g_2, \ldots, g_n\} \mid g_i \in G_i \text{ for } i = 1, \ldots, n\} \tag{1}$$

Each context $c \in \mathcal{C}$ produces a separate SQL query, and the final result is $\bigcup_{c \in \mathcal{C}} Q(c)$ where $Q(c)$ is the query for context c.

Axes and Aggregation The IFR includes axis definitions that describe how results should be grouped and measured:

- **X-Axis (Categories).** Defines grouping dimensions with optional bin size for histogram buckets and sort order.
- **Y-Axis (Measures).** Defines what to calculate with aggregation functions (count, count distinct, average, minimum, maximum, or string aggregation).

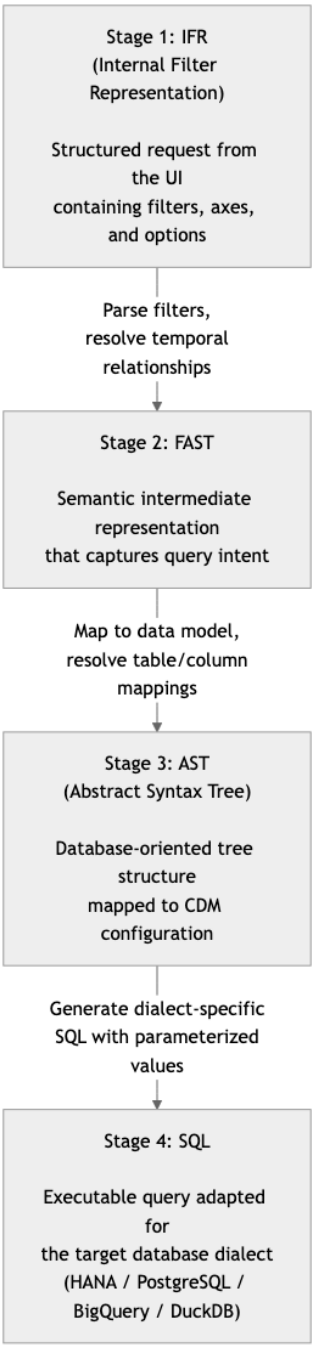

Figure 16: The Four-Stage Query Transformation Pipeline

When a bin size b is specified for a continuous attribute x, the system discretizes values into histogram buckets using:

$$\text{bin}(x) = \left\lfloor \frac{x}{b} \right\rfloor \cdot b \tag{2}$$

This groups all values within each interval $[kb,\ (k{+}1)b)$ into the same bucket, producing evenly spaced histogram bars.

Attribute Constraints Within each filter card, the system supports several constraint types:

- **Literal values.** Direct comparisons like diagnosis code equals E11.
- **Range values.** A minimum and maximum defining a bounded range.
- **Absolute time values.** Date-based constraints on event start and end times.
- **Duration between events.** Temporal distance constraints between clinical events.
- **Expression references.** Comparisons between two attributes rather than an attribute and a constant.
- **SQL functions.** Inline SQL function calls for specialized filtering.

Temporal Relationships The IFR supports two forms of temporal relationships:

- **Successor relationships.** Define minimum and/or maximum days between events, creating DAYS_BETWEEN calculations.
- **Advance time filters.** Complex temporal query expressions combining multiple time-based conditions with AND/OR logic.

Stage 2: FAST

The FAST is a semantic intermediate representation that captures the intent of the query in a database-independent way. The IFR-to-FAST transformation resolves filter cards, temporal relationships, and aggregation axes into a unified semantic tree.

The Transformation Process The IFR-to-FAST transformation proceeds in several phases:

1. **Match Any Expansion.** If the IFR contains multiple Match Any groups, the system computes the cartesian product. Each combination becomes a separate patient context.
2. **Patient Context Building.** For each combination, filter cards are grouped by interaction type, and each group becomes a set of filter nodes with their attribute constraints.
3. **Population Context Building.** Y-axis attributes become measures with aggregation functions, X-axis attributes become groupBy dimensions.
4. **Post-Processing.** Attributes referenced in axes but without corresponding filter cards get automatic LEFT JOIN filter nodes. Identical filter cards get additional join conditions to prevent duplicates.
5. **Entry/Exit Criteria.** If cohort entry/exit tracking is enabled, a special context captures start and end dates of defining clinical events.

Patient Context Structure Each patient context (ParserContainer) contains:

- **Filter section.** A dictionary mapping interaction types to their filter nodes with alias, template identifier, data type, attribute constraints, and relationship type.
- **Where section.** Patient-level attribute constraints (e.g., age or gender filters).
- **GroupBy section.** X-axis attributes defining how results are categorized.
- **Measure section.** Y-axis attributes with aggregation functions.
- **Having section.** Post-aggregation filters.
- **OrderBy section.** Sort specifications for the result set.

Definition Building Once the ParserContainers are built, they are transformed into a definition structure that directly maps to the AST. This transformation converts filter sections into relationship definitions (WITH, WITHOUT, or LEFTJOIN), performs topological sorting on relationships to ensure correct dependency order, detects circular dependencies, creates aggregate definitions for the population context, and creates Union definitions for multiple patient contexts.

Comparison with CQL ELM FAST was designed after the Expression Logical Model (ELM) from the HL7 Clinical Quality Language (CQL) specification [17]. It borrows several core structural concepts from ELM while adapting and extending them for population-level analytics and multi-database SQL generation.

High-Level Comparison At a high level, the two representations serve different audiences and goals:

Aspect	FAST	CQL ELM
Purpose	Intermediate representation for analytical queries (aggregations, patient lists, survival analysis) targeting clinical data warehouses.	Intermediate representation for clinical quality measures and clinical decision support logic targeting any CQL-compliant data source.
Scope	Focused on population-level analytics: grouping, aggregation, cohort filtering, and chart-oriented output.	General-purpose clinical logic: individual patient evaluation, quality measure calculation, and decision support rules.
Query model	Relational: patient contexts with interaction relationships (WITH/WITHOUT/LEFTJOIN), GROUP BY, measures, and HAVING clauses. Designed for SQL generation.	Functional: expression trees with retrieves, queries, and functions. Designed for abstract evaluation independent of any query language.
Data model coupling	Tightly coupled to the CDM configuration layer and OMOP placeholder system. Relationships reference specific interaction types and attributes.	Loosely coupled via data model-independent retrieves. Data types reference QDM, FHIR, or other models through model info declarations.

Aspect	FAST	CQL ELM
Aggregation	First-class concept: measures, groupBy, having, and orderBy are core components of the representation.	Not a core concern. Population aggregation is handled at a layer above ELM through measure evaluation logic.
Temporal logic	Successor relationships and duration-between constraints expressed as pairwise event comparisons.	Rich interval algebra with precise temporal operators (before, after, during, overlaps, starts, ends) based on the HL7 temporal specification.
Output	Produces parameterized, dialect-specific SQL (HANA, PostgreSQL, TrexSQL) via the AST stage. BigQuery is supported indirectly through the TrexSQL caching layer.	Produces an abstract expression tree that can be evaluated by any CQL engine; translation to SQL is optional and engine-specific.
Serialization	Internal JSON structure within the query pipeline; not a published standard.	Standardized XML or JSON serialization defined by the HL7 CQL specification.

The following sections provide a detailed feature-by-feature comparison identifying exactly which concepts FAST shares with ELM and where the two diverge.

Shared Concepts FAST adopts the following concepts directly from ELM:

Feature	How FAST uses it	ELM origin
Retrieve	DataModelTypeExpression with type set to Retrieve. Represents fetching records of a clinical data type (e.g., Condition, Procedure). FAST adds a templateId binding for OMOP entity resolution.	ELM Retrieve element fetches records of a given type, optionally filtered by a code path. FAST mirrors the concept but binds to CDM templates instead of FHIR/QDM profiles.
With relationship	Relationship with type With. Requires the patient to have at least one matching interaction. Contains a suchThat clause for join conditions.	ELM With clause in a Query. Nearly identical semantics: a source must have related records satisfying a suchThat condition.
Without relationship	Relationship with type Without. Excludes patients who have matching interactions. suchThat conditions define what constitutes a match.	ELM Without clause. Same exclusion semantics. FAST preserves the suchThat pattern from ELM.
suchThat clause	SuchThat conditions within With/Without relationships define the filtering logic for the join (e.g., temporal constraints, attribute comparisons).	ELM suchThat is an expression within With/Without that defines the relationship condition. FAST mirrors this directly.
Where clause	Where conditions filter records based on attribute constraints, using the same comparison operators as ELM.	ELM where clause filters query results. Same role and semantics.
Query structure	Query contains source, relationships, where, groupBy, measure, having, and orderBy. The source-relationship-where pattern follows ELM.	ELM Query contains source, let, relationship (with/without), where, return, aggregate, and sort. FAST follows the same structural pattern.
ExpressionRef	BaseOperandExpressionRef references a named definition by name, enabling composition of multiple queries.	ELM ExpressionRef references a named expression. FAST uses the same concept for cross-referencing CTEs.
Literal	Stores a typed value (string, integer, date).	ELM Literal element. Same concept.

Feature	How FAST uses it	ELM origin
Property	References an attribute by path and scope alias.	ELM Property expression. FAST mirrors this.
DurationBetween	Computes duration between two dates at day precision.	ELM DurationBetween with configurable precision; FAST is fixed at days.
IsNull / IsNotNull	Triggered by the special value NoValue. Generates null-check operators.	ELM IsNull and IsNotNull operators. Same semantics, different triggering mechanism.
Union	Combines multiple patient contexts into a single result set when Match Any logic produces multiple independent queries.	ELM Union expression combines two list expressions. Same set operation semantics.
Logical operators	And and Or operators combine conditions. Multiple filters on the same attribute use Or; cross-attribute filters use And.	ELM And and Or expressions. Same boolean logic.
Comparison operators	Equal, Less, Greater, LessOrEqual, GreaterOrEqual, NotEqual.	ELM uses the same operator names. FAST adopted ELM's naming convention directly.

Features where FAST diverges from ELM While FAST borrows ELM's structural vocabulary, it extends and modifies several areas to serve its analytical focus:

Feature	FAST approach	ELM approach
LeftJoin relationship	FAST adds a third relationship type (LeftJoin) that produces a SQL LEFT JOIN, allowing patients without matching records to appear with NULL attribute values. This is essential for optional data inclusion in analytics.	ELM does not define a LeftJoin relationship. With and Without are the only relationship types. Optional data inclusion is handled through expression-level null checking.
Aggregation model	First-class concept with separate Measure (what to aggregate: count, avg, min, max, countDistinct, string_agg) and GroupBy (how to group, with optional binsize for histograms) components. HAVING clause for post-aggregation filtering.	Aggregation is not a core ELM concern. ELM has an AggregateClause but population-level aggregation is handled by the measure evaluation layer above ELM, not within the expression model itself.
Histogram binning	GroupBy attributes support a binsize parameter that discretizes continuous values into evenly spaced buckets. This is a visualization-specific feature with no ELM equivalent.	No equivalent. ELM does not address visualization concerns.
Topological sorting	Relationships are topologically sorted to ensure dependency order. Without relationships declare dependencies on other relationships and are processed after them. Circular dependencies are detected and rejected.	ELM does not enforce or require ordering of With/Without clauses. Evaluation order is implicit.

Feature	FAST approach	ELM approach
Patient vs. population contexts	Explicit separation: patient contexts (ParserContainers) define individual-level queries, and a population context wraps them with aggregation. This two-level structure is built into the representation.	ELM does not distinguish patient from population contexts within the expression model. Context handling (Patient vs. Population) is defined at the library level, not within queries.
Entry/exit criteria	Special PatientRequestEntryExit context captures cohort entry and exit dates from designated clinical events, enabling temporal cohort analysis.	ELM does not have a built-in entry/exit concept. Cohort entry/exit logic is expressed through standard CQL expressions and measure timing definitions.
Type system	Minimal: tracks expression type (Property, Literal, Retrieve) and value type (String, Integer, SQLFunction) for semantic clarity. No type inference or validation.	Comprehensive: every expression has a resultType, with full type inference, validation, and a rich set of clinical types (DateTime, CodeableConcept, Quantity, etc.).
Temporal operators	Limited to DurationBetween (day precision only) and successor relationships (min/max days between events). Complex temporal queries use nested AND/OR structures.	Rich interval algebra: before, after, during, overlaps, starts, ends, meets, includes, with configurable precision from milliseconds to years. Full HL7 temporal specification support.
Set operations	Only Union is implemented, created implicitly when multiple patient contexts exist.	Full set: Union, Intersect, Except. All are explicit, composable binary operations.
Conditional logic	Not supported. No Case, If, or conditional expressions.	Full support: Case, If/Then/Else, Coalesce for null handling.
Output target	Designed specifically for SQL generation. The next stage (AST) produces parameterized, dialect-specific SQL for HANA, PostgreSQL, and TrexSQL. BigQuery is supported via the TrexSQL caching layer.	Database-agnostic. ELM produces an abstract expression tree evaluated by a CQL engine. SQL translation is optional and engine-specific.
Data model coupling	Tightly coupled to the CDM configuration and OMOP placeholder system. Retrieve references a templateId that maps to specific OMOP tables.	Loosely coupled. Retrieve references data types from pluggable model info (QDM, FHIR, etc.) without binding to physical storage.
Serialization	Internal JSON structure within the query pipeline. Not a published standard.	Standardized XML or JSON format defined by the HL7 CQL specification. Designed for interoperability.

Feature support summary

Feature	FAST	ELM
Retrieve (data access)	Yes (with templateId)	Yes (with code filter)
With relationship	Yes	Yes
Without relationship	Yes	Yes
LeftJoin relationship	Yes	No
suchThat conditions	Yes	Yes
Where filtering	Yes	Yes
GroupBy	Yes (with binsize)	Yes
Aggregation functions	Yes (count, avg, min, max, countDistinct, string_agg)	Yes (Count, Sum, Min, Max, Avg)
HAVING clause	Yes	No
OrderBy / Sort	Yes	Yes
Union	Yes (implicit)	Yes (explicit)
Intersect	No	Yes
Except	No	Yes
DurationBetween	Yes (day precision only)	Yes (configurable precision)
Temporal interval operators	No	Yes (before, after, during, overlaps, starts, ends)
Null handling	Basic (IsNull/IsNotNull via NoValue)	Advanced (IsNull, IsNotNull, Coalesce, three-valued logic)
Conditional logic (Case/If)	No	Yes
Type system	Minimal	Comprehensive with inference
Function definitions	No	Yes
Standardized format	No (internal)	Yes (HL7 specification)
SQL generation target	Yes (multi-dialect)	No (engine-agnostic)
Histogram binning	Yes	No
Entry/exit cohort criteria	Yes	No
Patient/population context split	Yes (explicit)	No (library-level)
Topological dependency sorting	Yes	No

In summary, FAST adopted ELM's query structure (Retrieve, With/Without, suchThat, Where, Property, Literal, ExpressionRef) as its foundation, then extended it with SQL-oriented features (LeftJoin, HAVING, binning, topological sorting) and analytics-specific concepts (Measure/GroupBy separation, patient/population contexts, entry/exit criteria) while deliberately omitting ELM's general-purpose features (rich temporal algebra, comprehensive type system, conditional logic, standardized serialization) that are not needed for its focused use case of generating analytical SQL against OMOP data warehouses.

Stage 3: Abstract Syntax Tree (AST)

The AST is a database-oriented tree structure that maps the semantic FAST representation onto the actual data model. It is the most architecturally rich stage of the pipeline, handling scope resolution, configuration lookups, and SQL generation in a single multi-phase traversal.

Relationship to Classical ASTs The name "AST" is borrowed from compiler design, where an Abstract Syntax Tree represents the structure of parsed source code. The Query Engine's AST follows a similar idea: it is an intermediate tree structure that gets traversed to produce output — in this case SQL rather than machine code. A typical compiler pipeline follows the pattern:

$$\text{Source Code} \xrightarrow{\text{Lexer}} \text{Tokens} \xrightarrow{\text{Parser}} \text{AST} \xrightarrow{\text{Code Gen}} \text{Target Code}$$

The Query Engine's pipeline mirrors this shape:

$$\text{IFR} \xrightarrow{\text{Parse}} \text{FAST} \xrightarrow{\text{Build}} \text{AST} \xrightarrow{\text{Traverse}} \text{SQL}$$

Several patterns from compiler design appear in the pipeline: a visitor pattern for tree traversal, alias-to-entity maps that serve as symbol tables, parent-chain walking for scope resolution, and multi-target code generation for emitting dialect-specific SQL (HANA, PostgreSQL, TrexSQL). While the Query Engine is not a general-purpose compiler, its architecture draws on the same principles to solve a similar problem — transforming a structured input through intermediate representations into executable output.

Tree Structure and Node Types The AST is composed of specialized node types, each responsible for generating a specific part of the final SQL:

Node Type	Role in SQL Generation
Statement	The root node. Orchestrates the overall SQL structure, manages the list of CTEs, and determines the output format.
Def (Definition)	Represents a single named CTE. Wraps a Query or Aggregation and tracks table alias-to-entity mappings.
Query	Represents a complete SELECT statement with all clauses (SELECT, FROM, JOIN, WHERE, GROUP BY, HAVING, ORDER BY).
Source	Represents the base table in the FROM clause. Manages additional table JOINs needed by attributes.
With (Relationship)	Handles three relationship types: WITH (inner join), WITHOUT (exclusion), and LEFTJOIN (optional).
Property	Represents an attribute/column reference. Resolves attribute configuration, applies default filters, handles date sentinels and concept hierarchies.
Operator	Generates SQL for comparisons, AND/OR logic, DAYS_BETWEEN, CONTAINS text search, UNION, and FULL OUTER JOIN.
Literal	Handles parameterization, date parsing, and case-insensitive string wrapping.
Aggregation	Implements four aggregation strategies: default, total patient count, Kaplan-Meier survival, and genomics.
ExpressionRef	A reference to another named definition (CTE name or inlined subquery).
Retrieve	Loads entity configuration for a clinical data type and registers database tables.

Three-Phase Traversal The AST uses a visitor pattern with a three-phase traversal:

- **Phase 1: Scope and Entity Resolution.** Each node registers its table aliases and entity mappings with its parent Definition.

- **Phase 2: Attribute Configuration.** Property nodes load their AttributeConfig, determine additional table JOINs, and apply default filters.
- **Phase 3: SQL Generation.** Each node generates its SQL fragment. Fragments are assembled bottom-up into the final output.

Table Alias Management A single analytical query often joins the same physical table multiple times. For example, a query asking for patients with both a primary diagnosis and a secondary diagnosis must join the condition_occurrence table twice, once for each diagnosis type. Without careful alias management, SQL column references would be ambiguous. The AST implements a multi-level aliasing system to handle this.

Aliases are generated and tracked at three levels:

1. **Source aliases.** The Source node (representing the FROM clause) assigns aliases to the base patient table and any patient-level attribute tables. The alias is derived from the table name: for example, the patient table might receive an alias like p1. Additional tables needed by patient-level attributes (such as @OBS for observations) are registered as join elements with their own aliases.
2. **Relationship aliases.** Each Relationship node (With/Without/LeftJoin) assigns aliases for its interaction's base table and any related tables. These aliases are namespaced under the relationship to avoid collisions. For example, if two diagnosis relationships exist in the same query, the first might alias the condition table as cond1 and the second as cond2. Within each relationship, attribute tables (like @CODE for concept codes) are further aliased relative to their parent interaction: cond1_code1, cond2_code1.
3. **Property-triggered aliases.** When a Property node resolves its attribute configuration, it may discover that the attribute requires a table not yet registered with the owning entity. For example, an attribute expression referencing @TEXT (for concept descriptions) triggers registration of the text table with the relationship that owns the attribute. The entity assigns a new alias and records the join condition (typically via INTERACTION_ID).

All aliases are tracked at the Definition (CTE) level in an alias-to-entity map. This map serves as the equivalent of a symbol table in a compiler: any node within the definition can look up which entity (Source or Relationship) owns a given alias, enabling Property nodes to resolve their attribute expressions to the correct table.

The aliasing system also handles a subtle complication: when the same physical table appears with different default filters, the system appends a hash of the filter to the table identifier before aliasing. This means that condition_occurrence filtered for primary diagnoses and condition_occurrence filtered for secondary diagnoses are treated as distinct tables with distinct aliases, even though they reference the same physical table. The hash-based disambiguation ensures correct SQL even in complex queries with many overlapping interactions.

Formally, the alias assignment can be described as a function:

$$\text{alias}(t, f, i) = \text{name}(t) \circ \text{hash}(f) \circ i \tag{3}$$

where t is the table placeholder, f is the default filter (empty if none), i is the occurrence index within the query, and $\circ$ denotes concatenation. This guarantees uniqueness: two references to the same table with different filters produce different aliases, and two references with the same filter but in different relationships produce different occurrence indices.

Relationship Join Logic The three relationship types produce different SQL patterns:

- **WITH (Inner Join).** The patient must have at least one record. Default filters are added to the ON clause.
- **WITHOUT (Exclusion).** Conditions are added to the WHERE clause as exclusion logic.
- **LEFTJOIN (Optional).** Patients without matching records appear with NULL values.

Default Filter Application Default filters distinguish between record types stored in the same table (e.g., primary vs. secondary diagnoses). They work at the interaction level (restricting rows by type) and the attribute level (additional restrictions for specific attributes). Placeholder references are replaced with actual table aliases before being added to join conditions.

Multi-Table Attribute Resolution Some attributes require data from multiple tables. The Property node identifies all required tables, registers them with aliases, and builds the final SQL expression by replacing placeholder references in the expression template.

Date Sentinel Values To prevent NULL comparisons from producing incorrect results, the AST applies sentinel values:

$$t^*_{\text{start}} = \text{IFNULL}\big(t_{\text{start}},\ 0001\text{-}01\text{-}01\big) \tag{4}$$

$$t^*_{\text{end}} = \text{IFNULL}\big(t_{\text{end}},\ 9999\text{-}12\text{-}31\big) \tag{5}$$

This ensures that $t^*_{\text{start}} \leq t$ for any real date t, and $t^*_{\text{end}} \geq t$ for any real date t, so temporal range comparisons produce correct results even when dates are missing.

Concept Hierarchy Navigation The OMOP vocabulary includes hierarchical concept relationships. The AST supports querying by ancestor concepts and automatically including all descendants through a multi-table join pattern involving the Concept, Concept Relationship, and Concept Ancestor tables.

Stage 4: SQL Generation

The final stage produces executable SQL. The generator supports multiple output formats:

- **Common Table Expressions (CTE)** – The default format using WITH clauses.
- **Anonymous Blocks** – Used with SAP HANA for procedural SQL (DO BEGIN...END).
- **Nested Subqueries** – Referenced definitions inlined as subqueries.
- **Combined Count** – Combines results from multiple count operations using FULL OUTER JOINs.

Aggregation Strategies The Aggregation node implements four distinct strategies:

- **Default Aggregation.** Standard SQL aggregation with GROUP BY, measures, and optional HAVING/ORDER BY.
- **Total Patient Count.** Uses a SQL window function (COUNT OVER PARTITION BY) for grouped patient counts.
- **Kaplan-Meier Survival.** Computes survival days, censoring status, event indicators, and start/end dates with a censoring threshold.
- **Genomics Values.** Extracts cohort and sample identifiers for genomics analysis pipelines.

Total Patient Count uses a window function to compute grouped counts without collapsing the result set:

$$\text{gr\_cnt}(g) = |\{p \in P \mid \text{group}(p) = g\}| = \text{COUNT}(1) \text{ OVER } (\text{PARTITION BY } g) \tag{6}$$

where P is the patient set and g represents the grouping key (X-axis value).

Kaplan-Meier Survival computes the following columns for each patient p:

$$\text{SurvivalDays}(p) = \text{DAYS\_BETWEEN}\big(t_{\text{start}}(p),\ t^*_{\text{end}}(p)\big) \tag{7}$$

$$\text{Censored}(p) = \begin{cases} 1 & \text{if } t_{\text{end}}(p) \text{ is NULL (event not observed)} \\ 0 & \text{otherwise} \end{cases} \tag{8}$$

$$t^*_{\text{end}}(p) = \begin{cases} t_{\text{end}}(p) & \text{if } t_{\text{end}}(p) \neq \text{NULL} \\ t_{\max} & \text{otherwise (last observed date)} \end{cases} \tag{9}$$

where $t_{\text{start}}(p)$ is the cohort entry date, $t_{\text{end}}(p)$ is the outcome event date (or NULL if censored), and $t_{\max}$ is the maximum observed date across all patients. Results are filtered by the censoring threshold τ: only groups where $\text{gr\_cnt}(g) \geq \tau$ are included, protecting patient privacy in small cohorts.

Parameterization All user-supplied values are parameterized using UUID-based placeholders to prevent SQL injection. The system supports typed parameterization for strings, dates, floating-point numbers, and integers.

20.4 Supported Query Types

Query Type	Purpose	Output
Aggregation Query	Group patients by dimensions and aggregate	Bar charts and histograms
Box Plot	Calculate statistical distribution	Quartiles, median, min, max, outliers
Kaplan-Meier	Compute survival curves with censoring	Time-to-event data
Patient Detail	Retrieve individual patient records	Tabular data for lists and CSV
Total Patient Count	Count distinct patients	Count value, optionally grouped
Plugin Query	Entity-specific queries	Flexible result sets

20.5 Handling Filters and Temporal Relationships

The service supports a rich set of filter operations:

- **Comparison Operators** – Greater than, less than, equal to, not equal to for numeric and date attributes.
- **Text Matching** – Contains and fuzzy matching with configurable fuzziness thresholds.
- **Null Handling** – IS NULL and IS NOT NULL checks with date sentinel values.
- **Set Operations** – UNION for match-any contexts, FULL OUTER JOIN for combining independent result sets.

- **Temporal Relationships** – Duration-based constraints using database-specific date arithmetic.

For fuzzy text matching, the system uses a similarity threshold $\alpha \in [0, 1]$ (default 0.7). A match is accepted when:

$$\text{similarity}(s_{\text{query}}, s_{\text{value}}) \geq \alpha \tag{10}$$

where the similarity function measures how closely the search term s_{query} matches the attribute value s_{value}. Higher values of α require closer matches, while lower values allow more approximate results.

For temporal relationships, the system computes the number of days between two clinical events and compares against user-specified bounds:

$$d_{\min} \leq \text{DAYS\_BETWEEN}(t_A, t_B) \leq d_{\max} \tag{11}$$

where t_A and t_B are the dates of events A and B, and $d_{\min}$, $d_{\max}$ are the user-specified minimum and maximum day constraints.

20.6 Multi-Dialect SQL Generation

The same user request produces correct SQL regardless of the target database:

- **SAP HANA** – Uses DAYS_BETWEEN for date arithmetic, supports anonymous blocks, UPPER() for case-insensitive comparisons.
- **PostgreSQL** [69] – Standard SQL date arithmetic, connection pooling, schema name adaptation.
- **TrexSQL** – The primary analytical database layer for all non-HANA deployments. Also serves as the execution dialect for Google BigQuery deployments via the TrexSQL caching layer.

21 Query Performance and In-Memory Optimization

21.1 Purpose

This chapter elaborates on the optimization passes referenced in Chapter 20. It is a deep dive into the AST-to-SQL stage [39], where the platform earns most of its query performance, and is intended for readers who want to understand why a given SQL pattern was chosen for the dialect they are seeing.

The earlier chapter described the four-stage pipeline at the level needed to follow a query end-to-end — enough to know where SQL is produced and how placeholders are resolved, but not why specific shapes are emitted. The material here is the next layer down: how columnar storage, vectorized execution, and dialect-specific physical layouts shape what the generator chooses to emit, and how those choices interact with the engines underneath. Readers chasing a performance question, debugging a slow query plan, or deciding how to lay out a new analytical table should find the reasoning here; readers following the platform's normal request flow can skip this chapter without losing the thread.

21.2 The Performance Challenge

Interactive clinical analytics imposes a demanding performance requirement: every query must return results fast enough that the researcher perceives the response as instantaneous. The Patient Analytics UI sends a new query on every filter change, every axis selection, and every chart type switch. If each query takes several seconds, the exploratory workflow — where a researcher iteratively refines filters to narrow a patient population — becomes frustratingly slow and cognitively disruptive. The target is sub-second response times for aggregation queries over datasets that routinely contain millions of clinical records.

This is a fundamentally different workload from transactional database operations. A transactional query retrieves a single patient's record by primary key — a microsecond operation on any database. An analytical query groups millions of records by a clinical dimension and computes aggregate statistics — a workload that must scan, filter, and aggregate across an entire table. The challenge is bridging the gap between these two worlds: providing the flexibility of ad-hoc analytical queries with the responsiveness of point lookups.

Traditional row-oriented databases store each record as a contiguous block of fields. When an analytical query needs only two columns out of fifty — say, a diagnosis code and a patient identifier for a frequency count — the database must still read all fifty columns from disk or memory for every row. For wide clinical tables with dozens of attributes, this means reading forty-eight columns of irrelevant data for every two columns of useful data. The waste is proportional to the table width, and clinical tables are wide.

The Query Engine addresses this through two complementary strategies: in-memory columnar storage that eliminates the irrelevant-data problem, and query-aware table design that aligns data layout with the access patterns the pipeline produces.

21.3 In-Memory Columnar Storage

Both SAP HANA and TrexSQL use columnar storage, where values from the same column are stored contiguously in memory rather than interleaved with values from other columns. This seemingly simple change in data layout has profound consequences for analytical query performance.

When a columnar engine executes an aggregation query that groups by one column and counts another, it reads exactly those two columns from memory — contiguous arrays of values that

fit neatly into CPU cache lines. The other columns in the table are never touched. For a table with fifty columns, this reduces memory bandwidth consumption by a factor of twenty-five compared to row storage. Since memory bandwidth is the primary bottleneck for analytical scans, this factor translates directly to query speed.

Columnar storage also enables compression techniques that exploit the statistical properties of individual columns. Dictionary encoding replaces repeated string values with small integer codes — a column of diagnosis codes that contains a few thousand unique values across millions of rows can be stored as a compact integer array with a small dictionary, reducing memory consumption by an order of magnitude. Run-length encoding compresses consecutive runs of identical values, which is common in sorted or clustered columns. Clinical data has naturally high redundancy: the same diagnosis codes, drug codes, visit types, and concept identifiers repeat across millions of patient records, making columnar compression exceptionally effective.

The compressed representation has a secondary benefit beyond space savings: many analytical operations can execute directly on compressed data. Filtering a dictionary-encoded column compares integer codes rather than decompressing strings, and aggregating a run-length-encoded column can skip entire runs of identical values. This means compression does not just reduce memory footprint — it accelerates computation.

Both engines also employ vectorized execution, processing data in batches of thousands of values at a time rather than one row at a time. Each operation — filter, compare, aggregate, join — runs over an entire vector of values in a tight loop that keeps data within CPU cache lines and can leverage SIMD (Single Instruction, Multiple Data) processor instructions. The per-row overhead of the traditional iterator model — function calls, virtual dispatch, branch prediction misses — is amortized across the entire vector, reducing it to negligible levels.

21.4 SAP HANA Engine Architecture

SAP HANA's architecture provides multiple execution engines, each optimized for different access patterns. Understanding which engine handles which part of a query is key to achieving the interactive performance that clinical analytics demands.

The column store is the foundation. It stores data column-by-column with dictionary encoding, advanced compression, and a delta merge architecture that separates recently inserted records (in a fast write-optimized delta store) from the bulk of historical data (in a highly compressed, read-optimized main store). The delta store absorbs inserts at transactional speed; a background merge process periodically compresses delta records into the main store. This dual-store design means that analytical queries scan the compressed main store at full speed while concurrent write operations (cohort materialization, data loading) proceed without blocking reads.

The HEX engine (HANA Execution Engine) is the query execution layer for column store data. It implements vectorized processing across CPU cores, with late materialization as a key optimization: during query execution, intermediate results are kept in their compressed columnar form for as long as possible. Full result rows are assembled only at the final projection step, after filters and joins have eliminated most of the data. This deferred materialization reduces memory bandwidth and keeps intermediate data in compact, cache-friendly representations.

D2E relies on the HEX engine as the execution path for its analytical queries against the column store. The other HANA engines (OLAP, ESX) are not used by D2E.

21.5 Choosing the Right Optimization Strategy

In HANA deployments, all D2E clinical data resides in the column store and is executed through the HEX engine. Performance comes from keeping data in the column store's compressed representation, exploiting late materialization, and aligning table layout and partitioning with the query pipeline's access patterns.

Vocabulary and reference tables — large, rarely updated, queried through both point lookups and hierarchical traversals — benefit from inverted indexes on high-cardinality columns. These indexes complement the implicit dictionary-based indexing that the column store provides, accelerating the concept hierarchy navigation and vocabulary lookups that the Terminology Service and Query Generation Service perform on every query.

Tables that serve dual roles — read-heavy for analytical access but written to during cohort materialization — benefit from the delta merge architecture. New cohort members are written to the fast delta store, allowing materialization to complete in seconds rather than minutes. Concurrent analytical queries read primarily from the compressed main store, unaffected by the ongoing writes. The background merge process compresses delta records into the main store during idle periods.

Partitioning strategy has a direct impact on scan performance. Partitioning clinical tables by time ranges enables partition pruning: queries restricted to a date window (which is common in clinical research, where studies focus on specific observation periods) skip entire partitions that fall outside the window. For datasets spanning many years, this can reduce the scan volume by an order of magnitude.

Getting these optimization choices wrong has measurable consequences. The same aggregation query can vary from sub-second to tens of seconds depending on whether the OLAP engine path is triggered, whether indexes exist for the join columns, and whether partitioning aligns with the query's filter predicates. The difference is not incremental — it is the difference between interactive analytics and batch processing.

21.6 TrexSQL Optimization

TrexSQL is an embedded columnar analytical engine designed for exactly the workload pattern the Query Engine produces. Its architecture combines several techniques that collectively deliver interactive performance for analytical queries.

Vectorized execution processes data in fixed-size vectors of approximately two thousand values at a time. Each operation — filter evaluation, aggregation, join probing — runs over the entire vector in a single pass, keeping data within CPU L1 cache and enabling SIMD instruction-level parallelism. This eliminates the per-row function call overhead and branch prediction penalties of traditional row-at-a-time execution engines.

Morsel-driven parallelism distributes work across CPU cores at a fine granularity. Data is divided into morsels — chunks that are processed independently by available threads. Unlike partition-based parallelism (where each thread is assigned a fixed data partition and fast threads sit idle waiting for slow ones), morsel-driven scheduling adapts dynamically: threads that finish their morsel pick up the next available one. Blocking operators like aggregation and sorting use a three-phase pattern — thread-local accumulation, completion signaling, and a single finalization step — that avoids centralized synchronization bottlenecks.

The execution model is push-based rather than pull-based. In the traditional Volcano model, each operator pulls data from its child by calling a "get next" method — a pattern that involves deep call stacks and frequent context switches. TrexSQL instead pushes data chunks through the operator pipeline as they become available, reducing scheduling overhead and

enabling streaming execution where intermediate results are processed incrementally without full materialization.

Columnar compression in TrexSQL is automatic and per-segment. The engine independently selects the optimal compression algorithm for each column segment based on the data distribution: dictionary encoding for columns with repeated values, bitpacking for narrow integers, run-length encoding for sorted or clustered data, and specialized algorithms for string and floating-point data. Because compression is lightweight, many operations execute directly on the compressed representation — a filter on a dictionary-encoded column evaluates against integer codes, and an aggregation on a run-length-encoded column can process entire runs as single operations.

Zone maps provide automatic range-based pruning. Every column chunk carries min-max statistics that the engine consults before reading data. When a query filter specifies a value range, the engine skips chunks whose min-max range does not overlap — without reading or decompressing the data. This is particularly effective for date-filtered clinical queries, where large portions of the dataset fall outside the query's time window and can be eliminated before any scan work begins.

TrexSQL works primarily in memory but handles datasets larger than available memory through adaptive spilling. Intermediate results that exceed the memory limit are partially written to temporary storage, with as much data as possible remaining in memory. Column chunks are loaded on demand, and only the columns actually referenced by the query are read — a property that interacts well with the Query Engine's tendency to produce queries that reference a small subset of a table's columns.

Full-text search indices on vocabulary tables enable fast concept name searches with English stemming and stopword filtering. These indices are created during cache provisioning and support the pattern-matching and fuzzy-search queries that power the terminology browser.

Selective column materialization during cache creation excludes columns not needed for analytical queries — large text fields, binary attachments, audit metadata — reducing cache size and improving scan throughput by ensuring that every byte read contributes to query results.

21.7 Query Structure Optimization

The Query Engine's four-stage pipeline produces SQL that aligns naturally with in-memory columnar execution.

Common Table Expressions structure each query as a pipeline of named intermediate results. Columnar engines execute these as a sequence of vectorized operations, where the output of one CTE feeds directly into the next without materializing to disk. The CTE structure also enables the engine's optimizer to identify opportunities for predicate pushdown, pushing filter conditions as early as possible in the pipeline to reduce the data volume before expensive join and aggregation operations.

The placeholder system enables query-aware table design without coupling the query generator to physical storage decisions. By mapping logical placeholders to physical tables, administrators can point the same analytical queries at table layouts optimized for the target engine — column-store tables with appropriate partitioning and indexing on HANA, or compressed columnar cache files on TrexSQL — without any change to the query generation logic.

Parameterized queries with typed placeholders enable prepared statement caching. The database compiles the query plan once — determining the join order, choosing indexes, selecting execution strategies — and reuses that plan across subsequent executions with different filter values. For interactive analytics, where the same query structure is executed repeatedly

as the researcher adjusts filters, prepared statement caching eliminates the compilation overhead that would otherwise add tens of milliseconds to every interaction.

21.8 The Interplay Between Storage and Query Design

The Query Engine's performance is not a property of the storage engine alone or the query generator alone. It emerges from the alignment between how queries are structured and how data is stored.

The placeholder system is the mechanism that enables this alignment. By abstracting physical table references behind logical placeholders, it allows the same query logic to run against table layouts optimized for different engines. A HANA deployment can use partitioned column-store tables with inverted indexes on join columns. A TrexSQL deployment can use compressed columnar cache files with zone maps. The query generator produces the same SQL in both cases; the performance comes from the storage layer's ability to execute that SQL efficiently.

The CDW configuration's default filters serve as a form of implicit partitioning. By filtering interaction tables by type — selecting only records of a specific clinical category — the query effectively accesses a logical subset of a physical table. Both HANA and TrexSQL can evaluate these predicates on compressed column data, skipping records that do not match without decompressing irrelevant columns. This means that the logical data model's categorization of clinical events (which exists for semantic clarity) also serves as a performance optimization (by reducing the scan scope of every query).

This co-design — where the query structure, the data model, and the storage engine reinforce each other — is what makes interactive analytical queries over millions of clinical records practical. No single layer provides the performance alone. The query generator produces efficient SQL. The placeholder system maps it to optimized physical tables. The columnar engine executes it with vectorized, parallel, cache-friendly processing. Together, they achieve sub-second response times that make exploratory clinical research feel instantaneous.

22 CDW Configuration Service

22.1 Origins and Rationale

Every analytical query needs to know what the data looks like: which tables hold which clinical events, how a "diagnosis" becomes a row, which placeholder names refer to which physical columns. The CDW Service owns that knowledge. It is the platform's registry of clinical data models — the configurations that say "in this dataset, a condition occurrence lives in `schema.condition_occurrence` and its concept identifier is in column `condition_concept_id`." Without this registry, the Query Generation Service has no way to translate IFR's clinical terms into SQL that hits real tables.

The CDW Service is also the editor for these configurations. Administrators use it to define new clinical data models, validate them against live databases, and publish them for use by analytical queries. It is therefore both a runtime metadata service and an authoring tool, with separate APIs for each role. The CDW (Clinical Data Warehouse) Configuration Service manages the Clinical Data Model configurations that tell the rest of the system how to interpret the data in each clinical database.

The shape of the service reflects a question the platform had to answer early: where do CDM configurations live, and who is allowed to change them? The obvious answer for a software-engineering audience is config-as-code. Configurations would be checked into a Git repository, reviewed in pull requests, deployed alongside the platform, and the audit trail would be the commit log. This was the first approach considered, and it was rejected. The people who maintain CDM configurations — clinical informaticists who know which OMOP table the local extraction populates and how a custom condition flag should map to a default filter — are not, in general, software engineers. Asking them to clone a repository, edit YAML, open a pull request, and wait for a deploy is friction the platform could not absorb. The cadence of clinical-data work is wrong for it: new data sources arrive, mappings need adjustment, a column changes name, and the people who notice are not the people with merge rights.

The decision was to put configurations in PostgreSQL [69] and make them mutable through the service's API. Versioning is tracked — every save produces a new revision, activations are timestamped, and previous versions remain queryable as inactive — but configurations can be updated without a deploy. A clinical informaticist edits a mapping in the UI, runs validation against the live database, sees that all attribute expressions parse and return the right types, and activates the new version. The platform never restarted; the next analytical query picks up the new configuration through the service's read API.

The cost of this decision is environmental drift. A configuration active in staging is not necessarily active in production, and a deploy does not bring the change. Operators have to use the platform's diff tools to compare staging and production configurations explicitly; "did this go out yet?" is not answered by "has the deploy completed?" but by "has somebody activated it in production?" This is a real operational burden, and the platform addresses it with diff endpoints, activation audit logs, and explicit promotion workflows rather than pretending the cost does not exist.

The benefit is that the people who understand the clinical data are the same people who change the mappings. A new data source arrives on Monday; the informaticist authors a configuration on Tuesday; validation runs against the live database and surfaces three attribute expressions that don't resolve; the informaticist fixes them and activates by Wednesday. A misconfiguration discovered after activation can be rolled back from the UI by reactivating the previous version — no engineering ticket, no hotfix branch, no deploy. The platform's ability to absorb new clinical data without an engineering cycle is, in practice, what makes multi-tenant

deployment tractable; each tenant's data has its own quirks, and forcing those quirks through a release process would have made onboarding measured in weeks instead of days.

This is an instance of principle 4 of 2: declarative over imperative. The platform stores declarations rather than code, and the runtime acts on them. CDM configurations are declarations of what clinical data looks like, in the same family as plugin manifests, route declarations, and Prefect [55] flow deployments. The runtime reads them and produces the running system. The CDW Service is the place where that declaration is honored for clinical data.

22.2 Clinical Data Model Configuration

A CDM configuration describes the structure of the patient data:

- **Patient Attributes** – Demographic and identifying attributes (age, gender, dates).
- **Conditions** – Logical groupings of clinical interactions.
- **Interactions** – Core clinical events (diagnoses, procedures, drug exposures, measurements, observations) with their database column expressions.

22.3 Configuration Lifecycle

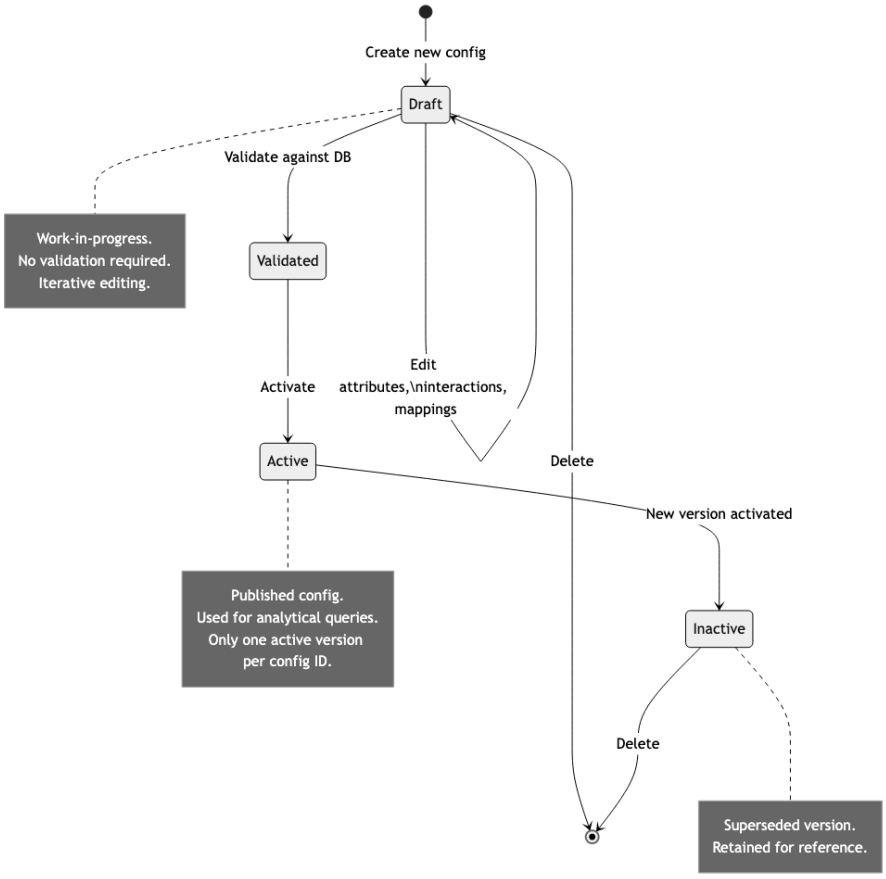

Figure 17: CDM Configuration Lifecycle

Status	Description
Draft	Work-in-progress configuration for editing. Can be saved without validation.
Active	Validated and published. Only one active version per configuration. Activating a new version marks the previous as inactive.
Inactive	Previously active, superseded by a newer version. Retained for reference.

22.4 Validation Pipeline

Structural Validation

Checks that the configuration conforms to the expected schema:

- Type constraints: fields must match declared types; attribute names must be alphanumeric with underscores.
- Mandatory fields: each attribute must define either an expression or measureExpression.
- Language validation: two-letter ISO codes with exactly one default entry.
- Uniqueness: all elements within a collection must have unique keys.
- Expression syntax: well-formed placeholder usage; system schemas prohibited.

Database Validation

Tests every configured attribute against the live database:

- Constructs and executes SQL queries using each attribute's expression and default filter.
- Checks returned data types match expected types (numeric for num, parseable dates for time).
- Reference expressions validated separately against vocabulary schemas.

22.5 The Placeholder System

The placeholder system provides abstraction between clinical concepts and physical database tables, enabling database-agnostic queries.

How Placeholders Work

Instead of referencing tables directly, the CDM configuration uses logical placeholders resolved to actual table names at query time:

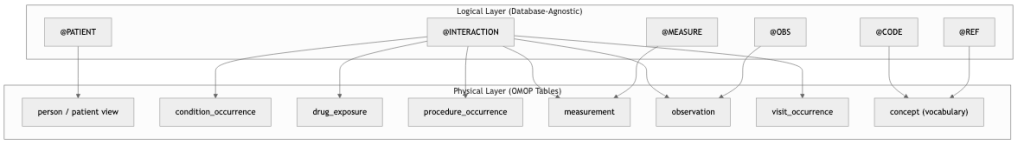

Figure 18: Placeholder-to-Table Mapping

Placeholder	Represents	Typical OMOP Mapping
@PATIENT	Patient dimension table	GDM Patient View
@INTERACTION	Clinical event tables	Condition, Procedure, Drug, etc.
@CODE	Coded attributes	Concept-related columns
@MEASURE	Numeric measurements	Measurement values
@OBS	Patient-level observations	Observation table
@TEXT	Text descriptions	Concept name lookups
@REF	Vocabulary reference	OMOP Concept table
@PDATA	Patient demographics	Person table attributes
@RELATED	Related records	Related entity references

OMOP Standard Placeholders

Pre-configured mappings cover all standard OMOP clinical tables: Condition Occurrence, Drug Exposure, Procedure Occurrence, Measurement, Observation, Visit Occurrence, Device Exposure, Specimen, Death, Condition/Drug/Dose Era, Observation Period, and Payer Plan Period.

Database-Agnostic Queries

The placeholder system combined with multi-dialect SQL generation means the same analytical query runs across SAP HANA, PostgreSQL, or Trex without modification. Google BigQuery deployments are supported through the TrexSQL caching layer.

22.6 Metadata Services

Attribute Information Service

Returns statistical metadata (COUNT, MIN, MAX) about an attribute's values. Operates in data mode (clinical tables) or reference mode (vocabulary tables). Performs type validation and applies access-control-based response filtering.

Domain Values Service

Returns distinct values for autocomplete dropdowns, with configurable suggestion limits, reference text resolution, and personalized placeholder mappings.

An audit threshold τ_{audit} prevents rare-value exposure. For each distinct value v of an attribute, the service computes its frequency $f(v)$ across the dataset, and only values meeting the threshold are returned:

$$V_{\text{visible}} = \{v \mid f(v) \geq \tau_{\text{audit}}\} \tag{12}$$

This prevents values associated with very few patients from appearing in dropdowns, protecting against indirect patient identification.

Table and Column Suggestion Services

Assist administrators building configurations by listing available columns (adapting queries for HANA, PostgreSQL, and TrexSQL) and suggesting distinct values for expressions.

22.7 Authorization and Access Control

Configuration-Level Privileges

Privilege	Allowed Actions
View (get)	Retrieve configurations assigned to the current user.
View as Admin	Retrieve any configuration with full detail.
Save	Create new configurations or save drafts.
Validate	Run the validation pipeline without saving.
Activate	Validate and publish a configuration.
Delete	Remove draft or inactive versions.
Suggest	Auto-generate configuration from database schema.
View All	List all configurations (admin only).

Data-Level Privileges

Privileged users see full attribute metadata; non-privileged users see only a permission flag, preventing indirect patient identification through metadata queries.

22.8 Configuration Auto-Suggest

Scans the database schema, identifies common OMOP patterns, and generates a template configuration that administrators can refine.

22.9 Advanced Settings

Setting	Default	Purpose
Fuzziness	0.7	Text search fuzziness threshold (0 to 1).
Max Result Size	5000	Maximum distinct values in suggestions.
Date Format	YYYY-MM-dd	Display format for dates.
Time Format	HH:mm:ss	Display format for times.
Enable Free Text	Enabled	Whether free-text search is available.
Variable Binding	Enabled	Whether variable binding is used in SQL.
SQL Return	Disabled	Debug: return generated SQL alongside results.
Error Details	Disabled	Debug: include error details in responses.
Stack Traces	Disabled	Debug: include stack traces in responses.

22.10 Integration Points

The CDW Service is read by the Query Generation Service on every analytical request — the CDM configuration is what allows IFR's clinical references to resolve to physical SQL. It is also read by the Patient Analytics frontend during configuration browsing, by the Dataset Service during dataset provisioning, and by the AI Services during AI-assisted ETL mapping. It does not call other services in the analytical hot path; its dependencies are the database (for storing configurations) and the Portal Service (for authentication and authorization).

22.11 Failure Modes

A CDW Service outage halts new analytical queries within seconds, since every query path begins by loading the relevant CDM configuration. Cached configurations in long-lived workers can mask a partial outage briefly, but new dataset access and any IFR referencing an unloaded configuration will fail. Validation failures during configuration authoring are the more common day-to-day failure mode: a placeholder that does not resolve to any column, a join condition that produces a Cartesian product, an SQL function not supported by the target dialect.

The validation step runs every defined expression against the live database, so misconfiguration surfaces during authoring rather than at query time — a deliberate design choice that pays off whenever a configuration would otherwise have shipped broken.

23 Logical and Physical Data Model

A defining architectural characteristic of the Query Engine is its strict separation between the logical data model (what clinical concepts exist and how they relate) and the physical data model (which database tables and columns store the actual data). This separation exists primarily to support different clinical data model layouts: the same analytical query logic can operate over an OMOP Common Data Model, a basic Entity-Attribute-Value (EAV) schema, a custom star schema, or any other table topology, without changing the query generation pipeline.

This is distinct from multi-database support (HANA, PostgreSQL, TrexSQL), which is handled separately by the SQL dialect generation in the AST stage. Google BigQuery is supported indirectly through the TrexSQL caching layer, which executes queries against a local cache of the BigQuery data. The logical-to-physical mapping solves a different problem: translating between data model structures, not between database engines.

23.1 The Problem: Diverse Clinical Data Layouts

Healthcare organizations store clinical data in widely varying schemas. Even within the OMOP standard, implementations differ in table naming, column naming, and view structures. Beyond OMOP, many organizations use:

- **OMOP CDM.** A standardized model with dedicated tables for each clinical domain: condition_occurrence for diagnoses, drug_exposure for medications, procedure_occurrence for procedures, each with domain-specific columns.
- **Entity-Attribute-Value (EAV) schemas.** A generic model where all clinical events are stored in a single observations table with type, key, and value columns. A diagnosis and a lab result occupy the same table, distinguished only by a type column.
- **Custom star schemas.** Organization-specific layouts with arbitrary table names, column names, and join structures that may not follow any published standard.
- **GDM (Generic Data Model) views.** A normalized view layer that sits between raw tables and the query engine, providing a consistent interface regardless of the underlying schema version or custom extensions.

The Query Engine must generate correct SQL for all of these layouts from the same user-facing filter interface. The CDM configuration and placeholder system provide this translation layer.

23.2 The Two-Layer Architecture

The mapping system operates in two layers:

- **Logical Layer (CDM Configuration).** Describes clinical data in business terms: patient attributes, clinical interactions (diagnoses, procedures, medications), attribute expressions, and filter conditions. This layer uses abstract placeholders (@PATIENT, @INTERACTION, @CODE) instead of real table names, making it independent of any specific physical schema.
- **Physical Layer (Placeholder Table Map).** Maps each logical placeholder to actual database tables and columns. Different placeholder maps can be provided for different data model layouts: one map for OMOP, another for an EAV schema, another for a custom star schema. The same logical configuration can be paired with any compatible map.

This dual abstraction means that switching from an OMOP schema to an EAV schema does not require changing the query generation pipeline or the analytical definitions. Only the placeholder table map needs to be updated to point placeholders at the new tables and columns.

Similarly, within a single configuration, individual interactions can override the default mapping via the from property, allowing a mix of data model patterns (e.g., standard OMOP tables for clinical data alongside a custom genomics table for variant data).

23.3 The Logical Data Model

Hierarchical Structure

The logical data model follows a hierarchical structure rooted at the patient level:

- **Patient.** The root entity representing a single person in the clinical dataset. Patient-level attributes include demographics such as age, gender, date of birth, and date of death.
- **Conditions.** Optional logical groupings that organize related interaction types. For example, a Cardiovascular condition might group together diagnoses, procedures, and medications related to heart disease. Conditions are purely organizational and do not map to database tables.
- **Interactions.** The core clinical events: diagnoses, procedures, drug exposures, laboratory measurements, observations, visits, and other temporal events. Each interaction type is defined with a name, a default filter (which distinguishes it from other record types in the same table), and a set of attributes.
- **Attributes.** Individual data points within an interaction or patient record. Each attribute defines an expression (a SQL column reference using placeholders), a data type (text, numeric, time, datetime), and optional metadata such as reference expressions for vocabulary lookups.

Interaction Definitions

Each interaction in the logical model specifies:

- A **default filter** that identifies which records in the physical database belong to this interaction type. For example, a Diagnosis interaction might use a filter like `@COND.CONDITION_TYPE = 5` to select only primary diagnoses from the condition occurrence table.
- An optional **parent interaction reference** for hierarchical relationships between events (e.g., a sub-procedure linked to a parent procedure).
- A set of **attributes**, each with an expression that describes how to extract the value from the database using placeholder references.
- An optional **from** override that allows this specific interaction to use a different placeholder-to-table mapping than the default.

Attribute Expressions

Attribute expressions use placeholder references to describe how to compute or extract a value. The format is:

```
@PLACEHOLDER.COLUMN
```

For example:

- `@INTERACTION.START` refers to the start date column of the interaction table
- `@CODE.VALUE` refers to the value column of the code/concept lookup table

- `@PATIENT.DOB` refers to the patient's date of birth
- `LEFT(@CODE.VALUE, 3)` applies a SQL function to the code value

Expressions can include SQL functions, enabling computed attributes like substring extraction, date arithmetic, or conditional logic. The placeholder references are resolved to actual table aliases during SQL generation.

Attributes support three types of expressions:

- **Data expression.** References columns in the clinical data tables (e.g., @INTERACTION.START). Used for direct data retrieval and filtering.
- **Measure expression.** An aggregate function applied to a data column (e.g., COUNT(@INTERACTION.INTERACTION_ID)). Used for computed metrics.
- **Reference expression.** References the vocabulary concept table (@REF) or cohort definition tables. Used for code-to-description lookups and terminology navigation.

23.4 The Placeholder System

Placeholder Categories

Placeholders are organized into categories based on the table type hierarchy:

Category	Placeholders	Role
Fact Table	@PATIENT	The root patient dimension table. Every query starts here. One row per patient.
Dimension Tables (temporal)	@COND, @VISIT, @PROC, @MEAS, @OBS, @DRUGEXP, @DRUGERA, @DOSEERA, @DEVEXP, @SPEC, @DEATH, @CONDERA, @OBSPER, @PPPER, @CONSENT, @RESPONSE, @PTOKEN	Clinical event tables. Each contains temporal records (with start/end dates) linked to patients via PATIENT_ID. One patient can have many records (1:N relationship).
Generic Dimension	@INTERACTION	A generic placeholder resolved at runtime to the specific dimension table based on the interaction's default filter. Acts as an alias for whichever OMOP table the interaction targets.
Attribute Tables	@CODE, @MEASURE, @TEXT	Detail tables providing additional information about an interaction (concept codes, measurement values, text descriptions). Joined to dimension tables via INTERACTION_ID.
Reference Tables	@REF	The OMOP Concept vocabulary table. Used for code-to-description lookups and concept hierarchy navigation. Resides in the vocabulary schema.

Category	Placeholders	Role
Cohort Tables	@COHORT, @CDM_COHORT_DEF, @RESULT_COHORT_DEF	Tables for cohort membership and cohort definitions. Reside in the CDM or results schema.

Standard Column Mappings

Every dimension table placeholder defines a standard set of column mappings that provide a uniform interface regardless of the underlying OMOP table:

Column Field	Purpose	Example (@COND)
.PATIENT_ID	Links record to patient table	PATIENT_ID
.INTERACTION_ID	Unique clinical event identifier	COND_OCCURRENCE_ID
.CONDITION_ID	Groups related events	COND_OCCURRENCE_ID
.PARENT_INTERACT_ID	Parent event reference	COND_OCCURRENCE_ID
.START	Start date/time	CONDITION_START_DATE
.END	End date/time	CONDITION_END_DATE
.INTERACTION_TYPE	Type classification	CONDITION_TYPE_NAME

This uniform column interface means that the query generation pipeline can reference .START, .END, .PATIENT_ID, etc., without knowing which specific OMOP table is being queried. The placeholder map translates these abstract column references to the actual column names in each table.

Schema Substitution

Table names in the placeholder map include schema substitution variables that are resolved at query execution time:

- `$$SCHEMA$$` resolves to the main CDM data schema (e.g., `omop_cdm`). Used for all clinical data tables.
- `$$VOCAB_SCHEMA$$` resolves to the vocabulary schema (e.g., `omop_vocab`). Used for concept and terminology tables.
- `$$RESULT_SCHEMA$$` resolves to the results schema (e.g., `omop_results`). Used for cohort storage and analysis outputs.

This three-schema separation allows organizations to share vocabulary tables across multiple studies while keeping clinical data and analysis results isolated.

23.5 The Table Type Hierarchy

The placeholder system defines a hierarchical structure that describes how tables relate to each other:

```
Fact Table: @PATIENT (one row per patient)
  |-- Attribute Tables: @OBS (1:N patient observations)
  |
  +-- Dimension Tables: @INTERACTION
        (hierarchy: true, time: true, 1:N, condition: true)
        |-- @CODE    (1:N concept codes)
        |-- @MEASURE (1:N measurement values)
        +-- @TEXT    (1:N text descriptions)
```

Each position in this hierarchy carries semantic flags:

- **hierarchy: true** indicates that the table supports parent-child relationships between records, using the PARENT_INTERACT_ID column. This enables tree-like navigation (e.g., a procedure with sub-procedures).
- **time: true** indicates that records have temporal extent (START and END dates), enabling date-range filters, temporal ordering, and survival analysis.
- **oneToN: true** indicates a one-to-many relationship with the parent table. One patient can have many interactions; one interaction can have many codes.
- **condition: true** indicates that the table can be used as the primary target for an interaction's default filter.

The query generation pipeline uses this hierarchy to determine how tables should be joined: fact-to-dimension joins use PATIENT_ID, dimension-to-attribute joins use INTERACTION_ID, and the hierarchy flag enables parent-child joins via PARENT_INTERACT_ID.

23.6 The Resolution Process

When the Query Generation Service produces SQL from a FAST representation, the following resolution steps transform logical references into physical SQL:

1. **Entity resolution.** The system loads the CDM configuration for the requested interaction path (e.g., patient.conditions.Cardiovascular.Diagnosis). It identifies the base entity placeholder by examining the interaction's default filter (e.g., @COND from a filter like @COND.CONDITION_TYPE = 5).
2. **Table resolution.** Each placeholder is resolved to its physical table name using the placeholder map. If the interaction has a default filter, a hash of the filter is appended to the table name, creating a unique identifier that allows the same physical table to appear multiple times in a query with different filters (e.g., one join for primary diagnoses and another for secondary diagnoses).
3. **Alias assignment.** The AST assigns a unique SQL alias to each table occurrence, tracked at the Definition level. This prevents alias collisions when the same table is joined multiple times.
4. **Expression substitution.** Placeholder references in attribute expressions (@PLACEHOLDER.COLUMN) are replaced with the assigned table alias and the resolved column name. For example, @COND.START becomes t1.CONDITION_START_DATE.
5. **Schema substitution.** The schema placeholder variables ($$SCHEMA$$, $$VOCAB_SCHEMA$$, $$RESULT_SCHEMA$$) are replaced with the actual schema names for the target study's database.
6. **Default filter injection.** The interaction's default filter expression is added to the JOIN's ON clause, with its placeholder references resolved to the assigned table aliases. This ensures that each interaction type only matches its intended records.

23.7 Interface Views vs Direct Tables

The system provides two complete sets of placeholder mappings:

- **Interface views mapping.** Uses database views that provide a uniform interface layer over the underlying tables. This is the default. It offers better abstraction (views can be modi-

fied without changing the configuration), easier maintenance, and a natural point for access control enforcement.

- **Direct table mapping.** References the underlying OMOP tables directly, bypassing the view layer. This offers lower latency but tighter coupling to the physical schema. It is used when performance is critical and the schema is stable.

Both mappings use identical column mappings; only the table references differ. The system also provides a guarded table mapping variant where the @PATIENT placeholder is replaced with a restricted view that includes row-level security filtering, preventing unauthorized access to patient records.

23.8 Per-Interaction and Per-Attribute Overrides

While the placeholder table map provides system-wide defaults, individual interactions and attributes can override specific placeholder mappings using a from property. The override is merged with the default mapping at entity resolution time, with the override taking precedence. This mechanism is central to the data-model-translation capability of the system.

Mixing Data Models Within a Single Configuration

The from override enables a single CDM configuration to span multiple data model patterns. For example:

- A **standard OMOP interaction** (e.g., Diagnosis) uses the default placeholder map, which points @INTERACTION to the condition_occurrence table with standard OMOP columns.
- A **genomics interaction** (e.g., Genetic Variant) overrides @INTERACTION to point to a custom genomics table with completely different columns (gene name, position, variant type). The query generation pipeline handles this transparently because it works with placeholders, not table names.
- A **custom collection attribute** overrides @OBS to point to an alternative observations table that stores research-specific data in an EAV-like format.

This flexibility means that organizations do not need to force all their data into a single table structure. The same analytical interface can query across OMOP tables, custom research tables, and legacy EAV schemas simultaneously, as long as each is mapped to the appropriate placeholders.

How Default Filters Adapt to Data Models

The default filter on an interaction serves as the bridge between the logical concept and the physical data. Its form varies depending on the underlying data model:

- In an **OMOP schema**, a default filter typically references a type column: `@COND.CONDITION_TYPE = 5` selects primary diagnoses from the condition occurrence table.
- In an **EAV schema**, a default filter might match on a type/key column: `@INTERACTION.ATTRIBUTE = 'DIAGNOSIS'` selects diagnosis records from a generic observations table.
- In a **custom schema**, a default filter uses whatever discriminator the schema provides: `@INTERACTION.RECORD_CLASS = 'LAB'` or even `1=1` when the interaction maps to a dedicated table that contains only the relevant records.

The query generation pipeline does not know or care which data model pattern is in use. It simply applies the default filter as a JOIN condition, using the resolved table alias. The CDM configuration author is the one who encodes the data model knowledge by choosing the right placeholders, expressions, and filters for their schema.

24 PA Configuration Service

24.1 Purpose

Patient Analytics is configurable per dataset: which interaction types are available, which attributes each interaction exposes, what the default bin sizes are, what concept hierarchies the autocompletes search. The PA Configuration Service owns those configurations — it is to the frontend what the CDW Service is to the SQL compiler. Without it, the Patient Analytics canvas has no items in its dropdowns and no rules for what its filter cards can express.

The PA (Patient Analytics) Configuration Service manages UI-level configurations that control how the analytics frontend displays and behaves. While the CDW Service defines what data is available, the PA Configuration Service defines how that data is presented — translating a raw clinical data model into an opinionated, task-oriented interface for researchers.

24.2 Relationship to CDW Configuration

Every PA configuration depends on a CDW configuration, and this dependency is structural rather than incidental. The CDW configuration defines the data model: which interactions (clinical event types) exist, what attributes each interaction carries, what data types those attributes have, and how interactions relate to each other. The PA configuration layers a presentation model on top of this foundation, specifying which of those interactions and attributes should be visible, how they should be labeled, how they should be grouped into filter panels, and what chart types can render them.

This layering means the PA configuration inherits the CDW's structural vocabulary — interaction paths, attribute identifiers, data types — but adds its own concerns: display order, visibility flags, chart bindings, and UI-specific metadata like bin sizes and reference text settings. When a PA configuration is loaded for the frontend, the service enriches it by resolving each source path against the CDW, pulling in display names, data type annotations, and parent interaction references. The result is a merged view where CDW-level structure and PA-level presentation are unified into a single object the frontend can consume without further lookups.

This dependency also means that a PA configuration is only valid relative to its CDW configuration. If the CDW changes — an interaction is removed, an attribute is renamed — the PA configuration may reference paths that no longer exist. The validation pipeline (described below) catches these inconsistencies before activation.

24.3 Filter Card Configuration

Filter cards are the primary unit of user interaction in the analytics frontend. Each filter card corresponds to a clinical concept (a diagnosis interaction, a procedure interaction, a lab result) and defines how that concept appears in the filter panel.

The core binding is the source path, which links the filter card to an interaction path in the CDW configuration. This path determines which database tables and columns back the filter card's attributes. When a user drags a filter card onto the query canvas, the frontend resolves this path to load the available attributes and their constraint options.

Each filter card carries visibility and initialization flags that control its lifecycle in the UI. The visibility flag determines whether the card appears in the filter panel at all — hidden cards exist in the configuration but are suppressed from the researcher's view, useful for interactions that are structurally necessary but not user-facing. The initialization flag controls whether the card is pre-loaded when the analytics view opens, allowing administrators to set up a default starting state for common workflows.

Display order is explicit: each filter card has a numeric order value that determines its position in the panel. The validation pipeline enforces uniqueness of these order values to prevent ambiguous layouts.

24.4 Attribute Settings

Within each filter card, individual attributes carry their own configuration layer. The most consequential setting is the category-versus-measure designation. Category attributes appear on the X-axis of charts, defining grouping dimensions — a diagnosis code, an age range, a gender. Measure attributes appear on the Y-axis, defining what gets aggregated — patient counts, lab values, durations. This designation directly controls how the Query Generation Service builds its GROUP BY and aggregation clauses.

Reference text and reference value flags control whether the frontend displays raw codes or human-readable descriptions. When reference text is enabled for a diagnosis attribute, the UI shows "Type 2 Diabetes Mellitus" rather than "E11." This setting flows through to the backend view, where the Analytics Service uses it to determine which columns to include in query results.

For continuous numeric attributes, the default bin size defines the histogram bucket width. When a researcher visualizes lab values on a bar chart, the bin size determines the granularity of the distribution display — a bin size of 10 for a glucose measurement groups values into 0–9, 10–19, 20–29, and so on.

Attributes also carry separate configuration blocks for their appearance in filter cards versus patient lists. The filter card settings control visibility and order within the filter panel. The patient list settings control whether and where the attribute appears in tabular patient views, including a link column designation that turns cell values into navigable references. Annotations provide an additional metadata layer, attaching tags that offer contextual information to researchers browsing the configuration.

24.5 Chart Options

Each chart type in the analytics frontend has its own configuration object within the PA configuration. These are not generic — each chart type carries settings specific to its visualization semantics.

The stacked bar chart configuration controls visibility, download capabilities (CSV, PDF, and image export), whether the chart supports adding results to a cohort, a fill-missing-values flag that inserts zero-count entries for empty bins (preventing misleading gaps in distributions), and initial attribute selections that pre-populate the axes when the chart loads.

The box plot configuration is simpler, controlling visibility, download options, and initial attributes for the distribution display. Because box plots compute their own statistical summaries (quartiles, median, outliers), they do not need bin size or fill-missing-values settings.

The Kaplan-Meier survival chart has the most specialized configuration. Beyond visibility and download settings, it includes a confidence interval toggle, selections for the start and end interactions that define the survival window, and filter definitions that constrain which events qualify as endpoints. These settings directly parameterize the Kaplan-Meier aggregation strategy in the Query Generation Service.

The patient list configuration controls visibility, download options (CSV and ZIP for large exports), and a configurable page size that determines how many rows the frontend fetches per

request. The variable browser configuration adds a reference name for genomic data integration, linking the analytics view to external genomics pipelines. Custom chart definitions provide an extensibility point for deployment-specific visualizations.

24.6 Panel Options

Panel options control system-wide behavior that cuts across individual charts and filter cards. The add-to-cohorts flag enables or disables the ability to save query results as materialized cohorts. The domain values limit caps the number of distinct values loaded for categorical dropdowns, preventing the UI from becoming unresponsive on high-cardinality attributes. The maximum filter card count limits how many filter cards a researcher can add to a single query, controlling query complexity. Cohort entry and exit tracking, when enabled, activates the temporal cohort analysis mode where specific interactions mark the beginning and end of a patient's membership in a cohort. Atlas cohort definition support enables integration with OHDSI's Atlas tool for importing externally defined cohort criteria.

24.7 Validation Pipeline

Before a PA configuration can be activated, it passes through a validation pipeline that checks consistency against the dependent CDW configuration. This validation is not merely syntactic — it verifies semantic correctness of the presentation layer relative to the data model.

The pipeline checks that every filter card's source path resolves to an existing interaction in the CDW configuration. If a CDW update removed an interaction that the PA configuration still references, validation catches the dangling reference. Initial attributes specified in chart configurations must be visible (not hidden by visibility flags) and must belong to filter cards that are themselves visible. At least one chart type must be enabled — a configuration with all charts disabled would render the analytics view empty. Display order values must be unique within their scope (filter cards within the panel, attributes within a filter card) to prevent layout ambiguity.

Validation runs as a separate step before activation, returning detailed errors and warnings that administrators can address. This separation allows draft configurations to exist in an inconsistent state during editing, with validation serving as a gate before publication.

24.8 Configuration Views

The service produces three distinct views of the same underlying configuration, each tailored to its consumer.

The frontend view is the richest. It merges PA settings with CDW metadata — display names, data types, parent interaction references — producing a self-contained object that the analytics UI can render without additional service calls. Internal metadata (version identifiers, CDW dependency details, validation state) is stripped to reduce payload size and prevent leakage of implementation details.

The backend view is consumed by the Analytics Service when processing queries. It includes reference flags (indicating which attributes use reference text versus raw codes) and chart enablement information (which chart types are active and their specific settings). This view omits display-oriented metadata like order values and annotations that are irrelevant to query execution.

The admin view exposes the complete configuration with all internal metadata: version history, CDW dependency identifiers, validation results, and draft status. This is the view used by the administrative UI for configuration editing and lifecycle management.

24.9 User Assignment

PA configurations are assigned at three levels of specificity. Individual assignment binds a configuration directly to a user, overriding all other assignments. Organization or study assignment applies a configuration to all users within an organizational unit, providing a shared analytical experience for a research team. System-wide defaults serve as the fallback when no more specific assignment exists. The resolution order is individual, then organization, then system default — the most specific applicable assignment wins.

Access control distinguishes between regular users, who can view and use their assigned configuration, and administrators, who have full CRUD access plus the ability to import and export configurations across environments.

24.10 Import and Export

Configurations can be exported as JSON and imported into other environments, supporting promotion workflows from development to staging to production, or sharing between institutions. The export format includes the complete filter card definitions, attribute settings, chart options, and panel options — everything needed to reconstruct the analytical experience.

However, the export deliberately excludes two categories of data. User assignments are excluded because user identities differ across environments. CDW-enriched metadata (display names, data types, annotations) is excluded because it is resolved dynamically at load time in the target environment by querying the target's CDW configuration. This means an imported configuration adapts to the target's CDW automatically, but it also means the target CDW must contain the interactions and attributes the configuration references — otherwise validation will fail after import, surfacing the incompatibilities for the administrator to resolve.

24.11 Integration Points

The PA Configuration Service is read by the Patient Analytics frontend on every dataset open. It coordinates with the CDW Service: a PA configuration references a CDW configuration, and the two are validated for consistency before publication. It does not participate in analytical query execution — once a UI session has loaded its PA config, all subsequent traffic flows through the Analytics Service.

24.12 Failure Modes

The most consequential failure mode is silent inconsistency between PA and CDW configurations: a PA config exposes an attribute that the CDW config does not map to a real column. The validation step at publication time prevents this for newly published configurations, but configurations that pre-date the validation rules can still hold latent inconsistencies. Researchers typically discover them through "no data" results in the chart pane.

25 Bookmark Service

The Bookmark Service manages saved filter definitions — bookmarks — that allow users to persist, retrieve, and share their query criteria. A bookmark captures the complete state of an analytical query so that it can be restored, shared with colleagues, or linked to a materialized cohort for downstream analysis.

25.1 What a Bookmark Stores

A bookmark is the persisted form of an Internal Filter Representation (IFR). This is not a summary or a simplified snapshot — it is the full query state. The stored IFR includes every filter card the user had on the canvas, each with its source path, instance number, and complete list of attribute constraints. It includes the operators connecting those constraints (equality, range, null checks), the combination logic (Match All versus Match Any groupings), temporal relationships between events, axis definitions for chart rendering, and exclusion flags. When a bookmark is loaded, the frontend receives exactly the query state the user saved, and the analytics view is restored to the same configuration without any re-specification.

Beyond the IFR itself, a bookmark carries metadata: a user-assigned name, a version number that tracks content revisions, a creation timestamp, references to the PA and CDM configurations that were active when the bookmark was created, the creating user's identity, and a boolean sharing flag.

25.2 Identifier Generation

When a bookmark is created, the service generates a unique identifier by combining a sanitized form of the user-provided name with a random value. The sanitization strips characters that would be problematic in URLs or database keys, and the random suffix guarantees uniqueness even when two users independently create bookmarks with the same name. This composite approach produces identifiers that are both human-recognizable (the name prefix hints at the bookmark's purpose in logs and database queries) and collision-free.

25.3 Version Management

The Bookmark Service tracks content revisions through a simple incrementing version number. When a user updates the filter definition — adding a filter card, changing a constraint, modifying axis settings — the service replaces the stored IFR with the new one and increments the version number. This version counter provides a lightweight audit trail: it tells consumers how many times the query definition has changed since creation, which is useful for understanding whether a bookmark represents a stable, well-refined query or one still under active iteration.

Renames are handled separately from content updates. Renaming a bookmark changes only the display name without touching the IFR or incrementing the version number. This distinction matters because renames propagate to linked cohorts (described below), and conflating a name change with a content revision would create misleading version histories. A researcher who renames a bookmark for clarity should not see the version number jump, which would imply the query logic changed.

25.4 Sharing Model

Bookmark sharing is controlled by a single boolean flag. When the flag is false, the bookmark is private — visible only to its creator. When true, the bookmark becomes visible to all authenticated users who have access to the same dataset, filtered by PA configuration ID. This means shared bookmarks appear only in listings for users whose assigned PA configuration matches

the one under which the bookmark was created, ensuring that shared queries make sense in the recipient's analytical context.

Despite visibility being shared, modification rights remain exclusive to the creator. Other users can load and execute a shared bookmark — restoring its filter state and running the query — but they cannot edit, rename, or delete it. This design prevents collaborative conflicts: a shared bookmark is a stable reference point that the creator controls, not a shared-editing artifact.

The sharing model interacts with the platform's dataset-level access control. A user must already have access to the dataset (through a study-level role) to see any bookmarks for that dataset, whether their own or shared ones. Sharing a bookmark does not grant data access — it merely makes the query definition visible to users who independently have the right to query the same data.

25.5 Bookmark-Cohort Relationship

Bookmarks can be linked to materialized cohorts, creating a "saved query to materialized result" lifecycle. When a user materializes a cohort from a bookmark, the cohort metadata stores the bookmark's identifier, establishing a bidirectional relationship: given a bookmark, the system can find its cohort; given a cohort, the system can retrieve the query that produced it.

This linkage enables two cascade behaviors. When a bookmark is renamed, the rename propagates to the linked cohort, keeping the display names synchronized. When a bookmark is deleted, the service first deletes any linked materialized cohorts via the Analytics Service before removing the bookmark record itself. This cascade prevents orphaned cohorts — materialized results whose defining query no longer exists and therefore cannot be refreshed or inspected.

The relationship also supports a common analytical workflow: a researcher builds a query, saves it as a bookmark, materializes a cohort, and later returns to refine the query. By loading the bookmark, they recover the exact filter state, adjust it, and can re-materialize. The version number on the bookmark tells them (and the system) whether the query has changed since the cohort was last materialized, enabling staleness detection.

25.6 Service Architecture

The Bookmark Service does not maintain its own database. All storage operations are proxied through the Portal Service, which provides the underlying PostgreSQL [69] persistence and handles authentication and authorization. The Bookmark Service acts as a domain-specific facade: it understands bookmark semantics (versioning, sharing rules, cohort linkage) while delegating storage mechanics to the Portal. This architecture avoids duplicating the Portal's authentication middleware and database connection management, at the cost of an additional network hop for every operation.

25.7 Multi-Bookmark Loading

For cohort comparison workflows, the service supports loading multiple bookmarks in a single request. A user comparing two patient populations — for example, patients with diabetes who received treatment A versus treatment B — needs both bookmarks loaded simultaneously so the frontend can render side-by-side charts. The multi-load endpoint accepts a list of bookmark identifiers and returns all requested bookmarks in parallel, avoiding the latency of sequential individual loads. This is a performance optimization rather than a semantic feature: the loaded bookmarks are independent objects with no special relationship to each other beyond appearing in the same comparison view.

26 User Management Service (alp-usermgmt)

26.1 Purpose

User Management is the platform's authorization plane, deliberately distinct from its authentication plane. Logto [33] handles identity extremely well — federated sign-in, password resets, MFA, account lockout, the long tail of credential-management features that no sensible platform writes from scratch — but its responsibility ends at the question "who is this user." The question that follows — "can this user analyse the diabetes dataset, and not the cardiology one" — is not Logto's job. It depends on facts that belong to D2E: which datasets exist, which study a researcher has been granted access to, which tenant they sit inside. This is principle 8 of 2: identity outside, authorization inside.

The obvious alternative was to shoehorn dataset-scoped authorization into Logto's scope and role model — one Logto role per dataset, or custom claims encoding dataset membership. That option was rejected on two grounds. First, dataset access changes orders of magnitude more frequently than Logto's role definitions: a researcher may be added to a study in the morning and removed by afternoon, while Logto's roles change at deployment cadence. Second, Logto's admin API is designed for occasional configuration writes, not for a high-frequency authorization service hammering it with grant and revoke calls. Treating Logto as a dataset-membership store would have coupled the platform's responsiveness to the latency and availability of an external identity provider — a tight coupling in exactly the dimension that matters most.

The decision is therefore to split the model along its natural seam. The User Management Service holds the authoritative `(user, role, dataset)` graph; Logto holds the authoritative identity record. They are linked by an IDP user identifier, and the two are reconciled at well-defined moments rather than continuously. At sign-in, Logto issues a JWT [25] enriched with platform-specific claims — the user's roles and dataset assignments, looked up via a hook into User Management. Downstream services validate the token and read the claims directly; they do not call User Management on every request. This token-enrichment step is the contract that lets the rest of the platform treat authorization as a property of the request, not of a remote service.

The cost of this split is a dual-record model: every user exists twice, once in Logto and once in the User Management database, and the two views can drift. An administrator who edits a user directly in the Logto admin console bypasses User Management's reconciliation logic, and the next sign-in may surface unexpected denials until the records realign. The benefit is that each system does only what it is good at. Logto is not asked to be a dataset-membership engine, and User Management is not asked to handle credentials. What the service forbids is any other component writing dataset assignments on its own; what it allows is the rest of the platform to treat authorization decisions as already settled by the time a request arrives at a route handler.

26.2 Overview

The User Management Service is the central authority for identity, role assignment, and access control within the D2E platform. It maintains its own authorization model while delegating authentication to an external identity provider (Logto). Every request to any D2E service ultimately depends on the user and role data this service manages: the JWT token issued by Logto identifies the user, and the User Management Service resolves that identity into a set of scoped permissions that govern what the user can see and do.

26.3 Role Hierarchy and Scoping

Permissions in D2E are not flat — they follow a three-tier hierarchy that mirrors the platform's organizational structure. Each permission grant is represented internally as a "group" record that encodes a role name together with a scope. The scope is what makes the same role mean different things in different contexts.

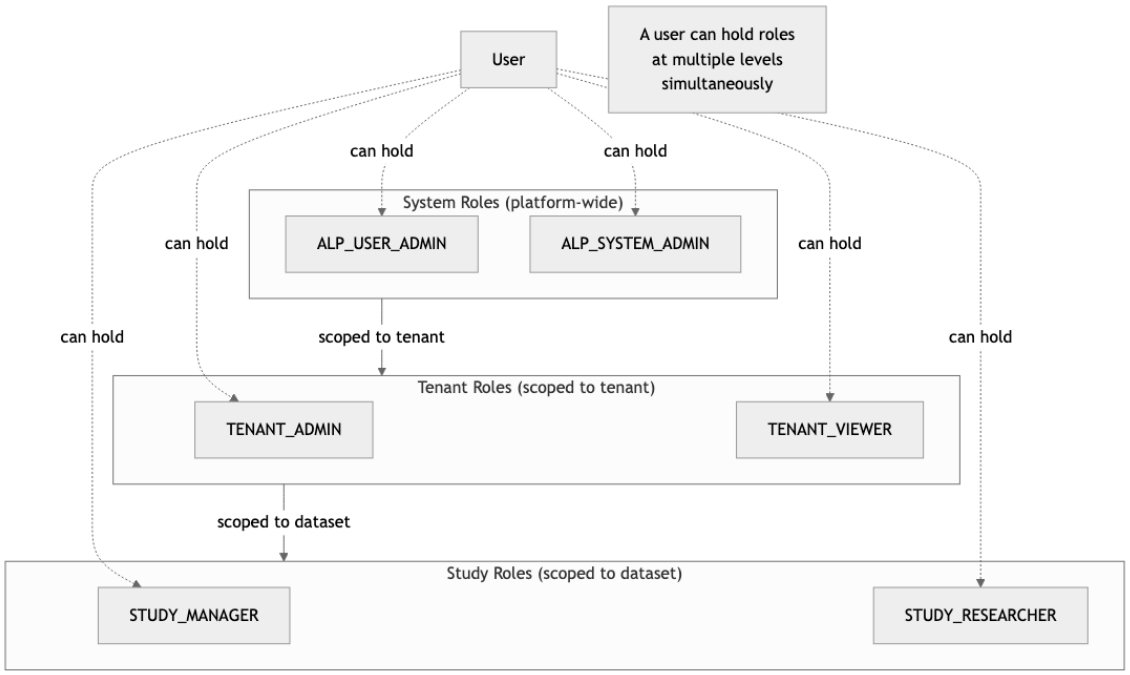

Figure 19: Three-Tier Role Hierarchy

1. **System Roles** operate across the entire platform. There are two system roles: one for user administration (managing all users, groups, and role assignments platform-wide) and one for system administration (managing configuration and infrastructure).
2. **Study Roles** are scoped to a specific dataset. Study managers control who can access the dataset, configure analyses, and grant permissions. Study researchers can view data and execute read-only analyses but cannot change access or configuration.

A single user may hold multiple roles simultaneously — for example, a system administrator role plus a study researcher role for a specific dataset. The system evaluates permissions by checking whether the user holds any role that satisfies the required scope for the operation being attempted.

26.4 Identity Provider Integration

The service maintains a dual-record model for every user. One record lives in the Logto identity provider and handles authentication concerns: credentials, password policies, and token issuance. The other record lives in the local PostgreSQL [69] database and handles authorization concerns: which roles the user holds, which tenants and studies they can access, and when their account was created or modified.

When an administrator creates a new user, the service first provisions the identity in Logto (which generates credentials or an invitation link), then creates the corresponding local record and links them via an IDP user identifier. This separation means the platform can switch identity providers without restructuring its authorization model, and it means authentication

failures (bad password, expired token) are handled entirely by the IDP while authorization failures (insufficient role) are handled by the User Management Service.

26.5 The User Lifecycle

A user's journey through the platform follows a predictable sequence of stages, each involving different actors and different parts of the service.

Registration. A system or tenant administrator creates the user account. This is a two-step process: the service provisions the identity in Logto and simultaneously creates the local user record. The user receives initial credentials or an invitation link depending on the tenant's configuration.

Role Assignment. After registration, the administrator assigns initial roles. Each assignment links the user to a group record that encodes the role and its scope.

Self-Service Access Requests. Once active, researchers can browse the platform's dataset listings and discover datasets marked as public. If they lack the study-level role needed to access a dataset's data, they can submit an access request. This request enters a pending state and is visible to study managers and tenant administrators, who can approve or reject it. Approval automatically grants the researcher the appropriate study-level role.

Active Use. During normal operation, the user authenticates through Logto and receives a JWT token. On every subsequent request to any D2E service, middleware decodes the token, resolves the local user record, and attaches the full set of role assignments to the request context. This happens transparently — downstream services simply check the attached roles without needing to query the User Management Service directly.

Deactivation and Deletion. Administrators can soft-disable a user by marking them inactive. Inactive users cannot authenticate, but their records and audit trail are preserved for compliance purposes. Full deletion removes both the IDP identity and all local records, cascading through group memberships and access requests.

26.6 Authorization Enforcement

The service enforces authorization through two complementary middleware functions that run before any route handler.

permittedUserCheck verifies that the caller holds the required system-level role. It is applied to administrative endpoints where the operation affects users or groups.

Self-access is a special case: any authenticated user can read and modify their own profile, change their own password, and view their own role assignments without needing any administrative role. This is enforced through dedicated self-service routes that operate on the identity extracted from the JWT token rather than requiring an explicit user ID.

26.7 Request Processing Pipeline

Every request passes through a layered middleware pipeline before reaching a route handler. The pipeline handles cross-cutting concerns in a fixed order: response compression, body parsing, structured logging with request correlation IDs, and a health check short-circuit (so monitoring probes do not require authentication). After these infrastructure concerns, the pipeline enters the security layer: it registers a scoped dependency injection container for the request lifecycle, decodes the JWT token, resolves the local user record with all role assignments, and attaches this context to the request object. Route-specific authorization middleware (the permittedUserCheck function described above) runs last, immediately before the handler.

26.8 Integration with Other Services

The User Management Service does not operate in isolation. It queries the Portal Service to retrieve tenant lists and dataset metadata when it needs to validate that a tenant or dataset exists before granting a scoped role. It calls the Logto identity provider for all credential management operations: creating users, resetting passwords, and deleting identities. It uses the OpenID Connect [52] discovery protocol to validate JWT tokens and ensure they were issued by the expected authority.

Other D2E services depend on the User Management Service indirectly: they consume the user context and role assignments that the middleware pipeline attaches to every authenticated request. This means the User Management Service's role model is the foundation of the entire platform's security posture.

26.9 Integration Points

User Management is called by Logto during login (to enrich tokens with platform-specific claims), by the Portal Service during user listing, by the Dataset Service during dataset-access changes, and by every authenticated endpoint indirectly — through the authorization middleware that consults User Management's data via cached tokens. It depends on Logto for identity primitives and on the platform database for its own state.

26.10 Failure Modes

User Management is in the critical path of every authentication. An outage prevents new logins; existing sessions continue working until tokens expire (typically minutes to an hour). Token-cache misses during an outage cascade into 401s. The most insidious failure mode is desynchronization between Logto's view of a user and User Management's view — usually after a manual change made in the Logto admin console without going through User Management. The reconciliation logic catches most cases on the next login, but stale Logto state can cause unexpected access denials until reconciled.

27 Dataset Service

27.1 Purpose

The Dataset Service owns the lifecycle of a dataset — creation, provisioning, publication, and retirement. It is the only path through which a new dataset becomes queryable on the platform. Behind a single creation call it threads together three concerns that must all succeed together: schema provisioning (the database structures that hold the data), metadata registration (telling the Portal Service that a new dataset exists and how to describe it), and access-grant initialization (coordinating with User Management so that the right roles can see and read the dataset from day one).

The service exists because each of those concerns lives in a different system, and a dataset is only useful when all three are in agreement. Without the Dataset Service, every new dataset would require a hand-coordinated dance between database administrators, the metadata catalog, and the access control layer. The cost is a single orchestrator that must be careful about partial failure; the benefit is that "a dataset exists" becomes a coherent platform-level fact rather than a checklist.

27.2 Overview

The Dataset Service is the primary interface for creating, configuring, and managing datasets within the D2E platform. While the Portal Service stores dataset metadata and the User Management Service controls who can access what, the Dataset Service orchestrates the operational side: provisioning database schemas, managing snapshots, and coordinating with infrastructure services to make datasets usable for analysis.

27.3 What is a Dataset?

A dataset is the fundamental unit of data organization in D2E. It is a logical container that binds together several concerns that would otherwise be managed separately:

- **Database connectivity**: Each dataset points to a specific database (PostgreSQL [69] or SAP HANA) via dialect-specific connection parameters. Credentials are not stored in the dataset record itself but are resolved at query time through a dedicated credentials service.
- **Schema structure**: A dataset typically encompasses three distinct schemas in its target database — a CDM schema holding source data in OMOP format, a vocabulary schema containing standardized terminologies, and a results schema where analysis outputs are written. This three-schema pattern separates concerns cleanly: source data is read-only, vocabularies are shared, and results are per-analysis.
- **Metadata**: Human-readable name, description, summary, tags, attributes, and release history. This metadata lives in the Portal Service's database but is managed through the Dataset Service's creation and update workflows.
- **Artifacts**: Dashboards, code snippets, parameterized queries, and Shiny applications that are associated with the dataset and provide researchers with ready-made analytical tools.
- **Cache layer**: TrexSQL-based cached representations that enable fast analytical queries without hitting the source database for every request.

The binding of all these concerns into a single addressable entity is what makes the dataset concept powerful. A researcher does not need to know which database host holds the data, which schema naming convention is in use, or how credentials are managed. They interact

with a dataset by its identifier or human-readable code, and the platform resolves everything else.

27.4 Dataset Types

D2E supports two dataset types that differ in how source data enters the platform.

Source datasets are backed by a direct database connection. Clinical or observational data already resides in a PostgreSQL or HANA database in OMOP CDM format, and the platform queries it in real time. This is the standard type for organizations that maintain their own CDM transformation pipelines.

FHIR datasets store clinical data in FHIR R4 format. The platform transforms this data into OMOP CDM via structure maps, making it available for the same analytical workflows that operate on source datasets. A FHIR dataset carries an additional project identifier that links it to the FHIR server's project structure.

Both types participate identically in the rest of the platform once their schemas are provisioned: the query engine, analytics services, and dashboard tools do not distinguish between source and FHIR datasets.

27.5 Schema Creation and Provisioning

Creating a dataset involves more than inserting a metadata record. The service must ensure that the database schemas the dataset points to actually exist and conform to the expected structure. The caller chooses one of four schema options at creation time, each reflecting a different operational scenario:

- **Create CDM**: The service provisions a new empty schema using the data model template (e.g., OMOP 5.4). It triggers a workflow through the JobPlugins infrastructure, which executes the DDL statements in the target database. This is the most common option for new datasets.
- **Existing CDM**: The dataset is pointed at a schema that already exists in the target database. The service validates that the schema conforms to the expected data model but does not modify it. This is used when data engineering teams have prepared schemas outside the platform.
- **Custom CDM**: The caller provides custom DDL in the request, and the service creates a schema using those statements. This supports non-standard data models or extensions to the OMOP schema.
- **No CDM**: The dataset record is created without any schema provisioning. This is used for metadata-only datasets or situations where schema creation is handled by an external process and will be linked later.

Schema creation is an asynchronous operation when it involves DDL execution. The service delegates to Prefect [55] workflows through the JobPlugins API, which means the dataset record may exist before its schemas are fully provisioned. Downstream services check schema readiness before allowing queries.

27.6 The Dataset Lifecycle

A dataset moves through several phases from creation to active use.

Creation. An administrator or automated process creates the dataset by specifying its type, target database, schema option, and initial metadata. The service validates the dataset's token

code (a URL-safe identifier that must be unique across the platform), provisions schemas if requested, and creates the metadata record in the Portal Service.

Metadata Enrichment. After creation, administrators and study managers add descriptive metadata: a human-readable name and description, tags for categorization, typed attributes for structured metadata (string, number, date, or boolean values), and release records to track dataset versions over time. This metadata drives the discoverability of datasets in researcher-facing listings.

Snapshot Creation. The service supports creating snapshots — point-in-time copies of an existing dataset. A snapshot creates a new dataset record with a reference back to the original and optionally copies the underlying database schema. Snapshots enable reproducible research by freezing a dataset's state at a specific point in time.

Active Use. Once provisioned and enriched, the dataset is available for queries, analyses, and dashboard rendering. Researchers interact with the dataset through the query engine, analytics services, and dashboard tools. The dataset's schemas are accessed through the credentials service, and its metadata is served from the Portal Service.

27.7 Visibility and Access Control

Datasets have a visibility status that controls their discoverability, independent of the fine-grained role-based access managed by the User Management Service.

Hidden datasets are invisible to researchers who lack explicit study-level roles. They do not appear in listings, search results, or filter options. Only users with direct study roles (study manager or study researcher) and system administrators can see them. This is appropriate for datasets containing sensitive data or datasets that are still being prepared.

Normal datasets are visible in listings to researchers who hold a study-level role on the dataset. They do not appear to researchers without such a role, and they do not expose a "Request Access" affordance to other users. This is the default visibility for operational datasets accessed by a defined group of researchers.

Public datasets are discoverable by all authenticated researchers. They appear in listings and can be browsed, but discovering a dataset does not grant data access. A researcher who finds an interesting public dataset must submit an access request through the User Management Service. A study manager reviews the request and, upon approval, the researcher receives a study-level role that grants actual data access.

This two-layer model — visibility for discoverability plus roles for data access — allows organizations to advertise the existence of datasets to encourage collaboration while maintaining strict control over who can actually query the data.

27.8 Integration with Other Services

The Dataset Service sits at the center of several service interactions:

- **Portal Service**: Stores and retrieves all dataset metadata. The Dataset Service creates, updates, and queries portal records as part of its own operations.
- **JobPlugins API**: Triggers Prefect workflows for schema creation, data management, and other database-level operations that cannot be performed synchronously.
- **Database Credentials Service**: Manages connection credentials for dataset access. The Dataset Service does not store credentials directly; it references databases by code and lets the credentials service resolve connection details at runtime.

- **Analytics Service**: Manages cohort definitions and analysis artifacts associated with datasets. The Dataset Service exposes cohort listings and delegates to the analytics service for cohort operations.
- **ShinyLive**: Manages interactive Shiny application deployments. The Dataset Service proxies requests to ShinyLive applications that are associated with specific datasets.

27.9 Integration Points

The Dataset Service writes to PostgreSQL (and in some deployments SAP HANA) during provisioning, executing the DDL that brings a new dataset's schemas into existence. It calls the Portal Service to register and update dataset metadata, and it coordinates with User Management for the initial access grants that make the dataset reachable by its first researchers. It is itself called by the ETL Prefect flows when they finalize a new dataset — the moment a load finishes, the flow notifies the Dataset Service so the dataset's status can move from provisioning to active.

27.10 Failure Modes

The most consequential failure mode is provisioning failure mid-flight: the database structures may have been partially created before an error interrupts the workflow, leaving orphan schemas or half-populated tables. The cleanup path is designed as an idempotent retry — the caller can re-issue the creation request and the service will detect existing structures, repair what is broken, and resume from where it left off, without producing duplicates. A subtler failure mode is permission-grant failure: the schemas can be created and the metadata registered, but if the call to User Management fails after the fact, the result is a dataset that exists in the catalog but that no one — not even its intended administrators — can read. Recovery requires re-running the access-grant step explicitly.

28 Portal Service

28.1 Purpose

The Portal Service is the platform's metadata hub: studies, datasets, user-to-dataset mappings, application configuration, feature flags, attribute definitions, tags, and Git configurations all live here. It is the place that answers "what datasets exist," "what is this study," "which feature flags are on," and "which users belong to which workspaces" on behalf of every other component. The frontend reads from it on every page load; backend services read from it on nearly every authenticated request.

The orthodox microservices instinct is "each service owns its data." Under that pattern, the Dataset Service would own dataset metadata, the User Management Service would own user-to-dataset mappings, a hypothetical Configuration Service would own platform configuration, and so on. The instinct is sound when writes dominate, because it confines each domain's mutations to one place and makes consistency a local problem. It is a poor fit for D2E because the access pattern is inverted: the cross-cutting *reads* dominate. Almost every service that does anything non-trivial needs to know about datasets, users, and configuration together, in the same request. Strict service ownership of metadata would force every consumer either to fan out across several services to assemble a coherent picture, or to re-implement local copies of the metadata it needs — the same drift problem that strict ownership was supposed to prevent, now distributed across half the platform.

The decision is therefore to centralize platform metadata in one service, and to invert the usual write pattern. Other services *write* into Portal through narrow, domain-specific APIs — the Dataset Service writes dataset records when datasets are created or deprovisioned, User Management writes user records during the user lifecycle, administrative tooling writes configuration — and *read* from it broadly. Each writer still owns the lifecycle of its domain; what is centralized is the projection that the rest of the platform consults. This is consistent with the broader spirit of 2: composition over construction (other services compose against a stable metadata surface rather than re-deriving it) and a single readable, writable, auditable canonical view of what the platform contains.

The cost is concentration risk. Portal sits in the critical path of nearly every authenticated request, and its database schema is the shape that the rest of the platform depends on. Cache layers in upstream services buy minutes of grace during a Portal outage, but sustained Portal failure brings the analytical surface down faster than the failure of any other single service. Schema migrations to the Portal database are correspondingly the riskiest deployment events on the platform. What the service forbids is other components maintaining shadow catalogs of platform metadata; what it allows is that "which service owns this metadata" is never a question anyone has to answer. When a new service needs to know about datasets, users, or platform configuration, the answer is always the same: ask the Portal.

28.2 Overview

The Portal Service is the central metadata and administration hub of the D2E platform. While other services handle operational concerns — the Dataset Service provisions schemas, the User Management Service manages identity and roles, the FHIR Service handles clinical data exchange — the Portal Service is the source of truth for what datasets exist, how they are described, and how the platform itself is configured. Nearly every other service in D2E queries the Portal at some point to resolve dataset metadata, retrieve configuration values, or look up tenant information.

Architecturally, the Portal sits between the frontend application and the rest of the backend. The frontend reads Portal metadata to populate the dataset catalog, render navigation, and determine which features are available. Backend services call the Portal to resolve dataset display names, check configuration values, and verify tenant membership. This central position makes the Portal the closest thing D2E has to a system-of-record service.

28.3 Dataset Metadata Management

The Portal Service owns the complete metadata record for every dataset in the platform. This record is organized into several complementary layers, each serving a different purpose.

Detail records capture the core identity of a dataset: its display name, a full description (supporting rich text), and a brief summary used in listing views. Each dataset also carries a flag controlling whether the "Request Access" button appears in the UI, allowing study managers to fine-tune the onboarding experience per dataset.

Typed attributes provide structured, extensible metadata. Rather than hard-coding metadata fields, the platform lets administrators define attribute types at the deployment level — specifying a name, data type (string, number, date, or boolean), a grouping category, and whether the attribute should appear in listing views. Study managers then set attribute values for individual datasets. The attribute type system is deliberately open-ended: administrators create the schema, and the platform enforces data types without prescribing what the schema should contain. A clinical research organization might define attributes like "IRB Approval Date" (date type) or "Patient Count" (number type). A pharmaceutical company might use "Therapeutic Area" (string) or "Trial Phase" (number). A health system might track "Data Refresh Frequency" (string) or "Source EHR System" (string). Because attribute types are configurable per deployment, the metadata model adapts to each organization's vocabulary without code changes. Attributes also participate in full-text search, so researchers can locate datasets by searching for attribute values directly. The grouping category on each attribute type controls how attributes are organized in the UI — attributes in the same category appear together, providing visual structure to what would otherwise be a flat list.

Tags offer a simpler, categorical layer of metadata. Administrators define tag categories (e.g., "Disease Area," "Data Source Type"), and study managers associate tags with individual datasets. Tags drive the faceted filtering in the researcher-facing catalog — selecting a tag narrows the visible dataset list immediately. Unlike attributes, which carry typed values, tags are binary associations: a dataset either has a given tag or it does not. This simplicity makes tags ideal for quick filtering, while attributes handle richer, value-bearing metadata.

Release records track versioned milestones. Each release carries a name and a date, providing a lightweight changelog that researchers can consult to understand when data was last refreshed or when significant structural changes occurred. Releases are informational rather than operational: they do not trigger schema changes or data movement, but they give researchers confidence about data currency. The release history accumulates over time, building a provenance narrative for the dataset — when it was first loaded, when corrections were applied, when new data sources were integrated.

28.4 Configuration Store

Beyond dataset metadata, the Portal Service manages platform-wide configuration through a key-value store. Each entry has a type identifier, a JSON value, and a scope designation that determines its visibility. The distinction between public and secret scope is the central design decision.

Public configurations are accessible to any authenticated user. The frontend reads them on startup to determine which features to render and how to present them. Typical public entries include feature flag definitions, UI theming and branding settings, supported CDM versions for dataset creation, and export templates. Because they are read on every page load, public configurations effectively control the shape of the user experience without requiring redeployment. Feature flags, stored as a public configuration value, deserve special mention: they allow platform operators to enable or disable capabilities (FHIR support, specific analytics tools, experimental UI components) at runtime through a simple enabled/disabled toggle per flag.

Secret configurations are restricted to system administrators and store sensitive operational values: database connection credentials, identity provider settings (Logto [33] client ID, secret, and endpoints), Prefect [55] workflow engine credentials, LLM API keys, and FHIR server connection details. The service applies a redaction protocol to protect these values even from the administrators who manage them.

The redaction protocol works as follows. When a secret configuration is read — even by an administrator — its value is replaced with a `[REDACTED]` placeholder in the API response. The actual secret never leaves the server in a read operation. When an administrator submits an update, the service inspects the incoming value. If it is the literal string `[REDACTED]`, the service recognizes the placeholder and preserves the existing secret in the database, leaving it unchanged. Only an explicitly provided new value causes the secret to be overwritten. This round-trip safety mechanism solves a specific problem: administrative UIs that display a form with all configuration entries. Without redaction awareness, submitting such a form would overwrite every secret with the placeholder text. The round-trip protocol means an administrator can update one entry in a form containing multiple secrets without inadvertently destroying the others. The service treats `[REDACTED]` as a sentinel value, not as valid configuration content — no legitimate secret would ever be that literal string.

28.5 Researcher vs. Administrator Views

The Portal Service serves two fundamentally different audiences, and its dataset listing behavior reflects this split.

Researchers see a filtered catalog. The visible datasets include those where the researcher holds a study-level role (granting data access) plus all non-hidden datasets (which can be discovered and requested). The filtering logic runs at query time: for each request, the service computes the set of datasets the user can see by joining role assignments, dataset visibility flags, and tenant membership. Filter scopes — the available values for tenant, data model, and tag filters — are computed dynamically from this visible dataset set. This dynamic computation means a researcher never sees a filter option that would return zero results. If no visible dataset carries the tag "Oncology," that tag does not appear in the filter dropdown. This prevents dead-end navigation and keeps the catalog interface clean. Full-text search spans dataset details and attribute values, letting researchers locate datasets by description keywords, attribute content, or tag names.

System administrators see an unfiltered, global view of every dataset across all tenants. This perspective is essential for platform governance, auditing, and cross-tenant operations. The administrator listing bypasses all visibility and role-based filtering, and full-text search operates across the entire corpus. Filter scopes for administrators are computed from the full dataset set, so every tag, tenant, and data model that exists in the platform appears as a filter option.

The filtering logic runs at query time rather than being materialized into separate tables. This means visibility is always consistent with the current state of role assignments — there is no

synchronization lag between a role change and the catalog view updating. A researcher who is granted a new study role sees the dataset immediately on their next catalog load.

28.6 Dashboard Code

The Portal Service stores dashboard configurations and executable code associated with datasets. A dashboard pairs a JSON layout definition with code written in R or Python. The code typically contains embedded SQL queries that pull data from the dataset's schema, transform it, and produce visualizations or summary statistics.

Storing dashboard code as Portal metadata — rather than as external files or repository artifacts — makes dashboards first-class entities in the platform. They inherit the same access control, soft-deletion, and audit-trail behavior as other dataset metadata. Study managers can update dashboard code through the administrative UI, and the changes take effect immediately for researchers viewing the dataset.

The SQL queries embedded in dashboard code use parameterized templates. Rather than hard-coding schema names or filter values, the queries contain placeholders that the execution environment resolves at runtime. A date range parameter, for instance, allows the same dashboard to display data for any user-selected time window. A cohort filter parameter restricts the query to patients in a specific materialized cohort. This parameterization serves two purposes: it enables interactive, data-driven exploration without granting researchers direct database access, and it keeps the data access boundary clean. The dashboard code runs with controlled permissions scoped to the dataset's schema, and the researcher interacts only with the parameterized interface — they can adjust parameters but cannot inject arbitrary SQL.

28.7 Soft Deletion and Audit Trail

All entities managed by the Portal Service use soft deletion: removing a dataset, attribute, tag, or configuration entry sets a `deleted_at` timestamp rather than physically removing the row. Soft-deleted records are excluded from normal queries but remain in the database for audit, compliance, and potential recovery.

This design reflects the platform's orientation toward regulated environments where data lineage and traceability are non-negotiable. In clinical research settings, the ability to demonstrate what metadata existed at a given point in time — and who changed it — is often a regulatory requirement. Soft deletion ensures that deleting a dataset's metadata does not destroy the evidence that the dataset once existed and was described in a particular way. Recovery is also straightforward: clearing the `deleted_at` timestamp restores the record to active status without any data reconstruction.

Every entity also carries a standard audit envelope — creation and modification timestamps plus the usernames of the creator and last modifier. Combined with soft deletion, this means the Portal maintains a complete provenance record: who created a dataset, who last changed its metadata, when the change occurred, and when (and by whom) it was deleted. These fields are available to administrators through the management interfaces and can be queried for compliance reporting. In deployments subject to audit requirements (GxP [72], HIPAA [71], institutional review board oversight), this built-in provenance eliminates the need for external audit logging infrastructure for metadata operations.

28.8 Integration Points

The Portal is read by virtually every other cloud function on virtually every request — dataset metadata lookups, study resolution, configuration reads, and tenant membership checks all

flow through it. Writes come from a smaller set of authoritative callers: User Management for user lifecycle events, the Dataset Service for dataset lifecycle events, and administrative tooling for configuration and catalog edits. The Bookmark Service uses the Portal as its persistence backend, treating it as a general-purpose metadata store rather than maintaining its own database.

28.9 Failure Modes

A Portal outage degrades the entire platform within a request cycle. Cache layers in the upstream services buy minutes of grace — recently-resolved metadata stays usable while the Portal is reachable again — but a sustained outage takes new requests offline as caches expire and the next call to the Portal fails. The riskiest deployment events on the platform are schema migrations to the Portal database: because so many services depend on the shape of its tables, an incompatible migration can break everything that reads from the Portal at once. Migrations are accordingly treated with disproportionate care, with backwards-compatible intermediate steps preferred over in-place schema rewrites.

29 D2E WebAPI

29.1 Purpose

The D2E WebAPI exists for one reason: OHDSI Atlas [44] compatibility. The OHDSI ecosystem — Atlas for cohort building, Hades [46] for analytical methods, Strategus [48] for orchestrated studies — speaks a particular HTTP dialect, the OHDSI WebAPI surface, and these tools are the lingua franca of observational research on the OMOP CDM. The D2E WebAPI implements that surface (cohort definitions, concept sets, vocabulary endpoints, generation status) against D2E's own data layer, so that OHDSI tools can talk to a D2E platform without translation, shims, or data export.

The cost of building a parallel API surface is significant; the benefit is that the entire OHDSI tool ecosystem becomes available to D2E users without modification. A researcher pointing Atlas at a D2E deployment sees their studies, their concept sets, and their cohorts exactly as Atlas would render them against a stock OHDSI WebAPI — with the difference that authentication, dataset scoping, and access control are all governed by D2E's rules rather than the OHDSI defaults.

29.2 Purpose and Role

The D2E WebAPI is the platform's bridge to the OHDSI ecosystem. It exposes an OHDSI-compatible interface that allows OHDSI front-end tools—most notably Atlas—to operate against D2E-managed datasets without modification. By implementing the same contract that Atlas expects from WebAPI, D2E can reuse the rich OHDSI UI for cohort building, concept exploration, and vocabulary browsing while keeping all data access under D2E's governance and multi-tenant security model.

The service runs as a Deno [8] cloud function inside Trex and is reached through the `/d2e-webapi` route. Every request passes through D2E's standard authentication and dataset-scoping middleware before reaching any business logic. The WebAPI also serves internationalisation strings and system notifications consumed by the Atlas front end, so that the UI can be presented in the user's locale and display platform-level alerts.

29.3 Cohort Definitions

A *cohort definition* is a declarative, JSON-based description of a patient population. It specifies inclusion criteria (e.g. "patients with at least two condition occurrences of Type 2 Diabetes within 365 days") using the OHDSI cohort expression language. D2E stores the expression alongside metadata such as name, description, and authoring source.

Atlas vs. Patient Analytics sources. Cohort definitions enter the system from two paths. *Atlas-authored* cohorts are built interactively in the Atlas cohort builder and saved through the WebAPI. *Patient Analytics (PA) cohorts* are created programmatically by D2E's own analytics pipelines. The WebAPI tags each definition with its source so that downstream tools can filter appropriately—Atlas displays only its own cohorts, while the analytics UI shows PA cohorts.

Generation workflow. Creating a cohort definition does not immediately produce a patient list. Generation is a separate, asynchronous step:

1. The user selects a target data source (identified by a *sourceKey*) and triggers generation.
2. The WebAPI converts the cohort expression into parameterised OHDSI SQL.

3. It submits the SQL execution as a Prefect [55] flow run through the JobPlugins service. Prefect handles scheduling, retry logic, and status tracking.
4. On completion the resulting patient set and record counts are written to the CDM results schema. The WebAPI then exposes generation status and summary statistics back to the caller.

Versioning and validation. Every update to a cohort definition's expression is versioned, allowing users to review or revert to earlier iterations. Before generation, the expression can be validated to catch structural errors or references to missing concepts. Cohort definitions can also be duplicated—creating a copy with a new identifier and a "(Copy)" suffix—so that users can fork an existing definition as the starting point for a variant without altering the original.

29.4 Concept Sets

A *concept set* is a reusable collection of OMOP concepts used as building blocks inside cohort definitions. Each item in the set carries flags that control resolution:

- **Include / Exclude** — whether the concept adds to or subtracts from the resolved set.
- **Descendants** — whether all descendant concepts in the vocabulary hierarchy should be pulled in.
- **Mapped** — whether source-to-standard mappings should be followed.

Because concept sets are shared across cohort definitions, the WebAPI enforces *deletion protection*: a concept set that is referenced by any cohort definition cannot be deleted. A dependency check runs before every deletion attempt, returning a conflict if references exist. This prevents silent breakage of downstream cohorts.

29.5 Vocabulary Access

The WebAPI provides vocabulary browsing capabilities scoped to a particular CDM source (identified by `sourceKey`). Key capabilities include:

- **Free-text and structured search** — users can search concepts by name, code, domain, vocabulary, or standard-concept status. Search results include record counts from the CDM so users can assess concept prevalence.
- **Hierarchy navigation** — given a concept, the service retrieves its ancestors and descendants via the `CONCEPT_ANCESTOR` table, enabling users to understand where a concept sits in the classification tree.
- **Relationship exploration** — related concepts are surfaced through `CONCEPT_RELATIONSHIP`, showing mappings, parent/child links, and cross-vocabulary equivalences.
- **Concept set resolution** — a concept set expression is expanded into a flat list of concept IDs by applying the include/exclude/descendant logic described above. This is the critical step that turns a human-readable concept set into the concrete ID list used in SQL generation.
- **Record counts** — for a given list of concept IDs the service returns the number of records in the CDM that reference each concept. This helps researchers gauge data availability and assess whether a concept is well-represented in the dataset before building a cohort around it.

The WebAPI also exposes metadata about the configured CDM sources—connection details, vocabulary versions, supported domains, and available vocabularies—so that front-end tools can present source-selection dialogs and display version information to users.

29.6 The TrexDAO Database Abstraction

All database access in the WebAPI goes through **TrexDAO**, a data-access layer that presents a single query interface regardless of the underlying engine. TrexDAO supports two backends:

- **TrexSQL** — the analytical database layer used for fast, local vocabulary queries. TrexSQL holds a replica of the vocabulary tables and is the default backend for search and hierarchy operations.
- **SAP HANA** — the authoritative CDM store used for cohort generation, record-count queries, and any operation that must reflect the latest clinical data.

TrexDAO selects the appropriate backend based on the operation type and the dataset's configuration. This dual-backend design lets the WebAPI serve interactive vocabulary exploration at low latency (via TrexSQL) while still executing heavy analytical queries against the full HANA-backed CDM.

The core vocabulary tables accessed through TrexDAO are `CONCEPT` (the master concept catalogue with names, domains, vocabularies, and validity periods), `CONCEPT_RELATIONSHIP` (pairwise relationships such as "Maps to" and "Is a" between concepts), and `CONCEPT_ANCESTOR` (the pre-computed transitive closure of hierarchical relationships, recording minimum and maximum levels of separation). These three tables together power search, hierarchy navigation, and concept set resolution.

29.7 Service Dependencies

The WebAPI delegates to several sibling services within the platform:

- **PortalServerAPI** — retrieves dataset metadata and tenant configuration.
- **AnalyticsAPI** — stores and retrieves cohort generation results and analysis artefacts.
- **JobPluginsAPI** — submits Prefect flow runs for asynchronous work such as cohort generation.
- **BookmarksAPI** — manages user-level bookmarks on cohort definitions and concept sets.
- **TerminologySvcAPI** — delegates complex vocabulary operations (hybrid search, FHIR expansion) to the dedicated Terminology Service.

29.8 Integration Points

The WebAPI is read by external OHDSI tools — the Atlas web UI, R packages such as the Hades suite, and study orchestration frameworks like Strategus. Internally it reads from the OMOP vocabulary tables (`CONCEPT`, `CONCEPT_RELATIONSHIP`, `CONCEPT_ANCESTOR`) and from the platform's cohort storage, both via TrexDAO. When a cohort materialization is requested through OHDSI conventions, the WebAPI coordinates with the Analytics Service: the heavy lifting of cohort generation is delegated through the JobPlugins API to a Prefect flow, while results are persisted in the results schema where the Analytics Service can also reach them.

29.9 Failure Modes

The most common failure mode is a vocabulary version mismatch: Atlas expects a particular OHDSI vocabulary release and D2E has not yet ingested it. Concept lookups silently return fewer results than the researcher expects, or a cohort definition that references a newer concept fails to resolve. Diagnosis usually requires comparing the `vocabulary_version` reported by D2E against the version Atlas was built against. Cohort definition compatibility is generally robust — the cohort expression schema changes slowly — but can break across major OHDSI WebAPI versions, which has historically required coordinated upgrades of the WebAPI implementation alongside Atlas.

30 Terminology Service

30.1 Purpose

The Terminology Service is the platform's runtime vocabulary lookup engine. It exists because every analytical query in D2E ultimately needs to translate between user-friendly clinical terms ("Type 2 Diabetes") and OMOP standard concept identifiers (`201826`), and that translation must be fast, accurate, and tolerant of partial matches. The service implements a hybrid search that combines SQL-based exact matching against the OMOP vocabulary tables with semantic similarity search for fuzzy queries. Without it, every autocomplete in the UI would return either too few candidates (missing legitimate clinical synonyms) or too many (drowning the user in unrelated near-matches).

The service is implemented in Express and mounted at `/terminology`. It is called both by the D2E WebAPI (which proxies vocabulary requests from Atlas [44]) and by other platform services that need concept lookup, mapping, or FHIR terminology capabilities. While the D2E WebAPI exposes OHDSI-compatible vocabulary endpoints, the heavier operations—semantic search, concept set resolution, and FHIR terminology—are delegated to this service, which is purpose-built for those workloads.

30.2 Dual-Backend Architecture

Like the WebAPI, the Terminology Service operates against two database backends, but it makes more deliberate use of each one depending on the access pattern:

- **TrexSQL** — the preferred backend for interactive queries. TrexSQL holds a local copy of the vocabulary tables plus pre-computed vector embeddings for semantic search. Because TrexSQL operates on columnar storage, it delivers sub-second response times for most search and hierarchy queries.
- **SAP HANA** — the authoritative backend. HANA is used when the query must reflect the most current CDM state, or when TrexSQL has not yet been populated for a given dataset. It is also the fallback when a query exceeds what TrexSQL can handle (e.g. very large descendant expansions on deeply nested hierarchies).

The service selects the backend transparently based on dataset configuration and query characteristics. Users and calling services do not need to specify which backend to target.

This architecture also provides resilience: if the HANA backend is temporarily unavailable for maintenance, vocabulary search and concept exploration can continue to operate against TrexSQL for datasets whose vocabulary data has been cached. Conversely, newly onboarded datasets that have not yet had their vocabulary replicated to TrexSQL are served directly from HANA until the cache is populated.

30.3 Concept Search

Concept search is the most frequently used capability of the Terminology Service, invoked every time a user types into the concept search bar in Atlas or the D2E analytics UI. It follows a three-phase pattern: *filter*, *search*, and *resolve*.

1. **Filter.** The caller first retrieves the available filter dimensions—domains, vocabularies, concept classes, standard-concept flags, and validity statuses. These dimensions are computed from the dataset's actual vocabulary content, so they reflect only concepts that exist in the data.

2. **Search.** The caller submits a search request that combines a free-text query string with any combination of the above filters. The service executes the search and returns matching concepts with metadata (concept name, domain, vocabulary, concept class, standard status, and concept code).
3. **Resolve.** For use in cohort building, the caller can search by concept ID or concept code directly, bypassing free-text matching. This is used when importing concept lists from external systems or when programmatically assembling concept sets.

Search also supports retrieving recommended standard concept mappings for non-standard source concepts, which is essential during ETL workflows where source codes must be mapped to the OMOP standard vocabulary.

Additionally, the service provides hierarchy exploration: given a concept, it returns the full ancestor and descendant tree drawn from `CONCEPT_ANCESTOR`. This lets users navigate from a broad clinical category (e.g. "Cardiovascular disease") down to specific diagnoses, or from a specific code up to its parent classifications—an essential capability when deciding how broadly or narrowly to define a concept set.

30.4 Hybrid Search Strategy

The free-text search phase uses a hybrid strategy that combines two complementary techniques:

Semantic search. Pre-computed vector embeddings are stored alongside concept records in TrexSQL. When a user searches for a term like "heart attack," the query is embedded into the same vector space and a nearest-neighbour lookup retrieves conceptually similar concepts—including synonyms like "myocardial infarction" and related terms across vocabularies. Semantic search excels at handling vocabulary variation and ambiguous or colloquial queries.

Traditional SQL matching. In parallel, the service runs pattern-based matching using SQL string functions (LIKE patterns and trigram similarity). Traditional matching provides high precision for exact concept names and codes, ensuring that a search for "ICD10CM 410.9" returns the exact concept rather than a semantically similar neighbour.

Result merging. Results from both strategies are merged into a single ranked list. The combined relevance score for a concept c given query q is:

$$\text{score}(q, c) = \alpha \cdot \text{cosine}(\mathbf{e}_q, \mathbf{e}_c) + (1 - \alpha) \cdot \text{trigram}(q, c) \tag{13}$$

where $\mathbf{e}_q$ and $\mathbf{e}_c$ are the query and concept embedding vectors, $\text{trigram}(q, c)$ is the SQL-level string similarity score, and α is a weighting factor that shifts toward semantic matching for natural-language queries and toward traditional matching for code-like queries. This combined approach ensures both recall (finding relevant concepts the user may not have known the exact name of) and precision (exact hits rank first).

The hybrid strategy is particularly valuable in the OMOP vocabulary context, where the same clinical idea may appear under dozens of names across different source vocabularies. A traditional search for "heart failure" would miss the SNOMED [65] synonym "cardiac insufficiency" or the ICD-10 [77] description "congestive heart failure, unspecified," but the semantic component surfaces these as closely related results.

30.5 Concept Set Resolution

Concept set resolution is the process of expanding a concept set expression into the concrete list of concept IDs it represents. The resolution algorithm applies the following logic for each item in the expression:

- If the item's **includeDescendants** flag is set, the service walks the `CONCEPT_ANCESTOR` hierarchy to collect all descendant concept IDs.
- If the item's **includeMapped** flag is set, source-to-standard mappings from `CONCEPT_RELATIONSHIP` are followed.
- Items marked as **excluded** are subtracted from the accumulated set after all inclusions have been processed.

The resolved set can be returned either as a flat list of concept IDs (for use in SQL generation) or as full concept records (for display in the UI). A count-only mode is also available for quick cardinality checks without transferring the full result.

Resolution is used in two main contexts. First, during cohort SQL generation, the WebAPI resolves every concept set referenced in the cohort expression so that the generated SQL contains concrete `concept_id` lists. Second, during interactive concept set editing in the UI, resolution is triggered on each change so the user can see exactly which concepts their expression captures and adjust inclusion/exclusion flags iteratively.

30.6 Integration Points

The Terminology Service sits in the read path of nearly every part of the platform that touches vocabulary. The Patient Analytics frontend calls it during autocomplete in the cohort and concept-set builders. The FHIR services call it whenever a coded value needs to be translated between source vocabularies and OMOP standard concepts. The Concept Mapping Service calls it during data onboarding so that suggested mappings are drawn from the same vocabulary view that researchers will later see at query time.

On the data side, the service reads from the OMOP vocabulary tables—`concept`, `concept_relationship`, and `concept_synonym`—together with the pre-computed embedding index that backs semantic search. It does not write to these tables; vocabulary refreshes are the responsibility of the Concept Mapping Service and the data-onboarding ETL flows.

30.7 Failure Modes

The dominant failure mode is vocabulary staleness. A researcher searches for a recently-added concept that the platform has not yet ingested—perhaps a new ICD-10 code or a SNOMED revision—and gets either no result or a semantically-adjacent older concept. The semantic-search fallback is a double-edged sword here: because it always returns *something* plausible, it can mask under-coverage that a strict-match service would have surfaced as an empty result. Operators should monitor the gap between OMOP vocabulary releases and platform updates as a first-order operational metric.

A subtler failure is corruption or drift in the semantic-search index. If the vector index is rebuilt from a partial extract, or if the embedding model is changed without re-embedding the vocabulary, queries silently return low-quality matches—the API still responds with concepts, ranked by score, and nothing in the response signals that the index is degraded. This is the hardest failure to detect from inside the platform; periodic ground-truth checks against a fixed query set are the most reliable guard.

31 Concept Mapping Service

31.1 Purpose

The Concept Mapping Service is the bridge between source data vocabularies and the OMOP standard vocabulary. Source clinical systems use whatever code sets they happen to use — ICD-9 in some hospitals, ICD-10 [77] in others, in-house code dictionaries everywhere — and analytical queries written against OMOP cannot run until those source codes have been mapped to OMOP standard concepts. This service is what turns "the source said E11.9" into "the OMOP concept ID is 201826."

It exists separately from the Terminology Service because the two have different audiences and different lifecycles. The Terminology Service is read-heavy and serves runtime vocabulary lookups for analytical queries. The Concept Mapping Service is write-heavy during data onboarding and is used by data engineers and ETL flows, not by researchers running queries. Splitting the two keeps both APIs focused.

31.2 Mapping Lifecycle

A typical mapping lifecycle has three phases. During *discovery*, the service ingests the source data's distinct code values and looks for any existing maps in OMOP's USAGI [49]-derived mapping tables. During *review*, codes without an existing mapping are queued for a clinical informaticist; the Terminology Service returns the nearest standard concepts by embedding similarity as a short candidate list, and the reviewer selects the best fit. During *publication*, reviewed mappings are written into the OMOP vocabulary tables so that downstream queries see them.

The service exposes endpoints for each phase, plus bulk operations for ETL pipelines that need to translate large volumes of source data in one pass.

31.3 What Source-to-Concept Mapping Is

In the OMOP CDM, every clinical event — a diagnosis, a lab result, a medication — is represented by a standard concept drawn from curated vocabularies like SNOMED [65], RxNorm, or LOINC [58]. However, source databases almost never use these standard concepts natively. A hospital system might record diagnoses as ICD-10 codes, medications as NDC codes, and lab tests using internal identifiers that exist nowhere outside that institution. Source-to-concept mapping is the process of defining how each of these local codes translates to its corresponding OMOP standard concept.

The stakes of this translation are high. The entire value proposition of the OMOP CDM rests on standardization: once data from multiple hospitals is mapped to the same vocabulary, it can be queried uniformly, compared across sites, and analyzed with shared tooling. A poorly mapped dataset undermines this promise. If one hospital maps its asthma ICD-10 code (`J45.0`) to the correct SNOMED concept while another leaves it unmapped, cross-site queries for asthma patients will silently miss the second hospital's data. The Concept Mapping Service exists to make this translation process manageable, auditable, and repeatable.

These mappings are stored in the OMOP `source_to_concept_map` table and are applied during the ETL transformation step. When the ETL pipeline encounters a source code, it looks up the mapping to find the equivalent standard concept. Each mapping record captures the source vocabulary identifier (which vocabulary the code comes from), the source code itself, the target concept ID in the OMOP vocabulary, the target vocabulary name, and a validity period that scopes the mapping to a date range. The validity period allows mappings to evolve

over time — a code that mapped to one concept before a vocabulary update can map to a different concept afterward, with both mappings coexisting in the table.

31.4 The Mapping Lifecycle

The Concept Mapping Service manages the full lifecycle of these mappings for a given dataset, from initial creation through to application during ETL.

Retrieval. A client requests all current mappings for a dataset, filtered by database dialect. The service reads the `source_to_concept_map` table and returns the complete mapping set. This is typically used by the frontend to display mappings for review and editing, presenting researchers with a table of source codes and their current target concepts.

Authoring. Mappings can be created through several paths. The most direct is manual authoring through the UI, where a researcher or ETL engineer selects a source code and associates it with a target OMOP concept. The UI integrates with the vocabulary system during this process: as the user searches for a target concept, the service queries the OMOP concept tables to provide autocomplete results with concept names, IDs, vocabulary sources, and domain classifications. This vocabulary integration is critical for mapping quality — without it, users would need to know OMOP concept IDs by heart. A second path is embedding-based suggestion: the service asks the Terminology Service to encode the source code's name into the shared sentence-embedding space and to return the nearest standard concepts by cosine similarity. The candidates (typically the top three) are presented to the user, who selects the best fit. The suggestion is never applied blindly — the embedding lookup narrows the search space; the human makes the mapping decision. A third path is bulk import, where a pre-built mapping set (perhaps from a previous ETL engagement or a shared mapping library) is loaded into the service.

Persistence. When mappings are saved, the service performs a full replacement rather than incremental updates. It deletes all existing mappings for the specified source vocabulary in the target dataset, then batch-inserts the new complete set. This atomic replacement strategy avoids a class of bugs that plague incremental approaches: partial updates that leave the mapping table in an inconsistent state where some codes reflect the old mapping and others the new one. After a save operation, the mapping table always reflects a single, consistent, complete mapping set for that vocabulary. The tradeoff is that every save transmits the full mapping set, even if only one entry changed — but for mapping tables, consistency is more valuable than bandwidth efficiency.

Application. During ETL execution, the transformation pipeline reads these mappings to translate source codes into standard concepts as data flows from source tables into OMOP CDM tables. The Concept Mapping Service is not directly involved in this step — the ETL pipeline reads the `source_to_concept_map` table directly — but the service's lifecycle management ensures that the table contains correct, complete, and current mappings when the ETL runs.

31.5 Compressed Payload Handling

Mapping sets can be large. A single source vocabulary might contain tens of thousands of code mappings — ICD-10 alone has over 70,000 codes, and institutional lab systems can have thousands of proprietary test identifiers. Transmitting these mapping sets as uncompressed JSON can exceed HTTP request size limits and introduce significant network latency.

The service addresses this with a transparent compression protocol. On the client side, the mapping array is serialized to JSON, compressed using zlib deflate (which typically achieves

85–95% size reduction on repetitive structured data like mapping tables), and then base64-encoded to produce a string safe for transmission in a JSON request body. The base64 encoding step is necessary because raw deflate output is binary data that cannot be embedded directly in JSON.

On the server side, the service detects compression through either a Content-Encoding header or a flag in the request body. When compression is detected, the service reverses the process: it base64-decodes the payload to recover the binary deflate stream, decompresses it to recover the original JSON string, and parses the JSON to obtain the mapping records. From this point forward, processing continues identically to an uncompressed request. The detection mechanism means clients can choose whether to compress based on payload size — small mapping sets can be sent uncompressed without any protocol negotiation.

31.6 Multi-Dialect SQL Generation

The service supports multiple database backends — currently SAP HANA and Trex — and generates dialect-appropriate SQL for the persistence operations. This matters because the DELETE and batch INSERT statements that implement the atomic replacement strategy use syntax that differs between databases.

For HANA, the DELETE statement uses standard SQL syntax targeting the `source_to_concept_map` table with a WHERE clause filtering by source vocabulary. The batch INSERT uses HANA's multi-row INSERT syntax, which allows multiple value tuples in a single statement. For Trex, the same logical operations use Trex-specific syntax for bulk loading.

The dialect is specified as a parameter on each request, allowing the same service instance to manage mappings across datasets stored in different database engines. This is a practical necessity in deployments where some datasets reside in HANA (the primary analytical database) while others use Trex (for lightweight local processing or as a caching layer). The service does not maintain persistent database connections — it generates the SQL statements and delegates execution to the appropriate database client based on the dialect parameter.

31.7 Integration with the Vocabulary System

The Concept Mapping Service does not operate in isolation from the OMOP vocabulary tables. During mapping authoring, when a user needs to find the correct target concept for a source code, the service queries the OMOP concept tables (`concept`, `concept_relationship`, `concept_synonym`) to support search and lookup. This integration allows the mapping UI to display concept names alongside IDs, show related concepts that might be better mapping targets, and validate that a selected target concept is a standard (not deprecated or non-standard) concept.

This vocabulary dependency means that the quality of available mappings is bounded by the vocabulary version loaded into the dataset. If the OMOP vocabulary tables have not been updated recently, newly added concepts will not appear as mapping targets. The Terminology Service (which manages vocabulary loading) and the Concept Mapping Service are thus complementary: the Terminology Service ensures the vocabulary is current, and the Concept Mapping Service uses that vocabulary to support accurate mapping authoring.

31.8 Integration Points

The Concept Mapping Service is called primarily by ETL Prefect flows during data onboarding and by the Concept Mapping frontend during human review, with the Terminology Service

queried inline for embedding-based candidate retrieval. It writes into the OMOP vocabulary tables, which are then read by the Terminology Service for runtime lookups. It does not participate in the analytical query hot path.

31.9 Failure Modes

The dominant failure mode is silent under-mapping: a source code that has no OMOP equivalent is mapped to OMOP's "no matching concept" sentinel, and queries downstream produce results that exclude legitimate clinical events. The service logs every such mapping for review, and a Prefect flow runs nightly to surface them, but operators should treat sustained growth in the no-match log as a warning that an ETL is feeding the platform code values it does not understand.

32 Job Plugins Service

32.1 Purpose

The Job Plugins Service is the platform's gateway in front of Prefect [55]. Direct Prefect API calls from analytics, dataset, and FHIR services would have worked technically — Prefect exposes a documented REST surface, and a thin client wrapper would have been trivial to write. That option was rejected. Prefect's API is unauthenticated by default and assumes its caller is trusted: it has no concept of platform tenants, no notion of per-user scope checks, no built-in retry semantics tied to platform job vocabulary, and no mechanism for enforcing per-tenant concurrency limits. Pointing production traffic at it directly would have meant either trusting every caller equally or duplicating authorization, retry, and quota logic into every service that submits jobs.

The decision is therefore a gateway service that translates platform job submissions into Prefect deployment runs, applying authentication, retry semantics, scope checks, and per-tenant concurrency limits before forwarding to Prefect. The Job Plugins Service is also where the platform's *job vocabulary* is defined: "characterize this dataset," "run this Strategus [48] study," "ingest this CSV," "generate this cohort." Each of these maps to one or more Prefect deployments, but the platform's API speaks in business operations rather than orchestrator primitives. A caller asks for a Kaplan–Meier analysis; it does not ask for `deployment_id=fa3c...` with a particular parameter envelope. This indirection is an instance of principle 1 of 2: the platform composes domain operations over a stable interface rather than wiring callers directly into the orchestrator's vocabulary.

The cost of the gateway is the cost any gateway carries: an extra service, an extra hop, an extra abstraction layer between callers and the orchestrator. Operators have one more component to deploy and monitor, and a new Prefect feature is not available until Job Plugins exposes it through the platform's vocabulary. In a system that ran one job a day this overhead would not be worth paying.

The benefit is that services like the Analytics Service, which submits Kaplan–Meier and other survival jobs, do not need to know that Prefect exists. They call Job Plugins with a domain operation and a dataset reference, and receive a job handle. Authentication tokens are scoped at submission time so that the container Prefect schedules can call back into D2E with only the permissions it needs — consistent with the credential discipline of principle 2. What the service forbids is other components reaching past it into Prefect; what it allows is that replacing Prefect with another orchestrator is a finite project rather than an open-ended one. It would require rewriting Job Plugins, not the entire platform.

32.2 The Orchestration Gateway

The Job Plugins Service is the central orchestration hub for all asynchronous and long-running operations in D2E. Many platform operations — cohort generation, ETL execution, data quality checks, Strategus analyses — take minutes or hours to complete. These cannot be handled synchronously within an HTTP request/response cycle. The Job Plugins Service bridges this gap: it accepts synchronous API calls from other services and the frontend, translates them into asynchronous Prefect flow runs, and returns identifiers that callers use to track progress.

This gateway exists because of a fundamental architectural tension: the rest of the platform speaks synchronous HTTP, while the work itself is inherently asynchronous and container-based. Without this service, every D2E component that needs to launch an asynchronous operation would have to understand Prefect's API, know the correct deployment names, construct parameter payloads in the format each flow expects, and manage its own token issuance for

callback authentication. That knowledge would be duplicated across the Portal, the cohort service, the ETL pipeline, and every future service that needs to trigger a flow. The Job Plugins Service eliminates this duplication by acting as the single translation layer between the synchronous HTTP world and the asynchronous Prefect world. Other services make a simple REST call describing what they want; the Job Plugins Service handles all the mechanics of how it gets executed.

This design also provides a single enforcement point for cross-cutting concerns. Authentication token scoping, parameter validation, deployment existence checks, and audit logging all happen in one place rather than being scattered across every service that might launch a flow.

32.3 Flow Execution Lifecycle

Every asynchronous operation follows the same lifecycle:

1. **Request** — A service or the frontend sends an HTTP request specifying the operation type, target dataset, and operation-specific parameters.
2. **Validation** — The appropriate controller validates input parameters against the JSON Schema registered for that operation in the plugin registry. This schema-based validation catches malformed requests before they reach Prefect, preventing flow runs that would fail immediately due to missing or ill-typed parameters. The controller also confirms that the target dataset exists and verifies that the caller has the required permissions.
3. **Deployment Lookup** — The controller queries the plugin registry to find the Prefect deployment that implements the requested operation. The lookup is keyed by flow type: the registry maps each logical operation name to a specific Prefect deployment ID. If the deployment does not exist or has been deactivated, the request is rejected with a descriptive error rather than silently failing inside Prefect.
4. **Flow Run Creation** — The controller submits a new flow run to Prefect with the operation parameters, including a scoped authentication token for callbacks. Prefect schedules the run, assigns it a unique `flow_run_id`, and manages container provisioning.
5. **Return** — The Prefect `flow_run_id` is returned to the caller immediately. The actual work proceeds asynchronously in a container managed by Prefect. This identifier becomes the handle that all downstream components use to refer to the operation: the frontend stores it alongside the user's request, the status polling mechanism queries Prefect by this ID, and WebSocket status updates include it so the frontend can match notifications to the correct in-progress operation.
6. **Status Tracking** — The frontend polls Prefect's API for status updates using the `flow_run_id`, or receives progress notifications via WebSocket. State transitions (Pending, Running, Completed, Failed) are surfaced in the UI so the user knows when results are ready or when an operation requires attention.

32.4 Plugin Registry

The service maintains a registry of available operations in the `trex.plugins` database table. Each row maps a logical operation name to its Prefect flow and deployment, along with a JSON Schema defining the expected parameters. At startup, the service verifies that all registered plugins have corresponding Prefect deployments, ensuring that no operation can be dispatched to a non-existent flow.

The registry is populated during platform initialisation by the `alp-dataflow-gen-init` process. This process scans the flows directory, creates deployment records in Prefect for each

discovered flow, and registers corresponding entries in `trex.plugins` with the deployment IDs and parameter schemas. This scan-and-register pattern makes the system genuinely extensible: adding a new asynchronous operation to the platform requires writing the flow code, placing it in the flows directory, and running the initialisation process. The new operation becomes available through the Job Plugins Service without any code changes to the service itself — the registry-based dispatch means the service does not contain hardcoded knowledge of which operations exist.

The JSON Schema stored alongside each plugin entry serves double duty. At request time, incoming parameters are validated against the schema before the flow run is created. At discovery time, the frontend can read the schema to understand what parameters each operation accepts, enabling dynamic form generation for operation configuration.

32.5 Controller Organisation

The service is organised into controllers that group related operations. Key controllers include:

- **CohortController** — Cohort generation and materialisation against CDM sources.
- **DqdController / DataCharacterizationController** — OHDSI Data Quality Dashboard [45] checks and Achilles [43]-based data profiling.
- **DataTransformationController** — ETL and source-to-OMOP data loading flows.
- **DataModelFlowController / DbSvcController** — OMOP schema provisioning, migration, and validation.
- **WhiteRabbitController / PerseusController** — Source database profiling and mapping visualisation, fronting White Rabbit [50] and Perseus [47].
- **StrategusResultsController / AnalysisController** — Strategus analysis execution and result retrieval.
- **CachedbController** — TrexSQL cache management: creation, refresh, and invalidation.
- **PrefectController** — Direct access to Prefect metadata (flow listings, deployment status, run history).

The reason for this fine-grained controller structure — rather than a single generic "run any flow" endpoint — is both security and correctness. Each controller understands the parameter semantics for its category of operations. The CohortController knows that a cohort generation request must include a valid cohort definition ID and a target data source; the DataTransformationController knows that an ETL request must reference an existing mapping specification. A generic endpoint would have to accept arbitrary JSON and pass it through without domain validation, creating a security risk: a malformed or malicious request could reach Prefect and consume resources before failing, or worse, execute with unintended parameters. Domain-specific controllers enforce that each flow receives exactly the parameters it expects, with the correct types and valid references.

All controllers share the same execution pattern described above — they differ only in the parameters they accept, the validation they perform, and the Prefect deployment they target.

32.6 Authentication Token Management

When the Job Plugins Service creates a Prefect flow run, the flow's container needs to call back into D2E services (for example, to update cohort status or store results). These containers run as separate Docker processes without access to the original user's HTTP session or bearer

token. To enable secure callbacks, the service generates a scoped access token and includes it in the flow run parameters.

These tokens follow the principle of least privilege: each token grants only the permissions required for the specific operation. A cohort generation flow receives a token that can update cohort status and write cohort membership records, but cannot modify dataset configurations or access unrelated studies. A data quality flow receives a token that can write DQD results, but cannot trigger ETL runs. This scoping is determined by the controller at submission time based on the operation type.

Token lifetime is matched to the expected duration of the flow. A cohort generation that typically completes in minutes receives a short-lived token; an ETL pipeline that may run for hours receives a longer-lived one. Once the flow completes or the token expires — whichever comes first — the token can no longer be used. This prevents a compromised or unexpectedly long-running container from retaining broad access to the platform. If a flow exceeds its expected duration and the token expires, the flow fails with an authentication error rather than silently continuing with stale credentials.

32.7 Integration Points

The Job Plugins Service is called by the Analytics Service for Kaplan–Meier and other survival jobs, by the Dataset Service for ETL flows, and by the FHIR services for caching and bulk-export jobs. It speaks downstream to Prefect: it submits flow runs to Prefect's API, polls Prefect for run status, and translates Prefect's run records back into the platform's job-status vocabulary. Result artifacts produced by flows — KM curve data, characterisation reports, ETL logs — are retrieved from Supabase Storage [66] via the Files Manager, not from Prefect itself.

The service does not own any analytical state of its own. It is a translation layer between the synchronous HTTP world above it and the asynchronous Prefect-and-storage world below.

32.8 Failure Modes

Three failure modes dominate. First, Prefect-server unavailability appears at the API surface as job-submission failures: new runs cannot be created, but in-flight runs continue to execute against their workers and complete normally. Status polling returns stale data until Prefect is reachable again. Second, worker-pool exhaustion causes runs to queue indefinitely. From the caller's perspective the run is "Pending" and looks healthy; in reality no worker has picked it up. Operators should monitor pending-state dwell time, not just submission success.

The most opaque failure is when a flow itself errors silently — a Python exception swallowed inside a task, or a container that exits before reporting state — and never reports back. The polling timeout (30 minutes by default) is the safety net that eventually marks such runs as failed, but the stuck-job problem can mask broader infrastructure issues such as worker crashes or network partitions, and a 30-minute window is long enough that the underlying cause has often disappeared by the time the alert fires.

33 OHDSI Tools

33.1 Purpose

This chapter aggregates three loosely related OHDSI-ecosystem tools — White Rabbit [50], Perseus [47], and Strategus [48] — under one roof. They are described together not because they share business logic, but because they share a deployment model and an authentication path within D2E: each runs as its own containerised service, each is launched through the Job Plugins Service and tracked through Prefect [55], and each authenticates against the platform's token-scoping mechanism in the same way. Beyond that, the tools have very little in common.

White Rabbit profiles source databases ahead of an ETL. Perseus visualises ER models for those same source schemas. Strategus runs standardised epidemiological studies against an OMOP dataset that has already been built. Each is described in its own subsection below; the shared infrastructure is the only thread that ties them together within D2E.

D2E integrates these OHDSI-ecosystem tools to support the data-engineering and analysis pipeline. Each tool runs as a separate service within the platform and communicates through Prefect for asynchronous work and the Files Manager for artefact storage.

33.2 White Rabbit

What It Does

White Rabbit profiles a source database and produces a *scan report*—a structured description of every table and column in the source schema. The scan report captures table row counts, column data types, value frequency distributions, and completeness statistics. This profile gives data engineers the information they need to plan an ETL from the source format into the OMOP CDM: which source columns contain clinical codes, what vocabularies are present, how complete the data is, and where quality issues may lurk.

The scan report is the primary input to source-to-OMOP mapping work, and is consumed by downstream ETL mapping tooling.

Asynchronous Scan Workflow

Database profiling can be time-consuming—scanning a large clinical database with hundreds of tables and millions of rows is not a request–response operation. White Rabbit therefore runs scans asynchronously through the following flow:

1. The user provides source database connection parameters (host, port, schema, credentials). A lightweight connectivity test validates that the database is reachable before committing to a full scan.
2. The service submits a scan job to the **JobPlugins** service, which creates a Prefect flow run. The caller receives a *conversionId* for tracking.
3. Prefect schedules the flow, which launches the White Rabbit Java process inside a Docker container that has network access to the source database. The Java process iterates over every table in the target schema, collecting column metadata and sampling value distributions.
4. On completion, the scan report is serialised to an Excel workbook and uploaded to the **Files Manager**. The conversion record is updated with the file reference so the report can be retrieved or downloaded later.

Users can poll the conversion status to check whether the scan is still running, has succeeded, or has failed. The final report is available both as structured data and as a downloadable Excel file for offline review.

White Rabbit tracks each scan as a conversion record that links the initiating user, the Prefect flow run identifier, and the resulting file reference. This record serves as the audit trail for the profiling operation and enables the UI to display scan history per dataset.

33.3 Strategus

What It Does

Strategus is the OHDSI framework for running standardised, reproducible epidemiological studies. It orchestrates multiple HADES [46] analytics modules—such as cohort diagnostics, incidence rate calculation, comparative cohort analysis, and self-controlled case series—within a single study definition. This lets researchers define a complete multi-method study as a single *analysis specification* and execute it consistently across one or more CDM data sources.

Analysis Specifications

An analysis specification is a JSON document that declaratively describes the study design. It identifies:

- Which HADES modules to run (e.g. CohortDiagnostics, CohortIncidence, CohortMethod).
- The target and comparator cohorts for each module.
- Module-specific parameters such as time-at-risk windows, outcome definitions, and covariate settings.
- The data source(s) against which the study should execute.

Researchers create and update analysis specifications through the D2E interface. The specification is stored and can be re-executed as data refreshes or study parameters evolve.

Results Visualisation with Shiny

Once a Strategus analysis completes, its results need to be explored interactively. D2E handles this by launching a per-study R Shiny viewer:

1. The user requests to view results for a specific study.
2. The service starts a dedicated Shiny container that loads the study's results and renders the appropriate OHDSI viewer application (e.g. cohort diagnostics explorer, incidence rate dashboard).
3. The Shiny application runs on an internal port and is proxied through Caddy [6] so that users access it seamlessly through their browser at the study's URL path.
4. When the user is finished, the container is stopped and cleaned up to free resources.

Each study gets its own isolated Shiny instance, preventing interference between concurrent users viewing different studies. The viewer code (R Shiny application source) is stored alongside the analysis specification, allowing custom visualisations to be bundled with specific study designs.

End-to-End Workflow

A typical Strategus workflow ties together several platform capabilities:

1. A researcher defines target and comparator cohorts using Atlas [44] (through the D2E WebAPI).
2. They compose an analysis specification that references those cohorts and selects the desired HADES modules.
3. The specification is submitted for execution. Strategus runs each module against the CDM, producing result artefacts.
4. The researcher opens the Shiny viewer to explore results—reviewing diagnostics, inspecting effect estimates, and iterating on the study design if needed.

This cycle can be repeated as cohort definitions evolve or new data becomes available, and the stored specification ensures full reproducibility of the analysis.

33.4 Integration Points

Each of the three tools integrates with D2E's authentication layer and with the dataset's database, but they touch different databases for different reasons. White Rabbit reads source databases for profiling; it needs read-only credentials for whatever JDBC-reachable system the data engineer points it at. Perseus also reads source databases, in its case for ER visualisation rather than statistical profiling. Strategus reads OMOP datasets and writes results back to the analytical database, where downstream Shiny viewers and the Analytics Service can consume them.

None of the three tools call each other. They share infrastructure — Prefect, the Files Manager, the authentication path — but their data flows are independent. A White Rabbit scan does not feed Perseus; a Perseus model does not feed Strategus. The integration is operational, not analytical.

33.5 Failure Modes

Each tool has its own failure surface. White Rabbit can run out of memory when profiling very wide tables: its sampling strategy holds per-column statistics in memory, and a source schema with thousands of columns or extreme cardinality will exhaust the JVM heap before the scan completes. Perseus has analogous Java-heap issues for very large schemas, surfaced as either an out-of-memory error or, worse, an unresponsive UI when the visualisation engine is asked to lay out too many tables at once.

Strategus runs are long — typically hours — and the most common failure is misconfiguration of the analysis specification rather than a runtime defect. An invalid covariate setting, a cohort reference that does not resolve, or a time-at-risk window that excludes all subjects will produce an error mid-flight, sometimes only after several modules have already executed. Because the error surfaces late, the cost of a misconfiguration is high; pre-flight validation of analysis specifications is the most effective mitigation.

34 AI Services

34.1 Purpose

This chapter covers the platform's newest functions, all of them rapidly evolving and all of them LLM-backed in some way. Three components live under the AI Services umbrella: the Code Suggestion Service, which provides AI assistance for analytical code and chat in notebooks; the MCP Server, which exposes platform tools to external LLM clients via the Model Context Protocol [3]; and the AI ETL mapping service (the Data Mapping Service), which drafts a source-schema-to-OMOP-CDM column mapping by prompting an LLM with the source schema and the target tables. The three are grouped together because they share the same dependency on external LLM providers, the same operational concerns around prompt and model drift, and the same maturity level relative to the rest of the platform. Concept mapping — the separate problem of turning a source vocabulary code into a standard OMOP concept — is not LLM-backed and is not covered here; it uses embedding-based nearest-neighbor suggestions from the Terminology Service, with a human reviewer choosing from the returned candidates (see Chapter 31).

This chapter is documented as a snapshot. The interfaces, prompts, and provider integrations described here are the ones in production at the time of writing, but a reader picking up the book a year from now should expect this material to have shifted — new providers, new tools exposed over MCP, and new prompts replacing older ones. The architectural shape (an agent loop, a tool-exposing MCP server, an LLM-driven mapping draft) is more durable than any specific model name or environment variable, and the rest of the chapter is best read with that distinction in mind.

34.2 Code Suggestion Service

The Code Suggestion Service provides AI-powered interactive chat for researchers working with CDM data. It is built on the LangChain agent framework, which allows the LLM to reason iteratively [75]: rather than producing a single response, the agent can plan a sequence of actions, invoke tools, observe results, and refine its answer before replying to the user.

Agent Architecture

A user message enters the service and is handed to a LangChain agent configured with a system prompt and a set of available tools. The agent operates in a ReAct (Reason + Act) loop [78], a pattern where the LLM alternates between thinking and acting. In each iteration, the agent first reasons about what it knows and what it still needs, then decides on an action — either calling a tool or producing a final response. If a tool is called, the agent observes the result, incorporates it into its context, and begins another reasoning step. This cycle continues until the agent determines it has enough information to answer the user's question.

The decision of when to call tools versus respond directly is governed by the system prompt and the agent's reasoning. The system prompt is tailored for the OHDSI ecosystem: it instructs the agent to prefer generating Strategus [48] R code and cohort definitions using OHDSI conventions, to consult the phenotype library before creating cohort definitions from scratch, and to use platform tools for CRUD operations rather than asking the user to perform them manually. This shaping means the agent behaves as a domain-aware assistant rather than a generic chatbot.

Multi-Model Support

The service abstracts the LLM provider behind a configuration layer controlled by the `AI_MODEL` environment variable. This variable selects both the provider and the specific model, and the service instantiates the appropriate LangChain LLM wrapper at startup. All providers support the same tool-calling interface, so the agent's behavior remains consistent regardless of which model backs it.

- **Anthropic Claude** [2] — frontier Claude models for high-quality code generation and clinical analysis guidance.
- **OpenAI / Azure OpenAI** — frontier OpenAI models, with the Azure variant available for enterprise compliance requirements. The Azure configuration accepts a separate endpoint URL and API version, allowing deployments to target organisation-specific Azure OpenAI instances.
- **Local Models** — an open-weights model for air-gapped deployments that cannot reach external APIs. Local model inference is served through Trex, which hosts the model weights and exposes an OpenAI-compatible API. This means the Code Suggestion Service connects to a local endpoint using the same client library it uses for cloud providers, with no code path differences.

The choice of provider is set at deployment time and is transparent to the user. Switching models requires only changing the environment variable and restarting the service; no code changes or prompt rewrites are needed, though the quality of tool-calling and code generation varies across models.

34.3 MCP Server

What MCP Is

The Model Context Protocol (MCP) is an open standard for giving LLMs structured access to external tools and data sources [61]. At the protocol level, MCP defines a JSON-RPC message format: the LLM sends a tool call request containing the tool name and typed parameters, the MCP server executes the operation, and returns a structured JSON response with the result. This is distinct from unstructured prompt engineering — instead of embedding API documentation in the prompt and hoping the LLM generates correct HTTP calls, MCP provides a formal contract. The LLM discovers available tools at runtime through a schema listing endpoint, understands their input/output types from JSON Schema definitions, and invokes them using the same structured calling convention regardless of what the tool does internally.

This decouples the AI capabilities from the underlying platform APIs. The LLM does not need to know HTTP endpoints, authentication details, or the internal structure of D2E services. It only sees named tools with typed parameters and receives typed responses. The MCP Server handles all the translation: it receives tool call requests from the LangChain agent, maps them to D2E backend API calls, executes those calls with appropriate authentication, and returns the results in the format the LLM expects.

Exposed Tools

The MCP Server exposes D2E platform operations as tools that the LangChain agent can call. These fall into three groups:

- **Cohort Operations** — CRUD operations on cohort definitions: creating a new cohort from a JSON expression, reading an existing definition by ID, updating its criteria, and deleting it. Beyond CRUD, the tools include triggering cohort generation against a specific data

source (which dispatches through the Job Plugins Service) and validating cohort expressions for syntactic and semantic correctness before generation. When the agent calls these tools, the MCP Server translates each call into the corresponding Portal API request, so the agent can manipulate cohorts without the LLM needing to know the Portal's REST endpoints or authentication scheme.

- **Phenotype Library** — Searching the OHDSI Phenotype Library for published cohort definitions by keyword, clinical domain, or concept ID, and retrieving specific phenotypes by their library identifier. This allows the agent to look up community-curated clinical definitions — for example, when a user asks "create a Type 2 Diabetes cohort," the agent can search the phenotype library first, find a validated definition, and adapt it rather than constructing one from scratch.
- **Strategus Analysis** — Creating Strategus analysis specifications by composing HADES modules (CohortGenerator, CohortDiagnostics, CohortIncidence, and others), executing them against a data source, and retrieving results. This lets the agent orchestrate full study workflows on behalf of the researcher: the user describes what they want to study in natural language, and the agent assembles the Strategus specification, triggers execution, and interprets the results.

Prompt Guidance

In addition to tools, the MCP Server defines prompts — blocks of guidance text injected into the LLM's context for specific tasks such as cohort definition authoring, data quality interpretation, and analysis planning. These prompts encode domain expertise: OHDSI conventions for cohort construction, OMOP CDM best practices for choosing the correct domain tables, and Strategus module configuration patterns. They shape how the agent approaches tasks so that it produces clinically sound outputs even when the underlying LLM has no specific medical training. For example, the cohort authoring prompt instructs the agent to use standard concept sets rather than individual concept IDs, to include descendant concepts for hierarchical vocabularies, and to apply appropriate observation period requirements.

34.4 Data Mapping Service

The Data Mapping Service uses LLM-powered semantic analysis to assist with the most labour-intensive part of ETL: mapping a source database schema to the OMOP CDM.

How AI-Assisted ETL Mapping Works

The service receives a JSON document describing the source schema — table names, column names, data types, and optionally sample values. It constructs a prompt that presents this source schema alongside the target OMOP CDM table definitions, then submits it to the configured LLM for semantic matching [31].

The LLM performs multi-signal reasoning to propose mappings. It analyzes column names for semantic similarity (a source column named `patient_dob` likely maps to `person.birth_datetime`), examines data types for compatibility (a VARCHAR column is unlikely to map to an integer field), and inspects sample values for domain clues (a column containing ICD-10 [77] codes belongs in the condition domain regardless of its name). The combination of these signals allows the LLM to produce reasonable mappings even for source schemas with non-standard or abbreviated column names.

The LLM's response is parsed into structured mapping rules and validated against known OMOP table and column definitions. Fields that cannot be confidently mapped receive a fall-

back `concept_id` of `-1`, a sentinel value that signals to the ETL engineer that manual review is needed. The `-1` convention is deliberate: it is an invalid OMOP concept ID that will fail validation if it reaches the CDM, ensuring that unmapped fields cannot silently propagate through the pipeline.

The complete mapping specification is returned as JSON, ready to be loaded into the concept mapping service and subsequently consumed by the data transformation flows. The mapping integrates with the broader ETL pipeline: once an engineer reviews and corrects the AI-generated draft, the finalised mapping feeds into the Data Transformation Controller in the Job Plugins Service, which executes the actual source-to-OMOP loading.

This approach does not replace human judgement — it accelerates the initial mapping by producing a reasonable first draft that an ETL engineer can review and correct, rather than starting from a blank slate. For a source schema with hundreds of columns, this can reduce the initial mapping effort from days to hours.

34.5 Integration Points

Each of the three services has a distinct caller. The Code Suggestion Service is invoked by Jupyter kernels through a small client library, so completions and chat responses appear inline as the researcher works. The MCP Server is called by external LLM clients — Claude Desktop, ChatGPT plugins, and any other Model Context Protocol consumer — and exposes platform endpoints as MCP tools that the external LLM discovers and invokes through the standard JSON-RPC schema. The Data Mapping Service is called by the Concept Mapping Service during the review phase of an ETL build, where the AI-generated draft is presented to the engineer alongside the source schema.

All three services call out to external LLM APIs configured per deployment via the `AI_MODEL` environment variable and its provider-specific companions, so the choice of Anthropic, OpenAI, Azure OpenAI, or a locally hosted model on Trex is a deployment decision rather than a code decision. Critically, the platform's authentication remains in the request path on the inbound side: a researcher's identity and dataset-scoped permissions are enforced before any prompt reaches the LLM, and tool calls dispatched by the MCP Server pass through the Portal's normal authorisation checks. Outbound calls to LLM providers carry no patient data unless the operator has explicitly configured a provider that has been approved for that data class.

34.6 Failure Modes

External LLM provider outages are the dominant failure mode for all three services. When the configured provider returns errors or times out, the Code Suggestion Service degrades to no completions, the MCP Server returns tool-call failures to its clients, and the Data Mapping Service surfaces the error to the ETL engineer who can either retry or fall back to manual mapping. Rate-limiting from the provider produces a similar but more insidious effect on interactive code suggestion — when token budgets are exhausted, latency climbs to the point where the feature is effectively unusable. The frontend handles this by hiding the suggestion UI gracefully rather than presenting a broken experience.

The most concerning failure mode is silent quality degradation. An LLM provider can update a model behind the same API endpoint, and outputs drift in ways that no HTTP status code reflects: cohort definitions become subtly wrong, mapping suggestions stop respecting OMOP conventions, or the agent's tool-calling becomes less reliable. Operators monitoring these services should not rely on HTTP-level signals alone. The right signal is downstream user feedback — thumbs-down rates on suggestions, manual override frequency on AI-generated

mappings, and qualitative reports from researchers — collected over time and reviewed when the underlying model is known to have changed.

35 File and Export Services

35.1 Purpose

The File and Export Services are a thin facade over Supabase Storage [66] and the Parquet [68] generation pipeline — small in code, but in the critical path for research output. Once a cohort has been built, a study has run, and the analytical results have been materialised, this is the surface researchers use to download their results in formats their downstream tools can consume. Without it, the rest of the platform's analytical machinery has no last mile.

The services in this chapter exist precisely because that last mile has its own concerns: streaming files larger than RAM, authenticating the download against the dataset that produced it, deduplicating uploaded scan reports, and cleanly tearing down temporary export files when a transfer completes or aborts. Each of those concerns is straightforward in isolation, but together they form a small, focused boundary between the platform's storage layers and the researcher's local toolchain.

35.2 Files Manager

The Files Manager provides centralised file storage for the D2E platform. Rather than storing files on disk or in object storage, it uses PostgreSQL [69] large objects — binary blobs managed directly by the database engine. This means file storage participates in the same transactional guarantees, backup strategy, and access control as the rest of the platform's data.

Storage Architecture

The choice of PostgreSQL large objects over the more common `BYTEA` column type is driven by streaming requirements. A `BYTEA` column loads the entire binary content into memory when read, which is acceptable for small files but problematic for multi-gigabyte scan reports or large data exports. PostgreSQL large objects, by contrast, support streaming reads and writes through a file-descriptor-like API: the service can read or write in chunks without ever holding the full file in memory. This is the same mechanism that allows the download endpoint to stream content directly from the database to the HTTP response.

Each uploaded file is stored as a PostgreSQL large object, with metadata (filename, uploader, logical grouping key, MD5 hash) tracked in a separate table. The grouping key — called the `dataKey` — categorises files by purpose. For example, White Rabbit [50] scan reports are grouped under one key, data exports under another, and uploaded mapping files under a third. This logical grouping allows services to query for all files relevant to a specific workflow without knowing individual file identifiers, and it enables bulk operations such as deleting all temporary exports for a completed study.

The separation between metadata and binary data enables MD5-based deduplication. When a file is uploaded, the service computes its MD5 hash and checks whether an identical blob already exists in the large object store. If a match is found, the service creates a new metadata record pointing to the existing large object rather than storing a duplicate copy. This deduplication is particularly effective for scan reports and configuration files that are frequently re-uploaded without changes — a common pattern when researchers iterate on ETL configurations. The deduplication is transparent to callers: each metadata record has its own unique identifier, so from the API perspective every upload produces a distinct file, even if the underlying storage is shared. When a deduplicated file is deleted, the service checks whether other metadata records still reference the same large object; the blob is only removed when the last reference is deleted.

Upload and Download Lifecycle

On upload, the service receives a multipart form containing the file, the uploading user's identity, and a data key. It computes the hash, checks for duplicates, writes the large object (if new), and creates the metadata record. The caller receives a unique identifier that can be used to retrieve or delete the file later.

On download, the service streams the large object content directly from PostgreSQL to the HTTP response, avoiding the need to buffer the entire file in memory. This allows the service to handle files larger than available RAM, which is critical for scan reports that can reach several gigabytes for large source databases.

Other D2E services use the Files Manager as a shared storage layer. White Rabbit stores scan reports here after profiling a source database. The Parquet Export Service (below) could store exported files for later retrieval. Any service that needs to persist a file for a user can delegate to the Files Manager rather than implementing its own storage.

35.3 Parquet Export Service

The Parquet Export Service enables researchers to extract cohort data from the CDM in Parquet or JSON format for use in external analysis tools such as Python, R, or Spark.

Parameterised SQL Templates

Rather than accepting arbitrary SQL from clients, the service works with pre-defined SQL templates stored in the Portal configuration. Each template is a parameterised query designed for a specific export use case — for example, extracting condition occurrences for a cohort within a date range. Templates contain named parameter placeholders (`cohortId`, `yearRange`, `conditions`) that the caller fills in when requesting an export.

Parameter validation is strict and multi-layered. Each parameter value is checked against an allowlist of safe patterns specific to its type: numeric parameters must match an integer or decimal pattern, date parameters must be valid ISO dates, and string parameters are checked against SQL injection signatures. The service rejects any input containing DDL statements, semicolons in parameter positions, comment sequences, or other constructs that could alter the query's intent. This defence-in-depth approach ensures that even if a caller crafts malicious parameters, the service will not execute dangerous SQL — the template system means users never supply raw SQL, and the parameter validation means they cannot smuggle SQL through parameter values.

Trex Export Pipeline

Once the query is validated, the service executes it through Trex's `COPY` command, which writes query results directly to a file in the requested format. Trex handles the columnar encoding for Parquet — organising data by column rather than by row, applying dictionary encoding for low-cardinality columns, and compressing each column independently. This columnar format is preferred for analytical workloads because downstream tools like Spark and pandas can read only the columns they need (column pruning) and skip row groups that do not match filter predicates (predicate pushdown), dramatically reducing I/O for large datasets. For JSON output, Trex handles the serialisation into well-formed JSON arrays.

The export pipeline follows a strict temporary file lifecycle. The service writes the output to a file in `/tmp`, streams the file to the client as the HTTP response body, and then deletes the temporary file immediately after the response completes — whether the transfer succeeded or

failed. A cleanup handler attached to the response ensures deletion even if the client disconnects mid-transfer or the stream errors. This prevents temporary files from accumulating on disk, even under high export volumes or interrupted connections, and avoids the need for a separate garbage collection process.

35.4 Integration Points

The File and Export Services serve files to the frontend over the standard authenticated route, which means downloads are issued by the Portal and inherit the platform's session and CSRF posture rather than introducing a parallel auth surface. On the storage side, they read from Supabase Storage for object content and coordinate with the Parquet-generation Prefect [55] flows that write the underlying snapshots — the export endpoint hands out files that an upstream flow has produced, rather than generating the snapshot itself in the request path. Access control is dataset-scoped: a request for a Parquet snapshot or an exported file is authorised against the dataset the artifact belongs to, using the same dataset permission model that gates analytical queries elsewhere in the platform.

35.5 Failure Modes

Supabase Storage unavailability or quota exhaustion are the only common failures. When Supabase Storage is unreachable, downloads fail immediately and the operator sees the failure in the service logs and in Supabase Storage's own health metrics; when its storage quota fills, uploads of new scan reports and snapshot writes from the Prefect flows begin to fail in a way that is also straightforward to diagnose.

The subtler failure mode is researcher-facing rather than operational. When a Parquet snapshot has not yet been generated — because the underlying Prefect flow has not run, or has not yet completed for the dataset and cohort the researcher is requesting — the service correctly returns a 404. This is the right behaviour: there is genuinely nothing to serve. But researchers who expect their results to be downloadable immediately after defining a cohort can read the 404 as a bug rather than as "the snapshot has not been built yet," and operators should be prepared to point them at the flow status rather than at the export service itself.

PART IV

Integrations

Part IV describes the integrations between D2E and external systems. The three main ones are the FHIR services, which expose clinical data over the HL7 R4 standard; the Jupyter notebook environment, which lets researchers run analytical code against the data; and the Prefect [55] workflow engine, which orchestrates ETL, characterization, and analysis pipelines.

The opening chapter covers the FHIR services: the gateway, the server, and the mapping from OMOP CDM to FHIR resources. The next chapter covers the Jupyter integration: how the Enterprise Gateway [56] provisions kernels, how notebooks are scoped per user and per dataset, and how the pgwire interface lets R kernels query the same data through SQL. The Prefect workflow chapter describes how D2E registers deployments, how flows are triggered from the platform, and how their results are returned. The remaining chapters cover the smaller integrations — DICOM [38] imaging and Supabase Storage [66] — and the individual Prefect flows, including CDM cache building, Achilles [43] characterization, and FHIR caching.

Together these integrations turn D2E from an OMOP query engine into a clinical analytics platform.

36 FHIR Services

36.1 Purpose

The FHIR services provide a FHIR-based interface to D2E. Clinical systems generally exchange data as FHIR resources, the HL7 R4 standard for clinical interoperability [18], rather than as OMOP CDM rows. The FHIR services give the platform a FHIR surface for ingesting data from hospital information systems, sending results to electronic health records, and interoperating with external clinical decision-support tools.

D2E has two FHIR-related components: the FHIR *server*, which exposes D2E's clinical data as FHIR resources for external clients to read, and the FHIR *gateway*, which translates incoming FHIR requests into platform operations. Together they let the platform participate in FHIR-based ecosystems without requiring other components to handle FHIR directly.

36.2 The Two-Component Design

The split between server and gateway corresponds to two interoperability scenarios. The server-mode case is read-heavy: an external system — for example an EHR or a research platform — queries D2E's clinical data through FHIR's REST API. The server translates each FHIR query into the appropriate database operation (often through the same Query Generation Service that powers Patient Analytics), formats the result as the requested FHIR resource type, and returns it. The gateway-mode case is the opposite: external systems push FHIR resources into D2E, and the gateway translates them into the platform's internal data structures, including OMOP CDM mapping where required.

Both components sit behind the same Trex routing layer and share the same authentication and authorization middleware. The split is logical rather than topological — in many deployments they share a single underlying service container.

36.3 FHIR-to-OMOP Mapping

The mapping between FHIR resources and OMOP CDM tables is the main work the FHIR services perform. The general approach — transforming FHIR resources into OMOP rows and serving OMOP data back as US Core conformant FHIR — has been characterized in published surveillance work such as MENDS-on-FHIR [10], which mapped eleven OMOP tables to ten FHIR resource types for chronic-disease surveillance at national scale. A FHIR `Patient`

resource maps to the OMOP `person` table, with identifier fields mapped to the person ID, demographic fields mapped to the demographic columns, and any extension elements stored in a sidecar table. A FHIR `Condition` maps to `condition_occurrence`, with the FHIR code translated through the Terminology Service to an OMOP concept identifier. A FHIR `Observation` maps to `measurement` or `observation` depending on the observation category.

The mapping is bidirectional but lossy. Some FHIR fields have no OMOP equivalent (FHIR's `contained` resources, for example, or its `narrative` text); these are preserved in extension columns when round-trip fidelity matters and discarded when it does not. Some OMOP fields have no FHIR equivalent (the OMOP concept hierarchy is richer than FHIR's coding model); when serializing OMOP to FHIR, these fields are encoded as FHIR coding extensions.

The Terminology Service handles code translation. Every coded value that crosses the boundary — a diagnosis code, a procedure code, a medication code — is translated through the OMOP vocabulary tables. SNOMED CT [65] codes from a FHIR resource become SNOMED concept identifiers in OMOP; ICD-10 [77] codes are mapped to their standard concept counterparts; LOINC [58] codes for measurements are resolved to their OMOP measurement concepts. Without this translation step the data would arrive in the right format but not be semantically aligned with the rest of the CDM.

36.4 Authentication and Authorization

The FHIR services use the same authentication and authorization model as the rest of the platform: OIDC [52] tokens issued by Logto [33], validated by Trex's authentication middleware, and matched against the platform's role-and-scope authorization. SMART-on-FHIR [19] is not supported in the current generation; FHIR's own authorization model is not used, in order to keep authentication consistent with the rest of the platform. External clients receive bearer tokens through the same OIDC flow as researchers using the web UI.

The trade-off is integration friction: FHIR clients that expect SMART-on-FHIR cannot connect without an OIDC adaptation layer. The benefit is that the platform's audit, scope, and revocation tooling apply uniformly to FHIR traffic.

36.5 Integration Points

The FHIR services integrate with the Terminology Service for code translation, with the database (via dataset-scoped credentials) for resource storage and retrieval, with the Query Generation Service for FHIR-Search-to-SQL compilation, and with Logto for authentication. They are called by external systems through the Caddy [6] reverse proxy and, less commonly, by internal services that need FHIR-formatted output for export.

36.6 Failure Modes

The most consequential FHIR failure mode is silent translation drift: a FHIR client sends a resource whose codes do not resolve to OMOP concepts, the Terminology Service falls back to a default mapping, and the resulting OMOP record is technically valid but clinically wrong. The platform logs unmapped codes for review, and a periodic Prefect [55] flow reconciles the unmapped-code log against new vocabulary releases, but the underlying risk remains: bad input plus permissive translation equals bad data. Operators should monitor the unmapped-code rate as a leading indicator.

The second failure mode is FHIR-Search query complexity. A FHIR `$everything` operation against a large patient record can produce a query with hundreds of joins; the Query Generation Service applies the same optimization passes it uses for IFR queries, but pathological FHIR

searches can still time out. Tuning happens through the same query-optimization mechanisms documented in Chapter 20 and Chapter 21.

36.7 Where FHIR is Going

The FHIR services are an active evolution surface. The genomics integration in development extends the FHIR resource set to include FHIR Genomics resources — `MolecularSequence`, `GenomicStudy`, the relevant Observation profiles. The companion OMOP-side work follows the Genomic Common Data Model proposed by Shin et al. [63], which extends OMOP with four genomic tables linked to the clinical CDM and standardizes gene and variant terminology so that sequencing data can be queried alongside clinical data. The mapping work for those resources is non-trivial and will land in a future revision of this chapter.

37 Enterprise Gateway (Jupyter Notebooks)

37.1 Overview

The Enterprise Gateway [56] provides Jupyter Notebook kernel management for the D2E platform. It runs two instances: a **researcher** instance with read/write capabilities and a **viewer** instance with read-only access. Kernels are spawned as Docker containers on dedicated networks, isolating notebook execution from the main application traffic.

37.2 WebSocket Authentication Flow

When a user opens a notebook in the D2E frontend, the browser initiates a WebSocket upgrade request to a path under `/jupyter/`. This request first arrives at Caddy [6], the platform's reverse proxy. Before upgrading the connection to WebSocket, Caddy performs a subrequest to the Trex UserMgmtAPI, passing the user's session token (extracted from the `authtoken` cookie). The UserMgmtAPI validates the token against Logto [33]'s JWKS [24] endpoint, checks that the user holds the appropriate dataset-scoped role (STUDY_RESEARCHER_ROLE for the researcher gateway, or a viewer role for the viewer gateway), and returns a success or failure response. Only if this validation succeeds does Caddy complete the WebSocket upgrade and proxy the connection to the Enterprise Gateway.

Once the WebSocket is established, the Jupyter protocol takes over. The browser sends execution requests, kernel info requests, and completion requests as Jupyter wire protocol messages over the WebSocket connection. The Enterprise Gateway deserializes these messages, identifies the target kernel from the session metadata embedded in each message, routes the message to the correct kernel container, and relays the responses back through the same WebSocket. The authenticated user identity is carried through as metadata on the kernel session, so audit logs can trace every cell execution back to a specific user.

This authentication flow means that kernel access is always mediated by the user's D2E session. If the session expires or is revoked (for example, because an administrator removes the user's dataset access), the WebSocket connection will fail on the next Caddy validation check, effectively disconnecting the user from their running kernel. The kernel itself continues to run until it is idle-culled, but no new messages can reach it without a valid session.

37.3 Kernel Container Lifecycle

When the Enterprise Gateway receives a request to start a kernel, it creates a new Docker container from the R kernel image (`r_ohdsi_docker`). The container is attached to the appropriate Docker network (researcher or viewer), given access to the shared `r-libs` volume, and injected with environment variables that carry database credentials, the Trex endpoint URL, and the user's session context.

The kernel container runs an R process that listens on ZMQ sockets for Jupyter protocol messages. When a user executes a code cell, the browser sends an `execute_request` message through the WebSocket, the Enterprise Gateway forwards it to the kernel container's ZMQ socket, the R process evaluates the code, and the result (text output, data frames, plots, or error messages) flows back through the same path as `execute_reply` and `display_data` messages.

Each kernel has a configurable launch timeout of 60 seconds — if the container fails to report readiness within that window, the launch is aborted and the user sees an error. The readiness check works by sending a `kernel_info_request` message and waiting for the corresponding `kernel_info_reply`; this reply confirms that the R process has started, loaded its base packages, and is ready to accept code cells.

Once running, kernels are monitored for idle activity. The cull idle timeout is set to 3600 seconds (one hour): if no execution requests arrive within that period, the Enterprise Gateway terminates the container and reclaims its resources. The idle check runs periodically (configurable via the cull interval) and inspects the timestamp of the last activity on each kernel. Each user is limited to one active kernel at a time, which prevents a single user from exhausting the host's memory or CPU allocation. If a user attempts to start a second kernel, the Enterprise Gateway returns an error rather than silently queueing the request.

37.4 R Library Persistence

The R kernel image ships with a pre-installed set of OHDSI packages, but researchers frequently need additional libraries for their analyses. When a user installs a package from a notebook cell, the package is downloaded, compiled (if necessary), and installed into a shared Docker volume. This volume is mounted into every kernel container, which means that installed packages survive kernel restarts, idle culling, and even platform upgrades — they accumulate over time as researchers add libraries for specific analyses.

This persistence model has an important implication: the `r-libs` volume is shared across all kernel instances on the same gateway. A package installed by one researcher is immediately available to all other researchers on the same instance. In practice this is convenient (common packages only need to be installed once) but requires coordination when incompatible package versions are needed.

37.5 Kernel Configuration

The R kernel (`r_ohdsi_docker`) comes pre-installed with OHDSI R libraries:

Package	Purpose
DatabaseConnector	Multi-dialect database connectivity (PostgreSQL, HANA, BigQuery) with connection pooling and credential management.
SqlRender	Translates OHDSI-standard SQL to database-specific dialects.
CohortGenerator	Materializes cohort definitions from JSON expressions into database tables.
FeatureExtraction	Extracts patient-level features for predictive modeling and characterization.
Achilles	Data profiling and characterization of OMOP CDM databases.
DataQualityDashboard	Automated data quality checks against CDM conformance rules.
Strategus	Orchestrates multi-method epidemiological studies.

37.6 Data Access Patterns

Kernel containers access clinical data through two mechanisms:

- **PG Wire Protocol (port 5433)** — Kernels connect to Trex's PostgreSQL-compatible wire protocol to query TrexSQL-cached CDW data directly. This provides fast, familiar SQL access using standard R database packages (`DBI`, `RPostgres`).

- **HTTP API** — Kernels can call D2E REST APIs (analytics, portal, cohort management) using the authenticated session token passed from the notebook frontend. This enables programmatic access to all platform features.

In practice, the most common data access pattern is a researcher connecting to Trex via the PG Wire protocol and executing SQL queries against the cached OMOP tables. A typical workflow involves establishing a database connection to Trex on port 5433 with the dataset's schema name, issuing SQL queries to extract cohort-level data, manipulating the resulting data frames in R, and producing visualizations or statistical summaries. Because the TrexSQL cache is optimized for analytical queries, even complex aggregations over millions of rows return in sub-second time, making interactive exploration practical.

For workflows that go beyond SQL — such as creating new cohort definitions, triggering Prefect [55] flows, or uploading results — researchers use the HTTP API path, constructing requests with the `httr` package and the session token that the notebook frontend injects into the kernel environment.

37.7 Template Notebooks

Notebook templates can be associated with datasets through the Portal. When a researcher opens a dataset for the first time, the platform can pre-populate their notebook environment with template notebooks that demonstrate common analytical patterns for that dataset's data model — for example, a template that connects to the dataset's TrexSQL cache, runs a basic demographic summary query, and plots the age and gender distribution. These templates are stored in Supabase Storage [66] and copied into the user's notebook workspace on first access. Templates are versioned alongside the dataset configuration, so updates to templates are reflected for new users without affecting notebooks that existing users have already customized.

37.8 Researcher vs. Viewer Instances

The two Enterprise Gateway instances run on separate Docker networks (`enterprise-gateway` and `enterprise-gateway-viewer`) with identical kernel specifications but fundamentally different access levels. The network separation is enforced at the Docker level: researcher kernel containers are attached to the `enterprise-gateway` network, which has routes to the database servers and internal APIs, while viewer kernel containers are attached to the `enterprise-gateway-viewer` network, which restricts outbound connections to read-only endpoints.

Researcher kernels have full read/write access to databases and APIs. They can create tables, write cohort results, install packages, and trigger analytical flows. Viewer kernels operate in read-only mode, suitable for reviewing shared notebooks and results without the ability to modify data. This separation ensures that sharing a notebook with a reviewer does not inadvertently grant them write access to the underlying dataset.

38 Prefect (Workflow Orchestration)

38.1 Overview

Prefect [55] serves as the asynchronous execution engine for D2E. Any operation that takes more than a few seconds to complete — ETL pipelines, data quality checks, cohort generation, statistical analyses, cache rebuilds — runs as a Prefect flow rather than blocking an HTTP request. This design choice is fundamental to the platform's architecture: clinical data operations routinely process millions of records and can run for minutes or even hours, making synchronous request-response patterns impractical. By delegating these operations to Prefect, the D2E frontend remains responsive, users can monitor progress, and the platform can execute multiple long-running operations concurrently without tying up API server resources.

The Job Plugins Service (see Part II) acts as the gateway between the D2E application layer and Prefect. When a user triggers an operation — for example, clicking "Run Data Quality Check" in the frontend — the request reaches Trex, which routes it to the Job Plugins Service. The service translates the request into a Prefect deployment run, submitting it with the appropriate parameters. From that point forward, Prefect manages the execution lifecycle: scheduling, container provisioning, progress tracking, and result storage. The frontend polls the Job Plugins Service for status updates, which in turn queries the Prefect API.

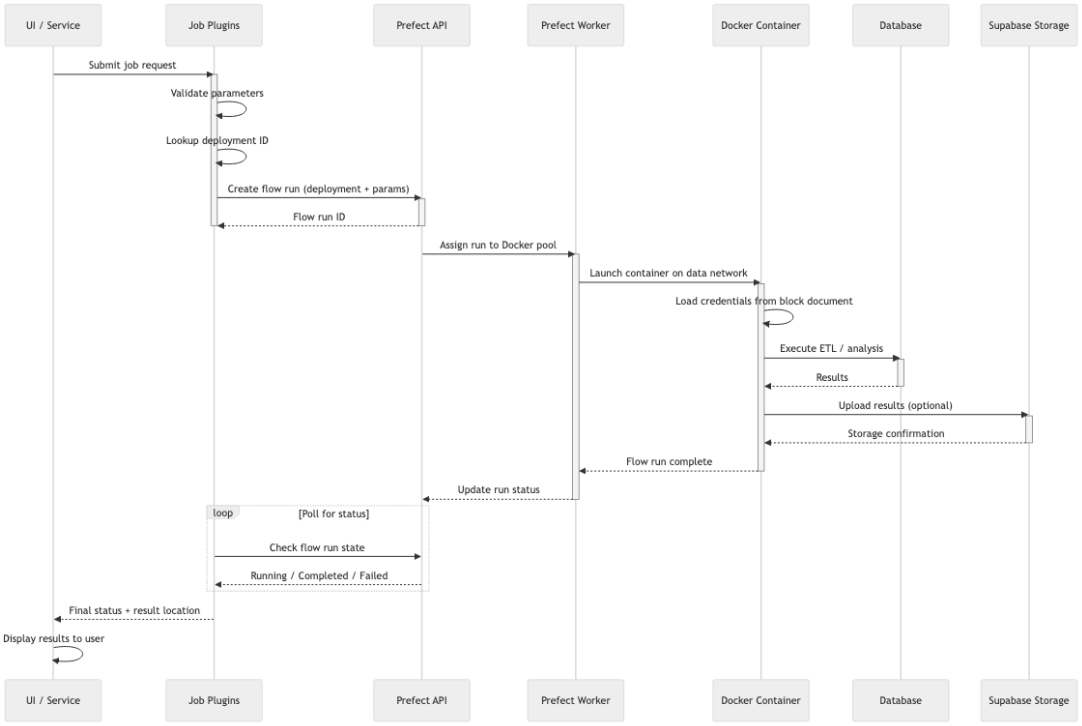

Figure 20: Prefect Flow Execution

38.2 The Prefect Server

The Prefect server (version 3.6.10, running on Python 3.12) is the central coordination point for all workflow activity. It exposes a REST API at `/d2e/api` on port 41120 and stores all of its state in PostgreSQL [69] using the `postgresql+asyncpg` async driver.

The server's database holds several categories of information. Flow metadata describes the registered Python flow functions — their names, source locations, and parameter signatures. Deployment configurations bind flows to specific execution infrastructure, defining which Docker

image to use, which work pool to target, and what default parameters to apply. Flow run state tracks every execution of every flow: when it was scheduled, when it started, its current status (pending, running, completed, failed, cancelled), and its result artifacts. Task run state provides finer-grained tracking within flows, recording the status of individual tasks that compose a flow. Artifacts store structured outputs from flow runs — data quality reports, row counts, error summaries — that can be retrieved later for display or downstream processing.

The server enforces an API record limit of 5000 records per query to prevent runaway result sets from consuming excessive memory. Server-level logging is set to DEBUG for operational visibility, while general Prefect logging runs at WARNING to keep log volume manageable in production.

The Prefect server depends on both the Logto [33] Post-Init and PG Management Init containers having completed before it can start, because it needs both a configured database schema and a functional identity provider for token validation.

38.3 The Worker Model

The Prefect worker operates as a Docker work pool, meaning it executes each flow run by spawning an isolated Docker container. This container-per-flow model provides strong isolation between concurrent executions: one flow's memory leak or crash cannot affect another flow, and each flow gets a clean execution environment.

When the worker receives a scheduled flow run from the server, it pulls the specified Docker image (which contains the flow's Python code and all its dependencies — Python packages, R libraries, Java runtimes as needed), injects the run parameters as environment variables, configures resource limits, and starts the container. Each flow container receives a default allocation of 16 GB of memory and 64 GB of swap (both configurable per deployment), providing enough headroom for large-scale data processing while preventing any single flow from consuming all host resources.

Flow containers are attached to the `data` Docker network, which gives them direct access to PostgreSQL, SAP HANA, and other databases without routing through the Trex API layer. This is a deliberate architectural choice: data-intensive flows that process millions of rows need direct database connectivity for performance. Going through Trex's HTTP API would add serialization overhead, connection multiplexing bottlenecks, and unnecessary memory copies. For operations that interact with Trex's TrexSQL cache, the Trex volume is mounted into the flow container, enabling direct file-based access to cached datasets without network overhead.

The Docker socket is mounted from the host into the worker container, which is what allows the worker to spawn sibling containers. This is a common pattern in Docker-based CI/CD and orchestration systems, though it does require that the host's Docker daemon is accessible to the worker process.

38.4 The Deployment Model

Each flow is registered as a Prefect deployment during platform initialization. The `alp-dataflow-gen-init` function, which runs as one of Trex's startup init functions, iterates over all known flows and creates a deployment for each one via the Prefect API.

A deployment is the binding between a flow's code and the infrastructure needed to run it. Each deployment defines several key properties. The flow name identifies which Python flow function to execute within the Docker image. The Docker image specifies the container image containing the flow code and all its runtime dependencies. The work pool is always

`docker-pool`, directing the deployment to the D2E worker. The parameter schema is a JSON Schema definition describing the expected input parameters for the flow, enabling both validation and UI generation. Infrastructure overrides allow per-flow customization of resource limits (memory, CPU), additional volume mounts (for example, mounting a shared data directory), and environment variables specific to that flow's needs.

When the Job Plugins Service creates a deployment run, it supplies the concrete parameter values for that specific execution. Prefect schedules the run on the worker's Docker pool. The worker pulls the flow image (or uses a cached copy), injects the run parameters, and starts the container. The flow's Python code executes within this isolated container, using the shared infrastructure layer — DBDao for database access, API clients for service communication, and the R bridge for statistical computations — to perform its work. As the flow progresses, it reports state transitions back to the Prefect server, and on completion (or failure), the final status and any artifacts are recorded for retrieval.

38.5 Flow Categories

D2E's 24 Prefect flows span six functional categories that cover the full lifecycle of clinical data analysis. Data Model Creation flows (4 flows) handle CDM schema creation for OMOP models, run Liquibase database migrations, manage HANA-specific data loading patterns, and build an i2b2 star schema from the CDM. Data Transformation flows (8 flows) handle the bulk of data ingestion: CSV file loading, FHIR resource import, DICOM [38] metadata extraction, MIMIC-to-OMOP conversion, NLP-based text extraction, and dynamic DAG execution for complex multi-step transformations. Data Quality flows (3 flows) run Achilles [43] profiling to characterize datasets, execute Data Quality Dashboard [45] checks against established rules, and generate White Rabbit [50] scan reports for source data profiling. Caching and Search flows (3 flows) build Trex's local TrexSQL cache from remote databases, create FHIR resource caches, and generate semantic embeddings for natural-language search. Cohort and Analysis flows (5 flows) run CohortGenerator for population selection, integrate with the Phenotype Library for standardized cohort definitions, execute Strategus [48] study packages, compute loyalty cohort scores, and share analysis artifacts between studies. Finally, a single Application flow handles ShinyLive app building and deployment for interactive data exploration tools.

For detailed coverage of each individual flow — including parameter schemas, execution logic, and error handling — see Chapter 40.

39 Other Integrations

39.1 Purpose

Beyond FHIR, Jupyter, and Prefect, D2E integrates with a small constellation of supporting services. Each is narrower in scope than the central three, but each fills a real role in the platform's operational surface. This chapter catalogues them and the patterns by which they connect.

39.2 DICOM Imaging

Some research studies require access to medical imaging data — CT, MRI, X-ray, and other modalities exchanged using the DICOM [38] standard. D2E supports this through an optional DICOM profile that adds an Orthanc server to the deployment, activated via the `dicom` Docker Compose profile.

Orthanc acts as a lightweight DICOM server that indexes metadata (patient ID, study date, modality, accession number, and similar DICOM tags) in an internal SQLite database while storing the pixel data in Supabase Storage [66]. Metadata queries hit the index; the heavy pixel data is streamed from object storage on demand.

A dedicated `medical_imaging` schema complements the OMOP CDM with imaging-specific acquisition parameters (slice thickness, pixel spacing, scanner model, magnetic field strength, etc.) that standard OMOP tables do not capture. Each row corresponds to a DICOM series and carries foreign keys to the OMOP `person` and `visit_occurrence` tables so that imaging attributes can be joined to clinical data in a single query. A `person_to_patient_mapping` table bridges the DICOM Patient ID and the OMOP `person_id` so that imaging records can be linked to the correct CDM patient.

The DICOM ETL Prefect [55] flow extracts study- and series-level tags from source files and populates these tables. It can run in metadata-only mode (tags only, original files remain external) or full-upload mode (files additionally uploaded to Orthanc for web-based viewing and PACS integration).

Imaging integration in D2E is deliberately thin. The platform does not interpret DICOM data; it stores it, secures access to it, and renders it through Orthanc's viewer. Image-based analytics — segmentation, feature extraction — are out of scope for the platform and live, when needed, in dedicated radiology pipelines that talk to the same DICOM store.

39.3 Supabase Storage

Object storage in D2E is provided by Supabase Storage [66], which exposes an authenticated, row-level-access-controlled API over an S3-compatible backend. Every file the platform produces or accepts — notebook attachments, Parquet exports, FHIR resource bundles, ETL source files, encrypted recontact packages, platform backups — lives there.

Two architectural choices are worth noting. First, Supabase Storage sits behind Trex, which means object access is mediated by the platform's authorization layer rather than by raw S3 IAM policies; this keeps the access model consistent across all D2E resources but adds a hop. Second, Supabase Storage was chosen because its row-level security maps cleanly onto D2E's per-dataset authorization, sparing the platform from reimplementing that logic at the storage edge. The S3-compatible backend underneath is an implementation choice that callers never see; from the platform's perspective the storage layer is Supabase Storage.

39.4 Logto Identity Provider

Logto [33] is treated as an integration rather than a core service because it is operationally separable from the rest of the platform — it has its own database, its own admin console, and its own upgrade cycle. D2E talks to Logto over HTTP for authentication and over its admin API for role and scope provisioning. The User Management Service in Part III is the platform's bridge to Logto.

The choice of Logto over alternatives (Keycloak, Auth0, custom OIDC) was made for the combination of OIDC [52] compliance, OSS licensing, and deployable footprint — Logto runs in a single container with PostgreSQL [69], fitting the same operational model as the rest of D2E.

39.5 Failure Modes Across Integrations

Each of these integrations has its own failure mode. DICOM outages affect imaging studies but not analytical queries. Supabase Storage outages disable file uploads, exports, and Parquet retrieval but leave database-backed analytics running. Logto outages prevent new logins; existing sessions persist until token expiry.

The pattern across all of them is that the platform degrades by feature rather than by user — the same researcher experiences different brokenness depending on what they try to do, rather than the platform being binarily up or down. Operators should monitor each integration's health independently rather than relying on a single platform health check.

40 Prefect Flows

40.1 Overview

The D2E platform includes 24 Prefect [55] flows (deployable flow plugins) that handle data ingestion, transformation, quality assessment, cohort generation, and analytical study execution. These flows are the primary mechanism for getting data into D2E and running computations that are too long-running or resource-intensive for synchronous API calls.

Every flow follows the same execution pattern: a user or service triggers an operation through the Job Plugins Service, which creates a Prefect deployment run. The Prefect worker spawns a Docker container on the `data` network, giving it direct access to databases and the Trex volume. The flow executes inside this container with its own Python environment, R runtime (when needed), and injected credentials. When the flow completes, its results are either written to the database, stored as files in Supabase Storage [66], or both.

The flows share a common infrastructure layer that solves three recurring problems. First, database access: a unified abstraction handles the differences between PostgreSQL [69], SAP HANA, Google BigQuery, and TrexSQL, so individual flows do not need dialect-specific code. Second, R integration: many OHDSI analytical tools are R packages, and flows call them through a Python-R bridge that embeds an R interpreter and translates data structures between the two languages. Third, resilience: complex flows that create database schemas attach failure hooks that automatically drop partially-created schemas if the flow fails mid-execution, preventing orphaned objects that would block future runs.

40.2 Data Model Creation

Before any clinical data can be loaded or analyzed, the target database needs the right schema structure. Four flows handle this concern, each addressing a different scenario.

OMOP CDM Creation

The primary entry point is the OMOP CDM creation flow. Given a target database and a CDM version (5.0, 5.3, or 5.4), it provisions the three-schema layout that every OMOP dataset requires: a CDM schema containing clinical tables (person, condition_occurrence, drug_exposure, and dozens more), a vocabulary schema containing terminology tables (concept, concept_relationship, concept_ancestor), and a results schema where analysis outputs are written.

The DDL generation does not use hand-written SQL. Instead, the flow invokes the OHDSI CommonDataModel R package through the Python-R bridge, which generates dialect-specific CREATE TABLE statements for the requested CDM version and database backend. This approach guarantees that the schema definition matches the canonical OHDSI specification exactly — including column types, nullability constraints, and naming conventions — without requiring D2E to maintain its own copy of the schema definition. The R package produces dialect-appropriate DDL automatically; for example, PostgreSQL DDL uses native auto-incrementing columns and text types, while HANA DDL uses identity columns and Unicode string types.

The three schemas are created in a specific order: vocabulary first (since clinical tables reference vocabulary concepts), then CDM, then results. Vocabulary schemas can be shared across datasets — a common pattern when multiple studies at the same institution use the same terminology version. When vocabulary sharing is configured, the flow skips vocabulary schema creation and instead validates that the referenced vocabulary schema exists and contains the

expected tables. The flow also supports creating "seed schemas" that bundle vocabulary and dataset creation into a single operation, useful for bootstrapping new environments where both need to be provisioned together.

Resilience is handled through failure hooks. If the flow fails at any point after creating a schema — for example, during table creation or vocabulary loading — the hook fires and drops the partially-created schema. Without this mechanism, a failed flow would leave behind an incomplete schema that would cause name collisions on retry. The hook captures the database connection and schema name at flow initialization time, so the cleanup code has everything it needs even if the flow's runtime state is corrupted.

Liquibase Schema Versioning

Schema versioning is managed through Liquibase, a database migration tool. The schema management flow wraps Liquibase operations and supports four actions: applying pending changelogs, rolling back a specified number of changesets, rolling back to a named tag, and marking all changelogs as applied without executing them (changelog sync).

Changelog files are stored in the flow's resource directory, organized by dialect. Each changelog file records a sequence of changesets that describe schema modifications — adding columns, creating indexes, altering data types. This per-dialect organization is necessary because changelogs can contain raw SQL that differs between databases.

The flow builds a JDBC connection dynamically from the dataset's database credentials and invokes the Liquibase process with the appropriate changelog path and action parameter. Any output containing database passwords is masked before being written to the Prefect log, preventing credential leakage in flow run histories visible to platform administrators.

The changelog sync action deserves special mention: it marks all changesets as "already applied" without executing them. This is essential when adopting Liquibase for an existing schema that was created by an earlier version of the OMOP CDM creation flow. Without this sync step, Liquibase would attempt to apply all historical migrations to a schema that already has the final structure, causing errors on every changeset.

HANA Demo Data Loading

For SAP HANA deployments, a specialized flow handles HANA-specific DDL and can optionally load Eunomia synthetic demo data — a small but realistic OMOP dataset useful for development and testing. The flow downloads CSV files from the Eunomia R package, converts them to HANA-compatible formats (uppercase column names, HANA-specific date formats), generates HANA-native DDL, and bulk-loads the data into the newly created schema.

40.3 Data Loading and Transformation

Getting data into a provisioned schema is the job of eight transformation flows, each handling a different source format or transformation pattern.

CSV Loading Pipeline

CSV loading is the most common ingestion path. The flow reads CSV files in configurable chunks, which prevents memory exhaustion when loading multi-gigabyte files. Each chunk undergoes several transformations before insertion.

Column name normalization adapts the CSV headers to the target database dialect. For PostgreSQL, all column names are lowercased; for SAP HANA, they are uppercased. This normal-

ization is essential because PostgreSQL folds unquoted identifiers to lowercase while HANA folds them to uppercase, and a mismatch between the CSV header and the table column name would cause the insert to fail.

Table-specific type coercions handle the data quality issues that are endemic in real-world CSV exports. Concept ID columns are coerced to integers (handling the common case where a CSV tool exports them as floats with trailing decimals), dosage and quantity fields are coerced to floats, and date columns are parsed with flexible format detection. These coercions are defined in a per-table configuration that maps each column to its expected type.

For PostgreSQL targets, the flow uses the COPY bulk loading protocol for insertion. Rather than generating individual INSERT statements, it streams each chunk directly into PostgreSQL's bulk loading pathway. This is typically 5–10x faster than row-by-row inserts for large datasets. For HANA targets, the flow falls back to ORM-based bulk inserts, which are slower but compatible with HANA's interface.

Each file can target a specific table with optional truncation before loading, allowing both fresh loads and incremental appends. The flow also handles the common case where a CSV file contains columns that do not exist in the target table — it silently drops unrecognized columns rather than failing, which makes it tolerant of schema version mismatches between the data source and the target CDM.

FHIR Resource Loading

The FHIR loading flow handles JSON and NDJSON files containing FHIR R4 resources. The flow extracts unique resource types from the files, optionally truncates existing tables by resource type, then posts each resource to the FHIR Gateway API. It supports FHIR Bundles (which contain multiple resources), resource lists, and individual resources.

FHIR-to-OMOP Transformation

The FHIR-to-OMOP flow bridges the gap between FHIR and OMOP formats. A Java-based microservice connects to the FHIR database as source and the OMOP CDM database as target, performing the structural transformation with configurable date filtering and batch sizes. The microservice reads FHIR resources, applies structure map rules maintained in the Git repository, resolves vocabulary mappings from FHIR coding systems to OMOP concept IDs, and writes the resulting rows to the target schema.

MIMIC-IV Conversion

The MIMIC-IV conversion flow transforms the widely-used MIMIC clinical database into OMOP CDM format, and its architecture illustrates a sophisticated staging strategy built around DuckDB [9].

The flow begins by creating a local DuckDB file that serves as the staging database. MIMIC source data and vocabulary files are loaded into this DuckDB instance, where they can be joined, filtered, and transformed using fast analytical SQL without touching the target database. The transformation logic is organized as 54 SQL scripts arranged in three phases: staging scripts that create intermediate tables from raw MIMIC data, ETL scripts that transform staged data into OMOP CDM format (resolving concept mappings, computing derived fields, handling NULL semantics), and unload scripts that export the final CDM tables.

The export path depends on the target database. For PostgreSQL, DuckDB attaches directly to the PostgreSQL instance using its database extension, enabling DuckDB to write transformed data to PostgreSQL tables without leaving the SQL layer — the data flows directly from

DuckDB's columnar storage into PostgreSQL's row-oriented tables through an optimized binary transfer path. For SAP HANA, which DuckDB cannot attach directly, the flow reads each table from DuckDB into DataFrames, converts column names to uppercase (HANA convention), and inserts the data in configurable chunks via the ORM layer. This chunked approach prevents memory exhaustion when exporting large tables like `measurement` or `drug_exposure`, which can contain tens of millions of rows. The chunk size is configurable per deployment, typically set between 10,000 and 100,000 rows depending on available memory and network bandwidth to the HANA instance.

Dynamic Dataflow Execution

Dynamic dataflow execution is the most flexible transformation mechanism. Users define arbitrary ETL pipelines as JSON directed acyclic graphs (DAGs), where each node represents an operation and edges represent data dependencies between them.

The flow parses the JSON specification and resolves execution order using topological sorting. Starting from nodes with no incoming edges, the algorithm repeatedly removes a node from the ready set, executes it, and adds any newly unblocked downstream nodes to the ready set. This guarantees that every node's input data is available before it executes, without requiring the user to specify an explicit ordering.

Node instantiation uses a factory pattern. The flow maintains a registry of node types: CSV readers, SQL transformers, Python code nodes, R script nodes, database readers, column mapping nodes, vocabulary lookup nodes, and FHIR-specific transformers. Each node type implements a common interface that accepts input DataFrames and produces output DataFrames.

SQL nodes use DuckDB as an in-memory SQL engine. The node registers its input DataFrames as virtual tables, executes the user-provided SQL query against them, and returns the result as a DataFrame. This means users can write standard SQL to join, filter, and aggregate data without needing a running database server. Python nodes provide a sandboxed execution environment where user-provided code can access the input DataFrames as variables.

For large-scale transformations, execution can be distributed across multiple workers via Dask. The flow supports three scheduler backends: local thread pools (for development and small datasets), Kubernetes pods (for cluster deployments where each node runs in its own container), and MPI processes (for high-performance computing environments where inter-node communication is fast). The choice of scheduler is transparent to the node implementations — they operate on DataFrames regardless of whether the execution is local or distributed.

All node executions are traced with timing metadata stored as Prefect artifacts, enabling post-hoc debugging of slow or failed transformations. Each artifact records the node type, input and output row counts, wall-clock execution time, and any warnings or errors produced during execution.

DICOM ETL

The DICOM [38] ETL flow extracts metadata from medical imaging files and loads it into both standard OMOP tables (`visit_occurrence`, `procedure_occurrence`) and a dedicated medical imaging schema with imaging-specific fields like slice thickness, pixel spacing, and scanner model. It maps DICOM tags to OMOP concepts and can optionally upload the original images to the Orthanc DICOM server.

NLP Entity Extraction

The NLP extraction flow processes clinical notes stored in the OMOP `note` table using two spaCy transformer models run in sequence. The first model, BC5CDR (trained on the BioCreative V CDR corpus), identifies diseases and chemical entities in the text; it is paired with a UMLS entity linker that resolves recognized entities to Unified Medical Language System concept identifiers. The second model, Med7 (trained on clinical text for drug name recognition), identifies medication mentions and is paired with an RxNorm linker that maps them to RxNorm concept codes.

Both models produce entity annotations with confidence scores. Only entities exceeding a 0.75 confidence threshold are retained. For each retained entity, the flow looks up the corresponding OMOP concept ID (via the UMLS or RxNorm code), constructs a `note_nlp` row recording the source note, the entity span, the recognized text, and the mapped concept, and writes it to the `note_nlp` table. This makes unstructured clinical text available for structured queries — a researcher can find all patients whose clinical notes mention a specific drug or disease without reading the notes manually.

Questionnaire Management

The questionnaire flow handles FHIR Questionnaire resources — structured survey instruments used in clinical research. The flow parses the hierarchical item structure of a FHIR Questionnaire and inserts it into GDM questionnaire tables, preserving the parent-child relationships between question groups and individual questions.

40.4 Data Quality and Profiling

Three flows assess data quality and generate profiling reports that help data engineers understand their data before and after transformation.

Achilles Data Characterization

The Achilles [43] flow invokes the OHDSI Achilles R package through the Python-R bridge. The flow first creates the results schema if it does not exist, then runs Achilles with configurable parameters: CDM version, thread count for parallel analysis execution, and a list of analysis IDs to exclude (useful for skipping analyses that are slow or irrelevant to the dataset).

Achilles performs over 3,000 individual analyses across the CDM schema, covering dashboard summaries, data density patterns, person demographics, visit distributions, and clinical domain breakdowns (conditions, procedures, drugs, measurements, observations, death). The results are written to 25 or more tables in the results schema, each containing aggregated counts and distributions for a category of analyses.

The flow does not abort the entire run when some analyses fail — partial Achilles results are still valuable for data characterization. It is common for a small number of analyses to fail on datasets that lack certain clinical domains (for example, drug era analyses fail if no drug exposure data is present). Failed analysis IDs are collected and logged as warnings.

Optionally, the flow exports results in ARES format — a set of CSV files organized by analysis category, structured for the ARES interactive visualization tool. These files are uploaded to Supabase Storage and linked to the dataset in the Portal. The flow can also compute per-concept record counts (how many records exist for each standard concept in each clinical table), which are stored in the results schema and power the Terminology Service's usage statistics. These counts allow the Terminology Service to show researchers which concepts are actually present in their data, helping them build more accurate cohort definitions.

Data Quality Dashboard

The DQD [45] flow invokes the OHDSI DataQualityDashboard R package, which validates CDM data against a library of conformance, completeness, and plausibility rules.

Checks are organized by three levels. TABLE-level checks verify structural properties (do required tables exist? are they non-empty?). FIELD-level checks validate individual columns (are concept IDs valid? are dates within plausible ranges? are required fields populated?). CONCEPT-level checks assess semantic consistency (do the concept IDs in condition_occurrence actually refer to condition concepts? are drug quantities within expected ranges?). The check set can be filtered to specific check names or scoped to a specific cohort, so that data quality is assessed only for the patient population of interest rather than the entire dataset.

The flow produces a JSON artifact containing the results of every check (pass, fail, or not applicable), along with threshold values and failure counts. This artifact can be rendered by the D2E frontend as an interactive data quality report. Optionally, detailed check results are also written to database tables for programmatic access. The flow supports both standard PostgreSQL connections and HANA JWT authentication.

WhiteRabbit Source Profiling

The WhiteRabbit flow runs the OHDSI White Rabbit [50] Java tool in headless mode. Because WhiteRabbit was originally designed as a desktop application with a graphical interface, the flow launches it under a virtual framebuffer that provides the display server the Java runtime requires without needing a physical screen. This allows the tool to run in a headless Docker container.

The flow operates in three modes: scanning database tables to profile their structure and value distributions, scanning uploaded CSV files, or generating ETL documentation as a Word document. For database scanning, the flow generates a configuration file specifying the connection parameters, table list, and scan options, then invokes WhiteRabbit's scanner. The scanner produces an Excel workbook containing one sheet per scanned table, with column-level statistics (data type, distinct count, NULL percentage, min/max values, and sample values).

The Excel scan report is then parsed to extract a JSON schema representation. This JSON schema is used by the D2E frontend to present the source data structure in the concept mapping interface, allowing users to visually map source fields to OMOP CDM targets. Source files are downloaded from Supabase Storage before scanning, and completed reports are uploaded back to storage for retrieval through the Files Manager.

40.5 Caching and Search Optimization

Three flows create performance-optimized data copies and search indices that accelerate the platform's query and discovery capabilities.

TrexSQL Cache Creation

Cache creation is central to D2E's query performance strategy. The flow uses DuckDB extensions to attach remote databases as virtual schemas: PostgreSQL databases via the PostgreSQL scanner extension and BigQuery datasets via the BigQuery extension. Once attached, the flow copies tables from the remote source into a local DuckDB file.

Not all columns are copied. A column filter configuration specifies which columns to include for each CDM table. For example, the `note` table might include structural columns like note

ID, person ID, and date, but exclude the free-text note body (which can be megabytes per row). This filtering serves two purposes: it reduces cache file size by excluding large text columns that are rarely queried analytically, and it avoids copying columns that contain sensitive free-text content not needed for the cached analytical workload.

For patient and timestamp filtering, the flow accepts optional parameters to restrict the cache to a subset of patients or a time range. These filters are applied during the copy, enabling both full cache refreshes and incremental updates.

Large tables pose a memory challenge: copying a 100-million-row measurement table in a single pass could exhaust the process's memory. The flow handles this through range-based chunking. It queries the source table's minimum and maximum values on the primary ID column, divides the range into chunks, and copies each chunk independently. Each chunk is small enough to process in memory, and the chunked approach also provides natural progress reporting.

After all tables are copied, the flow creates full-text search (FTS) indices on specified columns — typically concept names and descriptions in vocabulary tables. The indices are configured with an English stemmer (which reduces words to their root form, so "medications" matches "medication") and a stopword list (which excludes common words like "the" and "of"). These FTS indices power the fast text search in the Terminology Service, allowing users to find concepts by typing partial terms.

The resulting DuckDB file is written to a Docker volume that Trex mounts, making it accessible through Trex's PG Wire protocol on port 5433. The file path follows a convention that includes the dataset ID, so multiple datasets can have independent caches on the same host. When a cache is rebuilt, the flow writes to a temporary file and atomically renames it to the target path, ensuring that concurrent queries never see a partially-written cache.

FHIR Schema Caching

FHIR schema caching creates a Trex copy of a FHIR source schema through the Trex SQL interface. The flow connects to Trex's PostgreSQL-compatible port, clears any existing cache for the dataset, copies the FHIR schema tables, and commits the transaction. This enables FHIR resource data to be queried through the same fast analytical path as OMOP data.

Semantic Embedding Generation

The semantic embedding flow creates vector embeddings that enable concept search by meaning rather than by exact text match. The flow reads concept names from the vocabulary tables, generates embeddings using a small transformer model (Supabase/gte-small, 384 dimensions), stores them in a dedicated column, and builds a vector index for fast similarity search.

The embedding process works as follows. For each concept name, the model's tokenizer produces a sequence of token IDs, which are fed through the transformer to produce per-token hidden state vectors $\mathbf{h}_1, \ldots, \mathbf{h}_T$ of 384 dimensions each. The flow applies average pooling weighted by the attention mask m_t (so that padding tokens do not contribute), then L2-normalizes to unit length:

$$\mathbf{e} = \frac{\hat{\mathbf{e}}}{\|\hat{\mathbf{e}}\|_2}, \quad \text{where} \quad \hat{\mathbf{e}} = \frac{\sum_{t=1}^{T} \mathbf{h}_t \cdot m_t}{\sum_{t=1}^{T} m_t} \tag{14}$$

The resulting unit vector $\mathbf{e} \in \mathbb{R}^{384}$ captures the semantic meaning of the concept name in a form suitable for cosine similarity comparison.

Concepts are processed in batches to balance throughput against memory usage. Each batch is tokenized, embedded, pooled, and normalized in a single forward pass through the model.

Once all embeddings are computed and stored, the flow creates an HNSW (Hierarchical Navigable Small World) vector index with cosine distance as the metric. HNSW is an approximate nearest-neighbor algorithm that builds a multi-layer graph of vectors; searching the graph starts at the top layer (which contains a sparse subset of vectors for coarse navigation) and descends through progressively denser layers until it finds the nearest neighbors in the full set. This enables sub-millisecond similarity searches over hundreds of thousands of concept embeddings, which is what powers the Terminology Service's hybrid search — combining traditional SQL text matching with semantic similarity to find concepts even when the researcher's query uses different terminology than the vocabulary.

40.6 Cohort Generation and Analysis

Five flows handle the execution of cohort definitions and analytical studies, connecting the platform's query capabilities to the OHDSI analytical ecosystem.

Cohort Generation

The cohort generation flow materializes cohort definitions using the OHDSI CohortGenerator R package. Given a JSON cohort definition (containing an OHDSI cohort expression and a name), the flow first checks whether a cohort definition record already exists in the Analytics Service; if not, it creates one through the Analytics API. It then invokes the CohortGenerator through the Python-R bridge, passing a database connection, the target results schema, and the cohort definition SQL.

The R package translates the declarative cohort expression — which describes inclusion criteria, exclusion criteria, time windows, and required events in a JSON format — into a complex SQL query that identifies matching patients. A single cohort expression can generate SQL with dozens of CTEs (Common Table Expressions), each handling one aspect of the definition: initial event identification, inclusion rule application, temporal constraint evaluation, and era construction. The query is executed against the CDM schema, and the resulting patient IDs (along with cohort start and end dates) are written to the cohort table in the results schema.

This is the mechanism that converts a declarative cohort definition ("patients with diabetes diagnosed after 2020 who had at least two HbA1c measurements") into a concrete set of patient identifiers that can be used in downstream analyses. The cohort table uses a standard four-column schema: cohort definition ID (linking back to the definition), subject ID (the patient identifier), cohort start date, and cohort end date. Multiple cohort definitions can coexist in the same table, distinguished by their definition ID.

Phenotype Library Integration

The Phenotype Library flow connects D2E to the OHDSI Phenotype Library — a curated collection of validated cohort definitions maintained by the OHDSI community. The flow retrieves definitions from the library using the PhenotypeLibrary R package (either the default set or specific IDs requested by the user).

Retrieved definitions can follow two paths. The Portal artifact path creates metadata records in the Portal Service, making the definitions browsable and editable through the D2E frontend without executing them. The database materialization path goes further: it creates result tables in the target schema and executes the cohort SQL through R, immediately populating the co-

hort table with matching patient IDs. The dual-path design lets researchers browse the library before committing to the computational cost of materialization.

Strategus Analysis Orchestration

Strategus [48] is the most complex flow in the platform. It executes multi-method epidemiological studies using the OHDSI Strategus R framework. An analysis specification — a JSON document defining analysis nodes and their dependencies — drives the execution.

The specification is parsed as a directed acyclic graph. Each node represents an OHDSI analysis module, and the flow supports over 20 module types: CohortMethod for comparative effectiveness studies using propensity score matching, PatientLevelPrediction for building and validating risk prediction models, TreatmentPatterns for analyzing sequences of treatments over time, SelfControlledCaseSeries for within-patient safety signal detection, CohortIncidence for computing incidence rates, CohortDiagnostics for evaluating cohort definition quality, Characterization for generating descriptive statistics, and many others.

The flow instantiates each node and executes them in dependency order. Results are written to a versioned results schema (the version suffix enables multiple analysis runs to coexist). Optionally, the flow generates a Table 1 demographic summary — a standard epidemiological table comparing baseline characteristics (age, gender, comorbidities, prior medications) between the target and comparator cohorts — which is essential for assessing whether propensity score matching achieved adequate balance. The CohortMethod module follows the large-scale propensity score methodology evaluated by Tian, Schuemie, and Suchard [70], in which an L1-regularised model fit over thousands of baseline covariates, combined with negative-control outcomes, has been shown to reduce residual confounding relative to the high-dimensional propensity score.

The flow supports four operational modes. Full execution mode runs the entire analysis specification from scratch, creating all required schemas, executing all analysis nodes, and writing all results. Direct R kernel mode delegates execution entirely to the Strategus R package running in an R process; this mode is useful for debugging because it produces R-native error messages and stack traces. Results cleanup mode drops a versioned results schema and its contents, allowing a fresh re-run without leftover data from a previous execution. Import mode loads pre-computed results from external storage into the results schema, enabling collaboration between sites that run analyses locally and share results centrally — a common pattern in multi-site observational studies where patient data cannot leave the institution but aggregated results can.

Artifact Sharing

The artifact sharing flow manages the visibility of concept sets and cohort definitions. It processes lists of artifact IDs and sets their sharing flag through the Portal API, making them visible (or invisible) to other users with access to the same dataset.

Patient Loyalty Scoring

The loyalty scoring flow computes data completeness scores that estimate how reliably a patient's medical history is captured in the database. This is important because observational databases are rarely complete — patients move between healthcare systems, and their records may be fragmented across multiple institutions.

The flow extracts patient-level features that serve as proxies for data completeness: visit frequency patterns (regular visits suggest ongoing care at the institution), medication counts

(more prescriptions suggest more complete pharmacy data), diagnosis diversity (a patient with records spanning many clinical domains likely receives most of their care locally), and provider variety (seeing multiple specialists suggests the institution is the patient's primary care hub).

These features are combined using a Lasso regression model. For each patient p, the loyalty score is computed as:

$$\text{loyalty}(p) = \sigma\left(\sum_{i=1}^{n} w_i \cdot f_i(p) + b\right) \tag{15}$$

where $f_i(p)$ are the extracted feature values, w_i are the pre-trained Lasso coefficients, b is the intercept, and σ is the logistic function that maps the raw score to the $[0, 1]$ interval. The Lasso (Least Absolute Shrinkage and Selection Operator) model is well-suited for this task because it produces sparse coefficients — features that are not predictive of data completeness receive zero weight and are effectively ignored, which makes the model interpretable and robust to irrelevant features. Higher scores indicate patients whose records are more likely to be complete and therefore more suitable for observational studies that assume complete data capture.

Scores are written to a dedicated loyalty scoring table in the results schema, with one row per patient containing the score and the component feature values. Researchers can then use these scores as inclusion criteria when building cohorts — for example, restricting a study to patients with loyalty scores above 0.7 to reduce the risk of information bias.

A retraining mode is also available. In this mode, the flow trains a new Lasso regression model on labeled data (where completeness has been assessed by domain experts or by cross-referencing with insurance claims). The training uses cross-validation to select the regularization parameter, computes ROC-AUC (Receiver Operating Characteristic — Area Under Curve) performance metrics on a held-out test set to evaluate discrimination, and saves the updated coefficients for future scoring runs. The ROC-AUC metric indicates how well the model distinguishes between complete and incomplete patient records; a value above 0.8 is considered good discrimination. The retraining path allows each site to calibrate the model to its own patient population and data capture patterns, since the features predictive of completeness vary across institutions.

40.7 Application Deployment

One flow handles the building and deployment of interactive analytical applications. The Shiny Live deployment flow accepts R or Python Shiny application source code, compiles it into static web assets using the appropriate build tool, and uploads the result to Supabase Storage via the Portal API. The deployed application is then served through Trex's shiny-live proxy endpoint, making it accessible to researchers associated with the dataset.

40.8 Shared Infrastructure

All flows share a common infrastructure layer that handles three cross-cutting concerns.

Database Abstraction

The database abstraction layer provides a unified interface for SQL execution, schema management, and data loading across four database backends. A factory dispatches to the appropriate implementation based on the dataset's dialect: PostgreSQL (with connection pooling), SAP HANA (with support for both JWT and password authentication), TrexSQL (providing both

direct file-based access for cache databases and remote access through the PG Wire protocol), and BigQuery (via a database extension).

PostgreSQL benefits from a critical optimization: the COPY bulk loading protocol. Instead of generating INSERT statements, the PostgreSQL path streams data directly to the server via the bulk loading pathway. For large tables this is an order of magnitude faster than row-by-row inserts.

HANA connections support JWT-based session variables. When a flow connects to HANA with a JWT token, the abstraction layer sets session-level variables that HANA's row-level security can evaluate, ensuring that the flow only sees data it is authorized to access.

This abstraction means that a flow written for PostgreSQL works on HANA or BigQuery without modification — only the credentials and dialect identifier change.

R Integration

Flows that need OHDSI R packages (Achilles, DataQualityDashboard, CohortGenerator, Strategus, PhenotypeLibrary, DatabaseConnector) use a Python-R bridge that embeds a full R interpreter within the Python process. When a flow needs to call an R function, it imports the R package, constructs arguments as Python objects (strings, integers, DataFrames), and invokes the function. The bridge translates Python data types to R equivalents: Python strings become R character vectors, Python integers become R integer vectors, and Pandas DataFrames become R data.frames (with column types mapped automatically). Return values undergo the reverse conversion.

This bridge is what makes it possible to invoke OHDSI R packages from Python flows without rewriting any of the R analytical code. The R interpreter runs in-process (not as a subprocess), so there is no serialization overhead for passing large data structures.

Failure Hooks and Resilience

Complex flows that create database schemas — OMOP CDM creation, cache creation, Strategus results schema provisioning — use Prefect's failure hook mechanism to ensure cleanup on failure. The pattern is consistent across all such flows: during initialization, the flow registers a cleanup hook that captures the database connection and target schema name. If the flow raises an unhandled exception after the schema has been created, Prefect invokes the hook, which drops the partial schema.

This mechanism prevents a common operational problem: a flow fails halfway through schema population, leaving behind an empty or partially-populated schema. On retry, the flow tries to create the same schema name and fails with a "schema already exists" error. The failure hook ensures that retries start from a clean state.

Service Clients

Flows communicate with D2E services through dedicated API clients for metadata and file operations, cohort management, FHIR resource operations, the Phenotype Library, file storage, study tracking, scan reports, imaging, and OAuth token acquisition that authenticates flow-to-service calls.

PART V

Security & Operations

Part V concerns the operational lifecycle: how D2E authenticates users and authorizes their actions, how the platform is deployed and updated, how operators interact with it through the command-line interface, and how its correctness is verified through testing.

The opening chapter covers authentication and security: the OIDC [52] flow with Logto [33], the role-and-scope authorization model, the credential-encryption design, and the audit pathway. The deployment chapter covers the Docker Compose topology, the bootstrap sequence, the upgrade and rollback patterns. The CLI chapter documents the operator's primary interface for installing, configuring, and lifecycle-managing a deployment. The testing chapter covers the test architecture: unit tests, integration tests, end-to-end tests, and the policy on what gets tested where.

Read this part if you operate a D2E deployment, debug a security or deployment incident, or evaluate the platform's operational maturity.

41 Authentication and Security

41.1 Purpose

Security is not a feature of D2E; it is the precondition that makes everything else legal. A platform that lets researchers query identifiable patient records cannot exist in a regulatory regime — HIPAA [71], GDPR [11], GxP [72], institutional review — without a coherent story for who is allowed in, what they are allowed to see once they are in, where credentials are kept, and what record exists of every access. This chapter consolidates that story. It describes the authentication flow that establishes identity through Logto [33], the authorization model that decides what each identity may do at the platform and dataset levels, the credential-encryption design that keeps database passwords safe at rest, the audit logging that preserves a record of every patient-data access, and the transport security that wraps every call between services.

The single most important fact in the chapter is that database credentials never leave Trex's process unencrypted. Every other piece of the security model — the OIDC [52] flow, the authn middleware, the role-resolution logic, the per-dataset authorization checks, the request correlation IDs — is what makes that fact useful. Read in isolation, any one of those mechanisms is a generic OAuth-and-RBAC story; read together, they are the platform's answer to the question of why a clinical data warehouse can be safely exposed through a web application at all.

41.2 User Authentication Flow

User authentication in D2E follows the OAuth 2.0 [16] Authorization Code flow, a widely adopted protocol that keeps user credentials away from the application itself. The flow involves three parties — the user's browser, Trex (the application backend), and Logto (the identity provider) — and proceeds through a carefully choreographed sequence of redirects and token exchanges.

When a user first navigates to D2E and is not yet authenticated, the frontend application detects the absence of a valid session and initiates the login process. It constructs an authorization URL pointing to Logto's sign-in endpoint, embedding the client ID, a redirect URI (where the user should return after authentication), the requested scopes (openid, profile, email), and a cryptographically random state parameter that prevents cross-site request forgery. The browser redirects the user to this URL, leaving the D2E application entirely.

At Logto's sign-in page, the user authenticates. This may involve entering a username and password directly (basic authentication), or it may trigger a federated sign-in through Microsoft Entra ID [35] (formerly Azure Active Directory) if the organization has configured

the Entra ID connector. In the Entra ID case, Logto itself acts as a relaying party, redirecting the user to Microsoft's login page and receiving the result. Regardless of the authentication method, once Logto is satisfied that the user's identity has been verified, it generates a short-lived authorization code and redirects the browser back to D2E's redirect URI with this code appended as a query parameter.

The frontend receives the authorization code and sends it to Trex's `/oauth/token` endpoint. This is a critical step: the authorization code alone is not enough to obtain tokens. Trex takes the code and forwards it to Logto's token endpoint, appending the client secret — a value that only the server knows and that proves the request is coming from the legitimate application rather than an attacker who intercepted the code. Logto validates the code and client secret, and if both check out, responds with three tokens: an access token (a short-lived JWT [25] used for API authorization), an ID token (a JWT containing the user's identity claims such as name and email), and a refresh token (a long-lived opaque token used to obtain new access tokens without re-authentication).

Refresh Token Lifecycle Refresh tokens in D2E have a 14-day time-to-live. When a refresh token is used to obtain a new access token, Logto employs rotation: it issues a brand-new refresh token alongside the new access token and invalidates the old refresh token. This rotation policy limits the damage from a leaked refresh token. If an attacker uses a stolen refresh token, the legitimate user's next refresh attempt will fail (because the old token has been consumed), alerting them to the compromise. If the legitimate user uses it first, the attacker's copy becomes invalid. The 14-day TTL provides a hard upper bound: even if rotation fails to detect misuse, the token expires within two weeks.

41.3 Machine-to-Machine Authentication

Not all API calls originate from human users. Services within D2E also need to call each other — for example, a Prefect flow worker needs to call the Trex API to update job status, or the FHIR gateway needs to authenticate when accessing clinical data. These service-to-service calls use the OAuth 2.0 Client Credentials flow, which is designed for situations where there is no user to redirect to a sign-in page.

In the Client Credentials flow, the calling service presents its client ID and client secret directly to Logto's token endpoint and receives an access token in return. There is no authorization code, no redirect, and no refresh token — the service simply re-authenticates when its access token expires.

D2E maintains two M2M (machine-to-machine) applications in Logto, each serving a distinct purpose. The first, `alp-svc`, is used for general service operations: administrative tasks, user management lookups, and internal orchestration calls. The second, `alp-data`, is used specifically for data access operations where the calling service needs to read or write clinical data. This separation exists so that the two types of access can be independently audited, rate-limited, and revoked if necessary. Both M2M applications request the scope `https://alp-default`, which is the resource identifier registered in Logto for the D2E API. This scope tells Logto which resource the token is intended for and ensures the resulting JWT contains the correct audience claim.

41.4 Token Validation (authn Middleware)

Every request that reaches Trex passes through the authn (authentication) middleware, which determines whether the caller has presented a valid identity. The middleware follows a layered approach to token extraction and validation.

First, the middleware checks whether the request URL matches a public bypass list. Certain endpoints — health checks, the OAuth callback, static assets — must be accessible without authentication. If the URL matches a bypass pattern, the middleware allows the request through without further checks.

For all other requests, the middleware attempts to extract a token. It first looks for an `Authorization: Bearer` header, the standard mechanism for API clients. If no header is present, it falls back to checking cookies: specifically the `authtoken` cookie (used by the main D2E frontend) and the `fhirtoken` cookie (used by the FHIR interface). This cookie fallback allows browser-based applications to authenticate transparently without requiring JavaScript to manually attach headers to every request.

Once a token is extracted, the middleware validates its JWT signature using JSON Web Key Sets (JWKS) [24]. Rather than storing a static signing key, Trex fetches Logto's public key set from its standard JWKS discovery endpoint. This remote key set is cached locally but refreshed periodically, which means that if Logto rotates its signing keys, Trex will pick up the new keys automatically. The middleware verifies that the token's signature matches one of the keys in the set, that the token has not expired, and that the audience and issuer claims are correct. If any of these checks fail, the middleware returns a 401 Unauthorized response.

41.5 Authorization (authz Middleware)

Authentication establishes who the caller is; authorization determines what they are allowed to do. The authz middleware runs after authn and implements role-based access control with both platform-level and dataset-level granularity.

URL-to-Scope Mapping The core of the authorization system is the `REQUIRED_URL_SCOPES` configuration, a mapping that associates URL path patterns and HTTP methods with the roles required to access them. For example, a POST to `/api/admin/users` might require the `ALP_USER_ADMIN` role, while a GET to `/api/datasets/:id/cohorts` might require the `RESEARCHER` role. The middleware matches the incoming request against these patterns and determines which roles are needed.

Role Resolution How roles are resolved depends on the type of token. For Entra ID tokens and M2M client credential tokens, roles are embedded directly in the token claims — the JWT itself declares what the caller can do. For regular user tokens, the middleware takes a different approach: it calls the UserMgmtAPI to look up the user's group memberships from the `usermgmt` schema in PostgreSQL. These groups map to roles, and the middleware assembles the complete set of roles the user holds.

The platform defines several role tiers. Administrative roles like `ALP_USER_ADMIN` and `ALP_SYSTEM_ADMIN` grant broad platform-wide access to user management and system configuration respectively. Tenant-scoped roles like `TENANT_ADMIN` and `TENANT_VIEWER` restrict access to resources within a specific organizational boundary. The `ALP_DASHBOARD_VIEWER` role provides read-only access to dashboards and visualizations without the ability to modify data or configurations.

Dataset-Level Authorization Research-oriented endpoints require an additional layer of authorization beyond platform roles. When a request targets a dataset-specific resource, the middleware checks that the user holds the `STUDY_RESEARCHER_ROLE` for the specific dataset being accessed. The dataset identifier is extracted from the request — it may appear as a query parameter, in the request body, or in the `x-dataset-id` header, depending on the endpoint. The middleware then verifies that the user's role assignments include research access

to that particular dataset. This per-dataset check ensures that a researcher granted access to one study cannot accidentally or deliberately access data from another. Specialized variants like `STUDY_WRITE_DQD_RESEARCHER` (write access to Data Quality Dashboard [45] results) and `STUDY_RESULTS_READ_RESEARCHER` (read access to analysis results) provide finer-grained control over what researchers can do within their assigned datasets.

41.6 Entra ID Group Mapping

Organizations that use Microsoft Entra ID for identity management can synchronize their Entra group structure with D2E's role system. When Entra ID integration is enabled, D2E periodically queries Microsoft's Graph API [36] to fetch group memberships for authenticated users. The mapping between Entra ID groups and D2E roles is defined in a platform configuration that associates each D2E role name with one or more Entra ID group identifiers.

During user provisioning — which happens automatically when a user first signs in through Entra ID — the system reads the user's Entra group memberships, consults the mapping, and assigns the corresponding D2E roles. This auto-provisioning means administrators do not need to manually create user accounts or assign roles in D2E; the Entra group structure serves as the single source of truth. Role synchronization occurs on subsequent logins as well, so if a user is added to or removed from an Entra group, their D2E roles update accordingly the next time they authenticate.

41.7 Transport Security

All internal communication between D2E services uses TLS encryption with certificates generated by an internal Certificate Authority managed by Caddy [6]. The certificate chain follows a three-level hierarchy: a root CA certificate, an intermediate certificate, and per-service certificates. Services communicate using the `alp.local` domain, with each service presenting its own certificate during the TLS handshake.

The `x-tls` environment variable group distributes the CA certificate, intermediate certificate, domain name, and private key to every service that needs to verify or present certificates. This internal CA approach means the platform does not depend on external certificate authorities for service-to-service communication, and certificates can be rotated quickly without coordinating with third parties. The trade-off is that clients outside the platform (such as browser connections to the frontend) still use externally-issued certificates through Caddy's public-facing TLS termination.

41.8 Credential Encryption

Database credentials — the usernames, passwords, and connection strings that D2E uses to connect to clinical data warehouses — are encrypted at rest in PostgreSQL [69] using an RSA public/private key pair. The public key is distributed to services that need to store credentials, organized by key source (such as "DataPlatform" for external database connections and "Internal" for platform-managed databases). When a service stores credentials, it encrypts them with the public key before writing to PostgreSQL.

The private key is held exclusively by Trex. When Trex needs to connect to a database, it reads the encrypted credentials from PostgreSQL, decrypts them with the private key, and uses the plaintext values only in memory for the duration of the connection. This design ensures that even if the PostgreSQL database is compromised, the credentials remain unreadable without the private key. Per-study credential isolation takes this further: each dataset's database credentials are independently managed, so compromising the credentials for one study does not expose credentials for any other study.

42 Security and Observability

42.1 Database Credential Resolution

Each study has its own credentials resolved at request time. The Portal Service provides metadata for accessible studies, and the middleware extracts the dataset identifier from the request. Credentials are then assembled from the encrypted store: host address, schema names, database dialect, and connection pool parameters. For SAP HANA connections, schema names are automatically converted to uppercase to match HANA's case-sensitivity requirements. Credentials are resolved entirely server-side and are never exposed to the frontend application.

42.2 Role-Based Access Control

The platform enforces two primary access tiers. System Administrators have full access to all configurations, metadata, and administrative operations, including the ability to view complete attribute statistics across all datasets. Standard users (including researchers) have access limited to their assigned configurations and studies, and see restricted metadata designed to prevent indirect patient identification through small-cell suppression and similar privacy techniques.

42.3 Audit Logging

Audited Operations

The following operations generate audit records: patient list retrieval, patient list export, patient list streaming, patient summary access, and plugin data retrieval. These represent the operations most relevant to patient privacy, where a record of who accessed what and when is essential for regulatory compliance.

Captured Information

Each audit record captures the patient identifier, the access channel through which the data was retrieved, the identity of the user making the request, every accessed attribute tagged with its configuration ID and version, the success or failure status of the operation, and optional attachment metadata. Records are processed in batches of 10 to balance between timely logging and database write efficiency.

42.4 Request Correlation Tracking

Every request entering the platform receives a UUID v4 correlation ID. If a correlation ID is already present (for example, from an upstream load balancer), it is preserved rather than replaced. This identifier attaches to all log messages generated during request processing, propagates across service boundaries in the `x-req-correlation-id` header, and enables end-to-end request tracing through the distributed system.

42.5 Error Handling

Error Response Structure

The centralized error handler logs full diagnostic details — including stack traces and request context — with a unique GUID, then returns only the log ID, error type, and a localized message to the client. This approach prevents sensitive internal details from leaking to callers while providing operators with enough information to diagnose issues.

Multi-Language Error Messages

Error messages are generated using a text library that selects the appropriate template based on the user's language preference, extracted from their authentication token. This allows the platform to serve a multilingual user base without hardcoding error strings.

Input Validation and Security

The platform applies several layers of input validation to prevent common attack vectors. Prototype pollution prevention rejects request parameters containing dangerous property names such as `__proto__` or `constructor`. Schema and table name validation enforces a strict alphanumeric pattern, preventing SQL injection through dynamic identifier construction. Request body size is capped at 50MB to prevent denial-of-service through oversized payloads. Parameter type checking ensures values conform to expected types before any processing logic executes.

42.6 Failure Modes

Authentication failure manifests in two architectural shapes: identity-side problems (Logto unavailable, JWKS stale, Entra ID drift) which appear as 401s, and authorization-side problems (scope tables out of date, dataset grants missing) which appear as 403s. Credential-decryption failures fall into a third shape: the RSA keypair is wrong for the data it is being asked to decrypt. The operational documentation describes the triage procedures for each.

43 Deployment and Operations

43.1 Purpose

Deployment is the moment the platform exists. Until a `docker compose up` succeeds and the dependency chain from PostgreSQL [69] through Logto [33] through Trex completes its bootstrap sequence, none of the architecture described in earlier parts is more than source code. This chapter describes that moment in operational detail — the Docker Compose topology, the strict startup ordering, the volume layout that defines what survives a restart, the YAML anchors that distribute shared environment variables, and the plugin init functions that turn an empty database into a configured one.

Operators read this chapter twice: once before they deploy, to understand what the platform expects of its host, and again during incidents, to recognize which layer of the bootstrap sequence is failing. The chapter is structured to support both readings. The early sections describe the topology and the dependency graph in the order an operator would encounter them; the later sections describe the upgrade and rollback patterns that come into play once the platform is running and the question becomes how to evolve it without losing data or trust.

43.2 Container Topology

D2E runs as a set of Docker containers, divided conceptually into the always-on core and a small set of optional capabilities that are enabled only when needed. The core consists of Trex (the application server and plugin host), PostgreSQL (the platform's primary relational store), Logto (the identity provider), Caddy [6] (the reverse proxy and TLS terminator), Redis [57] (the caching layer), Supabase Storage [66] (the object-storage layer), the Prefect [55] server and worker (workflow orchestration), and Enterprise Gateway [56] (Jupyter kernel lifecycle management). Optional capabilities — DICOM [38] imaging, the OHDSI Atlas [44] vocabulary infrastructure — are activated by profile so that organizations that do not need them do not pay their cost.

The architectural observation behind the topology is that each container owns one concern. Identity is in Logto, not in Trex. Caching is in Redis, not in PostgreSQL. Object storage is in Supabase Storage, not on the application's local filesystem. This separation is what makes the platform recoverable: a damaged container can be replaced without rebuilding the rest, because state lives in volumes and identity lives in Logto.

43.3 Startup Dependency Chain

The platform follows a strict dependency-ordered startup. PostgreSQL must be ready before Logto, because Logto stores its identity data in a PostgreSQL schema. Logto must be ready before Trex, because Trex needs valid OAuth [16] client credentials to proxy token exchanges. Trex must be ready before Prefect, because Prefect's flow registrations are made by Trex's plugin init functions. Two one-shot init containers sit between these stages: a database-management init that creates the platform's schemas and database users before any application starts, and a Logto post-init that registers D2E's client applications, API resources, roles, and (optionally) the Entra ID [35] social connector before Trex comes online.

The architectural point is that the ordering is not arbitrary — each step depends on configuration the previous step established. The implementation details of how readiness is detected, how often it is polled, and how long the platform waits for each stage live in the Compose file and the operational documentation. The chain itself is the thing worth understanding.

43.4 Plugin Init Functions

Trex's plugin architecture includes an init function pattern: each plugin can register a function that runs exactly once during Trex's startup sequence. These functions handle schema creation, seed data insertion, flow registration with Prefect, bucket creation in object storage, and any other one-time setup a subsystem requires before it can serve requests. They are the mechanism by which an empty database becomes a configured platform.

Init functions matter architecturally because they keep the deployment surface uniform. Adding a new plugin does not require an operator to run a separate migration tool or remember to seed a table; the plugin declares its setup in code, and the platform runs it on first start. Each function operates within a transactional boundary with rollback on failure, so a partial deployment leaves no half-configured schema behind.

43.5 Persistent State

Persistent state lives in named Docker volumes; the platform's threat model assumes these volumes are backed up by the operator; the runtime treats their existence as a precondition. Anything in a volume — relational data, cached query results, uploaded files, internal CA material, installed R packages — survives a restart. Anything in a container's filesystem does not.

43.6 Environment Configuration

D2E's Docker Compose file uses YAML anchors to define reusable groups of environment variables shared across services. Two reasons drive the pattern: it keeps the configuration DRY (each database connection string, TLS path, or shared secret is defined once), and it enforces consistency (every service connecting to PostgreSQL inherits identical pool settings, so connection behaviour is predictable across the platform). When a new service is added, it references the relevant anchor groups rather than duplicating environment variables; rotating a shared password becomes a one-line edit.

43.7 Failure Modes

Deployment has three failure surfaces: bootstrap (the platform doesn't come up cleanly), runtime upgrade (a healthy platform breaks during an update), and rollback (recovering from the previous two requires coordination across plugin versions, schema migrations, and audit-log preservation). The operational documentation describes the procedures; the architectural observation is that these three surfaces exist because the platform is composed of many independently-versioned pieces that must agree at runtime.

44 Command-Line Interface

44.1 Overview

The D2E command-line interface is the primary tool for operators deploying and managing the platform. It wraps Docker Compose with platform-specific concerns that would otherwise require manual coordination: generating environment files with cryptographic secrets, selecting which optional services to include, allocating system resources, and managing the lifecycle of a multi-container deployment.

The design philosophy is that a new operator should be able to go from a fresh checkout to a running D2E instance with two commands: one to initialize the environment, and one to start the services. Everything needed for that transition — passwords, certificates, keys, resource

limits, service configuration — is generated automatically during initialization. Customization is available but never required for a basic deployment.

44.2 Lifecycle Surface

The CLI exists because the platform has an operational shape that no off-the-shelf orchestrator can manage on its own. Generating the cryptographic material that the credential boundary depends on, choosing which optional services to include, sequencing the initial bootstrap, and providing a recoverable upgrade path are all responsibilities that sit above Docker Compose's abstraction. The CLI is what fills that gap.

The interface itself is small — init, start, stop, build, clean — and most of the complexity lives in what each command sets up rather than in the surface. Init produces a complete environment file from zero state, including the cryptographic keys that anchor the credential-encryption design described in Chapter 41. Start brings services up in dependency order. Stop and clean unwind the deployment, with clean guarded by an interactive confirmation because volume removal destroys data that may not be recoverable. Build compiles images from source. Each command is one verb; behind each verb is a sequence the operator does not have to remember.

44.3 Why a Custom CLI

The alternative was vanilla Docker Compose plus a runbook. This was rejected because the runbook had a tendency to drift from the platform's actual setup requirements as plugins were added — a new plugin would introduce a new environment variable, a new migration step, a new init dependency, and the runbook would lag the code by weeks. The CLI codifies the requirements so they cannot drift: if a plugin needs a secret generated, the init command generates it; if a service depends on another, the start command sequences them. The platform's setup story lives in code that is exercised every time someone deploys, not in a document that is read once and forgotten.

45 Testing Infrastructure

45.1 Purpose

This chapter explains how the platform's correctness is verified. It distinguishes the four testing layers — unit, backend integration, end-to-end, and performance — and the policy on what each one catches. The aim is not to enumerate every specification file or list every browser scenario; it is to give a reader confidence that they could land a change without breaking something, by making clear which signal would catch which kind of regression.

The chapter belongs in Part V because testing is an operational concern as much as a development one. Test results are the primary mechanism by which operators decide whether to deploy a build; the snapshot-based environment management that the end-to-end runner provides is itself a deployment artefact; and the failure modes of the test suite map directly onto the failure modes that operators see in production. A reader who understands which test signal corresponds to which class of breakage can read a CI report the same way they read an incident dashboard.

45.2 Testing Strategy

D2E maintains four complementary testing layers because the classes of bugs each one catches are fundamentally different. Unit tests verify business logic in isolation from infrastructure. Backend integration tests verify that API endpoints behave correctly when called against running services with real database connections. End-to-end tests drive a real browser through complete user workflows. Performance tests measure response times under controlled conditions.

The justification for all four — rather than collapsing into one or two — is that each layer is blind to the failure modes the others are designed for. Unit tests are blind to protocol mismatches between services: a test can verify that a service correctly formats a request payload while the receiving service has silently changed the structure it expects. Integration tests close that gap by exercising real HTTP contracts, but they cannot detect user-experience regressions where the backend returns correct data, the frontend renders it correctly in isolation, and the combined flow still breaks on a timing bug or a CSS change that hides a critical element. End-to-end tests catch that, but they cannot tell whether a functionally correct system has degraded from sub-second to multi-second response times due to an unoptimised query or a missing index. Performance tests catch that.

The distribution of effort follows the testing pyramid: many cheap, fast unit tests at the base; fewer integration tests in the middle, each requiring a bootstrapped environment; still fewer end-to-end tests at the top, where browser automation is inherently slower and more brittle; and a small set of targeted performance tests at the apex, run during release validation rather than on every commit.

45.3 Unit Tests

Unit tests run on Jest, integrated with NestJS's testing module. The architecturally significant fact about the layer, however, is not the runner but the dependency-injection pattern it exploits. Every backend service receives its dependencies through injection rather than direct instantiation, which means the testing module can substitute any dependency at test time without touching the service's source code. A service that normally queries PostgreSQL [69] through a repository can be tested with an in-memory mock returning hardcoded results; a service that normally calls an external authentication provider can be tested with mocks for both success-

ful and failed authentication. The result is that each test exercises only the business logic of the service under test, with no possibility of interference from infrastructure concerns.

Because no external services are involved — no database connections to establish, no HTTP requests to send, no containers to start — the entire unit suite for a service typically completes in under a minute. That speed makes it practical to run unit tests continuously during development, catching errors at the moment they are introduced rather than discovering them minutes later in integration.

45.4 Backend Integration Tests

The defining property of the integration layer (Mocha as runner, Chai for assertions) is that it exercises the same code paths production traffic does: real HTTP endpoints, real database connections, real authentication. The most consequential design choice is in that last word. Integration tests do not bypass authentication by injecting tokens or skipping middleware. The test bootstrap performs a full OIDC [52] authorisation-code exchange — the same flow a human user would trigger by signing in through the browser — and uses the resulting bearer token for every subsequent test request.

The reason is straightforward: a test that bypasses authentication tests a different code path than the one production uses, and provides no assurance that the same request would succeed through the real authentication pipeline. By going through the real OIDC flow, integration tests verify in a single execution that the token exchange works, that middleware correctly extracts and validates the token, that role resolution produces the expected permissions, and that the endpoint returns the correct response.

The cost of that fidelity is setup complexity. Before any test runs, the bootstrap initialises dedicated test databases, starts all required services and waits for their health checks, performs the OIDC flow, and seeds the environment with the dataset records, user accounts, and configuration entries that the suite expects. Only then does the test run begin. The complexity is load-bearing: every layer of it exists because removing it would make a passing test less meaningful.

45.5 End-to-End Tests

The end-to-end layer (Playwright driving Chromium) exercises the platform through the same user interactions a human operator would perform. Three properties of how it is configured deserve explanation, because each reflects a deliberate trade-off rather than a default.

Single-worker execution. Tests run sequentially, one at a time, with no parallelisation. This is a deliberate trade of speed for correctness. Clinical data operations have side effects — creating a cohort modifies shared state, materialising a patient list writes results, modifying a configuration changes the behaviour of subsequent operations. Two parallel tests creating cohorts against the same dataset would produce unpredictable results: one might see the other's cohort in its result set, or both might encounter database conflicts. Sequential execution eliminates that entire class of race conditions at the cost of longer total run time.

Four diagnostic channels per failure. When a test fails, the infrastructure captures a screenshot at the point of failure, a video of the entire execution, a network HAR with every request and response, and the browser console log including JavaScript errors. The reason there are four rather than one is that each one isolates a different layer of failure: the screenshot tells you what the browser was showing, the video tells you how it got there, the HAR tells you whether the wire traffic was correct, and the console log tells you whether the frontend itself raised an error. Together they are usually enough to diagnose a failure without local reproduction.

Volume snapshots for fast iteration. The end-to-end runner can save and restore complete Docker volume snapshots — database contents, cache files, object storage [66], configuration — so that the multi-minute platform initialisation runs once rather than once per test execution. After the first run the runner saves the post-init state; subsequent runs restore from the snapshot in seconds and proceed directly to test execution. The same mechanism doubles as a deployment artefact, because a saved snapshot is a known-good platform state that an operator can roll back to.

45.6 Performance Tests

Performance tests use HAR replay: a real browser session is captured as an HTTP Archive file and replayed programmatically to measure backend response times under controlled conditions. The point of replaying real sessions rather than benchmarking individual endpoints is that real sessions contain the dependent and concurrent request patterns users actually produce, which synthetic benchmarks do not.

The replay runs in two modes: a cached mode with a warm TrexSQL cache (typical user experience) and a non-cached mode that clears the cache before measurement (worst case, useful for detecting query regressions the cache would otherwise mask). Per request, the tool records total response time and the execution times of individual database queries, aggregated across runs with threshold-based filtering to highlight regressions.

45.7 Failure Modes

From an operator's view, each test layer signals a different kind of breakage and the right diagnostic response depends on which layer is red.

A unit-test regression — the suite that runs in seconds against mocked dependencies — almost always means a recent code change broke local logic: a calculation, a conditional branch, an edge case. The first thing to check is the diff that landed since the previous green run; the failure is rarely environmental and almost never about deployment. A unit-test failure that cannot be reproduced locally is a strong signal that a developer committed without running tests, not that the test infrastructure is at fault.

An integration-test failure points further out. The unit suite is green, so local logic is intact, but the HTTP contract between services is broken. A 401 or 403 in an integration test that previously passed usually means an authentication or authorisation regression — a Logto [33] configuration drift, a role mapping change, a JWKS [24] issue. A 500 with a stack trace usually means a serialisation mismatch or a database schema drift. Because the integration bootstrap performs a full OIDC flow, an integration suite that fails to start at all (rather than failing inside specific tests) is an environmental problem — almost always Logto, PostgreSQL, or the test-database init step rather than the platform code.

End-to-end failures have the noisiest signature. The browser surface is sensitive to timing, CSS, and state-management bugs that the API tests cannot reach, so an E2E suite can go red on a build whose unit and integration tests are entirely green. The diagnostic discipline is to read the captured artefacts in order: the screenshot first (does the page look right?), then the video (where did the interaction diverge?), then the network HAR (did a request fail or return wrong data?), then the console log (was there a JavaScript error?). A pattern of failures clustered around a single feature usually means a frontend regression in that feature; a pattern spread across unrelated features usually means an environmental problem — a slow database, a service that did not pass health check, a snapshot restored from the wrong version. The single-worker execution model is also worth remembering during diagnosis: tests run sequen-

tially, so a single early failure that leaves shared state in an unexpected condition can cascade into many later failures that are not independent of the first.

A performance regression rarely shows up as a binary failure; it shows up as a gradient. The cached-mode numbers describe what most users will experience; the non-cached numbers describe the database's actual workload. A regression that appears only in non-cached mode is a SQL or index problem masked by the cache. A regression that appears in both is something heavier — a service-level change, a degraded dependency, or a query plan that flipped.

Case Study: A Query from Click to Chart

This case study traces a single analytical query through every layer of D2E. It is the platform doing what it was built to do, with each service named at the moment it acts. The reader can use this chapter as the path that ties Parts II–V together.

The researcher's intent

A clinical researcher wants the age distribution of patients diagnosed with Type 2 Diabetes (ICD-10 [77] code E11) who were also prescribed metformin in the same calendar year, broken down by gender. They have already loaded the diabetes dataset and have Patient Analytics open in their browser.

Building the query

In the Patient Analytics canvas, the researcher adds two filter cards: one for the Diagnoses interaction with a constraint of `condition_concept_id = 201826` (Type 2 Diabetes), and one for the Drug Exposure interaction with a constraint of `drug_concept_id = 1503297` (metformin). They define a successor relationship between the cards: drug exposure must occur within 365 days of the diagnosis. They set Age as the X-axis and Gender as a stack, choose a stacked bar chart, and click Run.

Behind the canvas, the Vuex `getIFR` getter walks the normalized entity tree and assembles an Internal Filter Representation. The IFR is converted to a bookmark via `IFR2Bookmark`, serialized to JSON, deflate-compressed with pako, and base64-encoded. The result is a 1.2 kB query string parameter named `mriquery`.

Crossing the wire

A POST request goes to `/analytics-svc/api/services/query` with the compressed query as its body. Caddy [6] receives it, terminates TLS, and forwards it to Trex on the internal Docker network.

Trex's authentication middleware extracts the OIDC [52] bearer token, validates it against Logto [33]'s JWKS [24], and constructs the request context. The authorization middleware matches the route against the SCOPES table and confirms that the researcher has the `analytics:read` scope on the diabetes dataset. The request enters the Analytics Service worker.

Inside the Analytics Service

The Analytics Service's middleware pipeline runs. The Study Resolution middleware calls the Portal Service to load the metadata for every dataset the researcher can access; the diabetes dataset is among them. The Credential Resolution middleware extracts the dataset identifier, looks up the encrypted database credentials in the platform database, and decrypts them using

Trex's RSA private key. The Connection Management middleware establishes a connection to the diabetes dataset's PostgreSQL [69] instance.

With the connection in hand, the Analytics Service decompresses the IFR and dispatches it to the Query Generation Service. Because both services are co-located inside Trex, the call goes through the internal IPC dispatch map rather than HTTP — the Analytics worker sends a message naming the target service, and Trex's listener routes it directly to the Query Generation worker's handler.

Inside the Query Generation Service

The Query Generation Service receives the IFR and runs it through its four-stage pipeline.

Stage 1 (IFR validation) confirms that the filter cards are well-formed and that the referenced configuration exists. It calls the CDW Service via internal IPC to load the diabetes CDM configuration, which describes how the OMOP `condition_occurrence` and `drug_exposure` tables map to the platform's logical placeholders.

Stage 2 (FAST construction) walks the IFR and builds the FAST semantic tree. Each filter card becomes a context node; the successor relationship becomes a temporal-window constraint between the two contexts; the X-axis becomes a grouping dimension; the gender stack becomes a secondary grouping dimension.

Stage 3 (AST emission) lowers the FAST tree into an abstract syntax tree — a database-shape representation where optimization passes run. The optimizer applies several passes: it pushes the diagnosis-code constraint into the join (rather than evaluating it as a post-filter), it rewrites the temporal window into a self-join with a date-difference predicate, and it adds an aggregation pass for the count-by-age-and-gender output shape.

Stage 4 (SQL generation) renders the AST into PostgreSQL-dialect SQL. Placeholder names from the CDM configuration resolve to actual schema-qualified table and column names: `@PATIENT` becomes `diabetes_data.person`, `@CONDITION` becomes `diabetes_data.condition_occurrence`, and so on. The final query is approximately 60 lines long, with two CTEs and three joins.

The SQL string returns to the Analytics Service through the IPC channel.

Executing the SQL

The Analytics Service executes the SQL on its open PostgreSQL connection. The query returns 14 rows — one per (age decile, gender) combination present in the cohort.

The result-formatting layer of the Analytics Service shapes the rows into the JSON structure the frontend's StackBarChart component expects: a categories array (the age deciles), a traces array (one per gender), and a values matrix.

Returning to the canvas

The JSON response travels back through the same path: Analytics Service → Trex → Caddy → the researcher's browser. The Vuex chart module updates with the new data; the StackBarChart component, watching for changes, re-renders. Plotly draws the stacked bars.

The researcher sees their answer.

What this case study illustrates

A single click traversed Caddy, Trex, two cloud functions (Analytics and Query Generation), the CDW Service, the Portal Service, Logto, and PostgreSQL. It crossed the platform's authentication boundary, its credential boundary, its IPC channel, and its database connection. It exercised the four-stage pipeline that gives the Query Engine its dialect independence. It produced a chart-shaped JSON response that the Vue [74] micro-frontend rendered inside a React [34] shell.

Every component named in this story has its own chapter in this book, and every chapter exists because the component plays an irreducible role in stories like this one.

Case Study: From Source Data to a Queryable Dataset

The first case study traced a query from click to chart — the platform serving the user. This second case study traces the path that has to complete before such a query can run — the platform admitting new data into itself. It is the same machinery in reverse, and reading it after the query case study clarifies what each component is for.

The starting state

A research-collaborating hospital has agreed to contribute observational data to the platform: five years of inpatient encounters covering roughly 80,000 patients. The data arrives as a tarball of CSV files, one per source table. The source vocabulary is mixed — ICD-10 [77] for diagnoses, source-specific codes for procedures, hospital-internal codes for some lab measurements. None of it is OMOP yet.

The operator's job is to turn this tarball into a dataset researchers can query through Patient Analytics. The platform's job is to make every step of that transition observable, recoverable, and authorised.

Provisioning the dataset

The operator opens the admin UI in the platform portal and registers a new dataset there. The portal authenticates the operator as an admin through the OIDC flow, then calls the Dataset Service through the standard authenticated route. The Dataset Service performs three operations in sequence: it creates the database schemas the new dataset will live in (a data schema for the OMOP tables, a vocabulary schema, a results schema for cohorts and characterization output), it registers metadata about the dataset with the Portal Service, and it initializes the access-grant table so that the operator's user account has researcher access to the dataset for testing.

At this point the dataset exists structurally but is empty. Querying it returns no rows because no rows have been written.

Triggering the ETL flow

The operator submits an ETL job through the Job Plugins Service, naming the new dataset and pointing at the tarball's location in Supabase Storage [66] (where it was uploaded earlier). Job Plugins translates the request into a Prefect [55] deployment trigger: it issues a flow-run create call to Prefect's API, passing the parameters Prefect's worker pool will receive. Prefect schedules the run against the dataflow worker pool, and within seconds a worker container picks it up.

The flow runs in five phases.

Phase 1: Extract and stage

The flow downloads the tarball from Supabase Storage into the worker's filesystem and extracts it. Each CSV file is loaded into a staging schema in the new dataset's database — a one-to-one copy of the source data, untransformed, with only the column types coerced from CSV strings to SQL types. The staging schema is throwaway; it exists so that subsequent phases can run multiple SQL passes over the source without re-reading flat files.

Phase 2: Concept mapping

The flow extracts the distinct values from each source-coded column and submits them to the Concept Mapping Service. For ICD-10 diagnosis codes, this is mostly automatic: the OMOP vocabulary already contains the standard mappings. For the hospital-internal procedure and lab codes, the service falls back to embedding-based suggestion: the source code name is encoded with the same sentence-embedding model used by the Terminology Service, and the nearest standard concepts in vector space are returned as a short candidate list (typically three) for a human reviewer to choose from. The flow continues with placeholder mappings that the review process will replace.

The Concept Mapping Service writes the resulting source-to-OMOP-concept mapping rows into the dataset's vocabulary schema. The Terminology Service can now resolve every source code the dataset contains to its OMOP concept identifier, with the caveat that low-confidence mappings will produce defensible-but-not-final results until reviewed.

Phase 3: OMOP transformation

The flow runs the OMOP transformation SQL: a sequence of `INSERT...SELECT` statements that read from the staging schema, join against the vocabulary mappings, and write into the OMOP CDM tables. Patient demographics become `person` rows. Encounters become `visit_occurrence` rows. Diagnosis line items become `condition_occurrence` rows with `condition_concept_id` populated from the vocabulary join. Procedures and labs follow the same pattern.

Each `INSERT...SELECT` runs in a transaction. If any step fails, the transaction rolls back and the flow halts; the partial OMOP state is discarded and a re-run starts from the staging schema (which is unchanged). This idempotent-from-staging design is what makes ETL recoverable: a failed run does not leave the dataset in a corrupt intermediate state.

Phase 4: Characterization

The Achilles [43] characterization flow runs against the populated OMOP tables. Achilles is an OHDSI tool; the platform invokes it through the same Job Plugins / Prefect path that brought us here, just with different parameters. The output is a set of pre-computed summary statistics — record counts per concept, demographic breakdowns, temporal distributions — that the data characterization reports in the Analytics Service consume.

This step is optional in the sense that researchers can query the dataset before Achilles finishes, but the characterization views in the UI will be empty until it does. In production deployments the characterization phase is part of the standard ETL pipeline because the up-front cost is small relative to the query-time cost of computing these statistics on demand.

Phase 5: Configuration generation

The flow constructs an initial CDM configuration from the OMOP tables it just populated. The CDW Service's auto-suggest endpoint inspects the database schema, identifies the OMOP tables and columns, and proposes a configuration mapping the platform's logical placeholders (`@PATIENT`, `@CONDITION`, `@DRUG`) to the physical table and column names. The generated configuration is saved as a draft.

The flow then triggers the CDW Service's validation pass, which executes every expression in the configuration against the live database and reports any failures. If validation passes — typically the case for cleanly-mapped OMOP datasets — the configuration is activated.

A PA configuration is authored next, deriving the available interactions from the active CDM configuration. The PA Configuration Service publishes it, validating consistency with the CDW configuration.

The dataset becomes queryable

At this point the dataset is structurally complete: schemas provisioned, OMOP tables populated, vocabulary mappings in place, characterization computed, CDM and PA configurations published, access grants initialized. A researcher with the appropriate role logs in, opens Patient Analytics, sees the new dataset in their dataset selector, and can construct queries against it. The first such query traverses the same path documented in the previous case study — click to chart — and returns answers about the contributing hospital's patient population.

What this case study illustrates

Where the query case study showed every component reading and serving, this case study shows every component writing and accepting. Dataset Service provisioned. Job Plugins orchestrated. Prefect executed. Concept Mapping interpreted. Terminology resolved. CDW Service generated and validated. PA Config Service published. Portal Service registered. The same components, the same boundaries, the same authorization layers — working in the opposite direction.

Two design principles dominate the ETL story. Principle 4 (declarative over imperative) shows up in how the configuration is generated: the auto-suggest path produces a declaration, validation tests the declaration, the Portal stores the declaration. Principle 9 (object storage canonical, database curated) shows up in how the source data is staged: it lives in Supabase Storage as files until the moment its meaning is established, at which point structured records flow into PostgreSQL [69].

A dataset that has been through all five phases is not just data the platform stores. It is data the platform reasons about.

Themes and Trajectories

A platform like D2E does not have a single design. It has a small number of recurring patterns that, once you have read enough chapters, begin to compose into a worldview. This closing chapter names them.

Composition over construction

D2E is built from packages. The Trex runtime contains no business logic; everything that the platform does, it does because some plugin's manifest told it to. The cloud functions, the UI applications, the Prefect [55] flows, even the database extensions all enter the system through the same uniform mechanism. This is the deepest design choice in the platform, and it shows up everywhere: in the way the security policy is declared (per-plugin scope tables, not central configuration), in how new capabilities are added (a new `package.json`, not a new branch in core), in how operators tune deployments (a JSON array in an environment variable, not code).

The cost is that the system is harder to debug at startup. The benefit is that it composes.

The credential boundary is the platform boundary

D2E's most consequential security decision is that database credentials are encrypted with an RSA key pair whose private half lives only inside Trex. Tenants cannot access each other's data because they cannot see each other's credentials, and credentials never leave the runtime. This is what makes the multi-tenant story credible. Everything in the security architecture — the layered authentication, the per-request credential resolution, the dataset-scoped authorization — exists to maintain this boundary.

If you remember one thing about the platform's security model, remember the encrypted credentials. Everything else follows.

The query pipeline is a compiler

The Query Engine's four-stage pipeline (IFR $\rightarrow$ FAST $\rightarrow$ AST $\rightarrow$ SQL) is a compiler. It has the structure of a compiler — semantic analysis, intermediate representation, optimization passes, code generation — and it has a compiler's virtues: testability at each stage, dialect independence, optimization that survives the change of any single backend. This is unusual in clinical analytics tools, which typically generate SQL strings directly. The investment paid off: the same IFR runs against PostgreSQL [69], HANA, BigQuery (via the cache), and DuckDB [9] without rewrites.

Frontend coexistence

The Vue [74]-inside-React [34] arrangement of the frontend is not an accident, and it is not waiting to be cleaned up. It reflects a pragmatic decision: rewriting Patient Analytics in the portal's framework would have cost months and added no functional value. The micro-frontend boundary is the cost of admitting that the most valuable code is sometimes the code you wrote five years ago in a different framework. The platform pays the cost so the user does not.

Where the platform is going

D2E continues to evolve along three axes. The AI services are the newest and the most rapidly changing — code suggestion, MCP [3] tool exposure, AI-assisted ETL mapping — and the chapters covering them in Part III will date faster than any others in this book. The integration surface continues to widen, particularly toward genomics data and toward more sophisticated FHIR-based workflows. And the operational surface — particularly observability and incident response — is the area where the platform's maturity has the most room to grow.

A reader returning to this book in two years should expect Part III to have shifted, Part IV to have grown, and Part V to have deepened. The platform's structural choices — the plugin system, the credential boundary, the four-stage pipeline, the micro-frontend — should be recognizable still.

Appendix A: Glossary

Term	Definition
AST	Abstract Syntax Tree — intermediate tree representation for SQL generation in the Query Engine.
CDM	Clinical Data Model — schema and mapping configuration for clinical concepts.
CDW	Clinical Data Warehouse — the database storing clinical and observational health data.
Cohort	A defined set of patients meeting specific criteria, stored as patient identifiers for reuse.
CQL ELM	Clinical Quality Language Expression Logical Model — HL7 standard that inspired FAST's design.
D2E	Data2Evidence — the platform name.
DQD	Data Quality Dashboard — OHDSI tool for data quality assessment.
FAST	Semantic intermediate representation capturing query intent in database-independent form.
FHIR	Fast Healthcare Interoperability Resources — HL7 R4 standard for healthcare data exchange.
HADES	Health Analytics Data-to-Evidence Suite — OHDSI R packages for analytics.
IFR	Internal Filter Representation — structured request format from the UI.
Kaplan-Meier	A statistical method for estimating survival functions from time-to-event data.
Logto	OpenID Connect identity provider used for D2E authentication.
MCP	Model Context Protocol — standard for exposing tools to LLMs.
OHDSI	Observational Health Data Sciences and Informatics — open-source analytics community.
OMOP	Observational Medical Outcomes Partnership — standardized common data model.
Perseus	OHDSI tool for source-to-OMOP CDM mapping visualization.
Placeholder	A logical identifier (e.g., @PATIENT) that abstracts database table references.
Prefect	Python workflow orchestration framework for ETL and analysis pipelines.
Strategus	OHDSI study execution framework for R-based analytical studies.
Trex	D2E runtime engine — Deno-based server on a Rust foundation. Hosts cloud functions as worker isolates and routes HTTP traffic.
Hono	The HTTP framework used inside Trex for route registration and middleware.
Deno	The JavaScript/TypeScript runtime that powers Trex's worker isolates.
Vuex	The state-management library used by the Vue-based Patient Analytics module.
ESZIP	A pre-bundled module format for fast Deno worker cold starts.
Micro-frontend	An architectural pattern where independent frontend applications mount inside a host shell at runtime.
SPA	Single-Page Application — a frontend architecture where the browser loads one HTML page and updates content via JavaScript.

Term	Definition
Work pool	In Prefect, a named pool of workers that execute flow runs of a particular type.
ChaCha20-Poly1305	The authenticated-encryption algorithm used for D2E's encrypted exports.
SCOPE	In Logto/OAuth terms, an authorization scope — a string that gates access to a specific platform capability.
White Rabbit	OHDSI data profiling tool for source data characterization.

Bibliography

[1] Alfred V. Aho, Monica S. Lam, Ravi Sethi, and Jeffrey D. Ullman. *Compilers: Principles, Techniques, and Tools*. Addison-Wesley, 2nd edition, 2006.

[2] Anthropic. Claude. https://www.anthropic.com/claude, .

[3] Anthropic. Model Context Protocol Specification. https://modelcontextprotocol.io/, .

[4] Andrew W. Appel. *Modern Compiler Implementation in ML*. Cambridge University Press, 1998.

[5] Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakantan, et al. Language models are few-shot learners. *Advances in Neural Information Processing Systems*, 2020.

[6] Caddy contributors. Caddy: extensible cross-platform web server. https://caddyserver.com/.

[7] Donald D. Chamberlin and Raymond F. Boyce. SEQUEL: A structured english query language. *Proceedings of the 1974 ACM SIGFIDET Workshop on Data Description, Access and Control*, pages 249–264, 1974.

[8] Deno Authors. Deno runtime documentation. https://docs.deno.com/.

[9] DuckDB Foundation. DuckDB: an in-process SQL OLAP database. https://duckdb.org/.

[10] Shahim Essaid, Jeffrey Andre, Ian M. Brooks, Katherine H. Hohman, Michael Hull, Sandra L. Jackson, Michael G. Kahn, Emily M. Kraus, Neha Mandadi, Andrey K. Martinez, Joyce Y. Mui, Bob Zambarano, and Andrey Soares. MENDS-on-FHIR: leveraging the OMOP common data model and FHIR standards for national chronic disease surveillance. *JAMIA Open*, 7(2):ooae045, 2024. doi: 10.1093/jamiaopen/ooae045.

[11] European Parliament and Council. Regulation (EU) 2016/679 (General Data Protection Regulation). https://eur-lex.europa.eu/eli/reg/2016/679/oj, 2016.

[12] David F. Ferraiolo, D. Richard Kuhn, and Ramaswamy Chandramouli. *Role-Based Access Control*. Artech House, 2nd edition, 2007.

[13] Roy Thomas Fielding. *Architectural Styles and the Design of Network-based Software Architectures*. PhD thesis, University of California, Irvine, 2000.

[14] Google. V8 JavaScript Engine. https://v8.dev/.

[15] Goetz Graefe and William J. McKenna. The volcano optimizer generator: Extensibility and efficient search. In *Proceedings of the Ninth International Conference on Data Engineering (ICDE)*, pages 209–218, 1993.

[16] D. Hardt. The OAuth 2.0 Authorization Framework. Request for Comments RFC 6749, Internet Engineering Task Force, 2012. URL https://www.rfc-editor.org/rfc/rfc6749.

[17] HL7 International. Clinical Quality Language Expression Logical Model. https://cql.hl7.org/.

[18] HL7 International. HL7 FHIR Release 4. https://www.hl7.org/fhir/R4/, 2019.

[19] HL7 International. SMART App Launch Framework. https://hl7.org/fhir/smart-app-launch/, 2023.

[20] Gregor Hohpe and Bobby Woolf. *Enterprise Integration Patterns: Designing, Building, and Deploying Messaging Solutions*. Addison-Wesley, 2003.

[21] Hono contributors. Hono: web framework on Web Standards. https://hono.dev/.

[22] George Hripcsak, Jon D. Duke, Nigam H. Shah, Christian G. Reich, Vojtech Huser, Martijn J. Schuemie, Marc A. Suchard, Rae Woong Park, Ian Chi Kei Wong, Peter R. Rijnbeek, et al. Observational health data sciences and informatics (OHDSI): Opportunities for observational researchers. *Studies in Health Technology and Informatics*, 216:574–578, 2015.

[23] George Hripcsak, Patrick B. Ryan, Jon D. Duke, Nigam H. Shah, Rae Woong Park, Vojtech Huser, Marc A. Suchard, Martijn J. Schuemie, Frank J. DeFalco, Adler Perotte, Juan M. Banda, Christian G. Reich, Lisa M. Schilling, Michael E. Matheny, Daniella Meeker, Nicole Pratt, and David Madigan. Characterizing treatment pathways at scale using the OHDSI network. *Proceedings of the National Academy of Sciences*, 113(27):7329–7336, 2016. doi: 10.1073/pnas.1510502113.

[24] M. Jones. JSON Web Key (JWK). Request for Comments RFC 7517, Internet Engineering Task Force, 2015. URL https://www.rfc-editor.org/rfc/rfc7517.

[25] M. Jones, J. Bradley, and N. Sakimura. JSON Web Token (JWT). Request for Comments RFC 7519, Internet Engineering Task Force, 2015. URL https://www.rfc-editor.org/rfc/rfc7519.

[26] E. L. Kaplan and Paul Meier. Nonparametric estimation from incomplete observations. *Journal of the American Statistical Association*, 53(282):457–481, 1958.

[27] Vladimir Karpukhin, Barlas Oguz, Sewon Min, Patrick Lewis, Ledell Wu, Sergey Edunov, Danqi Chen, and Wen-tau Yih. Dense passage retrieval for open-domain question answering, 2020.

[28] Andreas Knöpfel, Bernhard Gröne, and Peter Tabeling. *Fundamental Modeling Concepts: Effective Communication of IT Systems*. Wiley, 2005.

[29] Andreas Knöpfel, Bernhard Gröne, and Peter Tabeling. FMC: Fundamental Modeling Concepts. http://www.fmc-modeling.org/, 2005.

[30] James Lewis and Martin Fowler. Microservices: a definition of this new architectural term. https://martinfowler.com/articles/microservices.html, 2014.

[31] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems*, 2020.

[32] Linux Foundation. MLflow: an open source platform for the machine learning lifecycle. https://mlflow.org/.

[33] Logto Team. Logto documentation. https://docs.logto.io/.

[34] Meta Platforms. React. https://react.dev/.

[35] Microsoft. Microsoft Entra ID Documentation. https://learn.microsoft.com/en-us/entra/, .

[36] Microsoft. Microsoft Graph API. https://learn.microsoft.com/en-us/graph/, .

[37] MinIO, Inc. MinIO: high-performance, S3-compatible object storage. https://min.io/.

[38] NEMA. Digital Imaging and Communications in Medicine. https://www.dicomstandard.org/.

[39] Thomas Neumann. Efficiently compiling efficient query plans for modern hardware. In *Proceedings of the VLDB Endowment*, volume 4, pages 539–550, 2011.

[40] Y. Nir and A. Langley. ChaCha20 and Poly1305 for IETF Protocols. Request for Comments RFC 8439, Internet Engineering Task Force, 2018. URL https://www.rfc-editor.org/rfc/rfc8439.

[41] Observational Health Data Sciences and Informatics. Omop common data model. https://ohdsi.github.io/CommonDataModel/.

[42] Observational Health Data Sciences and Informatics. *The Book of OHDSI*. OHDSI, 2021. URL https://ohdsi.github.io/TheBookOfOhdsi/.

[43] OHDSI. Achilles: characterization of healthcare databases. https://github.com/OHDSI/Achilles, .

[44] OHDSI. ATLAS: a unified interface for the OHDSI tools. https://github.com/OHDSI/Atlas, .

[45] OHDSI. Data Quality Dashboard. https://github.com/OHDSI/DataQualityDashboard, .

[46] OHDSI. HADES: Health Analytics Data-to-Evidence Suite. https://github.com/OHDSI/Hades, .

[47] OHDSI. Perseus: source-to-OMOP mapping interface. https://github.com/OHDSI/Perseus, .

[48] OHDSI. Strategus: a coordinator for OHDSI analytic studies. https://github.com/OHDSI/Strategus, .

[49] OHDSI. USAGI: vocabulary mapping tool. https://github.com/OHDSI/Usagi, .

[50] OHDSI. WhiteRabbit: source data profiling. https://github.com/OHDSI/WhiteRabbit, .

[51] OHDSI. EUNOMIA Synthetic OMOP Dataset. https://github.com/OHDSI/Eunomia, 2023.

[52] OpenID Foundation. OpenID Connect Core 1.0. https://openid.net/specs/openid-connect-core-1_0.html, 2014.

[53] Long Ouyang, Jeff Wu, Xu Jiang, Diogo Almeida, Carroll L. Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, et al. Training language models to follow instructions with human feedback, 2022.

[54] J. Marc Overhage, Patrick B. Ryan, Christian G. Reich, Abraham G. Hartzema, and Paul E. Stang. Validation of a common data model for active safety surveillance research. *Journal of the American Medical Informatics Association (JAMIA)*, 19(1):54–60, 2012.

[55] Prefect Technologies, Inc. Prefect documentation. https://docs.prefect.io/.

[56] Project Jupyter. Jupyter Enterprise Gateway. https://jupyter-enterprise-gateway.readthedocs.io/.

[57] Redis Ltd. Redis. https://redis.io/.

[58] Regenstrief Institute. LOINC: Logical Observation Identifiers Names and Codes. https://loinc.org/.

[59] N. Sakimura, J. Bradley, and N. Agarwal. Proof Key for Code Exchange by OAuth Public Clients. Request for Comments RFC 7636, Internet Engineering Task Force, 2015. URL https://www.rfc-editor.org/rfc/rfc7636.

[60] Ravi S. Sandhu, Edward J. Coyne, Hal L. Feinstein, and Charles E. Youman. Role-based access control models. *IEEE Computer*, 29(2):38–47, 1996.

[61] Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. Toolformer: Language models can teach themselves to use tools, 2023.

[62] P. Griffiths Selinger, M. M. Astrahan, D. D. Chamberlin, R. A. Lorie, and T. G. Price. Access path selection in a relational database management system. *Proceedings of the 1979 ACM SIGMOD International Conference on Management of Data*, pages 23–34, 1979.

[63] Seo Jeong Shin, Seng Chan You, Yu Rang Park, Jin Roh, Jang-Hee Kim, Seokjin Haam, Christian G. Reich, Clair Blacketer, Dong-Sub Son, Sungho Oh, and Rae Woong Park. Genomic common data model for seamless interoperation of biomedical data in clinical practice: Retrospective study. *Journal of Medical Internet Research*, 21(3):e13249, 2019. doi: 10.2196/13249.

[64] single-spa contributors. single-spa: a JavaScript router for front-end microservices. https://single-spa.js.org/.

[65] SNOMED International. SNOMED CT. https://www.snomed.org/.

[66] Supabase, Inc. Supabase Storage. https://supabase.com/storage.

[67] Joel N. Swerdel, George Hripcsak, and Patrick B. Ryan. PheValuator: Development and evaluation of a phenotype algorithm evaluator. *Journal of Biomedical Informatics*, 97:103258, 2019. doi: 10.1016/j.jbi.2019.103258.

[68] The Apache Software Foundation. Apache Parquet. https://parquet.apache.org/.

[69] The PostgreSQL Global Development Group. PostgreSQL. https://www.postgresql.org/.

[70] Yuxi Tian, Martijn J. Schuemie, and Marc A. Suchard. Evaluating large-scale propensity score performance through real-world and synthetic data experiments. *International Journal of Epidemiology*, 47(6):2005–2014, 2018. doi: 10.1093/ije/dyy120.

[71] U.S. Department of Health and Human Services. Health Insurance Portability and Accountability Act of 1996. https://www.hhs.gov/hipaa/, 1996.

[72] U.S. Food and Drug Administration. Good Practice (GxP) Quality Guidelines. https://www.fda.gov/.

[73] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems*, volume 30, 2017.

[74] Vue.js contributors. Vue.js. https://vuejs.org/.

[75] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed H. Chi, Quoc V. Le, and Denny Zhou. Chain-of-thought prompting elicits reasoning in large language models. In *Advances in Neural Information Processing Systems*, 2022.

[76] WHATWG. Web Workers. `https://html.spec.whatwg.org/multipage/workers.html`.

[77] World Health Organization. International Statistical Classification of Diseases and Related Health Problems, 10th Revision. `https://icd.who.int/browse10/`, 2019.

[78] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations (ICLR)*, 2023.